

YRC1000 OPTIONS INSTRUCTIONS

Reference Manual For New Language Environment MotoPlus (API Function Specifications)

Upon receipt of the product and prior to initial operation, read these instructions thoroughly, and retain for future reference.

MOTOMAN INSTRUCTIONS

MOTOMAN-□□□ INSTRUCTIONS

YRC1000 INSTRUCTIONS

YRC1000 OPERATOR'S MANUAL (GENERAL) (SUBJECT SPECIFIC)

YRC1000 MAINTENANCE MANUAL

YRC1000 ALARM CODES (MAJOR ALARMS) (MINOR ALARMS)

THIS MATERIAL IS FOR STUDY PURPOSE ONLY.
YOU MUST READ THE MANUAL WHICH ENCLOSED
WITH A ROBOT.



DANGER

- This manual explains MotoPlus of the YRC1000 system. Read this manual carefully and be sure to understand its contents before handling the YRC1000. Any matter, including operation, usage, measures, and an item to use, not described in this manual must be regarded as "prohibited" or "improper".
- General information related to safety are described in "Chapter 1. Safety" of the YRC1000 INSTRUCTIONS. To ensure correct and safe operation, carefully read "Chapter 1. Safety" of the YRC1000 INSTRUCTIONS.



CAUTION

- In some drawings in this manual, protective covers or shields are removed to show details. Make sure that all the covers or shields are installed in place before operating this product.
- YASKAWA is not responsible for incidents arising from unauthorized modification of its products. Unauthorized modification voids the product warranty.

NOTICE

- The drawings and photos in this manual are representative examples and differences may exist between them and the delivered product.
- YASKAWA may modify this model without notice when necessary due to product improvements, modifications, or changes in specifications. If such modification is made, the manual number will also be revised.
- If your copy of the manual is damaged or lost, contact a YASKAWA representative to order a new copy. Be sure to tell the representative the manual number listed on the front cover.

Notes for Safe Operation

Read this manual carefully before installation, operation, maintenance, or inspection of your product.

In this manual, the Notes for Safe Operation are classified as “DANGER”, “WARNING”, “CAUTION”, or “NOTICE”.

**DANGER**

Indicates an imminently hazardous situation which, if not avoided, will result in death or serious injury. Safety Signs identified by the signal word DANGER should be used sparingly and only for those situations presenting the most serious hazards.

**WARNING**

Indicates a potentially hazardous situation which, if not avoided, will result in death or serious injury. Hazards identified by the signal word WARNING present a lesser degree of risk of injury or death than those identified by the signal word DANGER.

**CAUTION**

Indicates a hazardous situation, which if not avoided, could result in minor or moderate injury. It may also be used without the safety alert symbol as an alternative to “NOTICE”.

NOTICE

NOTICE is the preferred signal word to address practices not related to personal injury. The safety alert symbol should not be used with this signal word. As an alternative to “NOTICE”, the word “CAUTION” without the safety alert symbol may be used to indicate a message not related to personal injury.

Even items described as “CAUTION” may result in a serious accident in some situations.

At any rate, be sure to follow these important items.



To ensure safe and efficient operation at all times, be sure to follow all instructions, even if not designated as “DANGER”, “WARNING” and “CAUTION”.



DANGER

- Before operating the manipulator, make sure the servo power is turned OFF by performing the following operations. When the servo power is turned OFF, the SERVO ON LED on the programming pendant is turned OFF.
 - Press the emergency stop buttons on the front door of the YRC1000, on the programming pendant, on the external control device, etc.
 - Disconnect the safety plug of the safety fence. (when in the play mode or in the remote mode)

If operation of the manipulator cannot be stopped in an emergency, personal injury and/or equipment damage may result.

Fig. : Emergency Stop Button



- Before releasing the emergency stop, make sure to remove the obstacle or error caused the emergency stop, if any, and then turn the servo power ON.

Failure to observe this instruction may cause unintended movement of the manipulator, which may result in personal injury.

Fig. : Release of Emergency Stop



- Observe the following precautions when performing a teaching operation within the manipulator's operating range:
 - Be sure to perform lockout by putting a lockout device on the safety fence when going into the area enclosed by the safety fence. In addition, the operator of the teaching operation must display the sign that the operation is being performed so that no other person closes the safety fence.
 - View the manipulator from the front whenever possible.
 - Always follow the predetermined operating procedure.
 - Always keep in mind emergency response measures against the manipulator's unexpected movement toward a person.
 - Ensure a safe place to retreat in case of emergency.

Failure to observe this instruction may cause improper or unintended movement of the manipulator, which may result in personal injury.

- Confirm that no person is present in the manipulator's operating range and that the operator is in a safe location before:
 - Turning ON the YRC1000 power
 - Moving the manipulator by using the programming pendant
 - Running the system in the check mode
 - Performing automatic operations

Personal injury may result if a person enters the manipulator's operating range during operation. Immediately press an emergency stop button whenever there is a problem. The emergency stop buttons are located on the front panel of the YRC1000 and on the upper right of the programming pendant.

- Read and understand the Explanation of the Warning Labels before operating the manipulator.



WARNING

- Perform the following inspection procedures prior to conducting manipulator teaching. If there is any problem, immediately take necessary steps to solve it, such as maintenance and repair.
 - Check for a problem in manipulator movement.
 - Check for damage to insulation and sheathing of external wires.
- Always return the programming pendant to the hook on the YRC1000 cabinet after use.

If the programming pendant is left unattended on the manipulator, on a fixture, or on the floor, etc., the Enable Switch may be activated due to surface irregularities of where it is left, and the servo power may be turned ON. In addition, in case the operation of the manipulator starts, the manipulator or the tool may hit the programming pendant left unattended, which may result in personal injury and/or equipment damage.

Definition of Terms Used Often in This Manual

The MOTOMAN is the YASKAWA industrial robot product.

The MOTOMAN usually consists of the manipulator, the controller, the programming pendant, and supply cables.

In this manual, the equipment is designated as follows.

Equipment	Manual Designation
YRC1000 controller	YRC1000
YRC1000 programming pendant	Programming pendant
Cable between the manipulator and the controller	Manipulator cable

Descriptions of the programming pendant keys, buttons, and displays are shown as follows:

Equipment	Manual Designation
Programming Pendant	
Character Keys /Symbol Keys	The keys which have characters or symbols printed on them are denoted with []. e.g. [ENTER]
Axis Keys /Numeric Keys	[Axis Key] and [Numeric Key] are generic names for the keys for axis operation and number input.
Keys pressed simultaneously	When two keys are to be pressed simultaneously, the keys are shown with a "+" sign between them, e.g. [SHIFT]+[COORD].
Mode Switch	Mode Switch can select three kinds of modes that are denoted as follows: REMOTE, PLAY or TEACH. (The switch names are denoted as symbols)
Button	The three buttons on the upper side of the programming pendant are denoted as follows: START, HOLD, or EMERGENCY STOP. (The button names are denoted as symbols)
Displays	The menu displayed in the programming pendant is denoted with { }. e.g. {JOB}



Description of the Operation Procedure

In the explanation of the operation procedure, the expression "Select . . ." means that the cursor is moved to the object item and [SELECT] is pressed, or that the item is directly selected by touching the screen.

Registered Trademark

In this manual, names of companies, corporations, or products are trademarks, registered trademarks, or brand names for each company or corporation. The indications of (R) and TM are omitted.

Contents

1	List of Usable API in Each Mode.....	1-1
2	Task Control API.....	2-1
	mpCreateTask.....	2-1
	mpDeleteTask.....	2-3
	mpTaskSuspend.....	2-4
	mpTaskResume.....	2-5
	mpMsgQCreate.....	2-6
	mpMsgQDelete.....	2-7
	mpMsgQReceive.....	2-8
	mpMsgQSend.....	2-10
	mpErrMsgQRcv.....	2-12
	mpErrMsgQSend.....	2-14
	mpSemBCreate.....	2-16
	mpSemDelete.....	2-17
	mpSemTake.....	2-18
	mpSemGive.....	2-19
	mpErrSemTake.....	2-20
	mpTaskDelay.....	2-22
	mpGetRtc.....	2-23
	mpGetSysClkRate.....	2-24
	mpClkAnnounce.....	2-25
	mpClkGetPeriod.....	2-27
	mpStopWatchCreate.....	2-28
	mpStopWatchDelete.....	2-30
	mpStopWatchStart.....	2-31
	mpStopWatchStop.....	2-32
	mpStopWatchLap.....	2-33
	mpStopWatchReset.....	2-34
	mpStopWatchGetTime.....	2-35
	mpStopWatchGetLapNum.....	2-36
	mpStopWatchGetLapTime.....	2-37
	mpStopWatchGetAliveLapNo.....	2-38
3	User Watchdog API.....	3-1
	mpUsrWdogCreate.....	3-1
	mpUsrWdogDelete.....	3-3
	mpUsrWdogStart.....	3-4
	mpUsrWdogClear.....	3-5
4	Network API.....	4-1
	mpSocket.....	4-1
	mpListen.....	4-2
	mpAccept.....	4-3
	mpBind.....	4-4
	mpConnect.....	4-5
	mpRecv.....	4-6

Contents

mpRecvFrom.....	4-6
mpSend.....	4-7
mpSendTo.....	4-7
mpClose.....	4-8
mpHtonl.....	4-9
mpHtons.....	4-9
mpNtohl.....	4-9
mpNtohs.....	4-9
mpInetAddr.....	4-10
mpInetNtoa.....	4-10
mpInetNtoaB.....	4-10
mpGetsockname.....	4-11
mpGetpeername.....	4-12
mpSetsockopt.....	4-13
mpIoctl.....	4-14
mpSelect.....	4-15
 5 Serial Communication API (RS232C Communication).....	 5-1
mpRsInit.....	5-1
mpRsOpen.....	5-3
mpRsClose.....	5-4
mpRsSend.....	5-5
mpRsRecv.....	5-6
 6 Memory Pool Control API.....	 6-1
mpMalloc.....	6-1
mpFree.....	6-2
 7 File Control API.....	 7-1
mpCreate.....	7-3
mpOpen.....	7-5
mpRemove.....	7-7
mpClose.....	7-8
mpRename.....	7-9
mpRead.....	7-10
mpWrite.....	7-11
mpIoctl.....	7-12
mpLseek.....	7-14
mpFstat.....	7-15
mpStat.....	7-15
mpOpendir.....	7-17
mpReaddir.....	7-18
mpRewinddir.....	7-19
mpClosedir.....	7-20
mpCreatedir.....	7-21
mpRemovedir.....	7-22
mpRenamedir.....	7-23

Contents

8 Existing File Access API	8-1
mpLoadFile.....	8-2
mpSaveFile	8-4
mpRefreshFileList	8-6
mpGetFileCount	8-7
mpGetFileName	8-8
mpFdWriteFile	8-9
mpFdReadFile	8-11
mpFdGetJobList	8-13
9 System Monitor API	9-1
mpGetVarData	9-1
mpGetSVarInfo.....	9-3
mpReadIO	9-5
mpMonitor	9-7
mpGetPosVarData	9-9
mpGetUserVars.....	9-12
mpGetAlarmStatus	9-16
mpGetAlarmCode.....	9-17
mpGetMode.....	9-19
mpGetCycle.....	9-20
mpGetServoPower	9-21
mpGetPlayStatus	9-22
mpGetMasterJob	9-23
mpGetCurJob	9-25
mpGetSpecialOpStatus	9-27
mpGetJobDate	9-28
mpGetCartPos.....	9-30
mpGetCartPosEx	9-33
mpGetPulsePos	9-37
mpGetRadPos	9-40
mpGetRadPosEx.....	9-43
mpGetDegPos	9-46
mpGetDegPosEx.....	9-49
mpGetFBPulsePos	9-52
mpGetFBPulsePosEx.....	9-55
mpGetServoSpeed	9-58
mpGetFBSpeed.....	9-61
mpGetTorque	9-64
mpGetTorqueEx	9-67
mpGetSysTimes	9-70
mpGetSysVersionNo	9-72
mpGetJogSpeed	9-73
mpGetJogCoord	9-75
mpCtrlGrpId2GrpNo	9-78
mpGetToolData	9-80
mpGetEncoderTemp	9-82

Contents

mpGetCalendar	9-85
mpGetJobStepData	9-86
mpGetToolNo	9-92
 10 System Control API	 10-1
mpPutVarData	10-1
mpPutSVarInfo	10-4
mpWriteIO	10-6
mpPutPosVarData	10-8
mpPutUserVars	10-11
mpSetAlarm	10-15
mpCancelError	10-16
mpResetAlarm	10-17
mpSetCycle	10-19
mpSetServoPower	10-21
mpSetMasterJob	10-23
mpSetCurJob	10-25
mpStartJob	10-27
mpHold	10-30
mpWaitForJobEnd	10-32
mpDeleteJob	10-34
mpSetJogCoord	10-36
mpSetToolNo	10-39
mpSetJobPosData	10-41
mpParallelShiftJob	10-45
mpApplicationInfoNotify	10-49
 11 Motion Control API	 11-1
mpMotStart	11-1
mpMotStop	11-3
mpMotTargetClear	11-4
mpMotTargetSend	11-5
mpMotTargetReceive	11-8
mpMotTargetGetVacantBufNum	11-11
mpMotSetCoord	11-13
mpMotSetSpeed	11-15
mpMotSetTool	11-17
mpMotSetOrigin	11-18
mpMotSetTask	11-20
mpMotSetSync	11-22
mpMotResetSync	11-24
mpMotSetConfig	11-25
mpMotSetAccuracy	11-27
mpMotSetAccel	11-29
mpMotSetDecel	11-31
 12 Motion Monitor/Control API	 12-1

Contents

mpIMOV	12-1
mpMOVJ	12-6
mpMOVL	12-11
mpPulseMOVJ	12-16
mpPulseMOVL	12-20
mpManualMOV	12-25
13 Sensor Control API	13-1
mpReceiveSkillCommand	13-1
mpEndSkillCommandProcess	13-5
mpMeiGetJobExecTask	13-8
mpMeiGetInterpolation	13-9
mpMeiGetExecControlGroup	13-10
mpMeiPutCorrPath	13-12
mpMeiPutForcePathEnd	13-16
mpMeiPutSpdOverride	13-19
mpMeiIncrementMove	13-21
14 Robot Motion Control API	14-1
mpExRcsIncrementMove	14-1
15 Servo Monitor/Control API	15-1
mpSvsGetVelTrqFb	15-1
mpSvsStartTrqLimit	15-3
mpSvsSetTrqLimit	15-6
mpSvsEndTrqLimit	15-11
mpSvsStartTrqCtrl	15-13
mpSvsSetTrqCtrl	15-16
mpSvsEndTrqCtrl	15-20
mpSvsForceInit	15-22
16 Kinematics API	16-1
mpConvAxesToCartPos	16-1
mpConvCartPosToAxes	16-4
mpConvPulseToAngle	16-8
mpConvAngleToPulse	16-10
mpConvFBPulseToPulse	16-12
mpMakeFrame	16-14
mpInvFrame	16-16
mpRotFrame	16-18
mpMulFrame	16-20
mpZYXeulerToFrame	16-22
mpFrameToZYXeuler	16-24
mpCrossProduct	16-26
mpInnerProduct	16-27
17 Alarm List	17-1

Contents

18 Version Information for the MotoPlus API..... 18-1

18.1 Version Information for the YRC1000 MotoPlus API..... 18-1

18.2 Version Information for the YRC1000micro MotoPlus API 18-7

1 List of Usable API in Each Mode

1 List of Usable API in Each Mode

The follow table lists API available in MotoPlus.

The constant used for each API is identified as "symbolic constant". The definition calculates the constant value from arguments is identified as "macro".

Refer to the descriptions of each API for details on the symbolic constant and the macro.

API Name		Not when an alarm or error occurs				When an alarm or error occurs
		Teach mode		Play mode		
		Not in operation	In operation	Not in operation	In operation	
Task control API	mpCreateTask	○	○	○	○	○
	mpDeleteTask	○	○	○	○	○
	mpTaskSuspend	○	○	○	○	○
	mpTaskResume	○	○	○	○	○
	mpMsgQCreate	○	○	○	○	○
	mpMsgQDelete	○	○	○	○	○
	mpMsgQReceive	○	○	○	○	○
	mpMsgQSend	○	○	○	○	○
	mpErrMsgQRcv	○	○	○	○	○
	mpErrMsgQsnd	○	○	○	○	○
	mpSemBCreate	○	○	○	○	○
	mpSemDelete	○	○	○	○	○
	mpSemTake	○	○	○	○	○
	mpSemGive	○	○	○	○	○
	mpErrSemTake	○	○	○	○	○
	mpTaskDelay	○	○	○	○	○
	mpGetRtc	○	○	○	○	○
	mpClkAnnounce	○	○	○	○	○
	mpClkGetPeriod	○	○	○	○	○
	mpStopWatchCreate	○	○	○	○	○
	mpStopWatchDelete	○	○	○	○	○
	mpStopWatchStart	○	○	○	○	○
	mpStopWatchStop	○	○	○	○	○
	mpStopWatchLap	○	○	○	○	○
	mpStopWatchReset	○	○	○	○	○
	mpStopWatchGetTime	○	○	○	○	○
	mpStopWatchGetLapNum	○	○	○	○	○
	mpStopWatchGetLapTime	○	○	○	○	○
mpStopWatchGetAliveLapNo	○	○	○	○	○	
User watchdog API	mpUsrWdogCreate	○	○	○	○	○
	mpUsrWdogDelete	○	○	○	○	○
	mpUsrWdogStart	○	○	○	○	○
	mpUsrWdogClear	○	○	○	○	○

1 List of Usable API in Each Mode

API Name		Not when an alarm or error occurs				When an alarm or error occurs
		Teach mode		Play mode		
		Not in operation	In operation	Not in operation	In operation	
Network API	mpSocket	○	○	○	○	○
	mpListen	○	○	○	○	○
	mpAccept	○	○	○	○	○
	mpBind	○	○	○	○	○
	mpConnect	○	○	○	○	○
	mpRecv	○	○	○	○	○
	mpRecvFrom	○	○	○	○	○
	mpSend	○	○	○	○	○
	mpSendTo	○	○	○	○	○
	mpClose	○	○	○	○	○
	mpHtonl	○	○	○	○	○
	mpHtons	○	○	○	○	○
	mpNtohl	○	○	○	○	○
	mpNtohs	○	○	○	○	○
	mpInetAddr	○	○	○	○	○
	mpInetNtoa	○	○	○	○	○
	mpInetNtoaB	○	○	○	○	○
	mpGetsockname	○	○	○	○	○
	mpGetpeername	○	○	○	○	○
	mpSetsockopt	○	○	○	○	○
	mpIoctl	○	○	○	○	○
	mpSelect	○	○	○	○	○
Serial communication API	mpRsInit	○	○	○	○	○
	mpRsOpen	○	○	○	○	○
	mpRsClose	○	○	○	○	○
	mpRsSend	○	○	○	○	○
	mpRsRecv	○	○	○	○	○
Memory pool control	mpMalloc	○	○	○	○	○
	mpFree	○	○	○	○	○
File control API	mpCreate	○	○	○	○	○
	mpOpen	○	○	○	○	○
	mpRemove	○	○	○	○	○
	mpClose	○	○	○	○	○
	mpRename	○	○	○	○	○
	mpRead	○	○	○	○	○
	mpWrite	○	○	○	○	○
	mpIoctl	○	○	○	○	○
	mpLseek	○	○	○	○	○
	mpFstat	○	○	○	○	○
	mpStat	○	○	○	○	○
	mpOpendir	○	○	○	○	○
	mpReaddir	○	○	○	○	○
	mpRewinddir	○	○	○	○	○
	mpClosedir	○	○	○	○	○

1 List of Usable API in Each Mode

API Name		Not when an alarm or error occurs				When an alarm or error occurs
		Teach mode		Play mode		
		Not in operation	In operation	Not in operation	In operation	
Existing file access	mpLoadFile	○	2010	2080	2080	2060
	mpSaveFile	○	○	○	○	○
	mpRefreshFileList	○	○	○	○	○
	mpGetFileCount	○	○	○	○	○
	mpGetFileName	○	○	○	○	○
	mpFdWriteFile	○	2010	2080	2080	2060
	mpFdReadFile	○	○	○	○	○
	mpFdGetJobList	○	○	○	○	○
System monitor API	mpGetVarData	○	○	○	○	○
	mpGetSVarInfo	○	○	○	○	○
	mpReadIO	○	○	○	○	○
	mpMonitor	○	○	○	○	○
	mpGetPosVarData	○	○	○	○	○
	mpGetUserVars	○	○	○	○	○
	mpGetAlarmStatus	○	○	○	○	○
	mpGetAlarmCode	○	○	○	○	○
	mpGetMode	○	○	○	○	○
	mpGetCycle	○	○	○	○	○
	mpGetServoPower	○	○	○	○	○
	mpGetPlayStatus	○	○	○	○	○
	mpGetMasterJob	○	○	○	○	○
	mpGetCurJob	○	○	○	○	○
	mpGetSpecialOpStatus	○	○	○	○	○
	mpGetJobDate	○	○	○	○	○
	mpGetCartPos	○	○	○	○	○
	mpGetCartPosEx	○	○	○	○	○
	mpGetPulsePos	○	○	○	○	○
	mpGetRadPos	○	○	○	○	○
	mpGetRadPosEx	○	○	○	○	○
	mpGetDegPos	○	○	○	○	○
	mpGetDegPosEx	○	○	○	○	○
	mpGetFBPulsePos	○	○	○	○	○
	mpGetFBPulsePosEx	○	○	○	○	○
	mpGetServoSpeed	○	○	○	○	○
	mpGetFBSpeed	○	○	○	○	○
	mpGetTorque	○	○	○	○	○
	mpGetTorqueEx	○	○	○	○	○
	mpGetSysTimes	○	○	○	○	○
	mpGetSysVersionNo	○	○	○	○	○
	mpGetJogSpeed	○	○	○	○	○
	mpGetJogCoord	○	○	○	○	○
	mpCtrlGrpId2GrpNo	○	○	○	○	○
mpGetToolData	○	○	○	○	○	
mpGetJobStepData	○	○	○	○	○	
mpGetToolNo	○	○	○	○	○	
mpGetEncoderTemp	○	○	○	○	○	

1 List of Usable API in Each Mode

API Name		Not when an alarm or error occurs				When an alarm or error occurs
		Teach mode		Play mode		
		Not in operation	In operation	Not in operation	In operation	
System control API	mpPutVarData	○	○	○	○	○
	mpPutSVarInfo	○	○	○	○	○
	mpWriteIO	○	○	○	○	○
	mpPutPosVarData	○	○	○	○	○
	mpPutUserVars	○	○	○	○	○
	mpSetAlarm	○	○	○	○	○
	mpCancelError	○	○	○	○	○
	mpResetAlarm	○	○	○	○	○
	mpSetCycle	○	○	○	○	2060
	mpSetServoPower	○	○	○	○	3450
	mpSetMasterJob	○	2010	○	2010	2060
	mpSetCurJob	○	2010	○	2010	2060
	mpStartJob	2080	2080	○	2010	2060
	mpHold	○	○	○	○	○
	mpWaitForJobEnd	○	○	○	○	2060
	mpDeleteJob	○	2010	2080	2080	2060
	mpSetJogCoord	○	2010	2080	2080	2060
	mpSetToolNo	○	2010	2080	2080	2060
	mpSetJobPosData	○	2010	2080	2080	2060
	mpParallelShiftJob	○	2010	2080	2080	2060
	mpApplicationInfoNotify	○	○	○	○	○
Motion control API	mpMotStart	✕	✕	○	○	✕
	mpMotStop	○	○	○	○	○
	mpMotTargetClear	○	○	○	○	○
	mpMotTargetSend	○	○	○	○	○
	mpMotTargetReceive	○	○	○	○	○
	mpMotTargetGetVacantBufNum	○	○	○	○	○
	mpMotSetCoord	○	○	○	○	○
	mpMotSetSpeed	○	○	○	○	○
	mpMotSetTool	○	○	○	○	○
	mpMotSetOrigin	○	○	○	○	○
	mpMotSetTask	○	○	○	○	○
	mpMotSetSync	○	○	○	○	○
	mpMotResetSync	○	○	○	○	○
	mpMotSetConfig	○	○	○	○	○
	mpMotSetAccuracy	○	○	○	○	○
	mpMotSetAccel	○	○	○	○	○
	mpMotSetDecel	○	○	○	○	○
Motion monitor/ control API	mpIMOV	2080	2080	○	2010	2060
	mpMOVJ	2080	2080	○	2010	2060
	mpMOVL	2080	2080	○	2010	2060
	mpPulseMOVJ	2080	2080	○	2010	2060
	mpPulseMOVL	2080	2080	○	2010	2060
	mpManualMOV	○	2010	2080	2080	2060

1 List of Usable API in Each Mode

API Name		Not when an alarm or error occurs				When an alarm or error occurs
		Teach mode		Play mode		
		Not in operation	In operation	Not in operation	In operation	
Sensor control API	mpReceiveSkillCommand	○	○	○	○	○
	mpEndSkillCommandProcess	○	○	○	○	○
	mpMeiGetJobExecTask	○	○	○	○	○
	mpMeiGetInterpolation	○	○	○	○	○
	mpMeiGetExecControlGroup	○	○	○	○	○
	mpMeiPutCorrPath	○	○	○	○	○
	mpMeiPutForcePathEnd	○	○	○	○	○
	mpMeiPutSpdOverride	○	○	○	○	○
	mpMeiIncrementMove	×	○	×	○	×
Robot control API	mpExRcsIncrementMove	×	○	×	○	×
Servo monitor/ control API	mpSvsGetVelTrqFb	○	○	○	○	○
	mpSvsStartTrqLimit	○	○	○	○	○
	mpSvsSetTrqLimit	○	○	○	○	○
	mpSvsEndTrqLimit	○	○	○	○	○
	mpSvsStartTrqCtrl	○	○	○	○	○
	mpSvsSetTrqCtrl	○	○	○	○	○
	mpSvsEndTrqCtrl	○	○	○	○	○
	mpSvsForceInit	○	○	○	○	○
Kinematics API	mpConvAxesToCartPos	○	○	○	○	○
	mpConvCartPosToAxes	○	○	○	○	○
	mpConvPulseToAngle	○	○	○	○	○
	mpConvAngleToPulse	○	○	○	○	○
	mpConvFBPulseToPulse	○	○	○	○	○
	mpMakeFrame	○	○	○	○	○
	mpInvFrame	○	○	○	○	○
	mpRotFrame	○	○	○	○	○
	mpMulFrame	○	○	○	○	○
	mpZYXeulerToFrame	○	○	○	○	○
	mpFrameToZYXeuler	○	○	○	○	○
	mpCrossProduct	○	○	○	○	○
	mpInnerProduct	○	○	○	○	○

Alarm code

Meaning

○

: Executable

×

: Inexecutable (API Error)

2060

: Alarm or error occurrence

2080

: Different mode

2010

: Manipulator in operation

3450

: Failed to turn on the servo power

2 Task Control API

mpCreateTask

Generate the task of MotoPlus in the controller.

■ Syntax

```
int mpCreateTask
(
    int priority,           /* priority of new task */
    int stackSize,         /* size (bytes) of stack */
    FUNCPTR entryPt,       /* entry point of new task */
    int arg1,              /* 1st of 10 req'd task args to pass to func */
    int arg2,
    int arg3,
    int arg4,
    int arg5,
    int arg6,
    int arg7,
    int arg8,
    int arg9,
    int arg10
)
```

■ Parameter

[*priority*]

One of the following 4 types can be selected. If a type other than these is specified, the API returns an error.

The following four types of symbolic constants are defined.

- **MP_PRI_IO_CLK_TAKE**
This is the priority for a special task to receive the notification of the I/O control cycle.
- **MP_PRI_IP_CLK_TAKE**
This is the priority for a special task to receive the notification of the interpolation cycle.
- **MP_PRI_TIME_CRITICAL**
High priority task: Response time is from several milliseconds to several tens of milliseconds. The latency is minimal, but the processing time of this task must be short (1 msec or less) because it influences the whole system performance.
- **MP_PRI_TIME_NORMAL**
Normal priority task: Normally use this level. Response time is from several tens of milliseconds to several hundreds of milliseconds. The CPU time of this task is scheduled by the time-sharing system (TSS) with the system task.

2 Task Control API

mpCreateTask

[stack Size]

Make sure to set MP_STACK_SIZE for the *stackSize*. The stack size is set at 40 KBytes. If another size is set, the API returns an error.

■ Return Value

Task ID (TID) Normal end

-1 Error (cannot create a task due to insufficient memory; exceeding the maximum task number; wrong priority; or the specified stack size is not MP_STACK_SIZE.)

■ Note

This API creates and starts a task. The creation and the start-up are always done simultaneously. Normally, the task started can be suspended by becoming the waiting state after initialization (if needed).

The number of tasks which can be created is limited as shown below.

Priority	Description	Max. task number
MP_PRI_IO_CLK_TAKE	Notification of I/O control cycle	1
MP_PRI_IP_CLK_TAKE	Notification of interpolation cycle	1
MP_PRI_TIME_CRITICAL	High priority for general use	1
MP_PRI_TIME_NORMAL	Normal priority for general use	3



Use precautions when you select a priority other than MP_PRI_TIME_NORMAL. The priority MP_PRI_IO_CLK_TAKE, MP_PRI_IP_CLK_TAKE, and MP_PRI_TIME_CRITICAL have higher priorities than the man-machine control task. Thus, if unusually long processing time (100 microseconds or more) is used to describe a program with these priorities, critical problems may occur. (e.g. The system alarm occurs due to insufficient processing time for the robot control task; The programming pendant or the HOLD button freezes because the man-machine control task cannot be operated.)

MotoPlus detects such status (when the man-machine task does not run for 3 seconds or more), stops the robot motion by turning the servo power OFF, and turns ON the system output (\$50901) as the protective function to take counter-measures for peripheral devices. Thus, make sure to set 100 microseconds as the maximum processing time in the high-priority task (100 microseconds equals to approx. 1000 lines in the C language). The OS waiting time, the message/semaphore waiting time, and the task delay time are not included in this processing time.

mpDeleteTask

Deletes the task of the MotoPlus.

■ Syntax

```
STATUS mpDeleteTask  
(  
    int tid /* task ID of task to delete */  
)
```

■ Parameter

[*tid*]

Set the symbolic constant: TID_SELF to delete the calling task itself.

■ Return Value

Symbolic constant: OK (value: 0) : Normal end

Symbolic constant: ERROR (value: -1) : Error

■ Note

To delete a task other than the calling task itself by this API, the task ID returned when the task is created must be retained in the application.

mpTaskSuspend

Suspends the task of the specified MotoPlus.

■ Syntax

```
STATUS mpTaskSuspend  
(  
    int tid /* task I D of task to suspend */  
)
```

■ Parameter

[*tid*]

Suspends the task specified by *tid*. If *tid* is the symbolic constant: TID_SELF, the calling task itself is suspended.

■ Return value

Symbolic constant: OK (value: 0) : Normal end

Symbolic constant: ERROR (value: -1) : Error

■ Note

The specified task is suspended regardless of its status. Thus, if the task which occupies the system resource (e.g. the network or the system memory partition) is suspended, the function regarding the resource cannot be used. Then the system will deadlock.

mpTaskResume

Resumes the task of the specified MotoPlus.

■ Syntax

```
STATUS mpTaskResume  
(  
    int tid /* task ID of task to resume */  
)
```

■ Description

This API releases the specified task from the suspended state.

■ Parameter

[*tid*]
Task ID

■ Return Value

Symbolic constant: OK (value: 0) : Normal end
Symbolic constant: ERROR (value: -1) : Error

mpMsgQCreate

Creates and initializes a message queue of the MotoPlus.

■ Syntax

```
MSG_Q_ID mpMsgQCreate
(
  int maxMsgs,           /* max messages that can be queued */
  int maxMsgLength,      /* max bytes in a message */
  int options            /* message queue options */
)
```

■ Description

This API creates a message queue which can keep messages up to the number of *maxMsgs* (the size of each message is up to *maxMsgLength* bytes).

■ Parameter

[*maxMsgs*]

The max. number of messages that can be queued

[*maxMsgLength*]

The max. size of a message queue

[*options*]

- Symbolic constant: MSG_Q_FIFO (value: 0x00)
Suspended tasks wait in the sequence of FIFO.
- Symbolic constant: MSG_Q_PRIORITY (value: 0x01)
Suspended tasks wait in the sequence of priority.

■ Return Value

Nonzero	MSG_Q_ID (Normal end)
0	Error

mpMsgQDelete

Deletes the specified message queue.

■ Syntax

```
STATUS mpMsgQDelete  
(  
    MSG_Q_ID msgQId /* message queue to delete */  
)
```

■ Parameter

[msgQId]

The ID of the message queue to be deleted

■ Return Value

Symbolic constant: OK (value: 0) : Normal end

Symbolic constant: ERROR (value: -1) : Error

■ Note

This API deletes a message queue. If the specified task is blocked by mpMsgQSend() or mpMsgQReceive(), the task is released from the blocked state.

mpMsgQReceive

Receives a message from the message queue.

■ Syntax

```
int mpMsgQReceive
(
    MSG_Q_ID msgQId,      /* message queue from which to receive */
    char *buffer,          /* buffer to receive message */
    UINT maxNBytes,       /* length of buffer */
    int timeout            /* ticks to wait */
)
```

■ Description

This API receives the message from the message queue *msgQId*. The received message is written to the specified buffer. The length of the buffer is *maxNBytes*. If the message is longer than *maxNBytes*, the excess is discarded. If no message is registered in the queue when the *mpMsgQReceive()* is called, the system waits for the registration for the number of ticks set to *timeout*.

■ Parameter

[*msgQId*]

The ID of the message queue to be received

[*buffer*]

The buffer to receive the message

[*maxNBytes*]

The max length of the message to be received

[*timeout*]

The number of ticks to wait for the registration

Also the following can be set:

- Symbolic constant: *NO_WAIT* (value: 0)
End immediately even if the message is not arrived
- Symbolic constant: *WAIT_FOREVER* (value: -1)
Wait endlessly until the message arrives

■ Return Value

Value ≥ 0 Data length copied to *buffer* (bytes)

Symbolic constant: *ERROR* (value: -1) : Error
To retrieve the cause of error,
use *mpErrMsgQRcv()*

2 Task Control API

mpMsgQReceive

■ Note

The period of time per tick (msec) can be retrieved by the following API:

float mpGetRtc(void)

Return value: 0.001 or more
(unit: msec)

mpMsgQSend

Sends a message to the message queue.

■ Syntax

```
STATUS mpMsgQSend
(
    MSG_Q_ID msgQId, /* message queue on which to send */
    char *buffer,    /* message to send */
    UINT nBytes,     /* length of message */
    int timeout,     /* ticks to wait */
    int priority     /* MSG_PRI_NORMAL or MSG_PRI_URGENT */
)
```

■ Description

This API sends the message written to the buffer (length: nBytes) to the message queue msgQId. If the message queue is full, the system waits for the queue to become available for the number of ticks set to *timeout*. The priority of the message to be sent is set to *priority*.

■ Parameter

[msgQId]

The message queue ID

[buffer]

The message buffer to be sent

[nBytes]

The length (byte) of the message to be sent

[timeout]

The number of ticks to wait for the queue to be available if the message queue is full

Also the following symbolic constants (value) can be set:

- Symbolic constant: NO_WAIT (value: 0)
End immediately even if the message is not sent
- Symbolic constant: WAIT_FOREVER (value: -1)
Wait endlessly until the message arrives

[priority]

The priority of the message to be sent is specified in *priority* parameter. The following constants (values) can be specified.

- Symbolic constant: MSG_PRI_NORMAL (value: 0)
Normal: The message is set to the end of the queue.
- Symbolic constant: MSG_PRI_URGENT (value: 1)
Urgent: The message is set to the top of the queue.

2 Task Control API

mpMsgQSend

■ Return Value

Symbolic constant: OK (value: 0) : Normal end

Symbolic constant: ERROR (value: -1) : Error
To retrieve the error cause,
use mpErrMsgQsnd().

■ Note

The period of time per tick (msec) can be retrieved by the following API:

float mpGetRtc(void)

Return value: 0.001 or more
(unit: msec)

mpErrMsgQRcv

Retrieves an error when a message queue is received.

■ Syntax

```
int mpErrMsgQRcv(void)
```

■ Description

This API retrieves the cause of the error which occurs when mpMsgQReceive() is executed. Only when retrieving it immediately after mpMsgQReceive() returns the symbolic constant: ERROR (value: -1): Error, the reference value is correct.

■ Parameter

None

■ Return Value

One of the following values returns.

Value		Description
E_TIME_OUT	0x00000001	Timeout
E_NONEXIST	0x00000002	The message queue was deleted
E_OBJECTS_ID	0x00000004	Object ID error
E_OBJ_NO_METHOD	0x00000005	Other than the above

■ Note

<Sample Code>

```
int foo(void)
{
    if (mpMsgQReceive(msgQId, (char *)buffer, maxNBytes, timeout) == ERROR)
    {
        switch (mpErrMsgQRcv())
        {
            case E_TIME_OUT:
                // Some error processing is done.
                break;
            case E_NONEXIST:
                // Some error processing is done.
                break;
            case E_OBJECTS_ID:
                // Some error processing is done.
                break;
```

2 Task Control API

mpErrMsgQRcv

```
default: //case E_OBJ_NO_METHOD:
    // Some error processing is done.
    break;
    return (NG);
}
}
...
return (OK);
}
```

mpErrMsgQsnd

Retrieves an error when a message queue is sent.

■ Syntax

```
int mpErrMsgQsnd(void)
```

■ Description

This API retrieves the cause of the error which occurs when mpMsgQSend() is executed. Only when retrieving it immediately after mpMsgQSend() returns the symbolic constant: ERROR (value: -1): Error, the reference value is correct.

■ Parameter

None

■ Return Value

One of the following values returns.

Value		Description
E_TIME_OUT	0x00000001	A timeout occurs when other than NO_WAIT is set to <i>timeout</i> of mpMsgQSend().
E_NONEXIST	0x00000002	The message queue was deleted
E_LIMIT_EXCEEDED	0x00000003	A timeout occurs when NO_WAIT is set to <i>timeout</i> of mpMsgQSend(), or, a value larger than <i>maxMsgLength</i> of mpMsgQCreate() is set to <i>nBytes</i> of mpMsgQSend().
E_OBJECTS_ID	0x00000004	Object ID error
E_OBJ_NO_METHOD	0x00000005	Other than the above

■ Note

<Sample Code>

```
int foo(void)
{
    if (mpMsgQSend(msgQId, (char *)buffer, nBytes, timeout,
                  MSG_PRI_NORMAL) == ERROR)
    {
        switch (mpErrMsgQsnd())
        {
            case E_TIME_OUT:
                // Some error processing is done.
                break;
```

2 Task Control API

mpErrMsgQsnd

```
    case E_NONEXIST:
        // Some error processing is done.
        break;
    case E_LIMIT_EXCEEDED:// Overflow message queue num.
        // Some error processing is done.
        break;
    case E_OBJECTS_ID:
        // Some error processing is done.
        break;
    default://case E_OBJ_NO_METHOD:
        // Some error processing is done.
        break;
    return (NG);
}
}
...
return (OK);
}
```

mpSemBCreate

Creates and initializes a binary semaphore.

■ Syntax

```
SEM_ID mpSemBCreate  
(  
    int options,                /* semaphore options */  
    SEM_B_STATE initialState /* initial semaphore state */  
)
```

■ Description

This API allocates memory to a binary semaphore and initializes it. The semaphore is initialized to the value of *initialState*. Set SEM_FULL(1) or SEM_EMPTY(0) to *initialState*.

Set the queuing procedure of the blocked task to *options*.

Two queuing procedures are available: the priority procedure and the FIFO procedure. For the priority procedure, set SEM_Q_PRIORITY(0x1). For the FIFO procedure, set SEM_Q_FIFO(0x0).

■ Parameter

[options]

The queuing procedure of the blocked task

SEM_Q_PRIORITY(0x1): priority procedure

SEM_Q_FIFO(0x0): FIFO procedure

[initialState]

The initialized value: 0 or 1

SEM_FULL(1)

SEM_EMPTY(0)

■ Return Value

Nonzero Semaphore ID

0 ERROR: no memory

mpSemDelete

Deletes the specified semaphore.

■ Syntax

```
STATUS mpSemDelete  
(  
    SEM_ID semId    /* semaphore ID to delete */  
)
```

■ Description

This API deletes the specified semaphore and releases the memory allocated to it. The suspended task is released from the blocked state.

■ Parameter

[*semId*]

The semaphore ID to be deleted

■ Return Value

Symbolic constant: OK (value: 0) : Normal end

Symbolic constant: ERROR (value: -1) : Error (when the specified semaphore ID is invalid)

mpSemTake

Takes the specified semaphore.

■ Syntax

```
STATUS mpSemTake
(
    SEM_ID semId,      /* semaphore ID to take */
    int timeout        /* timeout in ticks */
)
```

■ Description

This API takes the specified semaphore. The timeout can be specified in ticks. If the task times out, mpSemTake() returns an error. As *timeout*, the symbolic constant: WAIT_FOREVER (symbolic constant that specifies waiting without a timeout) and NO_WAIT (symbolic constant that ends immediately even if the semaphore is not taken) can also be set.

■ Parameter

[*semId*]

The ID of the semaphore to be taken

[*timeout*]

Specifies the timeout in ticks.

If the task times out, mpSemTake() returns an error.

- NO_WAIT (value: 0)
End immediately even if the semaphore is not taken
- WAIT_FOREVER (value: -1)
Keep waiting without timeout

■ Return Value

Symbolic constant: OK (value: 0) : Normal end

Symbolic constant: ERROR (value: -1) : Error (when the semaphore ID is invalid or the task times out)
To retrieve the cause of the error, use mpErrSemTake().

mpSemGive

Sets the ID and gives a semaphore.

■ Syntax

```
STATUS mpSemGive  
(  
    SEM_ID semId /* semaphore ID to give */  
)
```

■ Parameter

[*semId*]

The ID of the semaphore to be given

■ Return Value

Symbolic constant: OK (value: 0) : Normal end

Symbolic constant: ERROR (value: -1) : Error (when the semaphore ID is invalid)

mpErrSemTake

Retrieves an error when a semaphore is retrieved.

■ Syntax

```
int mpErrSemTake(void)
```

■ Description

This API retrieves the cause of the error which occurs when mpSemTake() is executed. Only when retrieving it immediately after mpSemTake() returns the symbolic constant: ERROR (value: -1): Error, the reference value is correct.

■ Parameter

None

■ Return Value

One of the following values returns.

Value		Description
E_TIME_OUT	0x00000001	Timeout
E_NONEXIST	0x00000002	The semaphore was deleted
E_OBJECTS_ID	0x00000004	Object ID error
E_OBJ_NO_METHOD	0x00000005	Other than the above

■ Note

<Sample Code>

```
int foo(void)
{
    if (mpSemTake(semId, timeout) == ERROR)
    {
        switch (mpErrSemTake())
        {
            case E_TIME_OUT:
                // Some error processing is done.
                break;
            case E_NONEXIST:
                // Some error processing is done.
                break;
            case E_OBJECTS_ID:
                // Some error processing is done.
                break;
```

2 Task Control API

mpErrSemTake

```
        default://case E_OBJ_NO_METHOD:
            // Some error processing is done.
            break;
        return (NG);
    }
}
...
return (OK);
}
```

mpTaskDelay

Delays the task in which this API is processing.

■ Syntax

```
STATUS mpTaskDelay  
(  
  int ticks /* number of ticks to delay task */  
)
```

■ Description

This API prevents the CPU from performing the task for the time (the number of ticks) specified in the calling task. (The next line of this API is not executed until the specified time has passed.) This is generally called “manual schedule”, and can also be used when waiting for the external event which is not defined as an interruption.

■ Parameter

[ticks]

The number of ticks to delay the task

Cannot set to an endless wait.

■ Return Value

Symbolic constant: OK (value: 0) : Normal end

Symbolic constant: ERROR (value: -1) : Error

■ Note

The period of time per tick (msec) can be retrieved by the following API:

float mpGetRtc(void)

Return value: 0.001 or more
(unit: msec)

< Sample Code >

The time per tick depends on the controller. (The time per tick for the YRC1000 and YRC1000micro is the same value, but the time per tick for the DX200 is a different value.) For this reason, use this API together with mpGetRtc() as shown below to delay execution of the task for the same amount of time no matter what controller is being used.

```
float waitTime = 1000.0; /* Task delay time 1000[msec] = 1 sec */
```

```
...
```

```
mpTaskDelay((int)(waitTime / mpGetRtc()));
```

```
...
```


mpGetRtc

Retrieves the clock cycle (1 tick) of the controller OS.

■ **Syntax**

float mpGetRtc(void)

■ **Parameter**

None

■ **Return Value**

The OS clock cycle (unit: msec)

For the DX200 and FS100, mpGetRtc() returns an int type value.
However, for the YRC1000, YRC1000micro, and later robot controller,
mpGetRtc() returns a float type value that is less than 1.

mpGetSysClkRate

Retrieves the controller OS clock frequency.

■ **Syntax**

```
int mpGetSysClkRate(void)
```

■ **Parameter**

None

■ **Return Value**

The controller OS clock frequency [Hz]

mpClkAnnounce

Notifies the I/O or interpolation cycle event of the controller.

■ Syntax

```
STATUS mpClkAnnounce
(
    int clk_id
)
```

■ Parameter

[*clk_id*]

Sends one of the following values to *clk_id*

- Symbolic constant: MP_IO_CLK (value: 0)
The I/O control cycle is notified.
- Symbolic constant: MP_INTERPOLATION_CLK (value: 1)
The interpolation cycle is notified.

The task which called this API is suspended until each clock (event) is notified. (synchronized semaphore)

■ Return Value

Same as mpSemTake()

■ Note

[example code]

```
void mpUsrRoot(int arg1, int arg2,...,int arg10)
{
    int tid;

    tid = mpCreateTask(MP_PRI_IO_CLK_TAKE, MP_STACK_SIZE,
                      (FUNCPTR)mp_io_mon_task, 0, 0, 0, 0, 0,
                      0, 0, 0, 0, 0);
}

void mp_io_mon_task(void)
{
    FOREVER
    {
        mpClkAnnounce(MP_IO_CLK); // pend and arriving of clock

        // Something is done.
    }
}

// The interpolation cycle task is also similar.
```

2 Task Control API

mpClkAnnounce

NOTE

Use precautions when you select a priority other than `MP_PRI_TIME_NORMAL`. The priority `MP_PRI_IO_CLK_TAKE` and `MP_PRI_IP_CLK_TAKE` have higher priorities than the man-machine control task. Thus, if unusually long processing time (100 microseconds or more) is used to describe a program with these priorities, critical problems may occur. (e.g. The system alarm occurs due to insufficient processing time for the robot control task; The programming pendant or the HOLD button freezes because the man-machine control task cannot be operated.)

MotoPlus detects such status (when the man-machine task does not run for 3 seconds or more), stops the robot motion by turning the servo power OFF, and turns ON the system output (\$50901) as the protective function to take counter-measures for peripheral devices. Thus, make sure to set 100 microseconds as the maximum processing time in the high-priority task (100 microseconds equals to approx. 1000 lines in the C language). The OS waiting time, the message/semaphore waiting time, and the task delay time are not included in this processing time.

NOTE

The APIs which can be used in the task that performs fixed-cycle processing by using `mpClkAnnounce` are restricted. For details, refer to “Task Priority and Usable API” in chapter 16.5 of MotoPlus Programmer’s Manual¹⁾.

¹ For the YRC1000:
YRC1000 OPTIONS INSTRUCTIONS Programmer’s Manual For New Language Environment MotoPlus (HW1483598)
For the YRC1000micro:
YRC1000micro OPTIONS INSTRUCTIONS Programmer’s Manual For New Language Environment MotoPlus (HW1484526)

mpClkGetPeriod

■ Syntax

```
LONG mpClkGetPeriod (  
    int clk_id;  
);
```

■ Description

Returns the control cycle in μsec of the I/O control cycle or interpolation control cycle specified in the argument.

■ Parameter

[clk_id]

Pass one of the following values in *clk_id*

- Symbolic constant: MP_IO_CLK (value: 0)
This ID specifies the I/O control cycle.
- Symbolic constant: MP_INTERPOLATION_CLK (value: 1)
This ID specifies the interpolation cycle.

■ Return Value

>0

Returns the specified control cycle in μsec .

Symbolic constant: ERROR (value: -1): Abnormal termination (specified value is not a value listed above)

■ Note

<Sample code>

```
void SampleProc  
{  
    long InterpolationPeriod;  
    InterpolationPeriod = mpClkGetPeriod  
                        (MP_INTERPOLATION_CLK);  
}
```

mpStopWatchCreate

Creates a stopwatch of the MotoPlus.

■ Syntax

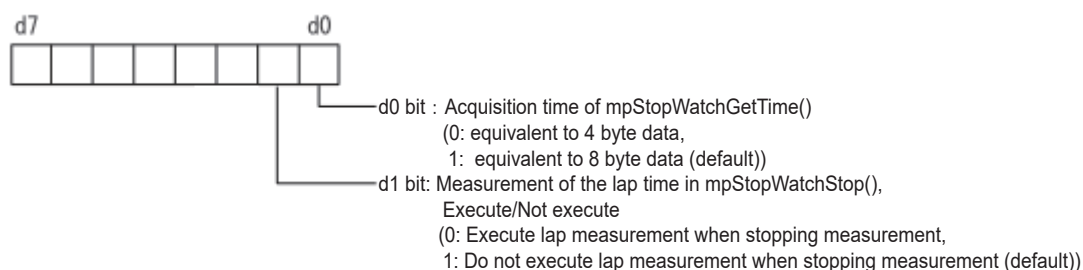
```
void * mpStopWatchCreate
(
  unsigned long maxLapTimeNum  /* Number of lap counter */
)
```

■ Description

APIs named mpStopWatchXxx() provide stopwatch functions using CPU time base. These functions are used to measure the time to execute processes. The data size of the time base is 64bits, and it is added by each CPU clock. mpStopWatchXxx APIs do not stop task scheduling and interruption. Thus, the measurement time may include the processing time of another task and interruption. The measurement time is in milliseconds (msec) and the floating point type.

For information on how to use mpStopWatchXxx() API, refer to chapter 2.6 “Processing Time Measurement” in MotoPlus Programmer’s Manual¹⁾.

If the compatibility specification (S2C1116) in the processing time measurement is set, the operation in the processing time measurement is changed. For the specification, use the bit specification.



■ Parameter

[*maxLapTimeNum*]

Number of lap time counters to be created

Creates the specified number of the stopwatches to store the lap time.

¹ For the YRC1000:
YRC1000 OPTIONS INSTRUCTIONS Programmer’s Manual For New Language Environment MotoPlus (HW1483598)
For the YRC1000micro:
YRC1000micro OPTIONS INSTRUCTIONS Programmer’s Manual For New Language Environment MotoPlus (HW1484526)

2 Task Control API
mpStopWatchCreate

■ Return Value

Stopwatch object ID	Normal end
0	Error: The stopwatch cannot be created. (insufficient memory)

■ Note

Make sure to call this API before using the stopwatch. After that, the stopwatch object ID created in this API must be specified as the first parameter for all of the stopwatch operations.

mpStopWatchDelete

Deletes the specified stopwatch.

■ Syntax

```
void mpStopWatchDelete  
(  
    void * objStopWatch    /* Stopwatch object ID */  
)
```

■ Description

Deletes the specified stopwatch.

■ Parameter

[objStopWatch]

Stopwatch object ID

The object ID of the stopwatch specifies the return value for mpStopWatchCreate().

■ Return Value

None

■ Note

This API releases the memory area allocated for the stopwatch.

mpStopWatchStart

Starts the measurement with the specified stopwatch.

■ Syntax

```
void mpStopWatchStart  
(  
    void * objStopWatch    /* Stopwatch object ID */  
)
```

■ Description

Starts the measurement with the specified stopwatch.

■ Parameter

[objStopWatch]

Stopwatch object ID

The object ID of the stopwatch specifies the return value for mpStopWatchCreate().

■ Return Value

None

mpStopWatchStop

Stops the measurement with the specified stopwatch.

■ Syntax

```
void mpStopWatchStop  
(  
    void * objStopWatch    /* Stopwatch object ID */  
)
```

■ Description

Stops the measurement with the specified stopwatch.

■ Parameter

[objStopWatch]

Stopwatch object ID

The object ID of the stopwatch specifies the return value for mpStopWatchCreate().

■ Return Value

None

mpStopWatchLap

Records the lap time with the specified stopwatch.

■ Syntax

```
void mpStopWatchLap  
(  
void * objStopWatch          /* Stopwatch object ID */  
)
```

■ Description

Records the lap time of the specified stopwatch.

■ Parameter

[*objStopWatch*]

Stopwatch object ID

The object ID of the stopwatch specifies the return value for mpStopWatchCreate().

■ Return Value

None

■ Note

If the lap time exceeds the number of lap time counter, the lap time records will be deleted in order from the oldest to the newest (from No. 0). Therefore, the number of lap time records to be stored will count back from the latest lap time counter.

mpStopWatchReset

Reset the lap time of the specified stopwatch.

■ Syntax

```
void mpStopWatchReset  
(  
    void * objStopWatch    /* Stopwatch object ID */  
)
```

■ Description

Reset the lap time of the specified stopwatch.

■ Parameter

[objStopWatch]

Stopwatch object ID

The object ID of the stopwatch specifies the return value for mpStopWatchCreate().

■ Return Value

None

■ Note

This API clears the lap time counter of the stopwatch. If the stopwatch does not have a lap time counter, this API does not run.

mpStopWatchGetTime

Retrieves the elapsed time of the specified stopwatch.

■ Syntax

```
float mpStopWatchGetTime  
(  
    const void * objStopWatch    /* Stopwatch object ID */  
)
```

■ Description

Retrieves the elapsed time of the specified stopwatch.

■ Parameter

[*objStopWatch*]

Stopwatch object ID

The object ID of the stopwatch specifies the return value for mpStopWatchCreate().

■ Return Value

This API returns the measured elapsed time in milliseconds. The elapsed time is the time from when the latest mpStopWatchStart() is called to when mpStopWatchStop() is called. However, even when mpStopWatchStop() is not executed after mpStopWatchStart() is executed, if mpStopWatchLap() is executed, the elapsed time until the latest mpStopWatchLap() is returned. In this case, the specified stopwatch continues to measure time since the state differs from that after mpStopWatchStop() is executed.

mpStopWatchGetLapNum

Retrieves the number of the lap time counter of the specified stopwatch.

■ Syntax

```
unsigned long mpStopWatchGetLapNum  
(  
    const void * objStopWatch    /* Stopwatch object ID */  
)
```

■ Description

Retrieves the number of the lap time counter of the specified stopwatch.

■ Parameter

[objStopWatch]

Stopwatch object ID

The object ID of the stopwatch specifies the return value for mpStopWatchCreate().

■ Return Value

The number of the lap time counter specified by mpStopWatchCreate() of the specified stopwatch is returned.

mpStopWatchGetLapTime

Retrieves the lap time from the specified stopwatch.

■ Syntax

```
float mpStopWatchGetLapTime
(
    const void * objStopWatch,    /* Stopwatch object ID */
    unsigned long lapNo           /* Lap number (zero origin) */
)
```

■ Description

Retrieves the lap time of the specified stopwatch.

■ Parameter

[*objStopWatch*]

Stopwatch object ID

The object ID of the stopwatch specifies the return value for `mpStopWatchCreate()`.

[*lapNo*]

Lap time number

When a number between 0 and the latest lap time number is specified, the lap time of the specified number is returned.

The following constants are provided.

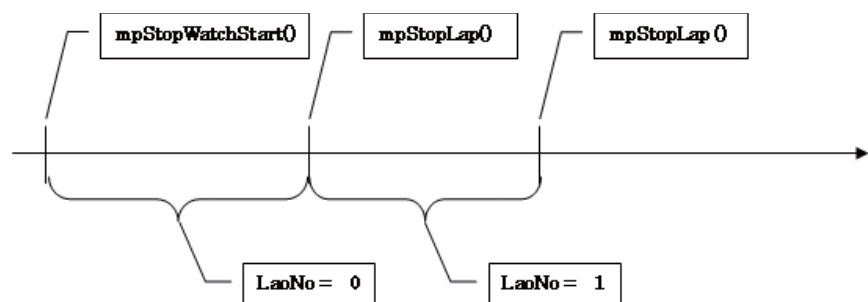
The latest lap time and the oldest lap time can be retrieved by using these constants.

- `MP_STOP_WATCH_GET_LAP_NEWEST`
The symbolic constant to retrieve the latest lap time
- `MP_STOP_WATCH_GET_LAP_OLDEST`
The symbolic constant to retrieve the oldest lap time

`mpStopWatchGetLapTime()` will always return 0 if the stopwatch does not have a lap time counter.

■ Return value

The lap time of the specified number is returned in milliseconds.



mpStopWatchGetAliveLapNo

Retrieves the range of the current valid lap time numbers from the specified stopwatch.

■ Syntax

```
void mpStopWatchGetAliveLapNo
(
    const void * objStopWatch,      /* Stopwatch object ID */
    unsigned long *oldestNo,         /* Number of the oldest lap */
    unsigned long *newestNo          /* Number of the last lap */
)
```

■ Description

Retrieves the range of the valid lap time number from the specified stopwatch.

The lap time number is returned to the area that returns the parameter number.

■ Parameter

[*objStopWatch*]

Stopwatch object ID

The object ID of the stopwatch specifies the return value for mpStopWatchCreate().

[*oldestNo*]

Oldest number of lap time set area

[*newestNo*]

Latest number of lap time set area

■ Return value

None

3 User Watchdog API

mpUsrWdogCreate

Creates a user watchdog timer.

■ Syntax

```

MP_WDG_HANDLE mpUsrWdogCreate
(
    int delay,                                /* timer value(0.001[sec]) */
    MP_USR_WDG_ROUTINE wdRoutine             /* routine to call upon expiration */
)

```

■ Description

Up to 2 user watchdog timers can be created.

If the user watchdog timer times out, the system turns OFF the servo power of all control groups. In addition to turning OFF the servo power, if necessary, the user can define another process to be executed when the timer times out. To do so, specify the process in the parameter, user watchdog time-out routine.

■ Parameter

[delay]

Specify the timer value of the user watchdog timer in milliseconds (1 msec or more).

[wdRoutine]

Specify a desired user watchdog time-out routine. Specify the process necessary when the user watchdog timer times out. If no process is necessary at the time-out, specify NULL.

The time-out routine does not have a return value. The time-out routine receives the handle value of the user watchdog timer which timed out as the parameter. Thus, since the timed-out user watchdog timer can be distinguished in the time-out routine, one time-out routine can be shared by two or more watchdog timers.

In addition, the time-out routine is executed in the task of the system. Therefore, if the task of the time-out routine is tried to be deleted in the time-out routine itself (if mpDeleteTask(TID_SELF) is tried to be executed), the symbolic constant: ERROR (value: -1): Error is returned.

- MP_USR_WDG_ROUTINE
Time-out routine

Syntax:

```
typedef void (*MP_USR_WDG_ROUTINE) (MP_WDG_HANDLE);
```

Example of time-out routine:

```

void fncwdRoutine0( MP_WDG_HANDLE handle )
{
    ...
}

```

3 User Watchdog API

mpUsrWdogCreate

■ Return Value

0 or more Normal end
The characteristic value of the created user watchdog timer is returned as the handle value.

Symbolic constant: ERROR (value: -1) : Error
In the case when the delay value is 0 or less, 2 user watchdog timers are already created, and the like.

mpUsrWdogDelete

Deletes the specified user watchdog timer.

■ Syntax

```
int mpUsrWdogDelete  
(  
    MP_WDG_HANDLE handle      /* specified watchdog timer */  
)
```

■ Description

Deletes the user watchdog timer specified in “*handle*”.

■ Parameter

[*handle*]

Specify the handle value of the user watchdog timer to be deleted.

“handle” uses the value returned by mpUsrWdogCreate().

■ Return Value

Symbolic constant: OK (value: 0) : Normal end

Symbolic constant: ERROR (value: -1) : Error
In the case when an invalid
handle value is specified, and
the like.

mpUsrWdogStart

Starts the specified user watchdog timer.

■ Syntax

```
int mpUsrWdogStart  
(  
    MP_WDG_HANDLE handle    /* specified watchdog timer */  
)
```

■ Description

Starts the user watchdog timer specified in “*handle*”.

Also, this API is used to start the user watchdog timer again when the user watchdog timer times out.

■ Parameter

[*handle*]

Specify the handle value of the user watchdog timer to be started.

“handle” uses the value returned by mpUsrWdogCreate().

■ Return Value

Symbolic constant: OK (value: 0) : Normal end

Symbolic constant: ERROR (value: -1) : Error
In the case when an invalid
handle value is specified, and
the like.

mpUsrWdogClear

Clears the specified user watchdog timer.

■ Syntax

```
int mpUsrWdogClear  
(  
    MP_WDG_HANDLE handle      /* specified watchdog timer */  
)
```

■ Description

Clears the user watchdog timer specified in “*handle*”.

This API must be executed periodically after the user watchdog timer is started.

■ Parameter

[*handle*]

Specify the handle value of the user watchdog timer to be cleared.

"handle" uses the value returned by mpUsrWdogCreate().

■ Return Value

Symbolic constant: OK (value: 0) : Normal end

Symbolic constant: ERROR (value: -1) : Error
In the case when an invalid
handle value is specified, and
the like.

4 Network API

mpSocket

Opens a socket.

■ Syntax

LONG mpSocket(LONG *domain*, LONG *type*, LONG *protocol*);

■ Description

This API opens a socket and returns a socket descriptor. The socket descriptor is passed to the other socket routines and used to identify the socket. The socket descriptor is a standard I/O system file descriptor (fd).

■ Parameter

[domain]

Specifies the protocol family for communication. The following can be set.

Symbolic constant	Meaning
AF_INET	IPv4 internet protocol

[type]

Specifies the communication type. The following can be set.

Symbolic constant	Meaning
SOCK_STREAM	TCP
SOCK_DGRAM	UDP

[protocol]

Protocol number

Fixed by 0

■ Return Value

Value ≥ 0 Socket descriptor

Symbolic constant: ERROR (value: -1) : Error



mpSocket() and mpAccept() use memory that is temporarily allocated from the heap. If mpSocket() and mpAccept() are called repeatedly without calling mpClose(), the heap memory will become exhausted and an insufficient memory error will occur. This may cause the controller to hang, communications errors to occur with the programming pendant, and robot operation to stop. If mpSocket() and mpAccept() must be called repeatedly, make sure to call mpClose() at an appropriate time.

mpListen

Gives the server attribute to the socket and waits for connection.

■ Syntax

```
LONG mpListen(LONG sockfd, LONG backlog);
```

■ Description

This API enables connections to a socket. After enabling connections with mpListen(), connections are actually accepted by mpAccept().

■ Parameter

[*sockfd*]

The descriptor which refers to the socket

Example: The socket descriptor returned by mpSocket()

[*backlog*]

The number of the I/O data

It also specifies the maximum number of unaccepted connections that can be suspended at one time (backlog).

■ Return Value

Symbolic constant: OK (value: 0) : Normal end

Symbolic constant: ERROR (value: -1) : Error

mpAccept

Accepts a connection from the client to the socket.

■ Syntax

```
LONG mpAccept(LONG sockfd, struct sockaddr *addr, LONG *addrlen);
```

■ Description

This API accepts a connection on a socket, creates a new socket created for the connection, and returns a new socket descriptor. The socket must be bound to an address with `mpBind()` and must also be enabled for connections by a call with `mpListen()`. `mpAccept()` removes the first connection request (deletes it from the queue) and creates a new socket with the same attributes as *sockfd*. It blocks the caller until a connection is present, unless the socket is marked as non-blocking.

The parameter *addrlen* must be initialized to the size of the available buffer pointed to by *addr*. Upon completion of the API, the size in bytes of the peer's address stored in *addr* is set to *addrlen*.

■ Parameter

[*sockfd*]

The socket descriptor waiting for a connection by `mpListen`

[*addr*]

[out] The pointer to the `sockaddr` structure for the socket address of the communication target

[*addrlen*]

At calling, this must have been initialized with the structure size (in bytes) specified by *addr*. Upon completion of the API, the actual size of the connecting target address is set. (This is used for input to or output from the API.)

■ Return Value

Value ≥ 0 The file descriptor of the socket

Symbolic constant: `ERROR` (value: -1) : Error



`mpSocket()` and `mpAccept()` use memory that is temporarily allocated from the heap. If `mpSocket()` and `mpAccept()` are called repeatedly without calling `mpClose()`, the heap memory will become exhausted and an insufficient memory error will occur. This may cause the controller to hang, communications errors to occur with the programming pendant, and robot operation to stop. If `mpSocket()` and `mpAccept()` must be called repeatedly, make sure to call `mpClose()` at an appropriate time.

mpBind

Sets an address to the socket.

■ Syntax

```
LONG mpBind(LONG sockfd, const struct sockaddr *addr, LONG addrlen);
```

■ Description

This API associates a network address (also referred to as its “name”) with a specified socket so that other processes can connect or send to it. When a socket is created with `mpSocket()`, it belongs to an address family but has no assigned name.

■ Parameter

[*sockfd*]

The socket descriptor created by `mpSocket`

[*addr*]

[in] The pointer of the `sockaddr` structure which stores the address associated with the socket.

The structure set to *addr* depends on the address family, but MotoPlus only supports `sockaddr_in`.

[*addrlen*]

The size of the address structure of the *addr* (unit: byte)

■ Return Value

Symbolic constant: OK (value: 0) : Normal end

Symbolic constant: ERROR (value: -1) : Error

mpConnect

Initiates a connection to the socket.

■ Syntax

```
LONG connect(LONG sockfd, const struct sockaddr *addr, LONG addrlen);
```

■ Description

If *sockfd* is a socket created by specifying SOCK_STREAM, this API establishes a virtual circuit between *sockfd* and another socket specified by the name. If *sockfd* is a socket created by specifying SOCK_DGRAM, this API permanently specifies the peer to which messages are sent. The name parameter specifies the address of the socket to be connected.

■ Parameter

[*sockfd*]

The socket descriptor created by mpSocket

[*addr*]

[in] The pointer of the sockaddr structure which keeps the socket address

The structure set to *addr* depends on address family but MotoPlus supports only sockaddr_in.

[*addrlen*]

The size of the address structure of *addr* (unit: byte)

■ Return Value

Symbolic constant: OK (value: 0) : Normal end

Symbolic constant: ERROR (value: -1) : Error

mpRecv

mpRecvFrom

Receives a message from the socket.

■ Syntax

```
LONG mpRecv(LONG sockfd, const void *buf, LONG len, LONG flags);  
LONG mpRecvFrom(LONG sockfd, const void *buf, LONG len, LONG flags,  
                 struct sockaddr *src_addr, LONG *addrlen);
```

■ Description

This API receives a message from the socket. The API to be used differs depending on the communication protocol to be used. `mpRecv()` is used by TCP. `mpRecvFrom()` is used by UDP.

■ Parameter

[*sockfd*]

Socket descriptor

[*buf*]

The pointer to the buffer for the received data

[*len*]

The maximum allowable length of received data to be stored in *buf* (unit: byte)

[*flags*]

Set "0".

[*src_addr*]

[in] The pointer to the `sockaddr` structure which stores the socket address

[*addrlen*]

At the execution of this API, it must be initialized with the structure size (in bytes) specified by *src_addr*. At normal completion of this API, the actual size of the connecting target address is set. (This is used for input to or output from the API.)

■ Return Value

Value ≥ 0 The number of bytes received (normal end)

Symbolic constant: `ERROR` (value: -1) : Error

mpSend

mpSendTo

Sends a message to the socket.

■ Syntax

```
LONG mpSend(LONG sockfd, const void *buf, LONG len, LONG flags);  
LONG mpSendTo(LONG sockfd, const void *buf, size_t len, LONG flags,  
               const struct sockaddr *dest_addr, LONG addrlen);
```

■ Description

Sends data to a socket. The API to be used differs depending on the communication protocol to be used. `mpSend()` is mainly used by TCP. `mpSendTo()` is mainly used by UDP.

■ Parameter

[*sockfd*]

Socket descriptor

[*buf*]

The pointer to the buffer for the sent data

[*len*]

The length of the sent data stored in *buf* (unit: byte)

[*flags*]

Set "0".

[*dest_addr*]

Specify the pointer [in] to the `sockaddr` structure which stores the socket address.

[*addrlen*]

The size of the address structure specified by *dest_addr* (unit:byte)

■ Return Value

Value ≥ 0 The number of bytes sent (normal end)

Symbolic constant: `ERROR` (value: -1) : Error

mpClose

Closes a socket.

■ Syntax

```
LONG    mpClose(LONG sockfd);
```

■ Description

Closes a socket.

■ Parameter

[*sockfd*]

Socket descriptor

■ Return Value

0 Normal end

-1 Error



mpClose() also exists in the File Control API. Make sure to set the correct descriptor as the mpClose() argument because this descriptor determines whether mpClose() is used as a Network API or a File Control API.



mpSocket() and mpAccept() use memory temporarily allocated from the heap. For this reason, if mpSocket() and mpAccept() are repeatedly called without calling mpClose(), the above working memory is consumed, and eventually the system runs out of memory, causing the controller to hang, programming pendant communication errors, and robots stopping operation. Therefore, if mpSocket() and mpAccept() must be called repeatedly, make sure to call mpClose() each time.

mpHtonl

mpHtons

mpNtohl

mpNtohs

Converts a value between the host byte order and the network byte order.

■ Syntax

```
ULONG    mpHtonl(ULONG    hostlong);  
USHORT   mpHtons(USHORT   hostshort);  
ULONG    mpNtohl(ULONG    netlong);  
USHORT   mpNtohs(USHORT   netshort);
```

■ Description

mpHtonl() converts the *hostlong* from the host byte order to the network byte order.

mpHtons() converts the *hostshort* from the host byte order to the network byte order.

mpNtohl() converts the *netlong* from the network byte order to the host byte order.

mpNtohs() converts the *netshort* from the network byte order to the host byte order.

mplnetAddr

mplnetNtoa

mplnetNtoaB

Converts the internet host address between the dot-decimal notation and the binary representation in network byte order.

■ Syntax

```
ULONG      mplnetAddr(const CHAR *cp);
CHAR       *mplnetNtoa(struct in_addr in);
void       mplnetNtoaB(struct in_addr inetAddress, char *pString)
```

■ Description

`mplnetAddr()` converts the internet host address *cp* from the dot-decimal notation of IPv4 to the binary representation in network byte order.

`mplnetNtoa()` converts the internet host address *in* from the binary representation in network byte order to the dot-decimal notation of IPv4.

`mplnetNtoaB()` converts the internet host address *inetAddress* from the binary representation in network byte order to the dot-decimal notation of IPv4, and returns the result to *pString*.

Although the functions of `mplnetNtoa()` and `mplnetNtoaB()` are the same, it is recommended to use `mplnetNtoaB()` because `mplnetNtoa()` is not thread-safe.

■ Note

<Sample Code>

```
#define HOST_IP_ADDRESS "192.168.1.1"
// IP address of network host (IPv4)

voidtest(void)
{
    ULONG host_ip; // Variable that holds the internet host address
    ...
    host_ip = mplnetAddr(HOST_IP_ADDRESS);
    ...
}
```

mpGetsockname

Gets the socket name.

■ Syntax

```
LONG mpGetsockname(LONG sockfd, struct sockaddr *addr, LONG *addrlen);
```

■ Description

The current socket address specified by *sockfd* is returned by storing in the buffer specified by *addr*. Before executing this API, the *addrlen* parameter must be initialized to the data size (in bytes) specified by *addr*. After the execution of the API, the actual size (in bytes) of the socket address is stored in *addrlen* and then returned.

■ Parameter

[*sockfd*]

Socket descriptor

[*addr*]

[out] The pointer of the sockaddr structure for the socket address

[*addrlen*]

At the execution of this API, it must be initialized to the data size (in bytes) specified by *addr*. At normal completion of this API, the actual size of the address is set. (This is used for input to or output from the API.)

■ Return Value

Symbolic constant: OK (value: 0) : Normal end

Symbolic constant: ERROR (value: -1) : Error

mpGetpeername

Gets the name of the peer connected to a socket.

■ Syntax

```
int mpgetpeername  
(  
    int s,  
    struct sockaddr *name,  
    int *namelen  
)
```

■ Parameter

[s]

Specifies the socket descriptor to get its peer name.

[name]

Specifies the address of the buffer which stores the address of the peer.

[namelen]

Specifies the size of the area which *name* indicates.

■ Return Value

Symbolic constant: OK (value: 0) : Normal end

Symbolic constant: ERROR (value: -1) : Error

mpSetsockopt

Sets socket options. (Sets options for disconnection.)

■ Syntax

```
STATUS mpSetsockopt
(
    int     sockfd,      /* target socket */
    int     level,       /* protocol level of option */
    int     optname,     /* option name */
    char * optval,       /* pointer to option value */
    int     optlen       /* option length */
)
```

■ Parameter

[*sockfd*]

The socket descriptor setting options

[*level*]

Specify *SOL_SOCKET* for the setting option layer *mpSetsockopt()*. Other layers cannot be set.

[*optname*]

Specify the manipulated option name. For *mpSetsockopt()*, set *SO_KEEPALIVE*. Other names cannot be specified.

SO_KEEPALIVE -- Disconnection detection

Enables the timer detecting disconnection during the TCP communication. The disconnection detection time is fixed to two hours. If a disconnection is detected, the socket specified as *sockfd* is closed.

[*optval*]

Specifies an appropriate type depending on the area address *optname* that stores the manipulating option values

[*optlen*]

Specifies the area size specified by the manipulating option size *optval*

Option name: <i>Optname</i>	The area type specified by <i>optval</i>	The value of <i>optval</i>	<i>Optlen</i>
<i>SO_KEEPALIVE</i>	int	1: ON 0: OFF	4

■ Return Value

Symbolic constant: OK (value: 0) : Normal end

Symbolic constant: ERROR (value: -1) : Error

mpioctl

Controls I/O devices (clears buffers, sets blocking / non-blocking)

■ **Syntax**

```
int mpioctl
(
    int fd,           /* file descriptor (target socket) */
    int function,     /* function code */
    int arg           /* arbitrary argument */
)
```

■ **Parameter**

[fd]

The file descriptor of the controlled I/O device
Set the socket descriptor, here.

[function]

Controlled function

Symbolic constant: FIOFLUSH (value: 2) -- Buffer flush

Symbolic constant: FIONBIO (value: 16) -- Blocking / non-blocking setting

[arg]

Control function option
Set the value passed to the controlled function.

Function name: <i>Function</i>	The area type specified by <i>arg</i>	The value of <i>arg</i>
FIOFLUSH	int	1: ON 0: OFF
FIONBIO	int	0: Blocking 1: Non-blocking

■ **Return Value**

Symbolic constant: OK (value: 0) : Normal end

Symbolic constant: ERROR (value: -1) : Error (Incorrect file descriptor, incorrect parameter specification)

mpSelect

Keeps a task waiting for multiple sockets.
Or as an another application, this API is used to examine whether mpSend and mpRecv can be executed to the socket with non-blocking.

■ Syntax

```
int mpSelect
(
    int          width,          /* number of bits to examine from 0 */
    fd_set *     pReadFds,       /* read fds */
    fd_set *     pWriteFds,      /* write fds */
    fd_set *     pExceptFds,     /* exception fds (unsupported) */
    struct timeval * pTimeOut    /* max time to wait, NULL = forever */
)
```

■ Parameter

[width]

This is the value to define the number of bits examined in fd_set. Set the value of the current maximum socket file descriptor value + 1, or specify the symbolic constant: FD_SETSIZE (value: 2048).

[pReadFds]

Specifies the fd_set structure that sets the bit of the socket file descriptor waiting for reading.

When not waiting for reading, set NULL.

At the end of execution, mpSelect() clears the socket file descriptor bit to 0, and sets only the bit corresponding to the socket file descriptor in ready state.

```
struct fd_set
{
    fd_mask fds_bits[howmany(FD_SETSIZE, NFDBITS)];
};
```

```
#define FD_SETSIZE2048
typedef longfd_mask;
#define howmany(x, y)((unsigned int)((((x)+((y)-1)))/(unsigned int)(y))
```

To set a bit to the fd_set structure, use the following macros.

Macro: FD_SET(fd, &fdset)

Macro: FD_CLR(fd, &fdset)

Macro: FD_ZERO(&fdset)

4 Network API

mpSelect

To check the socket file descriptor usable when finishing mpSelect(), use the following macro.

Macro: FD_ISSET(fd, &fdset)

[pWriteFds]

Specifies the fd_set structure that sets the bit of the socket file descriptor waiting for writing.

When not waiting for writing, set NULL.

At the end of execution, mpSelect() clears the socket file descriptor bit of the fd_set structure to 0, and sets only the bit corresponding to the socket file descriptor in ready state.

[pExeptFds]

Set NULL, always.

[pTimeout]

Specifies the timeval structure setting the maximum waiting time.

When waiting without timeout, set NULL.

The following members are provided for the timeval structure.

```
struct timeval
{
    long tv_sec; /* second */
    long tv_usec; /* micro second */
};
```

■ Return Value

0 Timeout

Value >= 0 The number of available sockets

-1 Error

5 Serial Communication API (RS232C Communication)

mpRsInit

Initializes the specified serial communication port.



This API cannot be used on the YRC1000micro.

■ Syntax

```
int mpRsInit
(
  const char * port,           /* name of port */
  const MP_RS_CONFIG * rs_config /* COM port parameter */
)
```

■ Parameter

The transmission parameter of the serial communication port with the character string specified by *port* is set to the conditions specified by *rs_config* parameter. This API is a prerequisite for using all serial communication related-APIs (RS232C communication). This API must be done before using another serial communication related-APIs. If the target serial communication port is already opened, this API returns an error. Only the ports defined by the symbolic constant: MP_RS_COM1 are supported. Make sure to set MP_RS_COM1 as *port*.

MP_RS_CONFIG, the structure of the *rs_config* parameter, has the following members. Set the member according to the communication setting to be used.

<MP_RS_CONFIG structure>

```
typedef struct {
  MP_RS_DATA_BIT      dataBit;
  MP_RS_STOP_BIT      stopBit;
  MP_RS_PARITY         parity;
  MP_RS_BAUDRATE       baudRate;
} MP_RS_CONFIG;
```

.dataBit : Specifies the data bit number. The following can be specified.

Symbolic constant: mpRsDataBit_7 (value: 7): Specified for 7 bits.

Symbolic constant: mpRsDataBit_8 (value: 8): Specified for 8 bits.

.stopBit : Specifies the data bit size. The following can be specified.

Symbolic constant: mpRsStopBit_one (value: 0): Specified for 1 bits.

Symbolic constant: mpRsStopBit_1point5 (value: 1): Specified for 1.5 bits.

Symbolic constant: mpRsStopBit_two (value: 2): Specified for 2 bits.

5 Serial Communication API (RS232C Communication)

mpRsInit

. parity : Specifies the parity. The following can be specified.

Symbolic constant: mpRsParity_none (value: 0): Specified when the parity is none.

Symbolic constant: mpRsParity_odd (value: 1): Specified when the parity is odd parity.

Symbolic constant: mpRsParity_even (value: 2): Specified when the parity is even parity.

. baudRate : Specifies the baud rate. The following can be specified.

Symbolic constant: mpRsParity_none (value: 0) mpRsBaudrate_150: Specified for 150 bps.

Symbolic constant: mpRsParity_none (value: 1) mpRsBaudrate_300: Specified for 300 bps.

Symbolic constant: mpRsParity_none (value: 2) mpRsBaudrate_600: Specified for 600 bps.

Symbolic constant: mpRsParity_none (value: 3) mpRsBaudrate_1200: Specified for 1200 bps.

Symbolic constant: mpRsParity_none (value: 4) mpRsBaudrate_2400: Specified for 2400 bps.

Symbolic constant: mpRsParity_none (value: 5) mpRsBaudrate_4800: Specified for 4800 bps.

Symbolic constant: mpRsParity_none (value: 6) mpRsBaudrate_9600: Specified for 9600 bps.

Symbolic constant: mpRsParity_none (value: 7) mpRsBaudrate_19200: Specified for 19200 bps.

■ Return Value

Symbolic constant: OK (value: 0) : Normal end

Symbolic constant: ERROR (value: -1) : Error

mpRsOpen

Opens the specified serial communication port.



This API cannot be used on the YRC1000micro.

■ Syntax

```
int mpRsOpen  
(  
  const char * port,    /* name of port */  
)
```

■ Parameter

This API opens the serial communication port with the character string specified by *port*. This API must be done before using the send/receive API. If the target serial communication port is already opened, this API returns an error. Make sure to set the symbolic constant: MP_RS_COM1 as *port*.

■ Return Value

Value ≥ 0 Handle value (normal end)

Symbolic constant: ERROR (value: -1) : Error

mpRsClose

Closes the specified serial communication port.



This API cannot be used on the YRC1000micro.

■ Syntax

```
int mpRsClose  
(  
  int port_handle,    /* handle value of port opened */  
)
```

■ Parameter

This API closes the specified serial communication port by port_handle.
port_handle specifies the handle value of mpRsOpen return values.

■ Return Value

Symbolic constant: OK (value: 0) : Normal end

Symbolic constant: ERROR (value: -1) : Error

mpRsSend

Sends data using the specified serial port.



This API cannot be used on the YRC1000micro.

■ Syntax

```
int mpRsClose
(
    int          port_handle,      /* handle value of port opened */
    const char   *buf,            /* send data buffer pointer */
    unsigned int buf_len          /* length of buffer data */
)
```

■ Description

mpRsSend sets the send data to the sending FIFO buffer (1 KByte). If the buffer does not have enough space specified for the data size (buf_len), an error will occur.

■ Parameter

port_handle : Handle value returned from mpRsOpen() execution
buf : Pointer of the send data
buf_len : Buffer size (unit: byte)

■ Return Value

0 : Normal end
-10 : The length of the send data exceeds the available buffer size.
The buffer size is 1 KByte.
-1 : Error (another parameter error)

mpRsRecv

Receives data using the specified serial port.



This API cannot be used on the YRC1000micro.

■ Syntax

```
int mpRsRecv
(
    int          port_handle, /* handle value of port opened */
    char         * buf,       /* receive data buffer pointer */
    unsigned int buf_len      /* length of buffer area */
)
```

■ Description

mpRsRecv reads the received FIFO buffer (1024 bytes).

■ Parameter

port_handle : Handle value returned from mpRsOpen() execution
buf : Pointer of the receive data readout area
buf_len : Allowable readout buffer size (Max *buf* size)



If nothing is received (nothing in the buffer), the return value is "0", not an error.

■ Return Value

Value ≥ 0 : Received data size (copied from the FIFO buffer)

Symbolic constant: ERROR (value: -1) : Error

6 Memory Pool Control API

mpMalloc

Reserves a memory area.

■ Syntax

```
void *mpMalloc(size_t size);
```

■ Description

Allocates a block with the size bytes from the heap memory. The reserved block is not cleared to 0.

This API makes it possible to reserve the memory with any necessary size at any necessary location, without allocating the memory in advance by using arrays in the program.

1 MB area is prepared in advance for MotoPlus. However, since the management domain is included in this area, all the 1 MB area cannot be used.

Release the memory, which is reserved with the mpMalloc API, after the memory is used, by using the mpFree API.

■ Parameter

[size]

The memory byte sizes to be reserved

■ Return Value

Success: The pointer to the reserved memory block

Failure: NULL (when the memory with the specified size cannot be reserved: insufficient memory)

mpFree

Releases the memory reserved with mpMalloc().

■ Syntax

```
void mpFree(void *ptr);
```

■ Description

Releases the memory in the area specified by ptr. After mpfree() has released the memory, the area is available for other allocations.

The function does nothing when ptr is an empty pointer.

■ Parameter

[ptr]

The pointer to the area to be released



If the area not reserved is specified, a serious problem such as task interruption may occur.

■ Return Value

None

7 File Control API

This service allows the user to freely create, read, write, and delete files on the SRAM drive in the controller's non-volatile memory, the DRAM drive in volatile memory, or the drive on a USB flash drive attached to the CPU circuit board¹⁾ by the user.

Files that are created on the SRAM drive and the USB drive are retained after the power supply is turned OFF, but files created on the DRAM drive are erased when the power supply is turned OFF.

The available memory that can be used on the SRAM drive depends on the system version of the YRC1000 or YRC1000micro as shown below.

For the YRC1000:

If the system version is earlier than YAS2.56-00, the memory expansion option (JANCD-AIF01-2E) for the robot I/F circuit board is required to use the SRAM drive. The available space is 512 KByte.

If the system version is YAS2.56-00 or later, the SRAM drive can be used without the memory expansion option. The available space depends on whether the memory expansion option is installed.

Available Space

Memory expansion option not installed 32 KByte

Memory expansion option installed 512 KByte

For the YRC1000micro:

If the system version is earlier than YBS1.03-00, the SRAM drive cannot be used.

If the system version is YBS1.03-00 or later, the SRAM drive can be used. The available space is 32 KByte.

(There is no memory expansion option available for the YRC1000micro.)

The available DRAM drive space depends on the software version.

Available Space

For YRC1000

Before YAS5.10-00-00: 512 KByte

YAS5.10-00-00 or later: 4 MByte

For YRC1000micro

Before YBS4.11.00-00: 512 KByte

YBS4.11.00-00 or later: 4 MByte

¹ For the YRC1000: JANCD-ACP01
For the YRC1000micro: JANCD-ACP31



Memory Expansion Option

For the YRC1000:

To add the memory expansion option (JANCD-AIF01-2E) after the YRC1000 has been purchased, the circuit board must be replaced and setup must be performed again. Contact your YASKAWA representative for details on replacing the circuit board.

For the YRC1000micro:

There is no memory expansion option available for the YRC1000micro.

mpCreate

Creates a new file.

If an existing file name is specified, the old file is overwritten.

■ Syntax

```
int mpCreate          /* return : fd */  
(  
    const char  * name, /* name of the file to create */  
    int         flags   /* O_RDONLY, O_WRONLY, or O_RDWR */  
);
```

■ Parameter

[name]

The full pathname (drive name + file name) of the file to be created

<Ex.> MPRAM1:0/TestFile.dat

Usable drive names are as follows.

- Symbolic constant: MP_SRAM_DEV_DOS (value: (MPSRAM1:0))
- Symbolic constant: MP_DRAM_DEV_DOS (value: (MPRAM1:0))
- Symbolic constant: MP_USB0_DEV_DOS (value: (MPUSB0))

If an existing file name is specified, the file is overwritten, and the previous file contents are deleted.

[flags]

The access right to the file

Symbolic constant	Value	Description
O_RDONLY	0	Read only
O_WRONLY	1	Write only
O_RDWR	2	Read and write

7 File Control API

mpCreate

■ Return Value

fd (>=0)	Normal end (file descriptor number)
-1	Error (Insufficient existing files or file descriptors, no specified directory)
-3	Specified drive name is invalid.

■ Note

The maximum number of file descriptors that can be used simultaneously is MP_FILES (16).



FileControl API is executed only with the priority MP_PRI_TIME_NORMAL. When a task with a higher priority than MP_PRI_TIME_NORMAL calls FileControl API, and the task priority limitation is set to "Automatic Adjustment" (S2C1101 = 1), the system temporarily lowers the priority to MP_PRI_TIME_NORMAL, and executes the function. If the task priority limitation is set to "limitation" (S2C1101 = 0), the API returns an error.

mpOpen

Opens a file.

■ Syntax

```
int mpOpen          /* return : fd */  
(  
    const char* name, /* name of the file to open */  
    int         flags, /* O_RDONLY, O_WRONLY, O_RDWR, or O_CREAT */  
    int         mode   /* mode of file to create (UNIX chmod style) */  
);
```

■ Parameter

[name]

The name of the file to be opened

Usable drive names are as follows.

- Symbolic constant: MP_SRAM_DEV_DOS (value: (MPSRAM1:0))
- Symbolic constant: MP_DRAM_DEV_DOS (value: (MPRAM1:0))
- Symbolic constant: MP_USB0_DEV_DOS (value: (MPUSB0))

[flags]

The access right to the file

Symbolic constant	Value	Description
O_RDONLY	0	Read only
O_WRONLY	1	Write only
O_RDWR	2	Read and write
O_CREAT	0x0200	Creates and opens a file (Use this in combination with one of the above three access rights.)

[mode]

Not used

■ Return Value

fd (>=0)	Normal end (file descriptor number)
-1	Error (Insufficient existing files or file descriptors, no specified directory)
-3	Specified drive name is invalid.
-20	Write in progress (write in progress due to other API, etc.)

7 File Control API

mpOpen

■ Note

The maximum number of file descriptors that can be used simultaneously is MP_FILES (16).



FileControl API is executed only with the priority MP_PRI_TIME_NORMAL. When a task with a higher priority than MP_PRI_TIME_NORMAL calls FileControl API, and the task priority limitation is set to "Automatic Adjustment" (S2C1101 = 1), the system temporarily lowers the priority to MP_PRI_TIME_NORMAL, and executes the function. If the task priority limitation is set to "limitation" (S2C1101 = 0), the API returns an error.

mpRemove

Removes the file.

■ Syntax

```
STATUS mpRemove      /* return : status */
(
    const char * name /* name of the file to remove */
);
```

■ Parameter

[name]

The file name to be removed

Usable drive names are as follows.

- Symbolic constant: MP_SRAM_DEV_DOS (value: (MPSRAM1: 0))
- Symbolic constant: MP_DRAM_DEV_DOS (value: (MPRAM1: 0))
- Symbolic constant: MP_USB0_DEV_DOS (value: (MPUSB0))

■ Return Value

- | | |
|----|---|
| 0 | Normal end |
| -1 | Error (No specified file or descriptor) |
| -3 | Specified drive name is invalid. |



FileControl API is executed only with the priority MP_PRI_TIME_NORMAL. When a task with a higher priority than MP_PRI_TIME_NORMAL calls FileControl API, and the task priority limitation is set to "Automatic Adjustment" (S2C1101 = 1), the system temporarily lowers the priority to MP_PRI_TIME_NORMAL, and executes the function. If the task priority limitation is set to "limitation" (S2C1101 = 0), the API returns an error.

Please check a drive name and a file name enough not to delete except the target file.

mpClose

Closes a file.

■ Syntax

```
STATUS mpClose          /* return : status */  
(  
    int fd              /* file descriptor to close */  
);
```

■ Parameter

[fd]

The file descriptor number of the file.

■ Return Value

Symbolic constant: OK (value: 0) : Normal end

Symbolic constant: ERROR (value: -1) : Error (Incorrect specified file descriptor)



mpClose() also exists in the Network API. Make sure to set the correct descriptor as the mpClose() argument because this descriptor determines whether mpClose() is used as a Network API or a File Control API.



FileControl API is executed only with the priority MP_PRI_TIME_NORMAL. When a task with a higher priority than MP_PRI_TIME_NORMAL calls FileControl API, and the task priority limitation is set to "Automatic Adjustment" (S2C1101 = 1), the system temporarily lowers the priority to MP_PRI_TIME_NORMAL, and executes the function. If the task priority limitation is set to "limitation" (S2C1101 = 0), the API returns an error.

mpRename

Renames a file.

■ Syntax

```
int mpRename          /* return : OK or ERROR */  
(  
    const char * oldName, /* name of file to rename */  
    const char * newName /* name with which to rename file */  
);
```

■ Parameter

[oldName]

Old file name

[newName]

New file name

■ Return Value

Symbolic constant: OK (value: 0) : Normal end

Symbolic constant: ERROR (value: -1) : Error (File is opened, no specified file)



FileControl API is executed only with the priority MP_PRI_TIME_NORMAL. When a task with a higher priority than MP_PRI_TIME_NORMAL calls FileControl API, and the task priority limitation is set to "Automatic Adjustment" (S2C1101 = 1), the system temporarily lowers the priority to MP_PRI_TIME_NORMAL, and executes the function. If the task priority limitation is set to "limitation" (S2C1101 = 0), the API returns an error.

mpRead

Reads data from a file.

■ Syntax

```
int mpRead          /* return : Number of bytes read (between 1 and maxbytes, */  
    (               /*      0 if end of file), or ERROR, by read() */  
    int  fd,         /* file descriptor from which to read */  
    char * buffer,    /* pointer to buffer to receive bytes */  
    size_t maxBytes /* max no. of bytes to read into buffer */  
    );
```

■ Parameter

[fd]

The file descriptor number of the file to be read

[buffer]

The address storing the data

[maxBytes]

The number of maximum bytes

■ Return Value

>=0	Normal end (The number of actually read bytes)
-1	Error (Specified file descriptor is incorrect or file for reading is not opened.)

■ Note

When this API is used, the file pointer of the file specified as fd moves to the end of the read data. When reading or writing after that, the process starts from there. To read or write from the beginning of the file, re-open the file, or move the file pointer by using mpLseek.



FileControl API is executed only with the priority MP_PRI_TIME_NORMAL. When a task with a higher priority than MP_PRI_TIME_NORMAL calls FileControl API, and the task priority limitation is set to "Automatic Adjustment" (S2C1101 = 1), the system temporarily lowers the priority to MP_PRI_TIME_NORMAL, and executes the function. If the task priority limitation is set to "limitation" (S2C1101 = 0), the API returns an error.

mpWrite

Writes data to a file.

■ Syntax

```
int mpWrite      /* return : Number of bytes written (if not equal to */  
(              /*      nbytes, an error has occurred), or ERROR */  
  int  fd,       /* file descriptor on which to write */  
  char * buffer, /* buffer containing bytes to be written */  
  size_t nBytes  /* number of bytes to write */  
);
```

■ Parameter

[fd]

The file descriptor number of the file to be written

[buffer]

The address where the data is written

[nBytes]

The number of bytes

■ Return Value

>=0	Normal end (The number of actually written bytes)
-1	Error (Specified file descriptor is incorrect, file for writing is not opened, or disk capacity is insufficient.)

■ Note

When this API is used, the file pointer of the file specified as fd moves to the end of the written data. When reading or writing after that, the process starts from there. To read or write from the beginning of the file, re-open the file, or move the file pointer by using mpLseek.



FileControl API is executed only with the priority MP_PRI_TIME_NORMAL. When a task with a higher priority than MP_PRI_TIME_NORMAL calls FileControl API, and the task priority limitation is set to "Automatic Adjustment" (S2C1101 = 1), the system temporarily lowers the priority to MP_PRI_TIME_NORMAL, and executes the function. If the task priority limitation is set to "limitation" (S2C1101 = 0), the API returns an error.

mpioctl

Controls devices.

■ **Syntax**

```
int mpiocbl          /* return : Return value of the driver, or ERROR */
(
    int fd,           /* file descriptor */
    int function,      /* function code */
    int arg            /* arbitrary argument */
);
```

■ **Parameter**

[fd]

The file descriptor number

[function]

The request code depending on the device

The support function is different according to the fd type.

fd: RAM drive

Symbolic constant	Description	ret = mpiocbl (fd, function, arg)
FIOATTRIBSET	Changes the file attribute byte in the directory entry to the attribute specified as newAttrib. MotoPlus supports only 0 (No attribute) or MP_RAMDRV_ATTR_RDONLY (read-only).	status = mpiocbl (fd, FIOATTRIBSET, newAttrib);
FIOGETNAME	Retrieves the file name specified by fd.	status = mpiocbl (fd, FIOGETNAME, &nameBuf);
FIOMOVE	Moves the file without renaming it.	
FIONCONTIG	Retrieves the maximum continuous area size on the device.	status = mpiocbl (fd, FIONCONTIG, bytesRequested);
FIONFREE	Retrieves the number of bytes in the unused area on the volume.	status = mpiocbl (fd, FIONFREE, &freeCount);
FIONREAD	Retrieves the number of unread bytes.	status = mpiocbl (fd, FIONREAD, &nBytesUnread);
FIOTRUNC	Truncates the file to the specified length.	status = mpiocbl (fd, FIOTRUNC, newLength);
FIOWHERE	Returns the current byte position of the file.	position = mpiocbl (fd, FIOWHERE, 0);

[arg]

The parameter corresponding to the request code

7 File Control API

mpioctl

■ Return Value

- >=0 Normal end (The return value depending on the device)
- 1 Error (Specified file descriptor is incorrect.)



FileControl API is executed only with the priority MP_PRI_TIME_NORMAL. When a task with a higher priority than MP_PRI_TIME_NORMAL calls FileControl API, and the task priority limitation is set to "Automatic Adjustment" (S2C1101 = 1), the system temporarily lowers the priority to MP_PRI_TIME_NORMAL, and executes the function. If the task priority limitation is set to "limitation" (S2C1101 = 0), the API returns an error.

mpLseek

Moves a file pointer.

■ Syntax

```
int mpLseek      /* return : New offset from the beginning of the file, or ERROR */  
(  
    int fd,      /* file descriptor */  
    long offset, /* new byte offset to seek to */  
    int origin   /* relative file position */  
);
```

■ Parameter

[fd]

File descriptor number

[offset]

The number of bytes from origin

[origin]

Specify the reference position.

- Symbolic constant: SEEK_SET (value: 0) The top of the file
- Symbolic constant: SEEK_CUR (value: 1) The current position of the file pointer
- Symbolic constant: SEEK_END (value: 2) The end of the file

■ Return Value

>=0	Normal end (The number of offset bytes from the top of the file to the new position)
-1	Error (Specified file descriptor is incorrect.)



FileControl API is executed only with the priority MP_PRI_TIME_NORMAL. When a task with a higher priority than MP_PRI_TIME_NORMAL calls FileControl API, and the task priority limitation is set to "Automatic Adjustment" (S2C1101 = 1), the system temporarily lowers the priority to MP_PRI_TIME_NORMAL, and executes the function. If the task priority limitation is set to "limitation" (S2C1101 = 0), the API returns an error.

mpFstat

mpStat

Retrieves file status information.

■ Syntax

```
STATUS mpFstat    /* return : OK or ERROR by fstat() */
(
    int          fd,          /* file descriptor for file to check */
    struct stat * pStat       /* pointer to stat structure */
);
```

```
STATUS mpStat     /* return : OK or ERROR by stat() */
(
    char *       name,       /* name of file to check */
    struct stat * pStat       /* pointer to stat structure */
);
```

■ Description

Retrieves the state information of file.

Use mpStat to specify the name of file with parameters. When using the file descriptor for parameters, use mpFstat.

■ Parameter

[fd]

File descriptor number

[name]

File name

[pStat]

The address to the stat structure

The structure of the file information status

```
struct stat
{
    unsigned long    st_dev;    /* device ID number */
    unsigned long    st_ino;    /* file serial number */
    int              st_mode;    /* file mode (see below) */
    unsigned long    st_nlink;  /* number of links to file */
    unsigned short   st_uid;    /* user ID of file's owner */
    unsigned short   st_gid;    /* group ID of file's group */
    unsigned long    st_rdev;    /* device ID, only if special file */
    long long        st_size;    /* size of file, in bytes */
    TIME             st_atime;    /* time of last access */
    TIME             st_mtime;    /* time of last modification */
};
```

7 File Control API

mpStat

```

TIME          st_ctime;    /* time of last change of file status */
long          st_blksize;
long          st_blocks;
unsigned char st_attrib;   /* file attribute byte (dosFs only) */
int           reserved1;  /* reserved for future use */
int           reserved2;  /* reserved for future use */
int           reserved3;  /* reserved for future use */
int           reserved4;  /* reserved for future use */
int           reserved5;  /* reserved for future use */
int           reserved6;  /* reserved for future use */
};

```

/* File mode (st_mode) bit masks */

```

#define S_IFMT 0xf000    /* file type field */
#define S_IFIFO 0x1000   /* fifo */
#define S_IFCHR 0x2000   /* character special */
#define S_IFDIR 0x4000   /* directory */
#define S_IFBLK 0x6000   /* block special */
#define S_IFREG 0x8000   /* regular */
#define S_IFLNK 0xa000   /* symbolic link */
#define S_IFSOCK 0xc000  /* socket */

```

■ Return Value

- 0 Normal end
- 1 Error (Incorrect specified file descriptor, I/F memory extension option is not used.)
- 3 Invalid drive name is specified.



FileControl API is executed only with the priority MP_PRI_TIME_NORMAL. When a task with a higher priority than MP_PRI_TIME_NORMAL calls FileControl API, and the task priority limitation is set to "Automatic Adjustment" (S2C1101 = 1), the system temporarily lowers the priority to MP_PRI_TIME_NORMAL, and executes the function. If the task priority limitation is set to "limitation" (S2C1101 = 0), the API returns an error.

mpOpendir

Opens the specified directory to examine the entry in the directory.

■ Syntax

```
DIR * mpOpendir          /* return : A pointer to a directory descriptor, */
(                        /*      or NULL if there is an error by opendir() */
  char * dirName         /* name of directory to open */
);
```

```
#define NAME_MAX 255 /* The file name length excluding the path */
struct dirent      /* dirent */
{
  char d_name [NAME_MAX + 1]; /* file name, null-terminated */
};
```

```
typedef struct      /* DIR */
{
  int dd_fd;        /* file descriptor for open directory */
  int dd_cookie;    /* filesystem-specific marker within dir */
  struct dirent dd_dirent; /* obtained directory entry */
} DIR;
```

■ Parameter

[dirName]

Directory name

■ Return Value

>0 Normal end (Directory descriptor address)
0 Error (No specified directory)



- FileControl API is executed only with the priority MP_PRI_TIME_NORMAL. When a task with a higher priority than MP_PRI_TIME_NORMAL calls FileControl API, and the task priority limitation is set to "Automatic Adjustment" (S2C1101 = 1), the system temporarily lowers the priority to MP_PRI_TIME_NORMAL, and executes the function. If the task priority limitation is set to "limitation" (S2C1101 = 0), the API returns an error.
- After opening a directory by mpOpendir, be sure to close it by mpClosedir when it is no longer needed.

mpReaddir

Reads one entry from the opened directory.

■ Syntax

```
struct dirent * mpReaddir      /* return : A pointer to a dirent structure, */
(                               /*      or NULL if there is an end-of-directory */
                               /*      marker or error */
    DIR * pDir                 /* pointer to directory descriptor */
);
```

```
#define NAME_MAX 255 /* The file name length excluding the path */
struct dirent
{
    char    d_name [NAME_MAX + 1]; /* file name, null-terminated */
};
```

```
typedef struct                /* DIR */
{
    int    dd_fd;             /* file descriptor for open directory */
    int    dd_cookie;         /* filesys-specific marker within dir */
    struct dirent dd_dirent;   /* obtained directory entry */
} DIR;
```

■ Parameter

[rDir]

Directory descriptor

■ Return Value

>0 Normal end (The address of the read directory entry)
0 Error (Specified file descriptor is incorrect.)



FileControl API is executed only with the priority MP_PRI_TIME_NORMAL. When a task with a higher priority than MP_PRI_TIME_NORMAL calls FileControl API, and the task priority limitation is set to "Automatic Adjustment" (S2C1101 = 1), the system temporarily lowers the priority to MP_PRI_TIME_NORMAL, and executes the function. If the task priority limitation is set to "limitation" (S2C1101 = 0), the API returns an error.

mpRewinddir

Resets the directory descriptor, and returns the read position to the top of the directory.

■ Syntax

```
void mpRewinddir          /* return : N/A */
(
    DIR * pDir            /* pointer to directory descriptor */
);

#define NAME_MAX 255      /* The file name length excluding the path */
struct dirent            /* dirent */
{
    char    d_name [NAME_MAX + 1];    /* file name, null-terminated */
};

typedef struct            /* DIR */
{
    int     dd_fd;          /* file descriptor for open directory */
    int     dd_cookie;      /* filesystem-specific marker within dir */
    struct dirent dd_dirent; /* obtained directory entry */
} DIR;
```

■ Parameter

[rDir]

Directory descriptor



FileControl API is executed only with the priority MP_PRI_TIME_NORMAL. When a task with a higher priority than MP_PRI_TIME_NORMAL calls FileControl API, and the task priority limitation is set to "Automatic Adjustment" (S2C1101 = 1), the system temporarily lowers the priority to MP_PRI_TIME_NORMAL, and executes the function. If the task priority limitation is set to "limitation" (S2C1101 = 0), the API returns an error.

mpClosedir

Closes the directory descriptor.

■ Syntax

```
STATUS mpClosedir          /* return : OK or ERROR by closedir() */
(
    DIR * pDir              /* pointer to directory descriptor */
);
```

```
#define NAME_MAX 255 /* The file name length excluding the path */
struct dirent      /* dirent */
{
    char    d_name [NAME_MAX + 1]; /* file name, null-terminated */
};
```

```
typedef struct      /* DIR */
{
    int    dd_fd;      /* file descriptor for open directory */
    int    dd_cookie;  /* filesys-specific marker within dir */
    struct dirent dd_dirent; /* obtained directory entry */
} DIR;
```

■ Parameter

[pDir]

Directory descriptor

■ Return Value

Symbolic constant: OK (value: 0) : Normal end
 Symbolic constant: ERROR (value: -1) : Error (Specified directory descriptor is incorrect.)



- FileControl API is executed only with the priority MP_PRI_TIME_NORMAL. When a task with a higher priority than MP_PRI_TIME_NORMAL calls FileControl API, and the task priority limitation is set to "Automatic Adjustment" (S2C1101 = 1), the system temporarily lowers the priority to MP_PRI_TIME_NORMAL, and executes the function. If the task priority limitation is set to "limitation" (S2C1101 = 0), the API returns an error.
- After opening a directory by mpOpendir, be sure to close it by mpClosedir when it is no longer needed.

mpCreatedir

Creates a directory in the USB drive.

■ Syntax

```
int mpCreatedir  
(  
    const char *dirName/* name of directory to create */  
);
```

■ Parameter

[dirName]

A name of the directory to be created

Input the full pathname (up to 255 bytes) including the drive name "MPUSB0".

<Ex.>

MPUSB0/directory

→ "directory" is created in the root of the USB drive 0.

MPUSB0/directory/sample

→ "sample" is created in "directory" of the USB drive 0.

■ Return Value

- | | |
|----|---|
| 0 | Normal end (Directory creation completed) |
| -1 | Error (The USB drive is not specified, The specified directory name has an error, or the specified directory already exists.) |

NOTE

FileControl API is executed only with the priority MP_PRI_TIME_NORMAL. When a task with a higher priority than MP_PRI_TIME_NORMAL calls FileControl API, and the task priority limitation is set to "Automatic Adjustment" (S2C1101 = 1), the system temporarily lowers the priority to MP_PRI_TIME_NORMAL, and executes the function. If the task priority limitation is set to "limitation" (S2C1101 = 0), the API returns an error.

NOTE

Any directory cannot be created in the SRAM drive or the DRAM drive.

The directory which has multiple hierarchies cannot be created simultaneously.

mpRemovedir

Deletes the directory in the USB drive.

■ Syntax

```
int mpRemovedir  
(  
    const char *dirName/* name of directory to remove */  
);
```

■ Parameter

[dirName]

A name of the directory to be deleted

Input the full pathname (up to 255 bytes) including the drive name "MPUSB0".

<Ex.>

MPUSB0/directory/sample

→ "sample" in the "directory" of the USB drive 0 is deleted.

MPUSB0/directory

→ "directory" of the USB drive 0 is deleted.

■ Return Value

0	Normal end (Directory deletion completed)
-1	Error (The USB drive is not specified, The specified directory name has an error, the specified directory does not exist, or any file and/or any directory exist(s) in the specified directory.)



FileControl API is executed only with the priority MP_PRI_TIME_NORMAL. When a task with a higher priority than MP_PRI_TIME_NORMAL calls FileControl API, and the task priority limitation is set to "Automatic Adjustment" (S2C1101 = 1), the system temporarily lowers the priority to MP_PRI_TIME_NORMAL, and executes the function. If the task priority limitation is set to "limitation" (S2C1101 = 0), the API returns an error.



The SRAM drive and the DRAM drive cannot be specified. Only when the targeted directory is empty, it can be deleted.

mpRenamedir

Renames the directory in the USB drive.

■ Syntax

```
int mpRenamedir
(
  const char *srcDirName/* name of directory to rename (source) */
  const char *destDirName/* name of directory to rename (destination) */
);
```

■ Parameter

[srcDirName]

A name of the directory (Change source)

[destDirName]

A name of the directory (Change destination)

Input the full pathname (up to 255 bytes) including the drive name "MPUSB0".

<Ex.>

[srcDirName] MPUSB0/directory

[destDirName] MPUSB0/sample

→ "directory" of the USB drive 0 is changed to "sample".

■ Return Value

- | | |
|----|---|
| 0 | Normal end (Directory renaming completed) |
| -1 | Error (The USB drive is not specified, The specified directory name has an error, the paths of the change source and the change destination are different, the directory of the change source does not exist, or the directory of the change destination already exists.) |

NOTE

FileControl API is executed only with the priority MP_PRI_TIME_NORMAL. When a task with a higher priority than MP_PRI_TIME_NORMAL calls FileControl API, and the task priority limitation is set to "Automatic Adjustment" (S2C1101 = 1), the system temporarily lowers the priority to MP_PRI_TIME_NORMAL, and executes the function. If the task priority limitation is set to "limitation" (S2C1101 = 0), the API returns an error.

NOTE

The SRAM drive and the DRAM drive cannot be specified.

8 Existing File Access API

This service allows the user to read and write the JOBS and various settings file in the controller. Data that was read from the controller can also be written to the SRAM drive in the controller's non-volatile memory, the DRAM drive in volatile memory, or the drive on a USB flash drive attached to the CPU circuit board¹⁾ by the user.

Files that are created on the SRAM drive and the USB drive are retained after the power supply is turned OFF, but files created on the DRAM drive are erased when the power supply is turned OFF.

The available memory that can be used on the SRAM drive depends on the system version of the YRC1000 or YRC1000micro as shown below.

For the YRC1000:

If the system version is earlier than YAS2.56-00, the memory expansion option (JANCD-AIF01-2E) for the robot I/F circuit board is required to use the SRAM drive. The available space is 512 KByte.

If the system version is YAS2.56-00 or later, the SRAM drive can be used without the memory expansion option. The available space depends on whether the memory expansion option is installed.

Available Space

Memory expansion option not installed 32 KByte

Memory expansion option installed 512 KByte

For the YRC1000micro:

If the system version is earlier than YBS1.03-00, the SRAM drive cannot be used.

If the system version is YBS1.03-00 or later, the SRAM drive can be used. The available space is 32 KByte.

(There is no memory expansion option available for the YRC1000micro.)

Memory Expansion Option

For the YRC1000:



To add the memory expansion option (JANCD-AIF01-2E) after the YRC1000 has been purchased, the circuit board must be replaced and setup must be performed again. Contact your YASKAWA representative for details on replacing the circuit board.

For the YRC1000micro:

There is no memory expansion option available for the YRC1000micro.

¹ For the YRC1000: JANCD-ACP01
For the YRC1000micro: JANCD-ACP31

mpLoadFile

Loads a file on the specified drive to the controller.

■ Syntax

```
LONG          mpLoadFile      /* return : status */  
(  
    LONG      mpRamDriveId,    /* ram drive id to read from */  
    CONST CHAR *loadPath,      /* file path name */  
    CONST CHAR *fileName       /* file name */  
);
```

■ Parameter

[mpRamDriveId]

ID of the drive

Symbolic constant: MP_DRV_ID_DRAM (value: 1)

Symbolic constant: MP_DRV_ID_SRAM (value: 2)

Symbolic constant: MP_DRV_ID_USB0 (value: 3)

[loadPath]

Folder name on the drive

To load a file on the root directory of the drive, specify a blank character string ("").

[fileName]

File name to load to the controller

A job file that already exists on the controller cannot be overwritten. In this case, the API returns an error. (The API returns with the return value "2040: Defined JOB name".)

First delete the relevant job on the controller with mpDeleteJob or another API, and then use this API.

8 Existing File Access API

mpLoadFile

■ Return Value

0 Normal end

One of the following values Error

Value (hexadecimal)	Value (decimal)	Description
0x1086 or less	4230 or less	Error related to the file For details, refer to the error numbers (decimal) in the "Chap. 9.1 Error Message" of "Chap. 9 Error" in the maintenance manual of the controller ¹⁾ .
0x2010	8208	The manipulator is in motion
0x2060	8288	Alarm or error status
0x2080	8320	Wrong operation mode
0x2090	8336	Data being accessed by another function Retry later.
0xffffffd	-3	Invalid drive name
0xffff 0xffffffff	65535	Other than the above

1 For the YRC1000: YRC1000 MAINTENANCE MANUAL (RE-CHO-A114)
For the YRC1000micro: YRC1000micro MAINTENANCE MANUAL (RE-CHO-A115)



- This API is executed only with the priority MP_PRI_TIME_NORMAL. When a task with a higher priority than MP_PRI_TIME_NORMAL calls FileControl API, and the task priority limitation is set to "Automatic Adjustment" (S2C1101 = 1), the system temporarily lowers the priority to MP_PRI_TIME_NORMAL, and executes the function. If the task priority limitation is set to "limitation" (S2C1101 = 0), the API returns an error.
- Calling mpLoadFile, mpSaveFile, mpFdWriteFile, or mpFdReadFile to load or save an existing file prohibits editing operation to prevent conflicts in the system's internal information. Thus, the editing mode of the JOB, variable, or I/O is initialized, and the programming pendant cannot be temporarily operated. Therefore, so as not to interfere with editing operation by using the programming pendant, use these APIs in the teach mode only when necessary, and do not load or save successively.

mpSaveFile

Saves a file on the controller to the specified folder on a drive.

■ Syntax

```
LONG          mpSaveFile          /* return : status */
(
    LONG          mpRamDriveId,    /* ram drive id to write to */
    CONST CHAR    *savePath,       /* file path name */
    CONST CHAR    *fileName        /* file name */
);
```

■ Parameter

[mpRamDriveId]

ID of the drive

Symbolic constant: MP_DRV_ID_DRAM (value: 1)

Symbolic constant: MP_DRV_ID_SRAM (value: 2)

Symbolic constant: MP_DRV_ID_USB0 (value: 3)

[savePath]

File name on the drive

To save a file to the root directory, specify a blank character string ("").

[fileName]

File name

If a file name that already exists on the drive is specified, that file is overwritten.

■ Return Value

0 Normal end

One of the following values Error

Value (hexadecimal)	Value (decimal)	Description
0x1086 or less	4230 or less	Error related to the file For details, refer to the error numbers (decimal) in the "Chap. 9.1 Error Message" of "Chap. 9 Error" in the maintenance manual of the controller ¹).
0x2060	8288	Alarm or error status
0x2090	8336	Data being accessed by another function Retry later.
0xffffffd	-3	Invalid drive name
0xffff 0xfffffff	65535 -1	Other than the above

¹ For the YRC1000: YRC1000 MAINTENANCE MANUAL (RE-CHO-A114)
For the YRC1000micro: YRC1000micro MAINTENANCE MANUAL (RE-CHO-A115)

8 Existing File Access API

mpSaveFile



- This API is executed only with the priority MP_PRI_TIME_NORMAL. When a task with a higher priority than MP_PRI_TIME_NORMAL calls FileControl API, and the task priority limitation is set to "Automatic Adjustment" (S2C1101 = 1), the system temporarily lowers the priority to MP_PRI_TIME_NORMAL, and executes the function. If the task priority limitation is set to "limitation" (S2C1101 = 0), the API returns an error.
- Calling mpLoadFile, mpSaveFile, mpFdWriteFile, or mpFdReadFile to load or save an existing file prohibits editing operation to prevent conflicts in the system's internal information. Thus, the editing mode of the JOB, variable, or I/O is initialized, and the programming pendant cannot be temporarily operated. Therefore, so as not to interfere with editing operation by using the programming pendant, use these APIs in the teach mode only when necessary, and do not load or save successively.

mpRefreshFileList

Updates a file list.

■ Syntax

```
LONG      mpRefreshFileList      /* return : status */  
(  
    SHORT  extension             /* file extension index */  
);
```

■ Description

Make sure to execute this API before executing mpGetFileCount() or mpGetFileName.

■ Parameter

[extensionId]

File type ID

Symbolic constant: MP_EXT_ID_JBI (value: 1): Specified for the independent job file (JBI).

Symbolic constant: MP_EXT_ID_JBR (value: 2): Specified for the related job file (JBR).

■ Return Value

Symbolic constant: OK (value: 0) : Normal end

Symbolic constant: ERROR (value: -1) : Error

mpGetFileCount

Retrieves a number of files of the file list.

■ Syntax

```
LONG mpGetFileCount      /* return : status */  
(  
    void  
);
```

■ Description

This API returns the number of files in the file list retrieved by mpRefreshFileList().
Thus, to retrieve the latest information, make sure to execute mpRefreshFileList() before executing this API.

■ Parameter

None

■ Return Value

Value ≥ 0 Number of files

Symbolic constant: ERROR (value: -1) : Error

mpGetFileName

Retrieves a file name from the file list.

■ Syntax

```
LONG    mpGetFileName    /* return : status */  
(  
    int        index,        /* index of file list */  
    CHAR        *fileName    /* pointer to buffer to write the file name */  
);
```

■ Description

This API retrieves the file name specified by *index* from the file list retrieved by `mpRefreshFileList()`.

Thus, to retrieve the latest information, make sure to execute `mpRefreshFileList()` before executing this API.

■ Parameter

[index]

The index in the list of the retrieved file name

[fileName]

The pointer to the buffer where the saved file name is output

■ Return Value

0 Normal end

Symbolic constant: ERROR (value: -1) : Error

mpFdWriteFile

Writes file data in the specified fd to the controller.

■ **Syntax**

```
long mpFdWriteFile                                /* return : status */
(
    int fd,                                       /* file descriptor to read from */
    MP_FILE_NAME_SEND_DATA *sData /* pointer to send file information */
);
```

■ **Parameter**

[fd]

The file descriptor that reads the written file data
Use the value that is retrieved in advance with mpOpen() of FileControl API.

[sData]

The pointer to the structure that stores the written file information (the following members are defined for MP_FILE_NAME_SEND_DATA structure.)

cFileName[TRANS_FILE_LEN + 1]

The array storing the character string of the written file

The file name is created when it is saved with the external memory function (name.JBI, etc.)

Symbolic constant: TRANS_FILE_LEN (value: 36) * The value indicates the sum of 32+1+3.

■ **Return Value**

0 Normal end

One of the following values Error

Value (hexadecimal)	Value (decimal)	Description
0x1086 or less	4230 or less	Error related to the file For details, refer to the error numbers (decimal) in the “Chap. 9.1 Error Message” of “Chap. 9 Error” in the maintenance manual of the controller ¹ .
0x2010	8208	The manipulator is in motion
0x2060	8288	Alarm or error status
0x2080	8320	Wrong operation mode
0x2090	8336	Data being accessed by another function Retry later.
0xffff 0xffffffff	65535 -1	Other than the above

1 For the YRC1000: YRC1000 MAINTENANCE MANUAL (RE-CHO-A114)
For the YRC1000micro: YRC1000micro MAINTENANCE MANUAL (RE-CHO-A115)

8 Existing File Access API

mpFdWriteFile



- This API writes to the file specified by the file descriptor. However, immediately before calling this API, make sure to open the target file and pass that file descriptor to this API. This API does not work correctly if an API is executed that performs a file operation (e.g., mpRead or mpWrite) on that file descriptor before calling these APIs.
- This API is executed only with the priority MP_PRI_TIME_NORMAL. When a task with a higher priority than MP_PRI_TIME_NORMAL calls FileControl API, and the task priority limitation is set to "Automatic Adjustment" (S2C1101 = 1), the system temporarily lowers the priority to MP_PRI_TIME_NORMAL, and executes the function. If the task priority limitation is set to "limitation" (S2C1101 = 0), the API returns an error.
- Calling mpLoadFile, mpSaveFile, mpFdWriteFile, or mpFdReadFile to load or save an existing file prohibits editing operation to prevent conflicts in the system's internal information. Thus, the editing mode of the JOB, variable, or I/O is initialized, and the programming pendant cannot be temporarily operated. Therefore, so as not to interfere with editing operation by using the programming pendant, use these APIs in the teach mode only when necessary, and do not load or save successively.

mpFdReadFile

Reads file data from the controller to the specified fd.

■ **Syntax**

```
long mpFdWriteFile                                /* return : status */
(
    int                fd,                        /* file descriptor to read from */
    MP_FILE_NAME_SEND_DATA *sData                /* pointer to send file information */
);
```

■ **Parameter**

[fd]

The file descriptor that writes the read file data

Use the value that is retrieved in advance with mpOpen() or mpCreate() of FileControl API.

[sData]

The pointer to the structure that stores the read file information
(The following member is defined for the MP_FILE_NAME_SEND_DATA structure.)

cFileName[TRANS_FILE_LEN + 1]

The array storing the character string of the read file

The file name is created when it is saved with the external memory function (name.JBI, etc.)

Symbolic constant: TRANS_FILE_LEN (value: 36) * The value indicates the sum of 32+1+3.

■ **Return Value**

0 Normal end

One of the following values Error

Value (hexadecimal)	Value (decimal)	Description
0x1086 or less	4230 or less	Error related to the file For details, refer to the error numbers (decimal) in the “Chap. 9.1 Error Message” of “Chap. 9 Error” in the maintenance manual of the controller ¹ .
0x2060	8288	Alarm or error status
0x2090	8336	Data being accessed by another function Retry later.
0xffff 0xffffffff	65535 -1	Other than the above

1 For the YRC1000: YRC1000 MAINTENANCE MANUAL (RE-CHO-A114)
For the YRC1000micro: YRC1000micro MAINTENANCE MANUAL (RE-CHO-A115)

8 Existing File Access API

mpFdReadFile



- This API reads from the file specified by the file descriptor. However, immediately before calling this API, make sure to open the target file and pass that file descriptor to this API. This API does not work correctly if an API is executed that performs a file operation (e.g., mpRead or mpWrite) on that file descriptor before calling these APIs.
- This API is executed only with the priority MP_PRI_TIME_NORMAL. When a task with a higher priority than MP_PRI_TIME_NORMAL calls FileControl API, and the task priority limitation is set to "Automatic Adjustment" (S2C1101 = 1), the system temporarily lowers the priority to MP_PRI_TIME_NORMAL, and executes the function. If the task priority limitation is set to "limitation" (S2C1101 = 0), the API returns an error.
- Calling mpLoadFile, mpSaveFile, mpFdWriteFile, or mpFdReadFile to load or save an existing file prohibits editing operation to prevent conflicts in the system's internal information. Thus, the editing mode of the JOB, variable, or I/O is initialized, and the programming pendant cannot be temporarily operated. Therefore, so as not to interfere with editing operation by using the programming pendant, use these APIs in the teach mode only when necessary, and do not load or save successively.

mpFdGetJobList

Retrieves a job list to the specified fd.

■ Syntax

```
long mpFdGetJobList                                /* return : status */
(
    int                fd,          /* file descriptor to write to */
    MP_GET_JOBLIST_RSP_DATA *rData /* pointer to response data */
)
```

■ Parameter

[fd]

The file descriptor that writes the job list

Use the value that is retrieved in advance with mpOpen() or mpCreate() of FileControl API.

[responseData]

The pointer to the structure that stores the read result
(The following members are defined for the structure.)

typedef struct

```
{
    USHORT    err_no;          /* Error number */
    USHORT    ulsEndFlag;      /* Always 1 with the normal end */
    USHORT    uListDataNum;    /* The number of read job names */
    UCHAR     cListData[MP_LIST_DATA_SIZE]; /* Work area used for reading */
    CHAR      reserved[2];
} MP_GET_JOBLIST_RSP_DATA;
```

```
#define MP_LIST_DATA_SIZE 1000 /*Size of work area used for reading */
```

■ Return Value

Symbolic constant: OK (value: 0) : Normal end

Symbolic constant: ERROR (value: -1) : Error

8 Existing File Access API

mpFdGetJobList

■ Note

- Job list format

Two-dimensional array with 32 characters +1 (NULL character is filled to the job name trail.)



This API is executed only with the priority MP_PRI_TIME_NORMAL. When a task with a higher priority than MP_PRI_TIME_NORMAL calls FileControl API, and the task priority limitation is set to "Automatic Adjustment" (S2C1101 = 1), the system temporarily lowers the priority to MP_PRI_TIME_NORMAL, and executes the function. If the task priority limitation is set to "limitation" (S2C1101 = 0), the API returns an error.

9 System Monitor API

mpGetVarData

Retrieves the values of variables (B, I, D, R) of the controller. Multiple variables can be specified.

Using mpGetUserVars is recommended for reading variables.



mpGetUserVars combines mpGetVarData (B, I, D, and R variables), mpGetSVarInfo (S variables), and mpGetPosVarData (position type variables) into a single API that greatly improves the speed of reading variables. This API remains available to maintain backwards compatibility with existing MotoPlus applications.

■ Syntax

```
LONG mpGetVarData (
MP_VAR_INFO*      sData,
LONG*              rData,
LONG              num
);
```

■ Parameter

[sData]

The pointer to the data structure which specifies variable

- MP_VAR_INFO
Variable setting data structure

Syntax: typedef struct
{
 USHORT usType;
 USHORT usIndex;
} MP_VAR_INFO;

Member: <usType>
Variable type

Symbolic constant	Description
MP_RESTYPE_VAR_B	Byte type
MP_RESTYPE_VAR_I	Integer type
MP_RESTYPE_VAR_D	Double-precision type
MP_RESTYPE_VAR_R	Real type

<usIndex>
Variable index

To set multiple variables, define *num* arrays, set the variable type and variable index of each array, then set the starting address of each array to *sData*.



If multiple variable data are retrieved by this API, these data are not synchronized. Thus, while retrieving multiple values, these may be changed by another application task or system task.

9 System Monitor API

mpGetVarData

[rData]

Pointer to the variable data

[num]

Number of variable data (up to 252)

■ Return Value

Symbolic constant: OK (value: 0) : Normal end

Symbolic constant: ERROR (value: -1) : Error



This API takes a few msec to process, so be sure not to use it in the I/O control cycle task or in the interpolation cycle task.

Is it is used, the process cannot be completed within the control cycle.

mpGetSVarInfo

Retrieves the value of S variable (S) of the controller.



Using mpGetUserVars is recommended for reading variables.

mpGetUserVars combines mpGetVarData (B, I, D, and R variables), mpGetSVarInfo (S variables), and mpGetPosVarData (position type variables) into a single API that greatly improves the speed of reading variables. This API remains available to maintain backwards compatibility with existing MotoPlus applications.

■ Syntax

```
LONG    mpGetSVarInfo
(
    MP_VAR_INFO          *sData,
    MP_SVAR_RECV_INFO    *rData,
    LONG                 num
)
```

■ Parameter

[sData]

Pointer to the data structure which specifies variable

typedef struct

```
{
    USHORT    usType;      /* Variable type (only MP_RESTYPE_VAR_S is available) */
    USHORT    usIndex;     /* Variable index */
} MP_VAR_INFO;
```

To set multiple variables, define *num* arrays, set the variable type and variable index of each array, and set the starting address of each array to *sData*.

[rData]

Pointer to the storage area for variable data

typedef struct

```
{
    UCHAR    ucValue[MAX_SVAR_SIZE+1]; /*MAX_SVAR_SIZE (32
characters)+null character\0*/
    UCHAR    reserved[3];
} MP_SVAR_RECV_INFO;
```

To set multiple variables, define *num* arrays.

[num]

Number of S variables (up to 10)

■ Return Value

Symbolic constant: OK (value: 0) : Normal end

Symbolic constant: ERROR (value: -1) : Error

9 System Monitor API

mpGetSVarInfo



- Since this API's processing time is a few milliseconds, do not use this API in a task with the I/O control cycle or interpolation cycle. If it is used in such a task, the process cannot be completed within the control cycle.
- If multiple S variables are retrieved by this API, these data are not synchronized. Thus, while retrieving multiple values, these may be changed by another application task or system task.
- When changing a parameter or loading a file from the external memory device, the process of this API cannot be executed because the system is busy. In this case, the error (-1) is returned. Retry this API after a while.

mpReadIO

Retrieves several I/O values of the controller at a time.

■ **Syntax**

```
LONG mpReadIO (  
  MP_IO_INFO*    sData,  
  USHORT*        rData,  
  LONG           num  
);
```

■ **Parameter**

[sData]

Pointer to the data structure that specifies I/O address

- MP_IO_INFO
Data structure that specifies I/O address
Syntax: typedef struct
 {
 ULONG ulAddr;
 } MP_IO_INFO;
Member: <ulAddr>
 I/O address

Value	Description
10 - 5127	Universal input #00010 - #05127(4096)
10010 - 15127	Universal output #10010 - #15127(4096)
20010 - 25127	External input #20010 - #25127(4096)
30010 - 35127	External output #30010 - #35127(4096)
40010 - 42567	Specific input #40010 - #42567(2048)
50010 - 55127	Specific output #50010 - #55127(4096)
60010 - 60647	Interface panel #60010 - #60647(512)
70010 - 79997	Auxiliary relay #70010 - #79997(7992)
80010 - 85127	Control input #80010 - #85127(4096)
87010 - 87207	Pseudo input #87010 - #87207(160)
27010 - 29567	Network input #27010 - #29567(2048)
37010 - 39567	Network output #37010 - #39567(2048)
1000000 - 1000999	Register M000 - M999 (1000)



M000 to M559 (1000000 to 1000559) is available for general registers. Later than that, from M560 to M999 (1000560 to 1000999) are an area reserved by the system and used by special functions. (For example, analog outputs are allocated to M560 to M562 in the instructions manual of JANCD-AEW02-E circuit board (for 3-channel analog output) for the YRC1000.) This area is not available for writing but reading. Refer to the instruction manual for information on how to use this area.

9 System Monitor API

mpReadIO

To set multiple I/Os, define *num* arrays, set the I/O address of each array, then set the starting address of each array to *sData*.



For the YRC1000/YRC1000micro, since the scanning time of the concurrent I/O is shorter than the I/O control cycle, the status changing of the signal may not be detected even if the priority is set to the notification task for the I/O control cycle.

For example, in the case of monitoring the pulse signal which is changing instantly OFF-ON-OFF, ON may not be acquired depending on the executing timing of the application. Monitor the signal by adjusting the pulse range or by preparing another signal to latch the pulse signal.

[rData]

I/O value

[num]

Number of I/O (up to 253)

■ **Return Value**

Symbolic constant: OK (value: 0) : Normal end

Symbolic constant: ERROR (value: -1) : Error

mpMonitor

Retrieves the variable and an I/O value of the controller at a time.

■ **Syntax**

```
LONG mpMonitor (  
MP_MONITOR_INFO*      sData,  
LONG*                  rData,  
LONG                   infonum  
);
```

■ **Parameter**

[sData]

Pointer to the monitor information structure

- MP_MONITOR_INFO

Monitor information structure

```
Syntax:  typedef struct  
        {  
            ULONG ulType;  
            ULONG ulIndex;  
        } MP_MONITOR_INFO;
```

Member: <ulType>
Variable, I/O type

Symbolic constant	Description
MP_RESTYPE_VAR_B (value: 1)	Byte type
MP_RESTYPE_VAR_I (value: 2)	Integer type
MP_RESTYPE_VAR_D (value: 3)	Double-precision type
MP_RESTYPE_VAR_R (value: 4)	Real type
MP_RESTYPE_CIO (value: 8)	IO

<ulIndex>
Variable number, I/O address

To set multiple variables and I/Os, define *infonum* arrays, set the variable type and variable index of each array, then set the starting address of each array to *sData*.



For the YRC1000/YRC1000micro, since the scanning time of the concurrent I/O is shorter than the I/O control cycle, the status changing of the signal may not be detected even if the priority is set to the notification task for the I/O control cycle.

For example, in the case of monitoring the pulse signal which is changing instantly OFF-ON-OFF, ON may not be acquired depending on the executing timing of the application. Monitor the signal by adjusting the pulse range or by preparing another signal to latch the pulse signal.

9 System Monitor API mpMonitor

[rData]

Monitor information value

[inforum]

Number of monitor (up to 126)

■ **Return Value**

Symbolic constant: OK (value: 0) : Normal end

Symbolic constant: ERROR (value: -1) : Error

mpGetPosVarData

Retrieves the value of the position-type variable of the controller.



Using mpGetUserVars is recommended for reading variables.

mpGetUserVars combines mpGetVarData (B, I, D, and R variables), mpGetSVarInfo (S variables), and mpGetPosVarData (position type variables) into a single API that greatly improves the speed of reading variables. This API remains available to maintain backwards compatibility with existing MotoPlus applications.

■ **Syntax**

```
LONG mpGetPosVarData (
MP_VAR_INFO*      sData,
LONG*             rData,
LONG              num
);
```

■ **Parameter**

[sData]

[in] The pointer to the data structure array that sets the position-type variable

- MP_VAR_INFO

[in] The data structure that sets the position-type variable

Syntax: typedef struct
 {
 USHORT usType;
 USHORT usIndex;
 } MP_VAR_INFO;

Member: <usType>
 Variable type

Symbolic constant	Description
MP_RESTYPE_VAR_ROBOT (value: 5)	Robot
MP_RESTYPE_VAR_BASE (value: 6)	Base
MP_RESTYPE_VAR_STATION (value: 7)	Station

<usIndex>
Variable number

9 System Monitor API

mpGetPosVarData

To set multiple position-type variables, define *num* arrays, set the variable type and variable index of each array, then set the starting address of each array to *sData*.



- If multiple position-type variables are retrieved by this API, these data are not synchronized. Thus, while retrieving multiple values, these may be changed by another application task or system task.
- Since this API's processing time is a few milliseconds, do not use this API in a task with the I/O control cycle or interpolation cycle. If it is used in such a task, the process cannot be completed within the control cycle.

[rData]

[out] The pointer to the position-type variable information (requires area for 10*num)

Array	Attribute	Description
rData[0]	D05 - D00	Variable type 0 Pulse 16 Cartesian (base coordinates) 17 Cartesian (robot coordinates) 18 Cartesian (tool coordinates) 19 Cartesian (user coordinates) 20 Cartesian (reserved for master tool)
	D07 - D06	Reserved by manufacturer
	D08	0: Front 1: Back
	D09	0: Upper arm 1: Lower arm
	D10	0: Flip 1: No flip
	D11	0: R < 180 deg 1: R >= 180 deg
	D12	0: T < 180 deg 1: T >= 180 deg
	D13	0: S < 180 deg 1: S >= 180 deg
	D14 - D15	Reserved by manufacturer
	D16 - D21	Tool number (0 - 63)
	D22 - D27	User coordinate number
	D28 - D31	Reserved by manufacturer
rData[1]		(Extended attribute)
Array	Pulse	Cartesian
rData[2]	1st axis (S) pulse value	X-axis coordinate (unit: micron)
rData[3]	2nd axis (L) pulse value	Y-axis coordinate (unit: micron)
rData[4]	3rd axis (U) pulse value	Z-axis coordinate (unit: micron)
rData[5]	4th axis (R) pulse value	Wrist angle Rx (unit: 0.0001 deg)
rData[6]	5th axis (B) pulse value	Wrist angle Ry (unit: 0.0001 deg)
rData[7]	6th axis (T) pulse value	Wrist angle Rz (unit: 0.0001 deg)
rData[8]	7th axis (E) pulse value	Angle Re (unit: 0.0001 deg)
rData[9]	8th axis pulse value	8th axis pulse value (micron in the case of traveling axis)

[num]

Number of array (up to 25)

9 System Monitor API

mpGetPosVarData

■ Return Value

Symbolic constant: OK (value: 0) : Normal end

Symbolic constant: ERROR (value: -1) : Error

mpGetUserVars

Retrieves the user variable of the controller.

To compare mpGetUserVars with the other variable access APIs, such as mpGetVarData(), mpGetPosVarData() and mpGetSVarInfo(), mpGetUserVars can retrieve user variable in high speed.

■ Syntax

```
STATUS mpGetUserVars(
  MP_USR_VAR_INFO*mp_usr_var_info
);
```

■ Parameter

[mp_usr_var_info]

[in] User variable access API interface (Input/Output)

- MP_USR_VAR_INFO

[in] The data structure that sets the user variable

```
Syntax:  typedef struct
          {
            int var_type;
            int var_no;
            union {
                MP_B_VAR_BUFF      b;
                MP_I_VAR_BUFF      i;
                MP_D_VAR_BUFF      d;
                MP_R_VAR_BUFF      r;
                MP_S_VAR_BUFF      s;
                MP_P_VAR_BUFF      p;
                MP_P_VAR_BUFF      bp;
                MP_P_VAR_BUFF      ex;
            } val;
          } MP_USR_VAR_INFO;
```

Member: <var_type>
Variable type (Input)

Symbolic constant	Description
MP_VAR_B	Byte Variable
MP_VAR_I	Integer Variable
MP_VAR_D	Double Variable
MP_VAR_R	Real Variable
MP_VAR_S	Character Variable
MP_VAR_P	Robot position Variable
MP_VAR_BP	Base position Variable
MP_VAR_EX	Station (LEX) Variable

<var_no>
Variable number (Input)

9 System Monitor API

mpGetUserVars

<val>

Variable data buffer (output)

Syntax: #define STR_VAR_SIZE 32
#define MP_GRP_AXES_NUM 8

```
typedef unsigned char MP_B_VAR_BUFF;
typedef short        MP_I_VAR_BUFF;
typedef long         MP_D_VAR_BUFF;
typedef float        MP_R_VAR_BUFF;
typedef char         MP_S_VAR_BUFF[STR_VAR_SIZE];
```

```
typedef struct {
    UNIT          dtype;
    int           tool_no;
    int           uf_no;
    BITSTRING     fig_ctrl;
    long          data[MP_GRP_AXES_NUM];
} MP_P_VAR_BUFF;
```

Member: *MP_P_VAR_BUFF*

<dtype>

Coordinate system type

Symbolic constant	Description
MP_PULSE_DTYPE	Pulse coordinate
MP_BASE_DTYPE	Base coordinate
MP_ROBO_DTYPE	Robot coordinate
MP_TOOL_DTYPE	Tool coordinate
MP_USER_DTYPE	User coordinate
MP_MTOOL_DTYPE	Master tool coordinate

<tool_no>

Tool file No. (0 to 63)

<uf_no>

User coordinate system file No. (0 to 62)

<fig_ctrl>

Figure information (bits)

Symbolic constant	Meaning
MP_FIG_SIDE (value: 0x00000001)	0: Front 1: Back
MP_FIG_ELBOW (value: 0x00000002)	0: Uper elbow 1: Lower elbow
MP_FIG_FLIP (value: 0x00000004)	0: Flip 1: No flip
MP_FIG_R180 (value: 0x00000008)	0: R<180 1: R>=180
MP_FIG_T180 (value: 0x00000010)	0: T<180 1: T>=180
MP_FIG_S180 (value: 0x00000020)	0: S<180 1: S>=180

<data>

Position data

9 System Monitor API

mpGetUserVars

■ Return value

0 : Normal end
Any of following negative values: : Abnormal end

Symbolic constant	Description
E_MP_VARS_ACCESS (value: -1)	An error except followings
E_MP_VAR_TYPE (value: -2)	Incorrect variable type
E_MP_VAR_NO_RANGE (value: -3)	Incorrect variable No.
E_MP_PVAR_DTYPE (value: -4)	Incorrect coordinate type of the position variable
E_MP_PVAR_TOOL_NO (value: -5)	Incorrect tool No. of the position variable
E_MP_PVAR_USER_NO (value: -6)	Incorrect user coordinate type of the position variable
E_MP_PVAR_UNSETUP (value: -7)	Undefined data of the position variable
E_MP_PMS_RET_VAL (value: -8)	Failure to retrieve the position variable

Data types which are used in this API are different from ones in the mpGetVarData(), mpGetPosVarData() and mpGetSVarInfo(). The following table shows the differences of the position variable.

Difference of the API Parameter

Position Variable Retrieved API	User Variable Retrieved API
mpGetPosVarData(MP_VAR_INFO *sData; LONG *rData; LONG num;)	mpGetUserVars(MP_USR_VAR_INFO *mp_usr_var_info;) Syntax typedef struct { int var_type; int var_no; union { MP_B_VAR_BUFF b; . . MP_P_VAR_BUFF p; MP_P_VAR_BUFF bp; MP_P_VAR_BUFF ex; } val; } MP_USR_VAR_INFO;



Setting Content of the Parameter

		mpGetPosVarData()		mpGetUserVars()	
Item		rData[10]	Bit No.	p, bp, ex	
Coordinate system		rData[0]	d0 to d5	.dtype	
Type	Front/Back		d08	.fig_ctrl	d0
	Upper elbow/ Lower elbow		d09		d1
	Flip/No flip		d10		d2
	R<180/R>=180		d11		d3
	T<180/T>=180		d12		d4
	S<180/S>=180		d13		d5
Tool No.			d16 to d21	.tool_no	
User coordinate No.			d22 to d27	.uf_no	
Position data		rData[2] to [9]		.data[0] to data[7]	

mpGetAlarmStatus

Retrieves the alarm status of the controller.

■ Syntax

```
LONG mpGetAlarmStatus(  
MP_ALARM_STATUS_RSP_DATA* rData  
);
```

■ Description

Retrieves the alarm status of system.

The value of the alarm information retrieved is input to the parameter structure.

■ Parameter

[rData]

[out] The pointer to the data structure which receives the alarm status

- MP_ALARM_STATUS_RSP_DATA

[out] The data structure which receives the alarm status

Syntax: typedef struct
{
 SHORT *sIsAlarm*;
 CHAR *reserved*[2];
} MP_ALARM_STATUS_RSP_DATA;

Member: <*sIsAlarm*>
Alarm status

Description of bits	Description
D00	Error (An error occurs when D00 bit is 1.)
D01	Alarm (An alarm occurs when D01 bit is 1.)
D02 - D15	Reserved by manufacturer

■ Return Value

Symbolic constant: OK (value: 0) : Normal end

Symbolic constant: ERROR (value: -1) : Error

mpGetAlarmCode

Retrieves the error code and alarm code of the controller.

■ Syntax

```
LONG mpGetAlarmCode
(MP_ALARM_CODE_RSP_DATA* rData
);
```

■ Description

Retrieves the error code and alarm code of the system.

The information retrieved is set in the corresponding member in parameter structure as a value.

■ Parameter

[rData]

[out] The pointer to the data structure which receives the alarm code

- MP_ALARM_CODE_RSP_DATA

[out] The data structure which receives the alarm code

```
Syntax: typedef struct
{
    USHORT      usErrorNo;
    USHORT      usErrorData;
    USHORT      usAlarmNum;
    CHAR        reserved[2];
    MP_ALARM_DATA AlarmData;
} MP_ALARM_CODE_RSP_DATA;
```

Member: <usErrorNo>
Error number

<usErrorData>
Error data

<usAlarmNum>
Number of alarm

<AlarmData>
Alarm data

- MP_ALARM_DATA

[out] Alarm data structure

Syntax: #define MAX_ALARM_COUNT (4)

```
typedef struct
{
    USHORT usAlarmNo[MAX_ALARM_COUNT];
    USHORT usAlarmData[MAX_ALARM_COUNT];
} MP_ALARM_DATA;
```

Member: <usAlarmNo[MAX_ALARM_COUNT]>
Alarm number (up to 4 numbers)

<usAlarmData[MAX_ALARM_COUNT]>
Alarm data (up to 4 data)

9 System Monitor API

mpGetAlarmCode

■ Return Value

Symbolic constant: OK (value: 0) : Normal end

Symbolic constant: ERROR (value: -1) : Error

mpGetMode

Retrieves the operation mode of the controller.

■ **Syntax**

```
LONG mpGetMode
(MP_MODE_RSP_DATA*    rData
);
```

■ **Description**

Retrieves the operation mode of the controller.

The controller has the following modes of operation: "teaching mode", "play mode", and "remote mode". The information retrieved is set in the corresponding member in parameter structure as a value.

■ **Parameter**

[rData]

[out] The pointer to the data structure which receives the operation mode

- MP_MODE_RSP_DATA

[out] The data structure which receives the operation mode

```
Syntax:  typedef struct
          {
            SHORT sMode;
            SHORT sRemote;
          } MP_MODE_RSP_DATA;
```

Member: <sMode>
Operation mode

sMode value	Description
1	TEACH
2	PLAY

<sRemote>
Remote mode

sRemote value	Description
0	Remote OFF
1	Remote ON

■ **Return Value**

Symbolic constant: OK (value: 0) : Normal end
Symbolic constant: ERROR (value: -1) : Error

mpGetCycle

Retrieves the JOB cycle mode of the controller.

■ **Syntax**

```
LONG mpGetCycle(  
MP_CYCLE_RSP_DATA* rData  
);
```

■ **Description**

Retrieves the cycle mode of the system.

The information retrieved is set in the corresponding member of the parameter structure as a value.

■ **Parameter**

[rData]

[out] The pointer to the data structure which receives the cycle mode

- MP_CYCLE_RSP_DATA

[out] The data structure which receives the cycle mode

```
Syntax: typedef struct  
{  
    SHORT sCycle;  
    CHAR reserved[2];  
} MP_CYCLE_RSP_DATA;
```

Member: <sCycle>
Cycle

sCycle value	Description
1	Step
2	1Cycle
3	Auto

■ **Return Value**

Symbolic constant: OK (value: 0) : Normal end
Symbolic constant: ERROR (value: -1) : Error

mpGetServoPower

Retrieves the ON/OFF status of the controller servo power.

■ **Syntax**

```
LONG mpGetServoPower
(MP_SERVO_POWER_RSP_DATA* rData
);
```

■ **Description**

Retrieves the ON/OFF status of the system servo power.
The information retrieved is set in the corresponding member of the parameter structure as a value.

■ **Parameter**

[rData]

[out] The pointer to the data structure which receives the ON/OFF status of servo power

- MP_SERVO_POWER_RSP_DATA

[out] The data structure which receives the ON/OFF status of servo power

```
Syntax: typedef struct
{
    SHORT sServoPower;
    CHAR reserved[2];
} MP_SERVO_POWER_RSP_DATA;
```

Member: <sServoPower>
Servo power

sServoPower value	Description
0	Servo power OFF
1	Servo power ON

■ **Return Value**

Symbolic constant: OK (value: 0) : Normal end
Symbolic constant: ERROR (value: -1) : Error

mpGetPlayStatus

Retrieves the play status of the job of the controller.

■ Syntax

```
LONG mpGetPlayStatus (  
    MP_PLAY_STATUS_RSP_DATA*    rData  
);
```

■ Description

Retrieves the play status information of the job (pause, playback). The information retrieved is set in the corresponding member in parameter structure as a value.

■ Parameter

[rData]

[out] The pointer to the data structure which receives the play status

- MP_PLAY_STATUS_RSP_DATA

[out] The data structure which receives the play status

Syntax: typedef struct
{
 SHORT sStart;
 SHORT sHold;
} MP_PLAY_STATUS_RSP_DATA

Member: <sStart>
Operation status

sStart value	Description
1	Start ON
0	Start OFF

<sHold>
Hold status

sHold value	Description
1	Hold ON
0	Hold OFF

■ Return Value

Symbolic constant: OK (value: 0) : Normal end

Symbolic constant: ERROR (value: -1) : Error

mpGetMasterJob

Retrieves the master job name of the controller.

■ **Syntax**

```
LONG mpGetMasterJob(  
    MP_TASK_SEND_DATA*          sData,  
    MP_JOB_NAME_RSP_DATA*      rData  
);
```

■ **Description**

Retrieves the master job name of the controller. When there is a sub task in the controller, the name of sub task can be retrieved by specifying the sub task for the parameter. The information retrieved is set in the corresponding member in parameter structure as a value.

■ **Parameter**

[sData]

[in] The pointer to the data structure which specifies the task number

- MP_TASK_SEND_DATA

[in] The data structure which specifies the task number

```
Syntax:  typedef struct  
        {  
            SHORT  sTaskNo;  
            CHAR   reserved[2];  
        } MP_TASK_SEND_DATA;
```

Member: <sTaskNo>
Task number

sTaskNo value	Description
0	Master task
1	Subtask 1
2	Subtask 2
3	Subtask 3
4	Subtask 4
5	Subtask 5
6	Subtask 6
7	Subtask 7
8	Subtask 8
9	Subtask 9
10	Subtask 10
11	Subtask 11
12	Subtask 12
13	Subtask 13
14	Subtask 14
15	Subtask 15

9 System Monitor API

mpGetMasterJob

[rData]

[out] The pointer to the data structure which receives the job name

- MP_JOB_NAME_RSP_DATA

[out] The data structure which receives the job name

Syntax: #define MAX_JOB_NAME_LEN (33)

```
typedef struct
```

```
{
```

```
    CHAR    cJobName[MAX_JOB_NAME_LEN];
```

```
    CHAR    reserved[3];
```

```
} MP_JOB_NAME_RSP_DATA;
```

Member: <cJobName[MAX_JOB_NAME_LEN]>

Job name (up to 32 characters for a job name)

■ Return Value

Symbolic constant: OK (value: 0) : Normal end

Symbolic constant: ERROR (value: -1) : Error

mpGetCurJob

Retrieves the name, line number, step number of the current job.

■ **Syntax**

```
LONG mpGetCurJob (  
    MP_TASK_SEND_DATA*    sData,  
    MP_CUR_JOB_RSP_DATA*  rData  
);
```

■ **Description**

Retrieves the name, line number, step number of the current job of the system.

The information retrieved is set in the corresponding member in parameter structure as a value.

■ **Parameter**

[sData]

[in] The pointer to the data structure which specifies the task number

- MP_TASK_SEND_DATA

[in] The data structure which specifies the task number

```
Syntax:  typedef struct  
        {  
            SHORT  sTaskNo;  
            CHAR   reserved[2];  
        } MP_TASK_SEND_DATA;
```

Member: <sTaskNo>
 Task number

sTaskNo value	Description
0	Master task
1	Subtask 1
2	Subtask 2
3	Subtask 3
4	Subtask 4
5	Subtask 5
6	Subtask 6
7	Subtask 7
8	Subtask 8
9	Subtask 9
10	Subtask 10
11	Subtask 11
12	Subtask 12
13	Subtask 13
14	Subtask 14
15	Subtask 15

9 System Monitor API

mpGetCurJob

[rData]

[out] The pointer to the data structure which receives the current job information

- MP_CUR_JOB_RSP_DATA

[out] The data structure which receives the current job information

Syntax: #define MAX_JOB_NAME_LEN (33)

```
typedef struct
{
    USHORT    usJobLine;
    USHORT    usStep;
    CHAR      cJobName[MAX_JOB_NAME_LEN];
    CHAR      reserved[3];
} MP_CUR_JOB_RSP_DATA;
```

Member: <usJobLine>
Job line

<usStep>
Step

<cJobName[MAX_JOB_NAME_LEN]>
Job name (up to 32 characters for a job name)

■ Return Value

Symbolic constant: OK (value: 0) : Normal end

Symbolic constant: ERROR (value: -1) : Error

mpGetSpecialOpStatus

Retrieves the playback special operation status.

■ **Syntax**

```
LONG mpGetSpecialOpStatus (  
MP_SPECIAL_OP_RSP_DATA*      rData  
);
```

■ **Description**

Retrieves the playback special operation status of the controller. The information retrieved is set to the corresponding member in parameter structure as a value. Refer to "Special Playback Operations" in chapter 4.2 of the operator's manual of the controller¹⁾ for details on "Special Playback Operations".

■ **Parameter**

[rData]

[out] The pointer to the data structure which receives the special operation status

- MP_SPECIAL_OP_RSP_DATA

[out] The data structure which receives the special operation status

```
Syntax:  typedef struct  
        {  
          SHORT  sSpecialOpStatus;  
          CHAR   reserved[2];  
        } MP_SPECIAL_OP_RSP_DATA;
```

Member: <sSpecialOpStatus>
Special operation (play mode only)

sSpecialOpStatus bit	Description
D00	Check operation
D01	Safety speed operation
D02	Dry-run speed operation
D03	Machine lock operation
D04	(Reserved)
D05	Low speed operation
D06	Weaving prohibited
D07-D09	(Reserved)
D010	Pressure instruction prohibited
D11-D15	(Reserved))

■ **Return Value**

Symbolic constant: OK (value: 0) : Normal end
Symbolic constant: ERROR (value: -1) : Error

1 For the YRC1000:
YRC1000 GENERAL OPERATOR'S MANUAL (RE-CSO-A051)
For the YRC1000micro:
YRC1000micro OPERATOR'S MANUAL (RE-CSO-A058)

mpGetJobDate

Retrieves the date when the job is changed.

■ Syntax

```
LONG mpGetJobDate (
    MP_JOB_NAME_SEND_DATA*    sData,
    MP_SYS_TIME_RSP_DATA*     rData
);
```

■ Parameter

[sData]

[in] The pointer to the data structure which specifies the job name

- MP_JOB_NAME_SEND_DATA

[in] The data structure which specifies the job name

Syntax: #define MAX_JOB_NAME_LEN (33)

```
typedef struct
{
    CHAR    cJobName[MAX_JOB_NAME_LEN];
    CHAR    reserved[3];
} MP_JOB_NAME_SEND_DATA;
```

Member: <cJobName[MAX_JOB_NAME_LEN]>
Job name (up to 32 characters for a job name)

[rData]

[out] The pointer to the data structure which receives the system time

- MP_SYS_TIME_RSP_DATA

[out] The data structure which receives the system time

Syntax: typedef struct

```
{
    SHORT    sStartYear;
    SHORT    sStartMonth;
    SHORT    sStartDay;
    SHORT    sStartHour;
    SHORT    sStartMin;
    SHORT    sStartSec;
    LONG     lElapsedTime;
} MP_SYS_TIME_RSP_DATA;
```

Member: <sStartYear>
Year

<sStartMonth>
Month

<sStartDay>
Day

<sStartHour>
Hour

9 System Monitor API
mpGetJobDate

<*sStartMin*>
Minute

<*sStartSec*>
Second

<*lElapsedTime*>
Not used

■ **Return Value**

Symbolic constant: OK (value: 0) : Normal end
Symbolic constant: ERROR (value: -1) : Error

mpGetCartPos

Retrieves the current position in Cartesian coordinates.

■ Syntax

```
LONG mpGetCartPos (
    MP_CTRL_GRP_SEND_DATA*    sData,
    MP_CART_POS_RSP_DATA*     rData
);
```

■ Description

Retrieves the current position in Cartesian coordinates of the system.

The information retrieved is set in the corresponding member in parameter structure as a value.



This API should not be used if possible. Compared to mpGetCartPos, mpGetCartPosEx described later has been optimized to retrieve the current position in Cartesian coordinates at a fast cycle. In particular, using mpGetCartPosEx is recommended to retrieve the current position in Cartesian coordinates in a high-priority task. This API remains available to maintain backwards compatibility with existing MotoPlus applications.

■ Parameter

[sData]

[in] The pointer to the data structure which specifies the control group

- MP_CTRL_GRP_SEND_DATA

[in] The data structure which specifies the control group

Syntax: typedef struct
{
CTRLG_T sCtrlGrp;
} MP_CTRL_GRP_SEND_DATA;

Member: <sCtrlGrp>

Control group

*This API returns an ERROR if the specified control group is not controlled by the target controller.

sCtrlGrp value	Description
0	R1 (Robot 1)
1	R2 (Robot 2)
2	R3 (Robot 3)
3	R4 (Robot 4)
4	R5 (Robot 5)
5	R6 (Robot 6)
6	R7 (Robot 7)
7	R8 (Robot 8)
8	B1 (Base 1)

9 System Monitor API

mpGetCartPos

sCtrlGrp value	Description
9	B2 (Base 2)
10	B3 (Base 3)
11	B4 (Base 4)
12	B5 (Base 5)
13	B6 (Base 6)
14	B7 (Base 7)
15	B8 (Base 8)

[rData]

[out] The pointer to the data structure which receives the cartesian position

- MP_CART_POS_RSP_DATA

[out] The data structure which receives the cartesian position

Syntax: #define MAX_CART_AXES (6)

```
typedef struct
{
    LONG   IPos[MAX_CART_AXES];
    SHORT  sConfig;
    CHAR   reserved[2];
} MP_CART_POS_RSP_DATA;
```

Member: <IPos[MAX_CART_AXES]>

Cartesian position (up to 6 arrays)

Array	Description
IPos[0]	X-axis coordinate (unit: micron)
IPos[1]	Y-axis coordinate (unit: micron)
IPos[2]	Z-axis coordinate (unit: micron)
IPos[3]	Wrist angle Rx (unit: 0.0001 deg)
IPos[4]	Wrist angle Ry (unit: 0.0001 deg)
IPos[5]	Wrist angle Rz (unit: 0.0001 deg)

<sConfig>

Relative position information

Attribute	Description
D00	0: Front 1: Back
D01	0: Upper arm 1: Lower arm
D02	0: Flip 1: No flip
D03	0: R < 180 1: R >= 180
D04	0: T < 180 1: T >= 180
D05	0: S < 180 1: S >= 180
D06 - D15	Reserved by manufacturer

9 System Monitor API

mpGetCartPos

For more information on the position data, refer to chapter 2 “Manipulator Coordinate Systems and Operations” in the operator's manual of the controller¹⁾.

■ Return Value

Symbolic constant: OK (value: 0) : Normal end

Symbolic constant: ERROR (value: -1) : Error

¹ For the YRC1000:
YRC1000 GENERAL OPERATOR'S MANUAL (RE-CSO-A051)
For the YRC1000micro:
YRC1000micro OPERATOR'S MANUAL (RE-CSO-A058)

mpGetCartPosEx

Retrieves the current position in Cartesian coordinates with a specified coordinate frame (Robot, Base, User).

■ Syntax

```
LONG mpGetCartPosEx (  
    MP_CARTPOS_EX_SEND_DATA*      sData,  
    MP_CART_POS_RSP_DATA_EX*      rData  
);
```

■ Description

Compared to mpGetCartPos, mpGetCartPosEx is optimized to retrieve the information at a fast cycle.

Using mpGetCartPosEx is recommended to retrieve the current position in Cartesian coordinates in a high-priority task.

Added feature: Previously it was always necessary to specify the tool number when retrieving the current value, but the API was changed so the tool number does not need to be specified and the current value can be retrieved with the tool number currently being used by the robot.

Usage conditions for added feature

For the YRC1000: System version YAS4.40-00 or later and MotoPlusIDE version 2.41 or later

For the YRC1000micro: System version YBS2.54-00 or later and MotoPlusIDE version 2.41 or later

■ Parameter

[*sData*]

[in] The pointer of the control group send data structure

- MP_CARTPOS_EX_SEND_DATA

[in] The present position send data structure

Syntax: typedef struct
 {
 SHORT *sRobotNo*;
 SHORT *sFrame*;
 SHORT *sToolNo*;
 CHAR *reserved*[2];
 } MP_CARTPOS_EX_SEND_DATA;

9 System Monitor API

mpGetCartPosEx

Member: *<sRobotNo>*
Control Group

Value	Description
0	R1 (Robot 1)
1	R2 (Robot 2)
2	R3 (Robot 3)
3	R4 (Robot 4)
4	R5 (Robot 5)
5	R6 (Robot 6)
6	R7 (Robot 7)
7	R8 (Robot 8)

Member: *<sFrameNo>*
Specified coordinate frame

Value	Description
0	Base coordinate
1	Robot coordinate
2,3, ..., 64	User coordinate number 1, 2,..., 63

Member: *<sToolNo>*
Tool number(0 - 63)

Added feature (This member can be used when the system software and MotoPlusIDE are specific versions¹⁾.)

Tool number (0 to 63) or CURRENT_TOOL (value: -1)

EXPLANATION:

If the CURRENT_TOOL symbolic constant (value: -1) is specified, the position in Cartesian coordinates of the tool with the current tool number is returned. Other values return the position in Cartesian coordinates in the tool file specified by the number. When using a system version or MotoPlusIDE version before this feature was added, specifying CURRENT_TOOL (value: -1) causes the API to return ERROR (value: -1): Error.

¹ For the YRC1000:
System version YAS4.40-00 or later and MotoPlusIDE version 2.41 or later
For the YRC1000micro:
System version YBS2.54-00 or later and MotoPlusIDE version 2.41 or later

[rData]

[out] The pointer of the Cartesian coordinate position receive data structure

- MP_CART_POS_RSP_DATA_EX

[out] The Cartesian coordinate position receive data structure

Syntax: #define MAX_CART_AXES_EX (12)

```
typedef struct
{
    LONG      IPos[MAX_CART_AXES_EX];
    SHORT     sConfig;
    CHAR      reserved[2];
} MP_CART_POS_RSP_DATA_EX;

(When the system software and MotoPlusIDE are specific
versions1.)
The sCurToolNo member was added to the
MP_CART_POS_RSP_DATA_EX structure.

typedef struct
{
    LONG      IPos[MAX_CART_AXES_EX];
    SHORT     sConfig;
    SHORT     sCurToolNo; /* If sToolNo in sData is
                           CURRENT_TOOL (value: -1),
                           the current tool number is
                           returned, otherwise "0" is
                           returned. */
} MP_CART_POS_RSP_DATA_EX;
```

Member: <IPos[MAX_CART_AXES_EX]>
Cartesian coordinate position (Maximum is 12)

Array	Description
IPos[0]	X-axis coordinate (unit: micron)
IPos[1]	Y-axis coordinate (unit: micron)
IPos[2]	Z-axis coordinate (unit: micron)
IPos[3]	Wrist angle Rx (unit: 0.0001 deg)
IPos[4]	Wrist angle Ry (unit: 0.0001 deg)
IPos[5]	Wrist angle Rz (unit: 0.0001 deg)
IPos[6]	Elbow angle E (unit: 0.0001 deg)
IPos[7]	Reserved
IPos[8]	1st external axis pulse value (micron in the case of traveling axis)
IPos[9]	2nd external axis pulse value (micron in the case of traveling axis)
IPos[10]	3rd external axis pulse value (micron in the case of traveling axis)
IPos[11]	Not in use

1 For the YRC1000:
System version YAS4.40-00 or later and MotoPlusIDE version 2.41 or later
For the YRC1000micro:
System version YBS2.54-00 or later and MotoPlusIDE version 2.41 or later

9 System Monitor API

mpGetCartPosEx

<sConfig>

Relative position information

Attribute	Description
D00	0: Front 1: Back
D01	0: Upper arm 1: Lower arm
D02	0: Flip 1: No flip
D03	0: R < 180 1: R >= 180
D04	0: T < 180 1: T >= 180
D05	0: S < 180 1: S >= 180
D06	0: L < 0 1: L >= 0
D07 - D15	Reserved by manufacturer

<sCurToolNo>

If sToolNo in sData is CURRENT_TOOL (value: -1), the current tool number is returned.

If sToolNo in sData is 0 to 63, "0" is returned.

For more information on the position data, refer to chapter 2 "Manipulator Coordinate Systems and Operations" in the operator's manual of the controller¹⁾.

■ Return Value

Symbolic constant: OK (value: 0) : Normal end

Symbolic constant: ERROR (value: -1) : Error



If an undefined user coordinate system is specified, the following error is displayed on the programming pendant.

Error 300 "Undefined user frame"

¹ For the YRC1000:
YRC1000 GENERAL OPERATOR'S MANUAL (RE-CSO-A051)
For the YRC1000micro:
YRC1000micro OPERATOR'S MANUAL (RE-CSO-A058)

mpGetPulsePos

Retrieves the current position in pulse counts.

■ **Syntax**

```
LONG mpGetPulsePos (  
    MP_CTRL_GRP_SEND_DATA*      sData,  
    MP_PULSE_POS_RSP_DATA*      rData  
);
```

■ **Description**

Retrieves the current position in pulse counts.

The information retrieved is set in the corresponding member in parameter structure as a value.

■ **Parameter**

[sData]

[in] The pointer to the data structure which specifies the control group

- MP_CTRL_GRP_SEND_DATA

[in] The data structure which specifies the control group

```
Syntax:  typedef struct  
        {  
            CTRLG_T  sCtrlGrp;  
        } MP_CTRL_GRP_SEND_DATA;
```

Member: <sCtrlGrp>
Control group

sCtrlGrp value	Description
0	R1 (Robot 1)
1	R2 (Robot 2)
2	R3 (Robot 3)
3	R4 (Robot 4)
4	R5 (Robot 5)
5	R6 (Robot 6)
6	R7 (Robot 7)
7	R8 (Robot 8)
8	B1 (Base 1)
9	B2 (Base 2)
10	B3 (Base 3)
11	B4 (Base 4)
12	B5 (Base 5)
13	B6 (Base 6)
14	B7 (Base 7)
15	B8 (Base 8)
16	S1 (Station 1)
17	S2 (Station 2)

9 System Monitor API

mpGetPulsePos

sCtrlGrp value	Description
18	S3 (Station 3)
19	S4 (Station 4)
20	S5 (Station 5)
21	S6 (Station 6)
22	S7 (Station 7)
23	S8 (Station 8)
24	S9 (Station 9)
25	S10 (Station 10)
26	S11 (Station 11)
27	S12 (Station 12)
28	S13 (Station 13)
29	S14 (Station 14)
30	S15 (Station 15)
31	S16 (Station 16)
32	S17 (Station 17)
33	S18 (Station 18)
34	S19 (Station 19)
35	S20 (Station 20)
36	S21 (Station 21)
37	S22 (Station 22)
38	S23 (Station 23)
39	S24 (Station 24)

[rData]

[out] The pointer to the data structure which receives the position coordinates in pulse

- MP_PULSE_POS_RSP_DATA

[out] The data structure which receives the position coordinates in pulse

Syntax: #define MAX_PULSE_AXES (8)

```
typedef struct
{
    LONG      IPos[MAX_PULSE_AXES];
} MP_PULSE_POS_RSP_DATA;
```

Member: <IPos[MAX_PULSE_AXES]>

Position coordinates in pulse (up to 8 arrays)

Array	Description
IPos[0]	1st axis (S) pulse value
IPos[1]	2nd axis (L) pulse value
IPos[2]	3rd axis (U) pulse value
IPos[3]	4th axis (R) pulse value
IPos[4]	5th axis (B) pulse value
IPos[5]	6th axis (T) pulse value
IPos[6]	7th axis (E) pulse value
IPos[7]	8th axis pulse value

9 System Monitor API

mpGetPulsePos

■ Return Value

Symbolic constant: OK (value: 0) : Normal end

Symbolic constant: ERROR (value: -1) : Error

mpGetRadPos

Retrieves the current position in radians.

■ Syntax

```
LONG mpGetRadPos (
    MP_CTRL_GRP_SEND_DATA*    sData,
    MP_RAD_POS_RSP_DATA*      rData
);
```

■ Description

Retrieves the current position in radians.

With this API, when linear motion axes such as servo tracks are included, -1 (error) is returned. Use mpGetRadPosEx() when linear motion axes are included. The information retrieved is set in the corresponding member in parameter structure as a value.



For robot controllers up to the DX200, there are axes on which the direction of the angle can be different from the one of the pulses. For the YRC1000/YRC1000micro, however, the direction of the angle is always the same as the direction of the pulses. For this reason, an application that moves a robot using absolute angles may reverse the movement direction of the robot if the DX200 program code runs on the YRC1000/YRC1000micro unmodified. When porting DX200 code to the YRC1000/YRC1000micro, check the direction of robot movement and correct the code if necessary.

A measure is available to keep the source code for DX200 and earlier robot controllers and for the YRC1000/YRC1000micro the same. Set parameter S2C1350 in the YRC1000/YRC1000micro to 0 (default value = 1) to use the same specification as the DX200 and earlier robot controllers. Then, the positive direction of the angle and the positive direction of the pulses for each axis become the same as those for the DX200.

■ Parameter

[*sData*]

[in] The pointer to the data structure which specifies the control group

- MP_CTRL_GRP_SEND_DATA

[in] The data structure which specifies the control group

```
Syntax:  typedef struct
          {
            CTRLG_T    sCtrlGrp;
          } MP_CTRL_GRP_SEND_DATA;
```

9 System Monitor API

mpGetRadPos

Member: <sCtrlGrp>
Control group

sCtrlGrp value	Description
0	R1 (Robot 1)
1	R2 (Robot 2)
2	R3 (Robot 3)
3	R4 (Robot 4)
4	R5 (Robot 5)
5	R6 (Robot 6)
6	R7 (Robot 7)
7	R8 (Robot 8)
8	B1 (Base 1)
9	B2 (Base 2)
10	B3 (Base 3)
11	B4 (Base 4)
12	B5 (Base 5)
13	B6 (Base 6)
14	B7 (Base 7)
15	B8 (Base 8)
16	S1 (Station 1)
17	S2 (Station 2)
18	S3 (Station 3)
19	S4 (Station 4)
20	S5 (Station 5)
21	S6 (Station 6)
22	S7 (Station 7)
23	S8 (Station 8)
24	S9 (Station 9)
25	S10 (Station 10)
26	S11 (Station 11)
27	S12 (Station 12)
28	S13 (Station 13)
29	S14 (Station 14)
30	S15 (Station 15)
31	S16 (Station 16)
32	S17 (Station 17)
33	S18 (Station 18)
34	S19 (Station 19)
35	S20 (Station 20)
36	S21 (Station 21)
37	S22 (Station 22)
38	S23 (Station 23)
39	S24 (Station 24)

9 System Monitor API

mpGetRadPos

[rData]

[out] The pointer to the data structure which receives the position coordinates in radians

- MP_PULSE_RAD_RSP_DATA

[out] The data structure which receives the position coordinates in radians

Syntax: #define MAX_PULSE_AXES (8)

```
typedef struct
{
    LONG      IRadPos[MAX_PULSE_AXES];
} MP_RAD_POS_RSP_DATA;
```

Member: <IRadPos[MAX_PULSE_AXES]>

Position coordinates in radians (up to 8 arrays)

Array	Description
IRadPos[0]	1st axis (S) rad value
IRadPos[1]	2nd axis (L) rad value
IRadPos[2]	3rd axis (U) rad value
IRadPos[3]	4th axis (R) rad value
IRadPos[4]	5th axis (B) rad value
IRadPos[5]	6th axis (T) rad value
IRadPos[6]	7th axis (E) rad value
IRadPos[7]	8th axis rad value

■ Return Value

Symbolic constant: OK (value: 0) : Normal end

Symbolic constant: ERROR (value: -1) : Error

mpGetRadPosEx

Retrieves the current position in radians.

■ Syntax

```
LONG mpGetRadPosEX (
    MP_CTRL_GRP_SEND_DATA*    sData,
    MP_RAD_POS_RSP_DATA_EX*   rData
);
```

■ Description

Retrieves the current position in radians. The information retrieved is set in the corresponding member in parameter structure as a value.

Linear motion axes such as servo tracks are retrieved not in radians, but in position (unit: 0.001 mm).



For robot controllers up to the DX200, there are axes on which the direction of the angle can be different from the one of the pulses. For the YRC1000/YRC1000micro, however, the direction of the angle is always the same as the direction of the pulses. For this reason, an application that moves a robot using absolute angles may reverse the movement direction of the robot if the DX200 program code runs on the YRC1000/YRC1000micro unmodified. When porting DX200 code to the YRC1000/YRC1000micro, check the direction of robot movement and correct the code if necessary.

A measure is available to keep the source code for DX200 and earlier robot controllers and for the YRC1000/YRC1000micro the same. Set parameter S2C1350 in the YRC1000/YRC1000micro to 0 (default value = 1) to use the same specification as the DX200 and earlier robot controllers. Then, the positive direction of the angle and the positive direction of the pulses for each axis become the same as those for the DX200.

■ Parameter

[sData]

[in] The pointer to the data structure which specifies the control group

- MP_CTRL_GRP_SEND_DATA

[in] The data structure which specifies the control group

```
Syntax:  typedef struct
          {
            CTRLG_T   sCtrlGrp;
          } MP_CTRL_GRP_SEND_DATA;
```

9 System Monitor API

mpGetRadPosEx

Member: <sCtrlGrp>
Control group

sCtrlGrp value	Description
0	R1 (Robot 1)
1	R2 (Robot 2)
2	R3 (Robot 3)
3	R4 (Robot 4)
4	R5 (Robot 5)
5	R6 (Robot 6)
6	R7 (Robot 7)
7	R8 (Robot 8)
8	B1 (Base 1)
9	B2 (Base 2)
10	B3 (Base 3)
11	B4 (Base 4)
12	B5 (Base 5)
13	B6 (Base 6)
14	B7 (Base 7)
15	B8 (Base 8)
16	S1 (Station 1)
17	S2 (Station 2)
18	S3 (Station 3)
19	S4 (Station 4)
20	S5 (Station 5)
21	S6 (Station 6)
22	S7 (Station 7)
23	S8 (Station 8)
24	S9 (Station 9)
25	S10 (Station 10)
26	S11 (Station 11)
27	S12 (Station 12)
28	S13 (Station 13)
29	S14 (Station 14)
30	S15 (Station 15)
31	S16 (Station 16)
32	S17 (Station 17)
33	S18 (Station 18)
34	S19 (Station 19)
35	S20 (Station 20)
36	S21 (Station 21)
37	S22 (Station 22)
38	S23 (Station 23)
39	S24 (Station 24)

9 System Monitor API
mpGetRadPosEx

[rData]

[out] The pointer to the data structure (expanded) which receives the position coordinates in radians

- MP_RAD_POS_RSP_DATA_EX

[out] The data structure (expanded) which receives the position coordinates in radians

```
Syntax:  #define MAX_PULSE_AXES (8)
         typedef struct
         {
             LONG      IRadPos[MAX_PULSE_AXES];
             LONG      IRadUnit[MAX_PULSE_AXES];
         } MP_RAD_POS_RSP_DATA_EX;
```

Member: <IRadPos[MAX_PULSE_AXES]>
Position coordinates in radians (up to 8 arrays)

Array	Description
IRadPos[0]	1st axis (S) rad or mm value
IRadPos[1]	2nd axis (L) rad or mm value
IRadPos[2]	3rd axis (U) rad or mm value
IRadPos[3]	4th axis (R) rad or mm value
IRadPos[4]	5th axis (B) rad or mm value
IRadPos[5]	6th axis (T) rad or mm value
IRadPos[6]	7th axis (E) rad or mm value
IRadPos[7]	8th axis rad or mm value

<IRadUnit[MAX_PULSE_AXES]>
Units (up to 8 arrays)
MP_POS_UNIT_DEGREE 1 Degree (0.0001 deg)
MP_POS_UNIT_DISTANCE 2 Distance (0.001 mm)
MP_POS_UNIT_RADIAN 3 Radian (0.0001 rad)

■ Return Value

Symbolic constant: OK (value: 0) : Normal end
Symbolic constant: ERROR (value: -1) : Error

mpGetDegPos

Retrieves the current position in degrees.

■ Syntax

```
LONG mpGetDegPos (
    MP_CTRL_GRP_SEND_DATA*    sData,
    MP_DEG_POS_RSP_DATA*      rData
);
```

■ Description

Retrieves the current position in degrees. The information retrieved is set in the corresponding member in parameter structure as a value.

When linear motion axes such as servo tracks are included, -1 (error) is returned.

Use mpGetDegPosEx() when linear motion axes are included.



For robot controllers up to the DX200, there are axes on which the direction of the angle can be different from the one of the pulses. For the YRC1000/YRC1000micro, however, the direction of the angle is always the same as the direction of always the pulses. For this reason, an application that moves a robot using absolute angles may reverse the movement direction of the robot if the DX200 program code runs on the YRC1000/YRC1000micro unmodified. When porting DX200 code to the YRC1000/YRC1000micro, check the direction of robot movement and correct the code if necessary.

A measure is available to keep the source code for DX200 and earlier robot controllers and for the YRC1000/YRC1000micro the same. Set parameter S2C1350 in the YRC1000/YRC1000micro to 0 (default value = 1) to use the same specification as the DX200 and earlier robot controllers. Then, the positive direction of the angle and the positive direction of the pulses for each axis become the same as those for the DX200.

■ Parameter

[*sData*]

[in] The pointer to the data structure which specifies the control group

- MP_CTRL_GRP_SEND_DATA

[in] The data structure which specifies the control group

Syntax: typedef struct
 {
 CTRLG_T *sCtrlGrp*;
 } MP_CTRL_GRP_SEND_DATA;

9 System Monitor API

mpGetDegPos

Member: <sCtrlGrp>
Control group

sCtrlGrp value	Description
0	R1 (Robot 1)
1	R2 (Robot 2)
2	R3 (Robot 3)
3	R4 (Robot 4)
4	R5 (Robot 5)
5	R6 (Robot 6)
6	R7 (Robot 7)
7	R8 (Robot 8)
8	B1 (Base 1)
9	B2 (Base 2)
10	B3 (Base 3)
11	B4 (Base 4)
12	B5 (Base 5)
13	B6 (Base 6)
14	B7 (Base 7)
15	B8 (Base 8)
16	S1 (Station 1)
17	S2 (Station 2)
18	S3 (Station 3)
19	S4 (Station 4)
20	S5 (Station 5)
21	S6 (Station 6)
22	S7 (Station 7)
23	S8 (Station 8)
24	S9 (Station 9)
25	S10 (Station 10)
26	S11 (Station 11)
27	S12 (Station 12)
28	S13 (Station 13)
29	S14 (Station 14)
30	S15 (Station 15)
31	S16 (Station 16)
32	S17 (Station 17)
33	S18 (Station 18)
34	S19 (Station 19)
35	S20 (Station 20)
36	S21 (Station 21)
37	S22 (Station 22)
38	S23 (Station 23)
39	S24 (Station 24)

9 System Monitor API

mpGetDegPos

[rData]

[out] The pointer to the data structure which receives the position coordinates in degrees

- MP_DEG_POS_RSP_DATA

[out] The data structure which receives the position coordinates in degrees

Syntax: #define MAX_PULSE_AXES (8)

```
typedef struct
{
    LONG      IDegPos[MAX_PULSE_AXES];
} MP_DEG_POS_RSP_DATA;
```

Member: <IDegPos[MAX_PULSE_AXES]>
Position coordinates in degrees (up to 8 arrays)

Array	Description
IDegPos[0]	1st axis (S) deg value
IDegPos[1]	2nd axis (L) deg value
IDegPos[2]	3rd axis (U) deg value
IDegPos[3]	4th axis (R) deg value
IDegPos[4]	5th axis (B) deg value
IDegPos[5]	6th axis (T) deg value
IDegPos[6]	7th axis (E) deg value
IDegPos[7]	8th axis deg value

■ Return Value

Symbolic constant: OK (value: 0) : Normal end

Symbolic constant: ERROR (value: -1) : Error

mpGetDegPosEx

Retrieves the current position in degrees.

■ Syntax

```
LONG mpGetDegPosEX (
    MP_CTRL_GRP_SEND_DATA*    sData,
    MP_DEG_POS_RSP_DATA_EX*   rData
);
```

■ Description

Retrieves the current position in degrees. The information retrieved is set in the corresponding member in parameter structure as a value.

When linear motion axes such as servo tracks are obtained, they are retrieved not in angles (unit: 0.0001 deg), but in position (unit: 0.001 mm).



For robot controllers up to the DX200, there are axes on which the direction of the angle can be different from the one of the pulses. For the YRC1000/YRC1000micro, however, the direction of the angle is always the same as the direction of the pulses. For this reason, an application that moves a robot using absolute angles may reverse the movement direction of the robot if the DX200 program code runs on the YRC1000/YRC1000micro unmodified. When porting DX200 code to the YRC1000/YRC1000micro, check the direction of robot movement and correct the code if necessary.

A measure is available to keep the source code for DX200 and earlier robot controllers and for the YRC1000/YRC1000micro the same. Set parameter S2C1350 in the YRC1000/YRC1000micro to 0 (default value = 1) to use the same specification as the DX200 and earlier robot controllers. Then, the positive direction of the angle and the positive direction of the pulses for each axis become the same as those for the DX200.

■ Parameter

[sData]

[in] The pointer to the data structure which specifies the control group

- MP_CTRL_GRP_SEND_DATA

[in] The data structure which specifies the control group

```
Syntax:  typedef struct
          {
            CTRLG_T    sCtrlGrp;
          } MP_CTRL_GRP_SEND_DATA;
```

9 System Monitor API

mpGetDegPosEx

Member: <sCtrlGrp>
Control group

sCtrlGrp value	Description
0	R1 (Robot 1)
1	R2 (Robot 2)
2	R3 (Robot 3)
3	R4 (Robot 4)
4	R5 (Robot 5)
5	R6 (Robot 6)
6	R7 (Robot 7)
7	R8 (Robot 8)
8	B1 (Base 1)
9	B2 (Base 2)
10	B3 (Base 3)
11	B4 (Base 4)
12	B5 (Base 5)
13	B6 (Base 6)
14	B7 (Base 7)
15	B8 (Base 8)
16	S1 (Station 1)
17	S2 (Station 2)
18	S3 (Station 3)
19	S4 (Station 4)
20	S5 (Station 5)
21	S6 (Station 6)
22	S7 (Station 7)
23	S8 (Station 8)
24	S9 (Station 9)
25	S10 (Station 10)
26	S11 (Station 11)
27	S12 (Station 12)
28	S13 (Station 13)
29	S14 (Station 14)
30	S15 (Station 15)
31	S16 (Station 16)
32	S17 (Station 17)
33	S18 (Station 18)
34	S19 (Station 19)
35	S20 (Station 20)
36	S21 (Station 21)
37	S22 (Station 22)
38	S23 (Station 23)
39	S24 (Station 24)

9 System Monitor API
mpGetDegPosEx

[rData]

[out] The pointer to the data structure (expanded) which receives the position coordinates in degrees

- MP_DEG_POS_RSP_DATA_EX

[out] The data structure (expanded) which receives the position coordinates in degrees

```
Syntax:  #define MAX_PULSE_AXES (8)
          typedef struct
          {
              LONG      IDegPos[MAX_PULSE_AXES];
              LONG      IDegUnit[MAX_PULSE_AXES];
          } MP_DEG_POS_RSP_DATA_EX;
```

Member: <IDegPos[MAX_PULSE_AXES]>
 Position coordinates in degrees (up to 8 arrays)

Array	Description
IDegPos[0]	1st axis (S) deg or mm value
IDegPos[1]	2nd axis (L) deg or mm value
IDegPos[2]	3rd axis (U) deg or mm value
IDegPos[3]	4th axis (R) deg or mm value
IDegPos[4]	5th axis (B) deg or mm value
IDegPos[5]	6th axis (T) deg or mm value
IDegPos[6]	7th axis (E) deg or mm value
IDegPos[7]	8th axis deg or mm value

Member: <IDegUnit[MAX_PULSE_AXES]>
 Position coordinates in degrees (up to 8 arrays)

MP_POS_UNIT_DEGREE	1	Degree (0.0001 deg)
MP_POS_UNIT_DISTANCE	2	Distance (0.001 mm)
MP_POS_UNIT_RADIAN	3	Radian (0.0001 rad)

■ Return Value

Symbolic constant: OK (value: 0) : Normal end
Symbolic constant: ERROR (value: -1) : Error

mpGetFBPulsePos

Retrieves the feedback position in pulse counts.

■ Syntax

```
LONG mpGetFBPulsePos (
    MP_CTRL_GRP_SEND_DATA*    sData,
    MP_FB_PULSE_POS_RSP_DATA* rData
);
```

■ Description

Retrieves the feedback pulses of the motor axis.

The reference pulses that can be retrieved with mpGetPulsePos() are the reference pulses for the load axis.

To retrieve the load axis feedback pulses, use mpConvFBPulseToPulse() to convert the motor axis feedback pulses retrieved with this API or use mpGetFBPulsePosEx().



Depending on the mechanisms of the robot, the motor axis feedback pulses and the load axis feedback pulses may not match.

For example, when the controller moves only B-axis, the T-axis motor may also move a small amount together with the B-axis motor depending on the mechanisms.

In this case, the motor axes for the B-axis and T-axis both change, but only the load axis for the B-axis changes and not the T-axis.

To confirm arrival at the target pulse position that was given with MOVx instructions in the JOB or with mpPulseMOVx instructions, use one of the following methods to retrieve the load axis feedback pulses.

- Use mpConvFBPulseToPulse() to convert the motor axis feedback pulses to load axis feedback pulses.
- Use mpGetFBPulsePosEx() with the data type set to load axis.

■ Parameter

[sData]

[in] The pointer to the data structure which specifies the control group

- MP_CTRL_GRP_SEND_DATA

[in] The data structure which specifies the control group

```
Syntax: typedef struct
{
    CTRLG_T  sCtrlGrp;
} MP_CTRL_GRP_SEND_DATA;
```

9 System Monitor API

mpGetFBPulsePos

Member: <sCtrlGrp>
Control group

sCtrlGrp value	Description
0	R1 (Robot 1)
1	R2 (Robot 2)
2	R3 (Robot 3)
3	R4 (Robot 4)
4	R5 (Robot 5)
5	R6 (Robot 6)
6	R7 (Robot 7)
7	R8 (Robot 8)
8	B1 (Base 1)
9	B2 (Base 2)
10	B3 (Base 3)
11	B4 (Base 4)
12	B5 (Base 5)
13	B6 (Base 6)
14	B7 (Base 7)
15	B8 (Base 8)
16	S1 (Station 1)
17	S2 (Station 2)
18	S3 (Station 3)
19	S4 (Station 4)
20	S5 (Station 5)
21	S6 (Station 6)
22	S7 (Station 7)
23	S8 (Station 8)
24	S9 (Station 9)
25	S10 (Station 10)
26	S11 (Station 11)
27	S12 (Station 12)
28	S13 (Station 13)
29	S14 (Station 14)
30	S15 (Station 15)
31	S16 (Station 16)
32	S17 (Station 17)
33	S18 (Station 18)
34	S19 (Station 19)
35	S20 (Station 20)
36	S21 (Station 21)
37	S22 (Station 22)
38	S23 (Station 23)
39	S24 (Station 24)

9 System Monitor API
mpGetFBPulsePos

[rData]

[out] The pointer to the data structure which receives the feedback position coordinates in pulse

- MP_FB_PULSE_POS_RSP_DATA

[out] The data structure which receives the feedback position coordinates in pulse

Syntax: #define MAX_PULSE_AXES (8)

```
typedef struct
{
    LONG      IPos[MAX_PULSE_AXES];
} MP_FB_PULSE_POS_RSP_DATA;
```

Member: <IPos[MAX_PULSE_AXES]>
Position coordinates in pulse (up to 8 arrays)

Array	Description
IPos[0]	1st axis (S) pulse value
IPos[1]	2nd axis (L) pulse value
IPos[2]	3rd axis (U) pulse value
IPos[3]	4th axis (R) pulse value
IPos[4]	5th axis (B) pulse value
IPos[5]	6th axis (T) pulse value
IPos[6]	7th axis (E) pulse value
IPos[7]	8th axis pulse value

■ Return Value

Symbolic constant: OK (value: 0) : Normal end
Symbolic constant: ERROR (value: -1) : Error

mpGetFBPulsePosEx

Retrieves the feedback position (the motor-axis, the load-axis) in pulse counts.

■ Syntax

```
LONG mpGetFBPulsePosEx(
    MP_FB_PULSE_POS_EX_SEND_DATA*   sData,
    /*the data structure which specifies the control group (FB expansion)*/
    MP_FB_PULSE_POS_RSP_DATA*       rData
    /* the data structure which receives the feedback position coordinates in pulse */
);
```

■ Description

Retrieves the feedback pulses of the load axis and motor axis.

The feedback pulses to the load axis can be retrieved by specifying the load axis and calling mpGetFBPulsePosEx(). The reference pulses to the load axis can be retrieved with mpGetPulsePos().

The feedback pulses for the motor axis can be retrieved by specifying the motor axis and calling mpGetFBPulsePosEx(), but this may differ from the pulses when the load axis is specified. If this API is called by specifying the motor axis, the same value is retrieved as when calling mpGetFBPulsePos().

Depending on the mechanisms of the robot, the motor axis feedback pulses and the load axis feedback pulses may not match.

For example, when the controller moves only B-axis, the T-axis motor may also move a small amount together with the B-axis motor depending on the mechanisms.

In this case, the motor axes for the B-axis and T-axis both change, but only the load axis for the B-axis changes and not the T-axis.



To confirm arrival at the target pulse position that was given with MOVx instructions in the JOB or with mpPulseMOVx instructions, use one of the following methods to retrieve the load axis feedback pulses.

- Use mpConvFBPulseToPulse() to convert the motor axis feedback pulses to load axis feedback pulses.
- Use mpGetFBPulsePosEx() with the data type set to load axis.

9 System Monitor API

mpGetFBPulsePosEx

■ Parameter

[sData]

[in] The pointer to the data structure which specifies the control group (FB expansion)

- MP_FB_PULSE_POS_EX_SEND_DATA

Syntax:

```
Typedef struct{
ULONG      grp_kind;    /* control group type (ROBOT/BASE/STATION) */
ULONG      grp_no;      /* control group number */
UCHAR      type;        /* data type */
UCHAR      reserved[3];
} MP_FB_PULSE_POS_EX_SEND_DATA;
```

Member:<grp_kind>

control group type

0: ROBOT

1: BASE

2: STATION

<grp_no>

control group number (beginning with 0)

when the control group type is ROBOT(0): 0 to 7

when the control group type is BASE(1): 0 to 7

when the control group type is STATION(2): 0 to 23

<type>

data type

0: motor axis

1: load axis



"0: motor axis" is the pulse position of the motor and "1: load axis" is the position of the angle of each axis of the robot converted to motor pulses.

Depending on the mechanisms of the robot, the positions of "0: motor axis" and "1: load axis" may not match.

For example, when the robot moves only its B-axis, the T-axis motor may also move a small amount together with the B-axis motor depending on the mechanisms.

In this case, the B-axis and T-axis both change with "0: motor axis", but only the B-axis changes and not the T-axis with "1: load axis".

To confirm arrival at the target pulse position that was given with MOVx instructions in the job or with mpPulseMOVx functions, use "1: load axis".

[rData]

[out] The pointer to the data structure which receives the feedback position coordinates in pulse

- MP_FB_PULSE_POS_RSP_DATA

[out] The data structure which receives the feedback position coordinates in pulse

Syntax: #define MAX_PULSE_AXES(8)
typedef struct

```
{  
    LONG          IPos[MAX_PULSE_AXES];  
} MP_FB_PULSE_POS_RSP_DATA;
```

Member: <IPos[MAX_PULSE_AXES]>
Position coordinates in pulse (up to eight)

Value	Description
IPos[0]	1st axis (S) pulse value
IPos[1]	2nd axis (L) pulse value
IPos[2]	3rd axis (U) pulse value
IPos[3]	4th axis (R) pulse value
IPos[4]	5th axis (B) pulse value
IPos[5]	6th axis (T) pulse value
IPos[6]	7th axis (E) pulse value
IPos[7]	8th axis pulse value

■ Return Value

Symbolic constant: OK (value: 0) : Normal end

Symbolic constant: ERROR (value: -1) : Error

mpGetServoSpeed

Retrieves the current servo speed in pulse counts (unit: second).

■ Syntax

```
LONG mpGetServoSpeed (  
    MP_CTRL_GRP_SEND_DATA*    sData,  
    MP_SERVO_SPEED_RSP_DATA*  rData  
);
```

■ Description

Retrieves the current servo speed in pulse counts (unit: second).

The information retrieved is set in the corresponding member in parameter structure as a value.

■ Parameter

[*sData*]

[in] The pointer to the data structure which specifies the control group

- MP_CTRL_GRP_SEND_DATA

[in] The data structure which specifies the control group

```
Syntax:  typedef struct {  
          CTRLG_T    sCtrlGrp;  
        } MP_CTRL_GRP_SEND_DATA;
```

Member: <*sCtrlGrp*>
Control group

Value	Description
0	R1 (Robot 1)
1	R2 (Robot 2)
2	R3 (Robot 3)
3	R4 (Robot 4)
4	R5 (Robot 5)
5	R6 (Robot 6)
6	R7 (Robot 7)
7	R8 (Robot 8)
8	B1 (Base 1)
9	B2 (Base 2)
10	B3 (Base 3)
11	B4 (Base 4)
12	B5 (Base 5)
13	B6 (Base 6)
14	B7 (Base 7)
15	B8 (Base 8)
16	S1 (Station 1)
17	S2 (Station 2)

9 System Monitor API

mpGetServoSpeed

Value	Description
18	S3 (Station 3)
19	S4 (Station 4)
20	S5 (Station 5)
21	S6 (Station 6)
22	S7 (Station 7)
23	S8 (Station 8)
24	S9 (Station 9)
25	S10 (Station 10)
26	S11 (Station 11)
27	S12 (Station 12)
28	S13 (Station 13)
29	S14 (Station 14)
30	S15 (Station 15)
31	S16 (Station 16)
32	S17 (Station 17)
33	S18 (Station 18)
34	S19 (Station 19)
35	S20 (Station 20)
36	S21 (Station 21)
37	S22 (Station 22)
38	S23 (Station 23)
39	S24 (Station 24)

[rData]

[out] The pointer to the data structure which receives the servo speed

- MP_SERVO_SPEED_RSP_DATA

[out] The data structure which receives the servo speed

Syntax: #define MAX_PULSE_AXES (8)

```
typedef struct
LONG    ISpeed[MAX_PULSE_AXES];
} MP_SERVO_SPEED_RSP_DATA;
```

Member: <ISpeed[MAX_PULSE_AXES]>
Speed (up to 8 arrays)

Array	Description
ISpeed[0]	In 1st axis (S) direction (unit: pulses per second - pps)
ISpeed[1]	In 2nd axis (L) direction (unit: pps)
ISpeed[2]	In 3rd axis (U) direction (unit: pps)
ISpeed[3]	In 4th axis (R) direction (unit: pps)
ISpeed[4]	In 5th axis (B) direction (unit: pps)
ISpeed[5]	In 6th axis (T) direction (unit: pps)
ISpeed[6]	In 7th axis (E) direction (unit: pps)
ISpeed[7]	In 8th axis direction (unit: pps)

9 System Monitor API

mpGetServoSpeed

■ Return Value

Symbolic constant: OK (value: 0) : Normal end

Symbolic constant: ERROR (value: -1) : Error

mpGetFBSpeed

Retrieves the feedback speed in pulse counts (unit: second).

■ **Syntax**

```
LONG mpGetFBSpeed (
    MP_CTRL_GRP_SEND_DATA*    sData,
    MP_FB_SPEED_RSP_DATA*    rData
);
```

■ **Description**

mpGetServoSpeed() retrieves the current commanded speed, and mpGetFBSpeed() retrieves the current actual speed of the manipulator (feedback value). The information retrieved is set in the corresponding member in parameter structure as a value.

■ **Parameter**

[sData]

[in] The pointer to the data structure which specifies the control group

- MP_CTRL_GRP_SEND_DATA

[in] The data structure which specifies the control group

```
Syntax:  typedef struct
          {
              CTRLG_T    sCtrlGrp;
          } MP_CTRL_GRP_SEND_DATA;
```

Member: <sCtrlGrp>
Control group

sCtrlGrp value	Description
0	R1 (Robot 1)
1	R2 (Robot 2)
2	R3 (Robot 3)
3	R4 (Robot 4)
4	R5 (Robot 5)
5	R6 (Robot 6)
6	R7 (Robot 7)
7	R8 (Robot 8)
8	B1 (Base 1)
9	B2 (Base 2)
10	B3 (Base 3)
11	B4 (Base 4)
12	B5 (Base 5)
13	B6 (Base 6)
14	B7 (Base 7)
15	B8 (Base 8)
16	S1 (Station 1)

9 System Monitor API

mpGetFBSpeed

sCtrlGrp value	Description
17	S2 (Station 2)
18	S3 (Station 3)
19	S4 (Station 4)
20	S5 (Station 5)
21	S6 (Station 6)
22	S7 (Station 7)
23	S8 (Station 8)
24	S9 (Station 9)
25	S10 (Station 10)
26	S11 (Station 11)
27	S12 (Station 12)
28	S13 (Station 13)
29	S14 (Station 14)
30	S15 (Station 15)
31	S16 (Station 16)
32	S17 (Station 17)
33	S18 (Station 18)
34	S19 (Station 19)
35	S20 (Station 20)
36	S21 (Station 21)
37	S22 (Station 22)
38	S23 (Station 23)
39	S24 (Station 24)

[rData]

[out] The pointer to the data structure which receives the feedback speed

• MP_FB_SPEED_RSP_DATA

[out] The data structure which receives the feedback speed

Syntax: #define MAX_PULSE_AXES (8)

```
typedef struct
{
    LONG    ISpeed[MAX_PULSE_AXES];
} MP_FB_SPEED_RSP_DATA;
```

Member: <ISpeed[MAX_PULSE_AXES]>
Speed (up to 8 arrays)

Array	Description
ISpeed[0]	In 1st axis (S) direction (unit: pulses per second - pps)
ISpeed[1]	In 2nd axis (L) direction (unit: pps)
ISpeed[2]	In 3rd axis (U) direction (unit: pps)
ISpeed[3]	In 4th axis (R) direction (unit: pps)
ISpeed[4]	In 5th axis (B) direction (unit: pps)
ISpeed[5]	In 6th axis (T) direction (unit: pps)
ISpeed[6]	In 7th axis (E) direction (unit: pps)
ISpeed[7]	In 8th axis direction (unit: pps)

9 System Monitor API
mpGetFBSpeed

■ **Return Value**

Symbolic constant: OK (value: 0) : Normal end

Symbolic constant: ERROR (value: -1) : Error

mpGetTorque

Retrieves the motor torque value as a percentage of the motor rating.



Using mpSvsGetVelTrqFb() is recommended as the API to retrieve the motor torque value. mpSvsGetVelTrqFb() can retrieve the torque value with a faster torque update cycle and at a finer resolution.

■ Syntax

```
LONG mpGetTorque (
    MP_CTRL_GRP_SEND_DATA*    sData,
    MP_TORQUE_RSP_DATA*       rData
);
```

■ Description

Retrieves the motor torque value as a percentage of the motor rating.

Unit: %

The information retrieved is set in the corresponding member in parameter structure as a value.

■ Parameters

[sData]

[in] The pointer to the data structure which specifies the control group

- MP_CTRL_GRP_SEND_DATA

[in] The data structure which specifies the control group

```
Syntax:  typedef struct
          {
            CTRLG_T  sCtrlGrp;
          } MP_CTRL_GRP_SEND_DATA;
```

Member: <sCtrlGrp>
Control group

sCtrlGrp value	Description
0	R1 (Robot 1)
1	R2 (Robot 2)
2	R3 (Robot 3)
3	R4 (Robot 4)
4	R5 (Robot 5)
5	R6 (Robot 6)
6	R7 (Robot 7)
7	R8 (Robot 8)
8	B1 (Base 1)
9	B2 (Base 2)
10	B3 (Base 3)
11	B4 (Base 4)
12	B5 (Base 5)
13	B6 (Base 6)

9 System Monitor API

mpGetTorque

sCtrlGrp value	Description
14	B7 (Base 7)
15	B8 (Base 8)
16	S1 (Station 1)
17	S2 (Station 2)
18	S3 (Station 3)
19	S4 (Station 4)
20	S5 (Station 5)
21	S6 (Station 6)
22	S7 (Station 7)
23	S8 (Station 8)
24	S9 (Station 9)
25	S10 (Station 10)
26	S11 (Station 11)
27	S12 (Station 12)
28	S13 (Station 13)
29	S14 (Station 14)
30	S15 (Station 15)
31	S16 (Station 16)
32	S17 (Station 17)
33	S18 (Station 18)
34	S19 (Station 19)
35	S20 (Station 20)
36	S21 (Station 21)
37	S22 (Station 22)
38	S23 (Station 23)
39	S24 (Station 24)

[rData]

[out] The pointer to the data structure which receives the torque value

- MP_TORQUE_RSP_DATA

[out] The data structure which receives torque the value

Syntax: #define MAX_PULSE_AXES (8)

```
typedef struct
{
    LONG    lTorquePcnt[MAX_PULSE_AXES];
} MP_TORQUE_RSP_DATA;
```

Member: <lTorquePcnt[MAX_PULSE_AXES]>
Torque percentage (up to 8 arrays)

Array	Description
lTorquePcnt[0]	In 1st axis (S) direction (unit: 1% of max. torque)
lTorquePcnt[1]	In 2nd axis (L) direction (unit: 1% of max. torque)
lTorquePcnt[2]	In 3rd axis (U) direction (unit: 1% of max. torque)

9 System Monitor API

mpGetTorque

Array	Description
lTorquePcnt[3]	In 4th axis (R) direction (unit: 1% of max. torque)
lTorquePcnt[4]	In 5th axis (B) direction (unit: 1% of max. torque)
lTorquePcnt[5]	In 6th axis (T) direction (unit: 1% of max. torque)
lTorquePcnt[6]	In 7th axis (E) direction (unit: 1% of max. torque)
lTorquePcnt[7]	In 8th axis direction (unit: 1% of max. torque)

■ Return Value

Symbolic constant: OK (value: 0) : Normal end

Symbolic constant: ERROR (value: -1) : Error

mpGetTorqueEx

Retrieves the motor torque value.



Using mpSvsGetVelTrqFb() is recommended as the API to retrieve the motor torque value. mpSvsGetVelTrqFb() can retrieve the torque value with a faster torque update cycle and at a finer resolution.

■ **Syntax**

```
LONG mpGetTorqueEx (  
MP_CTRL_GRP_SEND_DATA*      sData,  
MP_TORQUE_EX_RSP_DATA*      rData  
);
```

■ **Description**

The torque value of the motor can be retrieved. The data unit is 0.0001 Nm.

The retrieved value is a signed value.

The information retrieved is set in the corresponding member in parameter structure as a value.

■ **Parameters**

[sData]

[in] The pointer to the data structure which specifies the control group

- MP_CTRL_GRP_SEND_DATA

[in] The data structure which specifies the control group

```
Syntax:  typedef struct  
        {  
          CTRLG_T  sCtrlGrp;  
        } MP_CTRL_GRP_SEND_DATA;
```

Member: <sCtrlGrp>
Control group

sCtrlGrp value	Description
0	R1 (Robot 1)
1	R2 (Robot 2)
2	R3 (Robot 3)
3	R4 (Robot 4)
4	R5 (Robot 5)
5	R6 (Robot 6)
6	R7 (Robot 7)
7	R8 (Robot 8)
8	B1 (Base 1)
9	B2 (Base 2)
10	B3 (Base 3)
11	B4 (Base 4)

9 System Monitor API

mpGetTorqueEx

sCtrlGrp value	Description
12	B5 (Base 5)
13	B6 (Base 6)
14	B7 (Base 7)
15	B8 (Base 8)
16	S1 (Station 1)
17	S2 (Station 2)
18	S3 (Station 3)
19	S4 (Station 4)
20	S5 (Station 5)
21	S6 (Station 6)
22	S7 (Station 7)
23	S8 (Station 8)
24	S9 (Station 9)
25	S10 (Station 10)
26	S11 (Station 11)
27	S12 (Station 12)
28	S13 (Station 13)
29	S14 (Station 14)
30	S15 (Station 15)
31	S16 (Station 16)
32	S17 (Station 17)
33	S18 (Station 18)
34	S19 (Station 19)
35	S20 (Station 20)
36	S21 (Station 21)
37	S22 (Station 22)
38	S23 (Station 23)
39	S24 (Station 24)

9 System Monitor API
mpGetTorqueEx

[rData]

[out] The pointer to the data structure which receives the torque value

- MP_TORQUE_EX_RSP_DATA

[out] The data structure which receives torque the value

Syntax: #define MAX_PULSE_AXES (8)

```
typedef struct
{
    LONG    lTorqueNm[MAX_PULSE_AXES];
} MP_TORQUE_EX_RSP_DATA;
```

Member: <lTorqueNm[MAX_PULSE_AXES]>
Torque (up to 8 arrays)

Array	Description
lTorqueNm[0]	In 1st axis (S) direction (unit: 0.0001 Nm)
lTorqueNm[1]	In 2nd axis (L) direction (unit: 0.0001 Nm)
lTorqueNm[2]	In 3rd axis (U) direction (unit: 0.0001 Nm)
lTorqueNm[3]	In 4th axis (R) direction (unit: 0.0001 Nm)
lTorqueNm[4]	In 5th axis (B) direction (unit: 0.0001 Nm)
lTorqueNm[5]	In 6th axis (T) direction (unit: 0.0001 Nm)
lTorqueNm[6]	In 7th axis (E) direction (unit: 0.0001 Nm)
lTorqueNm[7]	In 8th axis direction (unit: 0.0001 Nm)

■ Return Value

Symbolic constant: OK (value: 0) : Normal end

Symbolic constant: ERROR (value: -1) : Error

mpGetSysTimes

Retrieves the system time of the controller.

■ Syntax

```
LONG mpGetSysTimes (  
    MP_SYS_TIME_SEND_DATA*      sData,  
    MP_SYS_TIME_RSP_DATA*       rData  
);
```

■ Description

Retrieves the system time when the API is executed. The information retrieved is set in the corresponding member in parameter structure as a value.

■ Parameters

[*sData*]

[in] The pointer to the data structure which specifies the system time to be retrieved

- MP_SYS_TIME_SEND_DATA

[in] The data structure which specifies the system time to be retrieved

Syntax: typedef struct
 {
 SHORT *sTimeType*;
 CHAR *reserved*[2];
 } MP_SYS_TIME_SEND_DATA;

Member: <*sTimeType*>
 Type of system time

Symbolic constant	Description
MP_POWER_ON_TIME_ID	Control power time
MP_SERVO_ON_TIME_ID	Servo power time
MP_PLAYBACK_TIME_ID	Playback time
MP_MOVING_TIME_ID	Moving time
MP_OPERATION_TIME_ID	Operating time

9 System Monitor API

mpGetSysTimes

[rData]

[out] The pointer to the data structure which receives the system time

- MP_SYS_TIME_RSP_DATA

[out] The data structure which receives the system time

Syntax: typedef struct
 {
 SHORT *sStartYear*;
 SHORT *sStartMonth*;
 SHORT *sStartDay*;
 SHORT *sStartHour*;
 SHORT *sStartMin*;
 SHORT *sStartSec*;
 LONG *lElapsedTime*;
 } MP_SYS_TIME_RSP_DATA;

Member: <*sStartYear*>
 Year

 <*sStartMonth*>
 Month

 <*sStartDay*>
 Day

 <*sStartHour*>
 Hour

 <*sStartMin*>
 Minute

 <*sStartSec*>
 Second

 <*lElapsedTime*>
 Elapsed time

■ **Return Value**

Symbolic constant: OK (value: 0) : Normal end

Symbolic constant: ERROR (value: -1) : Error

mpGetSysVersionNo

Retrieves the version number of system software (main CPU) in character strings.

■ Syntax

[rData]

```
LONG mpGetSysVersionNo
(
    MP_SYS_VERSION_NO_RSP_DATA* rData
)
```

■ Description

Retrieves the version number of the controller system software (main CPU) in character strings.

The information retrieved is set in the corresponding member in parameter structure as a value.

■ Parameters

[out] The pointer to the data structure which receives the system version

MP_SYS_VERSION_NO_RSP_DATA

[out] The data structure which receives the system version

Syntax: typedef struct

```
{
    CHAR sys_version[OFFLINE_SYSTEM_VERSION_SIZE];
                                                    /* System version */

    CHAR reserved[3];
}    MP_SYS_VERSION_NO_RSP_DATA;

#define OFFLINE_SYSTEM_VERSION_SIZE: System version character length 22
```

Member: <sys_version>

System version

(e.g.) For the YRC1000: "YAS1.00.00A(JP/EN)-00"

For the YRC1000micro: "YBS1.00.00A(JP/EN)-00"

■ Return Value

Symbolic constant: OK (value: 0) : Normal end

Symbolic constant: ERROR (value: -1) : Error (Version number could not be retrieved)

mpGetJogSpeed

Retrieves the current jog speed.

■ **Syntax**

```
LONG mpGetJogSpeed (  
MP_JOGSPEED_RSP_DATA*   rData  
);
```

■ **Description**

Retrieves the current jog speed.

The information retrieved is set in the corresponding member in parameter structure as a value.

■ **Parameter**

[rData]

[out] The pointer to the data structure which receives the jog speed

- MP_JOGSPEED_RSP_DATA

[out] The data structure which receives the jog speed

```
Syntax:  typedef struct  
        {  
        SHORT      sJogSpeed;  
        CHAR        reserved[2];  
        } MP_JOGSPEED_RSP_DATA;
```

Member: <sJogSpeed>
 Jog speed

sJogSpeed value	Meaning
0	Inching
1	Low speed
2	Medium speed
3	High speed
4	Maximum speed ([HIGH SPEED] key)

■ **Return Value**

Symbolic constant: OK (value: 0) : Normal end
Symbolic constant: ERROR (value: -1) : Error

9 System Monitor API

mpGetJogSpeed



Exclusive control by using semaphores is executed between this API and the APIs listed below.

Thus, if this API is executed while one of the following APIs is already being executed, the task may be suspended to wait for the API to be completed. (waiting for semaphore acquisition)

Therefore, DO NOT use the following APIs for the task which must be performed periodically with high priority.

mpCancelError(), mpResetAlarm(), mpSetCycle(),
mpSetServoPower(), mpSetMasterJob(), mpSetCurJob(),
mpStartJob(), mpHold(), mpWaitForJobEnd(),
mpDeleteJob(), mpGetJogSpeed(), mpGetJogCoord(),
mpIMOV(), mpMOVJ(), mpMOVL(), mpPulseMOVJ(),
mpPulseMOVL(), mpManualMOV()

mpGetJogCoord

Retrieves the coordinate system of the current jog operation.

■ Syntax

```
LONG mpGetJogCoord (  
    MP_JOGCOORD_RSP_DATA*    rData  
);
```

■ Description

Retrieves the coordinate system of the current jog operation.

The information retrieved is set in the corresponding member in parameter structure as a value.

A feature has been added to return the user coordinates file number being used if the coordinate system of the jog operation is user coordinates. The feature is supported by the system software and MotoPlusIDE versions shown below.

For the YRC1000:

System software: YAS2.60-00 or later

MotoPlusIDE: Ver2.22 or later

For YRC1000micro:

System software: YBS2.04-00 or later

MotoPlusIDE: Ver2.22 or later

9 System Monitor API

mpGetJogCoord

■ Parameter

[rData]

[out] The pointer to the data structure which receives the operational coordinates

• MP_JOGCOORD_RSP_DATA

[out] The data structure which receives the operational coordinates

System software version earlier than YAS2.60-00 and MotoPlusIDE version earlier than version 2.22¹⁾

```
Syntax:  typedef struct
          {
            USHORT    sJogCoord; /* Jog coordinate system */
            CHAR      reserved[2];
          } MP_JOGCOORD_RSP_DATA;
```

System software version YAS2.60-00 or later and MotoPlusIDE version 2.22 or later²⁾

```
typedef struct
{
  USHORT sJogCoord; /* Jog coordinate system */
  SHORT  sUsrFrameNo; /* User coordinates number (1-63)
                      (Always returned regardless of
                      the selected coordinates type.) */
} MP_JOGCOORD_RSP_DATA;
```

Member: <sJogCoord>
Operational coordinates

sJogCoord value	Meaning
0	Joint coordinates
1	Cartesian coordinates
2	Cylindrical coordinates
3	Tool coordinates
4	User coordinates
5	External reference point coordinates
6	Torch coordinates

<sUsrFrameNo>
User coordinates number (1-63)

■ Return Value

Symbolic constant: OK (value: 0) : Normal end

Symbolic constant: ERROR (value: -1) : Error

1 For the YRC1000micro:
System software version earlier than YBS2.04-00 and MotoPlusIDE version earlier than version 2.22

2 For the YRC1000micro:
System software version YBS2.04-00 or later and MotoPlusIDE version 2.22 or later

9 System Monitor API

mpGetJogCoord

■ Note

sUsrFrameNo can be used in a software configuration that satisfies all of the following conditions.

For the YRC1000:

- System software: YAS2.60.00A-00 or later
- MotoPlusIDE: Ver2.22 or later

For the YRC1000micro:

- System software: YBS2.10.00A-00 or later
- MotoPlusIDE: Ver2.22 or later

If these conditions are not satisfied, SHORT sUsrFrameNo is defined as CHAR reserved[2] and cannot be used.

sUsrFrameNo is set to a value regardless of the current coordinate system.

Use the value of sUsrFrameNo only when sJogCoord = 4.



Exclusive control by using semaphores is executed between this API and the APIs listed below. Thus, if this API is executed while one of the following APIs is already being executed, the task may be suspended to wait for the API to be completed. (waiting for semaphore acquisition)

Therefore, DO NOT use the following APIs for the task which must be performed periodically with high priority.

mpCancelError(), mpResetAlarm(), mpSetCycle(),
mpSetServoPower(), mpSetMasterJob(), mpSetCurJob(),
mpStartJob(), mpHold(), mpWaitForJobEnd(),
mpDeleteJob(), mpGetJogSpeed(), mpGetJogCoord(),
mpIMOV(), mpMOVJ(), mpMOVL(), mpPulseMOVJ(),
mpPulseMOVL(), mpManualMOV()



The value of sUsrFrameNo can be used only if sJogCoord = 4 (user coordinates).

mpCtrlGrpId2GrpNo

Converts the control group ID to the control group number.

■ Syntax

```
int mpCtrlGrpId2GrpNo  
(  
    MP_GRP_ID_TYPE grp_id          /* specified control group id's */  
)
```

■ Description

The control group ID is the characteristic number assigned to each control group. This value is not changed according to the configuration of the control group in the system, but is always the same value. For example, the control group ID for S1 is the same either in the system with R1+S1 or R1+R2+S1.

The control group number is the number assigned sequentially to the control group in the system. Thus, the number assigned to each control group differs according to the configuration of the control group in the system. For example, the control group number for S1 is 1 in the system with R1+S1, but it is 2 in the system with R1+R2+S1.

For some of the sensor control APIs or the servo control APIs, the control group number must be specified by the parameter. To use one of those APIs, retrieve the appropriate control group number by using this API (mpCtrlGrpId2GrpNo).

■ Parameter

[*grp_id*]

Specify the value in the following table.

Symbolic constant used by group ID	Description
MP_R1_GID	R1 (Robot 1)
MP_R2_GID	R2 (Robot 2)
MP_R3_GID	R3 (Robot 3)
MP_R4_GID	R4 (Robot 4)
MP_R5_GID	R5 (Robot 5)
MP_R6_GID	R6 (Robot 6)
MP_R7_GID	R7 (Robot 7)
MP_R8_GID	R8 (Robot 8)
MP_B1_GID	B1 (Base 1)
MP_B2_GID	B2 (Base 2)
MP_B3_GID	B3 (Base 3)
MP_B4_GID	B4 (Base 4)
MP_B5_GID	B5 (Base 5)
MP_B6_GID	B6 (Base 6)
MP_B7_GID	B7 (Base 7)
MP_B8_GID	B8 (Base 8)

9 System Monitor API

mpCtrlGrpId2GrpNo

Symbolic constant used by group ID	Description
MP_S1_GID	S1 (Station 1)
MP_S2_GID	S2 (Station 2)
MP_S3_GID	S3 (Station 3)
MP_S4_GID	S4 (Station 4)
MP_S5_GID	S5 (Station 5)
MP_S6_GID	S6 (Station 6)
MP_S7_GID	S7 (Station 7)
MP_S8_GID	S8 (Station 8)
MP_S9_GID	S9 (Station 9)
MP_S10_GID	S10 (Station 10)
MP_S11_GID	S11 (Station 11)
MP_S12_GID	S12 (Station 12)
MP_S13_GID	S13 (Station 13)
MP_S14_GID	S14 (Station 14)
MP_S15_GID	S15 (Station 15)
MP_S16_GID	S16 (Station 16)
MP_S17_GID	S17 (Station 17)
MP_S18_GID	S18 (Station 18)
MP_S19_GID	S19 (Station 19)
MP_S20_GID	S20 (Station 20)
MP_S21_GID	S21 (Station 21)
MP_S22_GID	S22 (Station 22)
MP_S23_GID	S23 (Station 23)
MP_S24_GID	S24 (Station 24)

■ Return Value

0 or more Normal end
Returns the control group number which begins with 0.

Symbolic constant: ERROR (value: -1) : Error

mpGetToolData

Retrieves the specified tool information.

■ Syntax

```
LONG mpGetToolData(
    SHORT          sTool/No,
    MP_TOOL_RSP_DATA* rData
)
```

■ Description

Retrieves the specified tool information.

The information retrieved is set in the corresponding member in parameter structure as a value.

■ Parameter

[sTool/No]

[in] Tool number

Set the tool number of the tool to be retrieved.

[rData]

[out] The pointer to the data structure which receives tools.

- MP_TOOL_RSP_DATA

[out] The data structure which receives tools

Syntax:

```
typedef struct
{
    USHORT    err_no;                /* error number */
    CHAR      reserved0[2];
    UCHAR     name[MAX_TOOL_NAME];  /* tool name */
    LONG      x;    /* position of the tool tip, X-direction (0.001mm) */
    LONG      y;    /* position of the tool tip, Y-direction (0.001mm) */
    LONG      z;    /* position of the tool tip, Z-direction (0.001mm) */
    LONG      rx;   /* posture of the tool, X-direction (0.0001deg) */
    LONG      ry;   /* posture of the tool, Y-direction (0.0001deg) */
    LONG      rz;   /* posture of the tool, Z-direction (0.0001deg) */
    LONG      xg;   /* center of gravity of the tool, X-direction (0.001mm) */
    LONG      yg;   /* center of gravity of the tool, Y-direction (0.001mm) */
    LONG      zg;   /* center of gravity of the tool, Z-direction (0.001mm) */
    LONG      w;    /* tool weight (0.001kg) */
    LONG      ix;   /* position of center of gravity on the tool,
                    X-direction moment of inertia (0.001mm) */
    LONG      iy;   /* position of center of gravity on the tool,
                    Y-direction moment of inertia (0.001mm) */
    LONG      iz;   /* position of center of gravity on the tool,
                    Z-direction moment of inertia (0.001mm) */
    CHAR      reserved1[8];
}MP_TOOL_RSP_DATA;
```

MAX_TOOL_NAME: The maximum number of characters of the tool name: 16

9 System Monitor API

mpGetToolData

Member: *<err_no>*
Error number

Value	Meaning
0x0000	Normal end
0x3620	Error in the specified tool file number

■ Return Value

Symbolic constant: OK (value: 0) : Normal end

Symbolic constant: ERROR (value: -1) : Error

mpGetEncoderTemp

Retrieves the celsius temperature of each axes of encoder.

■ Syntax

```
LONG mpGetEncoderTemp(
    MP_CTRL_GRP_SEND_DATA *sData,
    MP_ENCODER_TEMP_RSP_DATA *rData
);
```

■ Description

Retrieves the celsius temperature of each axes of encoder.

The information retrieved is set in the corresponding member in parameter structure as a value.

■ Parameters

[sData]

[in] The pointer to the data structure which specifies the control group

- MP_CTRL_GRP_SEND_DATA

[in] The data structure which specifies the control group

```
Syntax:  typedef struct
          {
            CTRLG_T sCtrlGrp;
          } MP_CTRL_GRP_SEND_DATA;
```

Member: <sCtrlGrp>
Control group

sCtrlGrp value	Description
0	R1 (Robot 1)
1	R2 (Robot 2)
2	R3 (Robot 3)
3	R4 (Robot 4)
4	R5 (Robot 5)
5	R6 (Robot 6)
6	R7 (Robot 7)
7	R8 (Robot 8)
8	B1 (Base 1)
9	B2 (Base 2)
10	B3 (Base 3)
11	B4 (Base 4)
12	B5 (Base 5)
13	B6 (Base 6)
14	B7 (Base 7)
15	B8 (Base 8)
16	S1 (Station 1)
17	S2 (Station 2)
18	S3 (Station 3)

9 System Monitor API

mpGetEncoderTemp

sCtrlGrp value	Description
19	S4 (Station 4)
20	S5 (Station 5)
21	S6 (Station 6)
22	S7 (Station 7)
23	S8 (Station 8)
24	S9 (Station 9)
25	S10 (Station 10)
26	S11 (Station 11)
27	S12 (Station 12)
28	S13 (Station 13)
29	S14 (Station 14)
30	S15 (Station 15)
31	S16 (Station 16)
32	S17 (Station 17)
33	S18 (Station 18)
34	S19 (Station 19)
35	S20 (Station 20)
36	S21 (Station 21)
37	S22 (Station 22)
38	S23 (Station 23)
39	S24 (Station 24)

[rData]

[out] The pointer to the data structure which receives the temperature value

- MP_ENCODER_TEMP_RSP_DATA

[out] The data structure which receives the temperature value

```
Syntax:  #define MAX_PULSE_AXES (8)
         typedef struct
         {
             LONG    ITemp[MAX_PULSE_AXES];
         } MP_ENCODER_TEMP_RSP_DATA;
```

Member: <ITemp[MAX_PULSE_AXES]>
Temperature of encoder (up to 8 arrays)

Array	Description
ITemp[0]	In 1st axis (S) direction (unit: 1 degree Celsius)
ITemp[1]	In 2nd axis (L) direction (unit: 1 degree Celsius)
ITemp[2]	In 3rd axis (U) direction (unit: 1 degree Celsius)
ITemp[3]	In 4th axis (R) direction (unit: 1 degree Celsius)
ITemp[4]	In 5th axis (B) direction (unit: 1 degree Celsius)
ITemp[5]	In 6th axis (T) direction (unit: 1 degree Celsius)
ITemp[6]	In 7th axis (E) direction (unit: 1 degree Celsius)
ITemp[7]	In 8th axis direction (unit: 1 degree Celsius)

9 System Monitor API

mpGetEncoderTemp

■ Return Value

Symbolic constant: OK (value: 0) : Normal end

Symbolic constant: ERROR (value: -1) : Error

■ Note

This API can be used in the software configuration that satisfies all of the following conditions.

For the YRC1000:

- System software: YAS2.41.00A-00 or later
- MotoPlusIDE: Ver2.01 or later

For the YRC1000micro:

- System software: YBS1.31.00A-00
- MotoPlusIDE: Ver2.01 or later

mpGetCalendar

Retrieves the date and time of the controller.

■ Syntax

```
LONG mpGetCalendar(  
    MP_CALEDAR_RSP_DATA*  rData  
);
```

■ Parameter

[*rData*]

[out] The pointer to the data structure which receives the calendar information

- MP_CALEDAR_RSP_DATA

Syntax:

```
typedef struct{  
    USHORT    usYear;           /* Year */  
    USHORT    usMonth;         /* Month */  
    USHORT    usDay;           /* Day */  
    USHORT    usHour;          /* Hour */  
    USHORT    usMin;           /* Minute */  
    USHORT    usSec;           /* Second */  
    CHAR      reserved[4];  
} MP_CALEDAR_RSP_DATA;
```

■ Return Value

Symbolic constant: OK (value: 0) : Normal end

Symbolic constant: ERROR (value: -1) : Error

mpGetJobStepData

Retrieves the information of the move instruction (step) corresponding to the specified job name and the step number.

■ Syntax

```
LONG mpGetJobStepData(
    MP_JOB_STEP_NO_SEND_DATA* sData
        /* the data structure which specifies the job step number */
    MP_JOB_STEP_RSP_DATA* rData
        /* the data structure which receives the move instruction information */
);
```

■ Parameter

[sData]

[in] The pointer to the data structure which specifies the job step number

- MP_JOB_STEP_NO_SEND_DATA

Syntax:

```
typedef struct{
    USHORT    usStep;                /* step number */
    CHAR      reserved0[2];
    CHAR      cJobName[MAX_JOB_NAME_LEN]; /* job name */
    CHAR      reserved1[3];
} MP_JOB_STEP_NO_SEND_DATA;
#define MAX_JOB_NAME_LEN 33 /* up to 32 characters + null character '\0' */
```

Member: <usStep>

step number (1 to 9998)

<cJobName[MAX_JOB_NAME_LEN]>

job name (up to 32 characters)

[rData]

[out] The pointer to data structure which receives the move instruction information

- MP_JOB_STEP_RSP_DATA

Syntax:

```
typedef struct{
    USHORT    err_no;                /* error number */
    CHAR      reserved0[2];
    ULONG     attr;                  /* step attribute (future function) */
    UCHAR     comment[STEP_COMM_SIZE+1]; /* comment */
    UCHAR     reserved1[3];
    ULONG     movCtrlDataNum; /* move instruction number in step */
    MP_MOV_CTRL_DATA movCtrlData[MAX_JOB_MOV_CTRL_DATA_NUM];
        /* data structure for every move instruction */
    LONG     posLevel;                /* setting value of positioning level (PL=) */
    LONG     cornerRadius;            /* setting value of conner radius (CR=) */
    LONG     accValue;                /* specified value of acceleration (ACC=) */
    LONG     decValue;                /* specified value of deceleration (DEC=) */
}
```


9 System Monitor API

mpGetJobStepData

```

UCHAR    reserved2[32];
} MP_JOB_STEP_RSP_DATA;
#define STEP_COMM_SIZE      32    /* up to 32 characters for a comment */
#define MAX_JOB_MOV_CTRL_DATA_NUM 4 /* maximum move instruction data number */

```

Member: <err_no>
error number

Value	Description
0x0000	Normal end
0x4040	Specified JOB not found
0x4130	No position data
0x5200	Out of data range

<attr>
step attribute

Value	Description	Definition
0x0001	Edit lock of line	JOB_LINE_ATTR_EDIT_LOCK
0x0002	Change line to comment	JOB_LINE_ATTR_COMMENT

<comment[STEP_COMM_SIZE+1]>

comment (up to 32 characters)

<movCtrlDataNum>

move instruction number in step

(e.g.) MOVJ: movCtrlDataNum = 1

SMOVL+MOVJ: movCtrlDataNum = 2

<movCtrlData>

data structure for every move instruction

<posLevel>

setting value of positioning level (PL=)

<cornerRadius>

setting value of conner radius (CR=)

<accValue>

specified value of acceleration (ACC=)

<decValue>

specified value of deceleration (DEC=)

*If the tag "PL=", "CR=", "ACC=" or "DEC=" is not in step, "-1" is set to posLevel, cornerRadius, accValue or decValue.

Also, in case a variable is used for the setting such as "PL=B000", "-1" is set.

• MP_MOV_CTRL_DATA

data structure for every move instruction

Syntax:

typedef struct{

UCHAR intpType; /* interpolation type */

UCHAR intpKind; /* move instruction type*/

UCHAR speedType; /* speed type */

UCHAR reserved0;

LONG speedValue; /* speed setting value (VJ=,V=,VR=,VE=,VS=) */

ULONG posNum; /* position data number in move instruction */

9 System Monitor API

mpGetJobStepData

```

MP_JOB_POS_DATA    posData[MAX_JOB_MOV_POS_NUM];
                    /* position data structure */

UCHAR    reserved1[32];
} MP_MOV_CTRL_DATA;
#define MAX_JOB_MOV_POS_NUM    3    /* maximum position data number */

```

Member: <intpType>
interpolation type

Value	Description	Definition
0	Joint interpolation	INTP_TYPE_JOINT
1	Linear interpolation	INTP_TYPE_LINEAR
2	Circular interpolation	INTP_TYPE_CIRCLE
3	Spline interpolation	INTP_TYPE_SPLINE
4	Reference point	INTP_TYPE_REF

<intpKind>
move instruction type

Value	Description	Definition
0	Normal	INTP_KIND_NORMAL
1	External reference point	INTP_KIND_EI
2	Coordination	INTP_KIND_SLAVE
3	Increment value move	INTP_KIND_INC
4	Increment value move (coordination)	INTP_KIND_SLAVE_INC
5	Conveyor synchronization	INTP_KIND_SYNCHRO
6	SVSPOTMOV	INTP_KIND_SVSPOTMOV
7	SVDRESMOV	INTP_KIND_SVDRESMOV
255	Others	INTP_KIND_OTHER

Combination of <intpType> and <intpKind> (movCtrlData[0])

		intpKind								
		0	1	2	3	4	5	6	7	255
intpType	0	MOVJ	N/A	N/A	N/A	N/A	SYMOVJ	N/A	N/A	N/A
	1	MOVL	EIMOVL	SMOVL	IMOV	SIMOV	SYMOVL	SVSPOTMOV	SVDRESMOV	FORMAPR
	2	MOVC	EIMOVC	SMOVC	N/A	N/A	SYMOCV	N/A	N/A	N/A
	3	MOVJ	N/A	SMOVS	N/A	N/A	N/A	N/A	N/A	N/A
	4	REFP	N/A	SREFP	N/A	N/A	N/A	N/A	N/A	N/A

9 System Monitor API

mpGetJobStepData

Combination of <intpType> and <intpKind> (movCtrlData[1 to 3])

		intpKind								
		0	1	2	3	4	5	6	7	255
intpType	0	+MOVJ	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A
	1	+MOVL	+EIMOVL	+SMOV, +SMOVX	+IMOV	N/A	N/A	N/A	N/A	N/A
	2	+MOVC	+EIMOV	+SMOVX	N/A	N/A	N/A	N/A	N/A	N/A
	3	+MOV	N/A	+SMOVX	N/A	N/A	N/A	N/A	N/A	N/A
	4	+REFP	N/A	+SREFP	N/A	N/A	N/A	N/A	N/A	N/A

*The interpolation of +SMOVX is the same as that of SMOVx instruction.

<speedType>

speed type

If there is no speed specified tag, speedType=0.

Value	Description	Definition
0	Joint speed (0.01[%])	SPEED_TYPE_VJ
1	Control point speed (0.1[mm/sec])	SPEED_TYPE_V
2	Position angular speed(0.1[mm/sec])	SPEED_TYPE_VR
3	External axis speed (0.1[mm/sec])	SPEED_TYPE_VE
4	Target speed (0.1[mm/sec])	SPEED_TYPE_VS

<speedValue>

speed setting value (VJ=, V=, VR=, VE=, VS=)

VJ=: 0.01[%] unit

Other than VJ=: 0.1[mm/sec] unit

*Regardless of the setting for speed data input form, the unit is [mm/sec].

If there is no speed specified tag, speedValue=-1.

<posNum>

registered number of the position data in the move instruction (each one for manipulator + base + station, three in total)

<posData>

position data structure

- MP_JOB_POS_DATA
the position data structure

Syntax:

```
typedef struct{
CTRLG_T   ctrl_grp;      /* control group */
LONG      posType;       /* position data type */
CTRLG_T   varIndex;     /* position variable number */
ULONG     attr;          /* data attribute */
ULONG     attrExt;       /* data attribute(expansion) */
LONG      pos[MP_GRP_AXES_NUM]; /*position data */
} MP_JOB_POS_DATA;
#define MAX_GRP_AXES_NUM 8 /* maximum control axis number */
```

9 System Monitor API

mpGetJobStepData

<ctrl_grp>
control group

Value	Description
0 to 7	R1(Robot1) to R8(Robot8)
8 to 15	B1(Base1) to B8(Base8)
16 to 39	S1(Station1) to S24(Station24)

<posType>
position data type

Value	Description	Definition
0	position data	JOB_POS_TYPE_POS
1	position variable	JOB_POS_TYPE_VAR
2	local position variable	JOB_POS_TYPE_LOCAL_VAR

<varIndex>
position variable number
When posType=1(position variable) or 2(local position variable), the position variable number is stored.
When posType=0 (position data), 0 is set.

<attr><attrExt><pos>
data attribute, data attribute (expansion), position data
The data structure is the same as that of mpGetPosVarData().

Member name		Description	
	mpGetPosVarData()	Bit No.	Contents
attr	rData[0]	D05-D00	Variable type 0 Pulse 16 Cartesian (base coordinates) 17 Cartesian (robot coordinates) 18 Cartesian (tool coordinates) 19 Cartesian (user coordinates) 20 Cartesian (master tool)
		D07-D06	Reserved by manufacturer
		D08	0:Front 1:Back
		D09	0:Upper arm 1:Lower arm
		D10	0:Flip 1:No flip
		D11	0:R<180deg 1:R>=180deg
		D12	0:T<180deg 1:T>=180deg
		D13	0:S<180deg 1:S>=180deg
		D14-D15	Reserved by manufacturer
		D16-D21	Tool number(0-63)
		D22-D27	User coordinate number
		D28-D31	Reserved by manufacturer
attrExt	rData[1]		(Expansion attribute)

9 System Monitor API

mpGetJobStepData

Member name		Pulse type	Cartesian type
	mpGetPos VarData()		
pos[0]	rData[2]	1st axis(S) pulse value	X-axis coordinate(unit:micron)
pos[1]	rData[3]	2nd axis(L) pulse value	Y-axis coordinate(unit:micron)
pos[2]	rData[4]	3rd axis(U) pulse value	Z-axis coordinate(unit:micron)
pos[3]	rData[5]	4th axis(R) pulse value	Wrist angle Rx(unit:0.0001deg)
pos[4]	rData[6]	5th axis(B) pulse value	Wrist angle Ry(unit:0.0001deg)
pos[5]	rData[7]	6th axis(T) pulse value	Wrist angle Rz(unit:0.0001deg)
pos[6]	rData[8]	7th axis(E) pulse value	angle Re(unit:0.0001deg)
pos[7]	rData[9]	8th axis pulse value	8th axis pulse value(micron in the case of traveling axis)

■ Return Value

Symbolic constant: OK (value: 0) : Normal end

Symbolic constant: ERROR (value: -1) : Error
(For details, check err_no.)

mpGetToolNo

Retrieves the operation tool number for the specified control group.

■ Syntax

```
LONG mpGetToolNo(
    ULONG                sRobotNo,
    /*the number for the manipulator which retrieves the tool number*/
    MP_GET_TOOL_NO_RSP_DATA* rData
    /* the data structure which receives the operation tool number */
);
```

■ Parameter

[sRobotNo]

[in] The number of the manipulator which retrieves the tool number

Value	Description
MP_R1_GID	R1
MP_R2_GID	R2
MP_R3_GID	R3
MP_R4_GID	R4
MP_R5_GID	R5
MP_R6_GID	R6
MP_R7_GID	R7
MP_R8_GID	R8

[rData]

[out] The pointer to the data structure which receives the operation tool number

- MP_GET_TOOL_NO_RSP_DATA

Syntax:

```
typedef struct{
    USHORT    err_no;      /* error number */
    SHORT     sToolNo;     /* operation tool number to be retrieved */
    CHAR      reserved[4];
}MP_GET_TOOL_NO_RSP_DATA;
```

■ Return Value

0: API execution processor received the command.

Return value = 0 and err_no = 0 : Normal end

Return value = 0 and err_no != 0 : Error (check err_no.)

-1: API execution processor does not respond.

10 System Control API

mpPutVarData

Writes the values for variables (B, I, D, R) of the controller.



Use of mpPutUserVars is recommended for writing the variables.
mpPutUserVars is the API, in which mpPutVarData (variables B, I, D, R), mpPutSVarInfo (variable S), and mpPutPosVarData (position-type variable) are integrated into a single API with the writing speed greatly accelerated.
To maintain the operation of existing applications, mpPutVarData is still retained.

■ Syntax

```
LONG mpPutVarData (  
MP_VAR_DATA*      sData,  
LONG              num  
);
```

■ Parameter

[sData]

Pointer to the data structure that writes the variable

- MP_VAR_DATA
Data structure that writes the variable

```
Syntax:  typedef struct  
        {  
        USHORT   usType;  
        USHORT   usIndex;  
        LONG      ulValue;  
        } MP_VAR_DATA;
```

Member: <usType>
Variable type

Value	Description
MP_RESTYPE_VAR_B	Byte type
MP_RESTYPE_VAR_I	Integer type
MP_RESTYPE_VAR_D	Double-precision type
MP_RESTYPE_VAR_R	Real type

<usIndex>
Variable index

<ulValue>
Variable value

To set multiple variables, define *num* arrays, set the variable type, variable index, and variable value of each array, then set the starting address of each array to *sData*.

10 System Control API

mpPutVarData

[num]

Number of the variable data (up to 126)

■ Return Value

Symbolic constant: OK (value: 0) : Normal end

Symbolic constant: ERROR (value: -1) : Error

<Sample Code>

10 is written to B000, -100 to I000, -1000 to D000, and 1.2345 to R000 by calling the following function.

```
void put_var_data_sample(void)
{
    MP_VAR_DATA    Data_type_B[1];
    MP_VAR_DATA    Data_type_I[1];
    MP_VAR_DATA    Data_type_D[1];
    MP_VAR_DATA    Data_type_R[1];

    LONG           tmp_type_B;
    LONG           tmp_type_I;
    LONG           tmp_type_D;
    FLOAT          tmp_type_R;

    //Write to B variable
    tmp_type_B = 10;
    sData_type_B[0].usType = MP_RESTYPE_VAR_B;
                                //Set variable type to B variable
    sData_type_B[0].usIndex = 0; //Set write destination to B000
    sData_type_B[0].ulValue = tmp_type_B;
    mpPutVarData(&sData_type_B[0], 1 );

    //Write to I variable
    tmp_type_I = -100;
    sData_type_I[0].usType = MP_RESTYPE_VAR_I;
                                //Set variable type to I variable
    sData_type_I[0].usIndex = 0; //Set write destination to I000
    sData_type_I[0].ulValue = tmp_type_I;
    mpPutVarData(&sData_type_I[0], 1 );

    //Write to D variable
    tmp_type_D = -1000;
    sData_type_D[0].usType = MP_RESTYPE_VAR_D;
                                //Set variable type to D variable
    sData_type_D[0].usIndex = 0; //Set write destination to D000
    sData_type_D[0].ulValue = tmp_type_D;
    mpPutVarData(&sData_type_D[0], 1 );

    //Write to R variable
    tmp_type_R = 1.2345;
    sData_type_R[0].usType = MP_RESTYPE_VAR_R;
                                //Set variable type to R variable
    sData_type_R[0].usIndex = 0; //Set write destination to R000
    /* Use memcpy and copy the data in tmp_type_R (FLOAT) to ulValue
    (LONG) */
    memcpy( &sData_type_R[0].ulValue, &tmp_type_R, sizeof(LONG) );
    mpPutVarData(&sData_type_R[0], 1 );
}
```


10 System Control API

mpPutVarData



- Immediately after the job starts, the system has access to the variable area, and the API to write to the variable area such as mpPutVarData(), mpPutSVarInfo() or mpPutPosVarData() may return an error. Thus, for the application which requires the job start-up, describe a code with which the value is repeatedly retrieved until the error does not return.
- There is no interlock between the MotoPlus API which writes variables, I/O, or registers and the following modification methods other than MotoPlus, thus the MotoPlus API can write them. Pay attention to the interference with the other methods so that the modification by MotoPlus will not affect the system's operation.
 1. Edit of variables, I/O, or registers in the teach mode by using the programming pendant
 2. Change of variables, I/O, or registers by job execution
 3. Change of I/O or registers by the concurrent I/O output
- When changing a parameter or loading a file from the external memory device, the process of this API cannot be executed because the system is busy. In this case, the error (-1) is returned. Retry this API after a while.

mpPutSVarInfo

Writes the value for variable (S).



Use of mpPutUserVars is recommended for writing the variables.

mpPutUserVars is the API, in which mpPutVarData (variables B, I, D, R), mpPutSVarInfo (variable S), and mpPutPosVarData (position-type variable) are integrated into a single API with the writing speed greatly accelerated.

To maintain the operation of existing applications, mpPutSVarInfo is still retained.

■ Syntax

```
int mpPutSVarInfo
(
    MP_SVAR_SEND_INFO    *sData,
    LONG                 num
)
```

■ Parameter

[sData]

Pointer to the data area which specifies variable

typedef struct

```
{
    USHORT  usType;    /* Variable type (Only MP_RESTYPE_VAR_S is valid) */
    USHORT  usIndex;   /* Variable index */
    UCHAR   ucValue[S_VAR_SIZE+1]; /*S_VAR_SIZE (32 characters)+null character\0*/
    CHAR     reserved[3];
} MP_SVAR_SEND_INFO;
```

To set multiple variables, define *num* arrays, set the variable type and variable index of each array, then set the starting address of each array to *sData*.

[num]

Number of S variables (up to 10)

■ Return Value

Symbolic constant: OK (value: 0) : Normal end

Symbolic constant: ERROR (value: -1) : Error

10 System Control API

mpPutSVarInfo



- Since this API's processing time is a few milliseconds, do not use this API in a task with the I/O control cycle or interpolation cycle. If it is used in such a task, the process cannot be completed within the control cycle.
- When changing a parameter or loading a file from the external memory device, the process of this API cannot be executed because the system is busy. In this case, the error (-1) is returned. Retry this API after a while.

mpWriteIO

Writes the value for I/O.

■ **Syntax**

```
LONG mpWriteIO (
MP_IO_DATA*      sData,
LONG             num
);
```

■ **Parameter**

[sData]

Pointer to the data structure that writes I/O

- MP_IO_DATA
Data structure that writes I/O

```
Syntax:  typedef struct
        {
            ULONG      ulAddr;
            ULONG      ulValue;
        } MP_IO_DATA;
```

Member: <ulAddr>
I/O address

Value	Description
10010 - 15127	Universal output #10010 - #15127(4096)
60010 - 60647	Interface panel #60010 - #60647(512)
27010 - 29567	Network input #27010 - #29567(2048)
1000000 - 1000559	Register M000 - M559(560)



M000 to M559 (1000000 to 1000559) is available for general registers. Later than that, M560 to M999 (1000560 to 1000999) are an area reserved by the system and used by specific functions. (For example, analog outputs are allocated to M560 to M562 in the instructions manual of JANCD-AEW02-E circuit board (for 3-channel analog output) for the YRC1000.) This area is not available for writing but reading using mpReadIO() and other APIs.

<ulValue>
I/O value

To set multiple I/Os, define *num* arrays, set the I/O address and I/O value of each array, then set the starting address of each array to *sData*.

10 System Control API

mpWriteIO



When writing multiple values simultaneously, this API temporarily makes the task priority of the application higher than the robot motion task, etc. so that the values will not be changed by another task while writing the values. Thus, writing many values may affect the robot motion. DO NOT specify a large value as *num* during the robot motion.

It may take time to reflect the written value contents. DO NOT read the same value by mpReadIO() or mpMonitor() soon after executing this API.

[num]

Number of the I/O data (up to 126)

■ **Return Value**

Symbolic constant: OK (value: 0) : Normal end

Symbolic constant: ERROR (value: -1) : Error



There is no interlock between the MotoPlus API which writes variables, I/O, or registers and the following modification methods other than MotoPlus, thus the MotoPlus API can write them. Pay attention to the interference with the other methods so that the modification by MotoPlus will not affect the system's operation.

1. Edit of variables, I/O, or registers in the teach mode by using the programming pendant
2. Change of variables, I/O, or registers by job execution
3. Change of I/O or registers by the concurrent I/O output

mpPutPosVarData

Writes the value for the position-type variable.



Use of mpPutUserVars is recommended for writing the variables.

mpPutUserVars is the API, in which mpPutVarData (variables B, I, D, R), mpPutSVarInfo (variable S), and mpPutPosVarData (position-type variable) are integrated into a single API with the writing speed greatly accelerated.

To maintain the operation of existing applications, mpPutSVarInfo is still retained.

■ Syntax

```
LONG mpPutPosVarData(
MP_POSVAR_DATA*      sData,
LONG                  num
);
```

■ Parameter

[sData]

[in] The pointer to the data structure array which specifies the position-type variable

- MP_POSVAR_DATA

[in] The data structure array which specifies the position-type variable

Syntax: typedef struct
{
 USHORT usType;
 USHORT usIndex;
 LONG ulValue[10];
} MP_POSVAR_DATA;

Member: <usType>
Variable type

Symbolic constant	Description
MP_RESTYPE_VAR_ROBOT (value: 5)	Robot
MP_RESTYPE_VAR_BASE (value: 6)	Base
MP_RESTYPE_VAR_STATION (value: 7)	Station

<usIndex>
Variable number

<ulValue[10]>
Setting value

To set multiple position-type variables, define *num* arrays, set the variable type, variable index, and position data of each array, then set the starting address of each array to *sData*.

10 System Control API

mpPutPosVarData

Array	Attribute	Description
rData[0]	D05 - D00	Variable type 0 Pulse 16 Cartesian (base coordinates) 17 Cartesian (robot coordinates) 18 Cartesian (tool coordinates) 19 Cartesian (user coordinates) 20 Cartesian (reserved for master tool)
	D07 - D06	Reserved by manufacturer
	D08	0: Front 1: Back
	D09	0: Upper arm 1: Lower arm
	D10	0: Flip 1: No flip
	D11	0: R < 180 deg 1: R >= 180 deg
	D12	0: T < 180 deg 1: T >= 180 deg
	D13	0: S < 180 deg 1: S >= 180 deg
	D14 - D15	Reserved by manufacturer
	D16 - D21	Tool number (0 - 63)
	D22 - D27	User coordinate number
	D28 - D31	Reserved by manufacturer
rData[1]		(Extended attribute)
Array	Pulse	Cartesian
rData[2]	1st axis (S) pulse value	X-axis coordinate (unit: micron)
rData[3]	2nd axis (L) pulse value	Y-axis coordinate (unit: micron)
rData[4]	3rd axis (U) pulse value	Z-axis coordinate (unit: micron)
rData[5]	4th axis (R) pulse value	Wrist angle Rx (unit: 0.0001 deg)
rData[6]	5th axis (B) pulse value	Wrist angle Ry (unit: 0.0001 deg)
rData[7]	6th axis (T) pulse value	Wrist angle Rz (unit: 0.0001 deg)
rData[8]	7th axis (E) pulse value	Angle Re (unit: 0.0001 deg)
rData[9]	8th axis pulse value	8th axis pulse value (micron in the case of traveling axis)

[num]

Number of array (up to 23)

Return Value

Symbolic constant: OK (value: 0) : Normal end

Symbolic constant: ERROR (value: -1) : Error



Immediately after the job starts, the system has access to the variable area, and the API to write to the variable area such as mpPutVarData(), mpPutSVarInfo(), or mpPutPosVarData() may return an error. Thus, for the application which requires the job start-up, describe a code with which the value is repeatedly retrieved until the error does not return.

10 System Control API

mpPutPosVarData



- There is no interlock between the MotoPlus function which writes variables, I/O, or registers and the following modification methods other than MotoPlus, thus the MotoPlus function can write them. Pay attention to the interference with the other methods so that the modification by MotoPlus will not affect the system's operation.

1. Edit of variables, I/O, or registers in the teach mode by using the programming pendant
 2. Change of variables, I/O, or registers by job execution
 3. Change of I/O or registers by the concurrent I/O output
- When changing a parameter or loading a file from the external memory device, the process of this function cannot be executed because the system is busy. In this case, the error (-1) is returned. Retry this function after a while.

mpPutUserVars

Writes the user variable.

To compare mpGetUserVars with the other variable access APIs, such as mpPutVardata(), mpPutPosVarData and mpPutSVarInfo(), mpGetUserVars can writes user variable in high speed.

■ Syntax

```
STATUS mpPutUserVars(  
MP_USR_VAR_INFO*mp_usr_var_info  
);
```

■ Parameter

[mp_usr_var_info]

[in] User variable access API interface (Input/Output)

- MP_USR_VAR_INFO

[in] The data structure that sets the user variable

```
Syntax: typedef struct  
{  
    int var_type;  
    int var_no;  
    union {  
        MP_B_VAR_BUFF      b;  
        MP_I_VAR_BUFF      i;  
        MP_D_VAR_BUFF      d;  
        MP_R_VAR_BUFF      r;  
        MP_S_VAR_BUFF      s;  
        MP_P_VAR_BUFF      p;  
        MP_P_VAR_BUFF      bp;  
        MP_P_VAR_BUFF      ex;  
    } val;  
} MP_USR_VAR_INFO;
```

Member: <var_type>
Variable type (Input)

Symbolic constant	Description
MP_VAR_B	Byte Variable
MP_VAR_I	Integer Variable
MP_VAR_D	Double Variable
MP_VAR_R	Real Variable
MP_VAR_S	Character Variable
MP_VAR_P	Robot position Variable
MP_VAR_BP	Base position Variable
MP_VAR_EX	Station (LEX) Variable

<var_no>
Variable number (Input)

10 System Control API

mpPutUserVars

<val>

Variable data buffer (Output)

Syntax: `#define STR_VAR_SIZE 32`
`#define MP_GRP_AXES_NUM 8`

```
typedef unsigned char MP_B_VAR_BUFF;
typedef short        MP_I_VAR_BUFF;
typedef long         MP_D_VAR_BUFF;
typedef float        MP_R_VAR_BUFF;
typedef char         MP_S_VAR_BUFF[STR_VAR_SIZE];
```

```
typedef struct {
  UINT      dtype;
  int       tool_no;
  int       uf_no;
  BITSTRING fig_ctrl;
  long      data[MP_GRP_AXES_NUM];
} MP_P_VAR_BUFF;
```

Member: `MP_P_VAR_BUFF`

<dtype>

Coordinate system type

Symbolic constant	Description
MP_PULSE_DTYPE	Pulse coordinate
MP_BASE_DTYPE	Base coordinate
MP_ROBO_DTYPE	Robot coordinate
MP_TOOL_DTYPE	Tool coordinate
MP_USER_DTYPE	User coordinate
MP_MTOOL_DTYPE	Master tool coordinate

<tool_no>

Tool file No. (0 to 63)

<uf_no>

User coordinate system file No. (0 to 62)

<fig_ctrl>

Figure information (bits)

Symbolic constant	Meaning
MP_FIG_SIDE	0: Front 1: Back
MP_FIG_ELBOW	0: Uper elbow 1: Lower elbow
MP_FIG_FLIP	0: Flip 1: No flip
MP_FIG_R180	0: R<180 1: R>=180
MP_FIG_T180	0: T<180 1: T>=180
MP_FIG_S180	0: S<180 1: S>=180

<data>

Position data

10 System Control API

mpPutUserVars

■ Return value

0 : Normal end
Any of following negative values: : Abnormal end

Symbolic constant	Description
E_MP_VARS_ACCESS	An error except followings
E_MP_VAR_TYPE	Incorrect variable type
E_MP_VAR_NO_RANGE	Incorrect variable No.
E_MP_PVAR_DTYPE	Incorrect coordinate type of the position variable
E_MP_PVAR_TOOL_NO	Incorrect tool No. of the position variable
E_MP_PVAR_USER_NO	Incorrect user coordinate type of the position variable
E_MP_PVAR_UNSETUP	Undefined data of the position variable
E_MP_PMS_RET_VAL	Failure to retrieve the position variable

10 System Control API

mpPutUserVars



Beware of changing the variable from this API during the JOB operation which uses a position variable as a target position of the move instruction. There is no guarantee for changing the position variable value which will reflect the motion of the robot.

Data types which are used in this API are different from ones in the mpPutVarData(), mpPutPosVarData() and mpPutSVarInfo(). The following table shows the differences of the position variable.

Difference of the API Parameter

Position Variable Retrieved API	User Variable Retrieved API
<pre>mpPutPosVarData(MP_POSVAR_DATA *sData; LONG num;) Syntax typedef struct { USHORT usType; USHORT usIndex; USHORT ulValue[10]; } MP_POSVAR_DATA;</pre>	<pre>mpGetUserVars(MP_USR_VAR_INFO *mp_usr_var_info;) Syntax typedef struct { int var_type; int var_no; union { MP_B_VAR_BUFF b; . . MP_P_VAR_BUFF p; MP_P_VAR_BUFF bp; MP_P_VAR_BUFF ex; } val; } MP_USR_VAR_INFO;</pre>



Setting Content of the Parameter

		mpGetPosVarData()		mpGetUserVars()	
Item		rData[10]	Bit No.	p, bp, ex	
Coordinate system		rData[0]	d0 to d5	.dtype	
Type	Front/Back		d08	.fig_ctrl	d0
	Upper elbow/ Lower elbow		d09		d1
	Flip/No flip		d10		d2
	R<180/R>=180		d11		d3
	T<180/T>=180		d12		d4
	S<180/S>=180		d13		d5
Tool No.			d16 to d21	.tool_no	
User coordinate No.			d22 to d27	.uf_no	
Position data		rData[2] to [9]		.data[0] to data[7]	

mpSetAlarm

Generates an application alarm in the controller.

■ Syntax

```
int mpSetAlarm
(
    short      alm_code,
    char       *alm_msg,
    UCHAR      sub_code
);
```

■ Description

Generates the alarm of the specified alarm code, alarm message, and subcode

If an alarm is generated using this API, all the jobs under execution are stopped.

■ Parameter

[alm_code]

Alarm code: 8000 to 8999

[alm_msg]

Alarm message (up to 32 characters)

Only ASCII characters can be used for the character string.

[sub_code]

Alarm sub code

■ Return Value

0 Normal end

ERROR Error (Alarm code range exceeded)



- Executing this API generates an alarm, and thus some operations are restricted, for example, editing is prohibited, JOB cannot be started, servo power cannot be turned ON.
Reset the alarm to cancel the restriction.
- The alarm generated by this API cannot be read by an alarm-reading API “mpGetAlarmStatus()” or “mpGetAlarmCode()” immediately after executing this API. This is because there is a time lag between the request of generating the alarm by this API and the actual detection of the alarm.

mpCancelError

Releases (resets) the error status.

■ Syntax

```
LONG mpCancelError (  
    MP_STD_RSP_DATA*    rData  
);
```

■ Parameter

[rData]

[out] The pointer to the data structure which receives the error information

- MP_STD_RSP_DATA

[out] The data structure which receives the error information

Syntax: typedef struct
 {
 USHORT *err_no*;
 CHAR *reserved*[2];
 } MP_STD_RSP_DATA;

Member: <*err_no*>
 Error number

Value	Description
0x0000	Normal end

■ Return Value

Symbolic constant: OK (value: 0) : Normal end

Symbolic constant: ERROR (value: -1) : Error



Exclusive control by using semaphores is executed between this API and the APIs listed below. Thus, if this API is executed while one of the following APIs is already being executed, the task may be suspended to wait for the API to be completed. (waiting for semaphore acquisition)

Therefore, DO NOT use the following APIs for the task which must be performed periodically with high priority.

mpCancelError(), mpResetAlarm(), mpSetCycle(),
mpSetServoPower(), mpSetMasterJob(), mpSetCurJob(),
mpStartJob(), mpHold(), mpWaitForJobEnd(),
mpDeleteJob(), mpGetJogSpeed(), mpGetJogCoord(),
mpIMOV(), mpMOVJ(), mpMOVL(), mpPulseMOVJ(),
mpPulseMOVL(), mpManualMOV()

mpResetAlarm

Resets the alarm.

■ **Syntax**

```
LONG mpResetAlarm (  
MP_STD_RSP_DATA*    rData  
);
```

■ **Description**

Resets the controller alarm.

The alarm may not be canceled by the reset operation depending on its type.

Refer to "Stop by Alarm" in chapter 4.3 of the operator's manual of the controller¹⁾.

■ **Parameter**

[rData]

[out] The pointer to the data structure which receives the error information

- MP_STD_RSP_DATA
- [out] The data structure which receives the error information

```
Syntax:  typedef struct  
        {  
            USHORT    err_no;  
            CHAR       reserved[2];  
        } MP_STD_RSP_DATA;
```

Member: <err_no>
Error number

Value	Description
0x0000	Normal end

■ **Return Value**

Symbolic constant: OK (value: 0) : Normal end

Symbolic constant: ERROR (value: -1) : Error

1 For the YRC1000:
YRC1000 GENERAL OPERATOR'S MANUAL (RE-CSO-A051)
For the YRC1000micro:
YRC1000micro OPERATOR'S MANUAL (RE-CSO-A058)

10 System Control API

mpResetAlarm



Exclusive control by using semaphores is executed between this API and the APIs listed below.

Thus, if this API is executed while one of the following APIs is already being executed, the task may be suspended to wait for the API to be completed. (waiting for semaphore acquisition)

Therefore, DO NOT use the following APIs for the task which must be performed periodically with high priority.

mpCancelError(), mpResetAlarm(), mpSetCycle(),
mpSetServoPower(), mpSetMasterJob(), mpSetCurJob(),
mpStartJob(), mpHold(), mpWaitForJobEnd(),
mpDeleteJob(), mpGetJogSpeed(), mpGetJogCoord(),
mpIMOV(), mpMOVJ(), mpMOVL(), mpPulseMOVJ(),
mpPulseMOVL(), mpManualMOV()

mpSetCycle

Sets the operation cycle.

■ **Syntax**

```
LONG mpSetCycle (
    MP_CYCLE_SEND_DATA*    sData,
    MP_STD_RSP_DATA*       rData
);
```

■ **Description**

Sets the "operation cycle" for the controller.

For details on the "operation cycle", refer to "Operation Cycle" in chapter 4.1 of the operator's manual of the controller¹⁾.

■ **Parameter**

[sData]

[in] The pointer to the data structure which transmits the cycle mode to be set

- MP_CYCLE_SEND_DATA

[in] The data structure which transmits the cycle mode to be set

```
Syntax:  typedef struct
          {
              SHORT    sCycle;
              CHAR      reserved[2];
          } MP_CYCLE_SEND_DATA;
```

Member: <sCycle>
Cycle

Constant	Description
1	Step
2	1Cycle
3	Auto

1 For the YRC1000:
YRC1000 GENERAL OPERATOR'S MANUAL (RE-CSO-A051)
For the YRC1000micro:
YRC1000micro OPERATOR'S MANUAL (RE-CSO-A058)

10 System Control API

mpSetCycle

[rData]

[out] The pointer to the data structure which receives the error information

- MP_STD_RSP_DATA

[out] The data structure which receives the error information

Syntax: typedef struct
 {
 USHORT err_no;
 CHAR reserved[2];
 } MP_STD_RSP_DATA;

Member: <err_no>
 Error number

Value	Description
0x0000	Normal end
0x2060	In error/alarm status

■ Return Value

- 0 API execution processor received the command.
 Return value = 0 and err_no = 0 Normal end
 Return value = 0 and err_no != 0 Error (Check err_no.)
- 1 API execution processor does not respond.



Exclusive control by using semaphores is executed between this API and the APIs listed below. Thus, if this API is executed while one of the following APIs is already being executed, the task may be suspended to wait for the API to be completed. (waiting for semaphore acquisition)

Therefore, DO NOT use the following APIs for the task which must be performed periodically with high priority.

mpCancelError(), mpResetAlarm(), mpSetCycle(), mpSetServoPower(), mpSetMasterJob(), mpSetCurJob(), mpStartJob(), mpHold(), mpWaitForJobEnd(), mpDeleteJob(), mpGetJogSpeed(), mpGetJogCoord(), mpIMOV(), mpMOVJ(), mpMOVL(), mpPulseMOVJ(), mpPulseMOVL(), mpManualMOV()

mpSetServoPower

Sets the ON/OFF of the servo power.

■ **Syntax**

```
LONG mpSetServoPower (  
    MP_SERVO_POWER_SEND_DATA*    sData,  
    MP_STD_RSP_DATA*              rData  
);
```

■ **Description**

Sets ON (turned ON)/OFF (turned OFF) of the controller servo power. For details on the state where the servo power can be turned ON, refer to chapter 5.2 "Turning On the Servo Power" in the instructions manual of the controller¹⁾.

■ **Parameter**

[sData]

[in] The pointer to the data structure which transmits the ON/OFF of the servo power

- MP_SERVO_POWER_SEND_DATA

[in] The data structure which transmits the ON/OFF of the servo power

```
Syntax:  typedef struct  
        {  
            SHORT    sServoPower;  
            CHAR      reserved[2];  
        } MP_SERVO_POWER_SEND_DATA;
```

Member: <*sServoPower*>
Servo power

Constant	Description
1	Servo power ON
0	Servo power OFF

[rData]

[out] The pointer to the data structure which receives the error information

- MP_STD_RSP_DATA

[out] The data structure which receives the error information

```
Syntax:  typedef struct  
        {  
            USHORT    err_no;  
            CHAR      reserved[2];  
        } MP_STD_RSP_DATA;
```

1 For the YRC1000: YRC1000 INSTRUCTIONS (RE-CTO-A221)
For the YRC1000micro: YRC1000micro INSTRUCTIONS (RE-CTO-A222)

10 System Control API

mpSetServoPower

Member: `<err_no>`
Error number

Value	Description
0x0000	Normal end
0x2060	In error/alarm status
0x3450	Failed (Unable to turn servo on)

■ Return Value

- 0 API execution processor received the command.
Return value = 0 and err_no = 0 Normal end
Return value = 0 and err_no != 0 Error (Check err_no.)
- 1 API execution processor does not respond.



Exclusive control by using semaphores is executed between this API and the APIs listed below. Thus, if this API is executed while one of the following APIs is already being executed, the task may be suspended to wait for the API to be completed. (waiting for semaphore acquisition)

Therefore, DO NOT use the following APIs for the task which must be performed periodically with high priority.

mpCancelError(), mpResetAlarm(), mpSetCycle(),
mpSetServoPower(), mpSetMasterJob(), mpSetCurJob(),
mpStartJob(), mpHold(), mpWaitForJobEnd(),
mpDeleteJob(), mpGetJogSpeed(), mpGetJogCoord(),
mpIMOV(), mpMOVJ(), mpMOVL(), mpPulseMOVJ(),
mpPulseMOVL(), mpManualMOV()

mpSetMasterJob

Registers the specified job as the master job.

■ **Syntax**

```
LONG mpSetMasterJob (  
    MP_MASTER_JOB_SEND_DATA*    sData,  
    MP_STD_RSP_DATA*            rData  
);
```

■ **Parameter**

[sData]

[in] The pointer to the data structure which transmits the master job information

- MP_MASTER_JOB_SEND_DATA

[in] The data structure which transmits the master job information

Syntax: #define MAX_JOB_NAME_LEN (33)

```
typedef struct  
{  
    SHORT    sTaskNo;  
    CHAR     cJobName[MAX_JOB_NAME_LEN];  
    CHAR     reserved[5];  
} MP_MASTER_JOB_SEND_DATA;
```

Member: <sTaskNo>
Task number

Constant	Description
0	Master task
1	Subtask 1
2	Subtask 2
3	Subtask 3
4	Subtask 4
5	Subtask 5
6	Subtask 6
7	Subtask 7
8	Subtask 8
9	Subtask 9
10	Subtask 10
11	Subtask 11
12	Subtask 12
13	Subtask 13
14	Subtask 14
15	Subtask 15

<cJobName[MAX_JOB_NAME_LEN]>
Job name (up to 32 characters for a job name)

10 System Control API

mpSetMasterJob

[rData]

[out] The pointer to the data structure which receives the error information

- MP_STD_RSP_DATA

[out] The data structure which receives the error information

Syntax: typedef struct
 {
 USHORT err_no;
 CHAR reserved[2];
 } MP_STD_RSP_DATA;

Member: <err_no>
 Error number

Value	Description
0x0000	Normal end
0x2060	In error/alarm status
0x2010	Robot is in operation
0x3400	Cannot operate MASTER JOB
0x3410	The JOB name is already registered in another task.
0x4040	Specified JOB not found

■ Return Value

- 0 API execution processor received the command.
 Return value = 0 and err_no = 0 Normal end
 Return value = 0 and err_no != 0 Error (Check err_no.)
- 1 API execution processor does not respond.

NOTE

- Exclusive control by using semaphores is executed between this API and the APIs listed below. Thus, if this API is executed while one of the following APIs is already being executed, the task may be suspended to wait for the API to be completed. (waiting for semaphore acquisition)
 Therefore, DO NOT use the following APIs for the task which must be performed periodically with high priority.

mpCancelError(), mpResetAlarm(), mpSetCycle(),
 mpSetServoPower(), mpSetMasterJob(), mpSetCurJob(),
 mpStartJob(), mpHold(), mpWaitForJobEnd(),
 mpDeleteJob(), mpGetJogSpeed(), mpGetJogCoord(),
 mpIMOV(), mpMOVJ(), mpMOVL(), mpPulseMOVJ(),
 mpPulseMOVL(), mpManualMOV()

- In case a task No. which exceeds the sub task number designated by the independent control function is specified, it is not registered as the master job. To use this function, the independent control function (optional) is required separately.

mpSetCurJob

Sets the job name and line number to the current job of the master task.

■ **Syntax**

```
LONG mpSetCurJob (  
  MP_CUR_JOB_SEND_DATA*      sData,  
  MP_STD_RSP_DATA*           rData  
);
```

■ **Parameter**

[sData]

[in] The pointer to the data structure which transmits the current job information

- MP_CUR_JOB_SEND_DATA

[in] The data structure which transmits the current job information

Syntax: #define MAX_JOB_NAME_LEN (33)

```
typedef struct  
{  
  USHORT   usJobLine;  
  CHAR     cJobName[MAX_JOB_NAME_LEN];  
  CHAR     reserved[5];  
} MP_CUR_JOB_SEND_DATA;
```

- Member: <usJobLine>
Job line
- <cJobName[MAX_JOB_NAME_LEN]>
Job name (up to 32 characters for a job name)

[rData]

[out] The pointer to the data structure which receives the error information

- MP_STD_RSP_DATA

[out] The data structure which receives the error information

```
Syntax: typedef struct  
{  
  USHORT   err_no;  
  CHAR     reserved[2];  
} MP_STD_RSP_DATA;
```

- Member: <err_no>
Error number

Value	Description
0x0000	Normal end
0x2060	In error/alarm status
0x2010	Robot is in operation
0x2110	Inaccessible data
0x4040	Specified JOB not found
0x5200	Over data range

10 System Control API

mpSetCurJob

■ Return Value

- 0 API execution processor received the command.
Return value = 0 and err_no = 0 Normal end
Return value = 0 and err_no != 0 Error (Check err_no.)
- 1 API execution processor does not respond.



Exclusive control by using semaphores is executed between this API and the APIs listed below. Thus, if this API is executed while one of the following APIs is already being executed, the task may be suspended to wait for the API to be completed. (waiting for semaphore acquisition)

Therefore, DO NOT use the following APIs for the task which must be performed periodically with high priority.

mpCancelError(), mpResetAlarm(), mpSetCycle(),
mpSetServoPower(), mpSetMasterJob(), mpSetCurJob(),
mpStartJob(), mpHold(), mpWaitForJobEnd(),
mpDeleteJob(), mpGetJogSpeed(), mpGetJogCoord(),
mpIMOV(), mpMOVJ(), mpMOVL(), mpPulseMOVJ(),
mpPulseMOVL(), mpManualMOV()

mpStartJob

Starts a job.

■ Syntax

```
LONG mpStartJob (
    MP_START_JOB_SEND_DATA*    sData,
    MP_STD_RSP_DATA*           rData
);
```

■ Description

This API executes the job specified for *cJobName* in MP_START_JOB_SEND_DATA from the beginning.

The registration of the master job is also changed to the job specified for *cJobName*.

If *cJobName* is specified as NULL, the job suspended by using the hold function, etc., resumes from where it stopped without changing the registration of the master job. If multiple jobs were running before execution was held, all of the suspended jobs are resumed.

If multiple jobs are operating before the hold function is executed, all of the suspended jobs resume.

■ Parameter

[*sData*]

[in] The pointer to the data structure which transmits the job to be started

- MP_START_JOB_SEND_DATA

[in] The data structure which transmits the job to be started

Syntax: #define MAX_JOB_NAME_LEN (33)

```
typedef struct
{
    SHORT  sTaskNo;
    CHAR    cJobName[MAX_JOB_NAME_LEN];
    CHAR    reserved[5];
} MP_START_JOB_SEND_DATA;
```

Member: <*sTaskNo*>

Task number: Always specify the master task, "0".

<*cJobName*[MAX_JOB_NAME_LEN]>

Job name (up to 32 characters for a job name)

10 System Control API

mpStartJob

[rData]

[out] The pointer to the data structure which receives the error information

- MP_STD_RSP_DATA

[out] The data structure which receives the error information

Syntax: typedef struct
 {
 USHORT err_no;
 CHAR reserved[2];
 } MP_STD_RSP_DATA;

Member: <err_no>
 Error number

Value	Description
0x0000	Normal end
0x2010	Robot is in operation
0x2030	In HOLD status (PP)
0x2040	In HOLD status (External)
0x2050	In HOLD status (Command)
0x2060	In error/alarm status
0x2070	In SERVO OFF status
0x2080	Wrong operation mode
0x3040	The home position is not registered
0x3050	Out of range (ABSO data)
0x3400	When calling the master in play mode is prohibited
0x3410	The JOB name is already registered in another task.
0x4040	Specified JOB not found

■ Return Value

0 API execution processor received the command.
 Return value = 0 and err_no = 0 Normal end
 Return value = 0 and err_no != 0 Error (Check err_no.)

-1 API execution processor does not respond.

■ Note

To execute an arbitrary job without changing the registration of the master job, the API must be called with NULL specified for cJobName. First specify the job name and starting line number with mpSetCurJob() and then call mpStartJob() with NULL specified for cJobName. This executes the job that was specified with mpSetCurJob() from the specified line number.

10 System Control API

mpStartJob

<Sample Code>

The specified job can be executed by calling the following function.

- To change the master job registration and execute the job.

```
void start_job_change_master()
{
    MP_START_JOB_SEND_DATA start_job_data = {sTaskNo = 0,
                                              //Always specify "0".
                                              .cJobName = "RC_JOB" };
                                              //Specify job name to
                                              //execute.

    MP_STD_RSP_DATA          status_job_data;
    LONG                     rc;

    rc = mpStartJob( &start_job_data, &status_job_data );
    // "RC_JOB" is registered for the master job and the job is
    // executed from the beginning.
}
```

- To execute the job without changing the master job registration.

```
void start_job_not_change_master()
{
    MP_CUR_JOB_SEND_DATA set_job_data = {.cJobName = "RC_JOB",
                                         //Specify job name to
                                         //execute.
                                         .usJobLine = 10 };
                                         //Specify line number to
                                         //execute.

    MP_START_JOB_SEND_DATA start_job_data = {sTaskNo = 0,
                                              //Always specify "0".
                                              cJobName = NULL };
                                              //Initialize with NULL.

    MP_STD_RSP_DATA          status_job_data;
    LONG                     rc;

    rc = mpSetCurJob( &set_job_data, &status_job_data );
    //Set line 10 of "RC_JOB" as the current task of
    //the master task.

    rc = mpStartJob( &start_job_data, &status_job_data );
    // "RC_JOB" is executed from line number 10.
    //The master job registration is not changed.
}
```



Exclusive control by using semaphores is executed between this API and the APIs listed below. Thus, if this API is executed while one of the following APIs is already being executed, the task may be suspended to wait for the API to be completed. (waiting for semaphore acquisition)

Therefore, DO NOT use the following APIs for the task which must be performed periodically with high priority.

mpCancelError(), mpResetAlarm(), mpSetCycle(),
 mpSetServoPower(), mpSetMasterJob(), mpSetCurJob(),
 mpStartJob(), mpHold(), mpWaitForJobEnd(),
 mpDeleteJob(), mpGetJogSpeed(), mpGetJogCoord(),
 mpIMOV(), mpMOVJ(), mpMOVL(), mpPulseMOVJ(),
 mpPulseMOVL(), mpManualMOV()

mpHold

Turns ON/OFF the hold function.

■ Syntax

```
LONG mpHold (  
    MP_HOLD_SEND_DATA*    sData,  
    MP_STD_RSP_DATA*      rData  
);
```

■ Parameter

[sData]

[in] The pointer to the data structure which transmits the ON/OFF of the hold function

- MP_HOLD_SEND_DATA

[in] The data structure which transmits the ON/OFF of the hold function

Syntax: typedef struct
{
 SHORT *sHold*;
 CHAR *reserved*[2];
} MP_HOLD_SEND_DATA;

Member: <*sHold*>
Hold status

Constant	Description
1	Hold ON
0	Hold OFF

[rData]

[out] The pointer to the data structure which receives the error information

- MP_STD_RSP_DATA

[out] The data structure which receives the error information

Syntax: typedef struct
{
 USHORT *err_no*;
 CHAR *reserved*[2];
} MP_STD_RSP_DATA;

Member: <*err_no*>
Error number

Value	Description
0x0000	Normal end

■ Return Value

Symbolic constant: OK (value: 0) : Normal end

Symbolic constant: ERROR (value: -1) : Error

10 System Control API

mpHold



Exclusive control by using semaphores is executed between this API and the APIs listed below.

Thus, if this API is executed while one of the following APIs is already being executed, the task may be suspended to wait for the API to be completed. (waiting for semaphore acquisition)

Therefore, DO NOT use the following APIs for the task which must be performed periodically with high priority.

mpCancelError(), mpResetAlarm(), mpSetCycle(),
mpSetServoPower(), mpSetMasterJob(), mpSetCurJob(),
mpStartJob(), mpHold(), mpWaitForJobEnd(),
mpDeleteJob(), mpGetJogSpeed(), mpGetJogCoord(),
mpIMOV(), mpMOVJ(), mpMOVL(), mpPulseMOVJ(),
mpPulseMOVL(), mpManualMOV()

mpWaitForJobEnd

Waits for completion of the job with the specified task number for the specified time.

■ Syntax

```
LONG mpWaitForJobEnd (
    MP_WAIT_JOB_SEND_DATA*    sData,
    MP_STD_RSP_DATA*          rData
);
```

■ Description

This API waits for completion of the job with the specified task number for only the specified time. If the job completes within the specified time, the result is normal end (return value = 0 and err_no = 0). For other values, refer to the err_no result.

■ Parameter

[sData]

[in] The pointer to the data structure which transmits the WaitJOB

- MP_WAIT_JOB_SEND_DATA

[in] The data structure which transmits the WaitJOB

```
Syntax:  typedef struct
          {
            SHORT sTaskNo; /* Task number for which to check
                           completion of job execution */
            SHORT sTime;   /* Time to wait for completion of job execution */
          } MP_WAIT_JOB_SEND_DATA;
```

Member: <sTaskNo>
Task number

Constant	Description
0	Master task
1	Subtask 1
2	Subtask 2
3	Subtask 3
4	Subtask 4
5	Subtask 5
6	Subtask 6
7	Subtask 7
8	Subtask 8
9	Subtask 9
10	Subtask 10
11	Subtask 11
12	Subtask 12
13	Subtask 13
14	Subtask 14
15	Subtask 15

10 System Control API

mpWaitForJobEnd

<*sTime*>

Specify the time to wait for job completion.
Time (unit: sec, from 0 to 32767)

[*rData*]

[out] The pointer to the data structure which receives the error information

- MP_STD_RSP_DATA

[out] The data structure which receives the error information

Syntax:

```
typedef struct
{
    USHORT    err_no;
    CHAR      reserved[2];
} MP_STD_RSP_DATA;
```

Member: <*err_no*>
Error number

Value	Description
0x0000	Normal end
0x2030	In HOLD status (PP)
0x2040	In HOLD status (External)
0x2050	In HOLD status (Command)
0x2060	In error/alarm status
0x2070	In SERVO OFF status
0xFFFF	mpWaitForJobEnd() timed out or the job execution interrupted

■ Return Value

- 0 API execution processor received the command.
 Return value = 0 and err_no = 0 Normal end
 Return value = 0 and err_no != 0 Error (Check err_no.)
- 1 API execution processor does not respond.



Exclusive control by using semaphores is executed between this API and the APIs listed below. Thus, if this API is executed while one of the following APIs is already being executed, the task may be suspended to wait for the API to be completed. (waiting for semaphore acquisition)

Therefore, DO NOT use the following APIs for the task which must be performed periodically with high priority.

mpCancelError(), mpResetAlarm(), mpSetCycle(),
 mpSetServoPower(), mpSetMasterJob(), mpSetCurJob(),
 mpStartJob(), mpHold(), mpWaitForJobEnd(),
 mpDeleteJob(), mpGetJogSpeed(), mpGetJogCoord(),
 mpIMOV(), mpMOVJ(), mpMOVL(), mpPulseMOVJ(),
 mpPulseMOVL(), mpManualMOV()

mpDeleteJob

Deletes the job.

■ Syntax

```
LONG mpDeleteJob (  
    MP_DELETE_JOB_SEND_DATA*    sData,  
    MP_STD_RSP_DATA*            rData  
);
```

■ Parameter

[sData]

[in] The pointer to the data structure which transmits the job name to be deleted

- MP_DELETE_JOB_SEND_DATA

[in] The data structure which transmits the job name to be deleted

Syntax: #define MAX_JOB_NAME_LEN (33)

```
typedef struct  
{  
    CHAR    cJobName[MAX_JOB_NAME_LEN];  
    CHAR    reserved[7];  
} MP_DELETE_JOB_SEND_DATA;
```

Member: <*cJobName*[MAX_JOB_NAME_LEN]>
Job name (up to 32 characters for a job name)

[rData]

[out] The pointer to the data structure which receives the error information

- MP_STD_RSP_DATA

[out] The data structure which receives the error information

```
Syntax: typedef struct  
{  
    USHORT    err_no;  
    CHAR      reserved[2];  
} MP_STD_RSP_DATA;
```

Member: <*err_no*>
Error number

Value	Description
0x0000	Normal end
0x2010	Robot is in operation; or cannot delete JOBS during the robot operation
0x2060	In error/alarm status
0x2080	Wrong operation mode
0x4020	Edit lock job
0x4040	Specified JOB not found

10 System Control API

mpDeleteJob

■ Return Value

- | | |
|----|---|
| 0 | API execution processor received the command.
Return value = 0 and err_no = 0 Normal end
Return value = 0 and err_no != 0 Error (Check err_no.) |
| -1 | API execution processor does not respond. |



Exclusive control by using semaphores is executed between this API and the APIs listed below. Thus, if this API is executed while one of the following APIs is already being executed, the task may be suspended to wait for the API to be completed. (waiting for semaphore acquisition)

Therefore, DO NOT use the following APIs for the task which must be performed periodically with high priority.

mpCancelError(), mpResetAlarm(), mpSetCycle(),
mpSetServoPower(), mpSetMasterJob(), mpSetCurJob(),
mpStartJob(), mpHold(), mpWaitForJobEnd(),
mpDeleteJob(), mpGetJogSpeed(), mpGetJogCoord(),
mpIMOV(), mpMOVJ(), mpMOVL(), mpPulseMOVJ(),
mpPulseMOVL(), mpManualMOV()

mpSetJogCoord

Sets the operation coordinates for the specified control group.

■ Syntax

```
LONG mpSetJogCoord(
    MP_JOGCOORD_SEND_DATA*    sData,
    MP_STD_RSP_DATA*          rData
)
```

■ Parameter

[*sData*]

[in] The pointer to the data structure which sends the operation coordinates

- MP_JOGCOORD_SEND_DATA

[in] The data structure which sends the operation coordinates

Syntax:

```
typedef struct
{
    ULONG    sCtrlGrp;        /* control group to set the operation coordinate */
    SHORT    sJogCoord;       /* operation coordinate to be set */
    SHORT    sUsrFrmNo;       /* user coordinate number */
} MP_JOGCOORD_SEND_DATA;
```

Member:

<*sCtrlGrp*>

The control groups which specifies the operation coordinates

Symbolic constant	Description
MP_R1_GID	R1
MP_R2_GID	R2
MP_R3_GID	R3
MP_R4_GID	R4
MP_R5_GID	R5
MP_R6_GID	R6
MP_R7_GID	R7
MP_R8_GID	R8
MP_B1_GID	B1
MP_B2_GID	B2
MP_B3_GID	B3
MP_B4_GID	B4
MP_B5_GID	B5
MP_B6_GID	B6
MP_B7_GID	B7
MP_B8_GID	B8

10 System Control API

mpSetJogCoord

<sJogCoord>

The operation coordinates to be set.

Symbolic constant	Description
MP_JOINT_COORD	Joint coordinates
MP_RECT_COORD	Cartesian coordinates
MP_CYLIND_COORD	Cylindrical coordinates
MP_TOOL_COORD	Tool coordinates
MP_USER_COORD	User coordinates
MP_EXT_REF_COORD	External reference point coordinates

<sUsrFrmNo>

The user coordinates number (1~63)

Sets the user coordinates number in case the operation coordinate is set as "User coordinates" or "External reference point coordinates".

[rData]

[out] The pointer to the structure of the standard received data

- MP_STD_RSP_DATA

[out] The structure of the standard received data

```
Syntax:  typedef struct
          {
            USHORT   err_no;
            CHAR      reserved[2];
          } MP_STD_RSP_DATA
```

Member: <err_no>
Error number

Constant	Description
0x0000	Normal end
0x2010	Manipulator is in operation
0x2060	In error/alarm status
0x2080	Wrong operation mode
0x3350	User coordinate is not registered
0x3360	Error in user coordinate file
0x3370	Wrong control group
0x3460	Coordinate system is not set
0x3600	User coordinate number is out of range

■ Return Value

0 : Normal end
None zero : Error

10 System Control API

mpSetJogCoord



- Operation coordinates system needs to be specified to each control group.
 - The initial value of the operation coordinate is set as “Link Coordinate” for every groups. When the control power of the controller is turned ON, the operation coordinate system setting turns back to initial value.
 - In the case of the followings, error number:0x3460 (Coordinate system is not set) occurs and the coordinate system cannot be changed.
 - The “external reference point” is specified while the “external reference point (optional)” is invalid.
 - In the “select cartesian/cylindrical” (=parameter: S2C196), which is to be specified in the teaching condition,
 - The “cylindrical coordinate” is specified while the “cartesian” (S2C196=1) is selected.
 - The “cartesian coordinate” is specified while the “cylindrical” (S2C196=0) is selected.
 - In the “coordinate switch prohibited” (=parameter: S2C197),
 - The “tool coordinate” is specified while the “tool coordinate switch prohibited” (S2C197=1 or 3) is selected.
 - The “user coordinate” is specified while the “user coordinate switch prohibited” (S2C197=2 or 3) is selected.
- S2C197:coordinate switch prohibited
- 0: user/tool coordinate switch is available.
 - 1: tool coordinate switch is prohibited.
 - 2: coordinate switch is prohibited.
 - 3: user/tool coordinate switch is available.

mpSetToolNo

Sets the jog tool number for the specified control group.

■ **Syntax**

```
LONG mpSetToolNo(  
    MP_SET_TOOL_NO_SEND_DATA*  sData,  
    MP_STD_RSP_DATA*           rData  
)
```

■ **Parameter**

[sData]

[in] The pointer to the data structure which sends jog tool numbers

- MP_SET_TOO_NO_SEND_DATA

[in] The data structure which sends jog tool numbers

Syntax:

```
typedef struct  
{  
    ULONG   sRobotNo;    /* robot number to set the jog tool number*/  
    SHORT   sToolNo;     /* jog tool number to be set */  
    SHORT   reserved[2];  
} MP_SET_TOOL_NO_SEND_DATA;
```

Member:

<sRobotNo>

The control group which sets the operation coordinate

Value	Description
MP_R1_GID	R1
MP_R2_GID	R2
:	:
MP_R8_GID	R8

<sToolNo>

The jog tool number to be set (0~63).

[rData]

[out] The pointer to the structure of the standard received data

- MP_STD_RSP_DATA

[out] The structure of the standard received data

Syntax:

```
typedef struct  
{  
    USHORT   err_no;  
    CHAR     reserved[2];  
} MP_STD_RSP_DATA
```

10 System Control API

mpSetToolNo

Member: <err_no>
Error number

Value	Description
0x0000	Normal end
0x2010	Robot is in operation
0x2060	In error/alarm status
0x2080	Wrong operation mode
0x3370	Wrong control group (Setting error in sRobotNo)
0x3470	Tool is not changed

■ Return Value

0: Normal end
None zero: Error



- Jog tool number is memorized to each robot, so it needs setting each.
- Initial values are set as R1:tool#0, R2:tool#1, ... etc.
The tool number is memorized even if the control power is turned OFF.
- When “tool number switch: invalid” (=S2C431: tool switch permit specification=0) is specified, error number: 0x3470 (Tool is not changed) appears, and tools are not switched.

mpSetJobPosData

Changes the teaching position data of the step number in the specified job name.

■ Syntax

```
LONG mpSetJobPosData(
    MP_JOB_POS_SEND_DATA*   sData
    /* the data structure which specifies the teaching position */
    MP_SET_JOB_POS_RSP_DATA* rData
    /* the data structure which receives the change result of teaching position data */
);
```

■ Parameter

[sData]

[in] The pointer to the data structure which specifies the teaching position

- MP_JOB_POS_SEND_DATA

Syntax:

```
typedef struct{
    USHORT    usStep;          /* step number */
    CHAR      reserved0[2];
    CHAR      cJobName[MAX_JOB_NAME_LEN]; /* job name*/
    CHAR      reserved1[3];
    LONG      posNum;          /*teaching position data number */
    MP_JOB_POS_DATA posData[MAX_JOB_POS_NUM];
                                /* teaching position data structure */
} MP_JOB_POS_SEND_DATA;
#define MAX_JOB_NAME_LEN 33 /* up to 32 characters + null character '\0' */
#define MAX_JOB_POS_NUM 12 /* maximum teaching position data number */
```

Member: <usStep>

step number (1 to 99988)

<cJobName[MAX_JOB_NAME_LEN]>

job name (up to 32 characters)

<posNum>

teaching position data number

Set the number of all the teaching position data in step.

<posData>

teaching position data structure

- MP_JOB_POS_DATA

teaching position data structure

Syntax:

```
typedef struct{
    CTRLG_T    ctrl_grp; /* control group */
    LONG       posType; /* position data type */
    CTRLG_T    varIndex; /* position variable number */
    ULONG      attr; /* data attribute */
    ULONG      attrExt; /* data attribute (expansion) */
}
```

10 System Control API

mpSetJobPosData

```
LONG      pos[MP_GRP_AXES_NUM];    /* position data */
} MP_JOB_POS_DATA;
#define MAX_GRP_AXES_NUM  8      /* maximum control axis number*/
```

<ctrl_grp>
control group

Constant	Description
0 to 7	R1(Robot1) to R8(Robot8)
8 to 15	B1(Base1) to B8(Base8)
16 to 39	S1(Station1) to S24(Station24)

<posType>

position data type

A local position variable cannot be specified.

When the permission to change the local variable (S2C1115=1) in mpSetJobPosData(), the local variable can be changed.

When the prohibition to change the local variable (S2C1115=0) in mpSetJobPosData(), the local variable cannot be changed

Constant	Description	Definition
0	Position data	JOB_POS_TYPE_POS
1	Position variable	JOB_POS_TYPE_VAR

<varIndex>

position variable number

When posType=1 (position variable), the index for the position variable data to be changed is specified.

When posType=0(position data), the position variable number is not used.

<attr><attrExt><pos>

data attribute, data attribute (expansion), position data

The data structure is the same as that of mpPutPosVarData().

Member name		Description	
	mpPutPosVarData()	Bit No.	Contents
attr	rData[0]	D05-D00	Variable type 0 Pulse 16 Cartesian (base coordinates) 17 Cartesian (robot coordinates) 18 Cartesian (tool coordinates) 19 Cartesian (user coordinates) 20 Cartesian (master tool)
		D07-D06	Reserved by manufacturer
		D08	0:Front 1:Back
		D09	0:Upper arm 1:Lower arm
		D10	0:Flip 1:No flip
		D11	0:R<180deg 1:R>=180deg
		D12	0:T<180deg 1:T>=180deg
		D13	0:S<180deg 1:S>=180deg
		D14-D15	Reserved by manufacturer

10 System Control API

mpSetJobPosData

Member name		Pulse type		Cartesian type
	mpPutPosVarData()			
attr	rData[0]	D16-D21	Tool number(0-63)	
		D22-D27	User coordinate number	
		D28-D31	Reserved by manufacturer	
attrExt	rData[1]		(Expansion attribute)	
pos[0]	rData[2]	1st axis(S) pulse value	X-axis coordinate(unit:micron)	
pos[1]	rData[3]	2nd axis(L) pulse value	Y-axis coordinate(unit:micron)	
pos[2]	rData[4]	3rd axis(U) pulse value	Z-axis coordinate(unit:micron)	
pos[3]	rData[5]	4th axis(R) pulse value	Wrist angle Rx(unit:0.0001deg)	
pos[4]	rData[6]	5th axis(B) pulse value	Wrist angle Ry(unit:0.0001deg)	
pos[5]	rData[7]	6th axis(T) pulse value	Wrist angle Rz(unit:0.0001deg)	
pos[6]	rData[8]	7th axis(E) pulse value	angle Re(unit:0.0001deg)	
pos[7]	rData[9]	8th axis pulse value	8th axis pulse value(micron in the case of traveling axis)	

[rData]

[out] The pointer to the data structure which receives the change result of teaching position data

- MP_SET_JOB_POS_RSP_DATA

Syntax:

```
typedef struct{
    USHORT    err_no;           /* error number */
    USHORT    err_sub;         /* error sub-number */
    USHORT    err_index;       /* error status index */
    CHAR      reserved[2];
} MP_SET_JOB_POS_RSP_DATA;
```

Member: <err_no>
error number

Value	Description
0x0000	Normal end
0x2010	Robot is in operation
0x2060	In error/alarm status
0x2080	Wrong operation mode
0x3370	Wrong control group
0x3470	Tool is not changed
0x3600	User coordinate number is out of range
0x3620	Error in specifying tool file number
0x3630	Error in data type
0x4020	Edit lock job
0x4040	Specified JOB not found
0x4130	No position data
0x4140	Wrong position data type

10 System Control API

mpSetJobPosData

Value	Description
0x5200	Out of data range
0x5210	Data does not coincide
0x6052	Unable to change/delete this line

<err_sub>
error sub-number

Error number	Error sub-number	Description
0x3630	0x0010	Error in specifying data type
	0x0020	Data type does not coincide between manipulator and base
	0x0030	Error in specifying master tool coordinate
0x5200	0x0010	Error in specifying step number
	0x0040	Error in specifying position data type
	0x0100	Error in specifying position data (1st axis (S) pulse value)
	0x0101	Error in specifying position data (2nd axis (L) pulse value)
	0x0102	Error in specifying position data (3rd axis (U) pulse value)
	0x0103	Error in specifying position data (4th axis (R) pulse value)
	0x0104	Error in specifying position data (5th axis (B) pulse value)
	0x0105	Error in specifying position data (6th axis (T) pulse value)
	0x0106	Error in specifying position data (7th axis (E) pulse value)
	0x0107	Error in specifying position data (8th axis pulse value)
	0x0110	Error in specifying position data (Cartesian type)
0x5210	0x0020	Teaching position data number does not coincide
	0x0030	Control group does not coincide
	0x0040	Position data type does not coincide
	0x0050	Position variable number does not coincide
	0x0060	User coordinate number does not coincide

<err_index>
error status index
The index which has an error in posData (the teaching position data structure) is specified.

- **Return Value**
 - Symbolic constant: OK (value: 0) : Normal end
 - Symbolic constant: ERROR (value: -1) : Error
(For details, check err_no.)

mpParallelShiftJob

Shifts the specified step by the same deviation and converts the job.

■ Syntax

```
LONG mpParallelShiftJob(
    MP_PARALLEL_SHIFT_JOB_SEND_DATA*  sData,
    /* the data structure which specifies the parallel shift job conversion */
    MP_STD_RSP_DATA*                  rData
    /* the data structure which receives the error information */
);
```

■ Parameter

[sData]

[in] The pointer to the data structure which specifies the parallel shift job conversion

- MP_PARALLEL_SHIFT_JOB_SEND_DATA

Syntax:

```
typedef struct{
    CHAR      srcJobName[MAX_JOB_NAME_LEN]; /* source job name */
    CHAR      reserved0[3];
    CHAR      dstJobName[MAX_JOB_NAME_LEN]; /* destination job name*/
    CHAR      reserved1[3];
    USHORT    usStartStep;                  /* conversion start step */
    USHORT    usEndStep;                    /* conversion end step */
    USHORT    usCoord;                      /* conversion coordinate */
    USHORT    usUsrFrmNo;                   /* user coordinate number */
    USHORT    usStationUnit;                /* station data unit */
    CHAR      reserved2[2];
    LONG      shiftNum;                      /* shift data number */
    MP_SHIFT_VALUE_DATA shiftData[MAX_SHIFT_VALUE_NUM];
    /* shift data structure */
} MP_PARALLEL_SHIFT_JOB_SEND_DATA;
#define MAX_JOB_NAME_LEN 33 /* up to 32 characters + null character '\0' */
#define MAX_SHIFT_VALUE_NUM 12 /* maximum shift data number */
```

Member: <srcJobName[MAX_JOB_NAME_LEN]>
source job name (up to 32 characters)

<dstJobName[MAX_JOB_NAME_LEN]>
destination job name (up to 32 characters)

<usStartStep>
conversion start step number(1 to 9998)

<usEndStep>
conversion end step number(1 to 9998)

<usCoord>
conversion coordinate
0: pulse
16: base
17: manipulator
18: tool

10 System Control API

mpParallelShiftJob

19: user
20: master tool

<usUsrFrmNo>
user coordinate number (1 to 63)

<usStationUnit>
future function (Set 0.)

<shiftNum>
shift data number
Set the number of control groups in the source job.

only R1: 1
R1+B1: 2
R1+R2: 2

• MP_SHIFT_VALUE_DATA

```
typedef struct{
    CTRLG_T      ctrl_grp;          /* control group*/
    LONG          val[MP_GRP_AXES_NUM]; /* shift data*/
} MP_SHIFT_VALUE_DATA;
#define MAX_GRP_AXES_NUM 8 /* maximum control axis number */
```

<ctrl_grp>
control group

Constant	Description
0 to 7	R1(Robot1) to R8(Robot8)
8 to 15	B1(Base1) to B8(Base8)
16 to 39	S1(Station1) to S24(Station24)

<val>
shift data

Shift data	Pulse type	Cartesian type
val[0]	1st axis (S) pulse value	X-axis coordinate (unit: micron)
val[1]	2nd axis (L) pulse value	Y-axis coordinate (unit: micron)
val[2]	3rd axis (U) pulse value	Z-axis coordinate (unit: micron)
val[3]	4th axis (R) pulse value	Wrist angle Rx(unit:0.0001deg)
val[4]	5th axis (B) pulse value	Wrist angle Ry(unit:0.0001deg)
val[5]	6th axis (T) pulse value	Wrist angle Rz(unit:0.0001deg)
val[6]	7th axis (E) pulse value	angle Re(unit:0.0001deg)
val[7]	8th axis pulse value	8th axis pulse value (micron in the case of traveling axis)

10 System Control API

mpParallelShiftJob

[rData]

[out] The pointer to the data structure which receives the error information

- MP_STD_RSP_DATA

Syntax:

```
typedef struct{
    USHORT          err_no;          /* error number */
    CHAR            reserved[2];
} MP_STD_RSP_DATA;
```

Member: <err_no>
error number

Constant	Description
0x0000	Normal end
0x2010	Robot is in operation
0x2060	In error/alarm status
0x2080	Wrong operation mode
0x2090	Data being accessed by another function
0x3350	User coordinate is not registered
0x3370	Wrong control group
0x3600	User coordinate number is out of range
0x3630	Error in data type
0x4020	Edit lock job
0x4030	The JOB name is already registered in another task.
0x4031	Error in job name
0x4040	Specified JOB not found
0x5200	Out of data range
0x5210	Data does not coincide
0x6010	Lack of job capacity

■ Return Value

- 0: API execution processor received the command.
Return value = 0 and err_no = 0: Normal end
Return value = 0 and err_no != 0: Error (Check err_no.)
- 1: API execution processor does not respond.

10 System Control API

mpParallelShiftJob



- The parallel shift job conversion using the conversion reference point is not available.
- The shift amount specification of the parallel shift job conversion (parameter S2C652) is not available.
- The position type variable used in the job is not subject to the parallel shift job conversion.
- The following jobs are not converted.
 - A job without a group
 - A concurrent job (option)
- The conversion process may take time depending on the number of step registered in job.
- The following operation cannot be performed in the conversion process.
 - Job execution
 - Displaying the job window (When the conversion process is executed while the job window is displayed, the job window is closed and a blank window is displayed.)
- The following sub menus are concealed. (job contents, select job, create new job, call master job, supervisory and select micro job)
- If the operation mode is changed in the conversion process, the process is canceled. (The conversion cannot be executed in the play mode. In addition, the contents in the middle of conversion are abandoned.)

mpApplicationInfoNotify

Notify MotoPlus application information to the controller.

The notified application information is displayed on the version information window of the programming pendant.

■ Syntax

```
int mpApplicationInfoNotify(
    MP_APPINFO_SEND_DATA*    sData,
    MP_STD_RSP_DATA*          rData
);
```

■ Parameter

[sData]

[in] The pointer to the data structure which sends application information

- MP_APPINFO_SEND_DATA

[in] The data structure which sends application information

Syntax:

```
#define MP_MAX_APP_NAME 32 // Max length of characters for application name
#define MP_MAX_APP_VERSION 16 // Max length of characters for version number
#define MP_MAX_APP_COMMENT 32 // Max length of characters for comment
```

typedef struct

```
{
    CHAR    AppName[MP_MAX_APP_NAME+4]; /* Name of Application */
    CHAR    Version[MP_MAX_APP_VERSION+4]; /* VersionNo. */
    CHAR    Comment[MP_MAX_APP_COMMENT+4]; /* Comment */
    CHAR    reserved[36];
}
```

Member:

<AppName>

Application name Character strings of max 32 characters

<Version>

Version number Character strings of max 16 characters

<Comment>

Comment (free space) Character strings of max 32 characters

[rData]

[out] The pointer to the structure of the standard received data

- MP_STD_RSP_DATA

[out] The structure of the standard received data

Member: <err_no>

Error number

Normal end 0x0000

Error 0x0001 The number of characters is exceeded.

 0x0002 Application name is not input.

10 System Control API

mpApplicationInfoNotify

■ Return Value

0: Normal end
-1: Error

■ Note

Only ASCII characters can be displayed.

11 Motion Control API

Using the Motion Control API can achieve the same motion as when movement instructions are continuously executed in a job.

However, when using JOB functions, it may actually turn out to be easier to build the overall application by using job movement instructions. Consider the suitability of the function in the final application, and then review whether this service or a job is optimal for movement instructions.

If the goal is to specify only the target value of a movement instruction with MotoPlus, this can also be implemented by using a position variable as the target value of a job movement instruction and changing that position variable with MotoPlus.

Additionally, the Motion Control API cannot be executed while a job is running. Do not operate a robot with a job in a system that uses the Motion Control API.

mpMotStart

Starts movement to the sent targets (motion command information for multiple target positions).

■ Syntax

```
int mpMotStart
(
    int options /* option */
)
```

■ Description

This API starts robot movement by starting execution of the sent targets (motion command information for multiple target positions) stored in the buffer. The target that has been executed(arrived) is deleted, then the next target is executed. When all the stored targets are executed, the manipulator stops operation. When new targets are sent and stored in the buffer, the manipulator starts operation automatically.

■ Parameter

[options]

Specify 0.

■ Return Value

≥ 0	Success
< 0	Failure

Negative value means failure.

11 Motion Control API

mpMotStart

Returned Value	Meaning	Remedy
E_MP_MOT_FAILURE (-1)	Error other than those listed below	
E_MP_MOT_SVOFF (-2)	Servo power is OFF.	Turn ON the servo power.
E_MP_MOT_HOLD (-3)	In HOLD status.	Release the HOLD status.
E_MP_MOT_ALARM_OCCUR (-4)	In alarm or error status.	Release the alarm or error.
E_MP_MOT_HOME_UNSET (-5)	Home position is unset.	Register the home position.
E_MP_MOT_HOME_UNCHK (-6)	Position confirmation is undone.	Move to the second position, and confirm the position.
E_MP_MOT_START_UNREADY (-7)	Not in play mode.	Set to the play mode.
E_MP_MOT_START_COLLISION (-8)	During job playback.	Start operation after the job playback.
E_MP_MOT_GRP_DUPLICATE (-9)	Duplicates to the control group of the job.	Select job or master job.

mpMotStop

Stops the operation(target execution).

■ Syntax

```
int  mpMotStop
(
    int    options /* option */
)
```

■ Description

This routine stops the manipulator's operation and the successive execution of targets.

■ Parameter

[options]

Specify 0.

■ Return Value

≥ 0	Success
< 0	Failure

mpMotTargetClear

Clears the target already sent.

■ Syntax

```
int    mpMotTargetClear
(
    CTRLG_T    grp,    /* target control group */
    int         options /* option */
)
```

■ Description

This routine clears the target buffer of the control group that is specified in *grp*.

■ Parameter

[*grp*]

Specify the target control group. Each bit indicates the control group. Specify in bits by using the control group number retrieved by the mpCtrlGrpId2GrpNo().

Bit	Meaning
d0	The first control group
d1	The second control group
d2	The third control group
d3	The fourth control group

[*options*]

Specify 0.

■ Return Value

≥ 0	Success
< 0	Failure

mpMotTargetSend

Sends a target (motion command information to a target position (e.g., interpolated motion type and target position information)).

■ Syntax

```
int mpMotTargetSend
(
    CTRLG_T   grp,      /* target control group */
    MP_TARGET *target,  /* target pointer */
    int        timeout  /* waiting time before the completion of sending */
)
```

■ Description

This routine sends the target to the control group specified in *grp*.

The target is the motion command information that includes the target point ID, interpolated motion type, and target point position information.

The target point ID is used in mpMotTargetReceive() to check arrival at the target point.

The *target* is the head pointer of target array. It composes the array by making the first control group being the first element and the second control group being the second element,..., then sends each head pointer.

In the system with only a manipulator(R1), sends the pointer of the target to be sent.

In the system with multiple control groups, sends the head pointer of the array to be sent. This API refers to the element that *grp* indicates.

When specify the 0x02 to *grp*, the array that *target* indicates needs to have at least two elements.

timeout is the waiting time(the number of ticks) before completion of sending. If the buffer is full, waits until there is a vacancy.

■ Parameter

[grp]

Specify the target control group. Each bit indicates the control group. Specify in bits by using the control group number retrieved by the mpCtrlGrpId2GrpNo().

Bit	Meaning
d0	The first control group
d1	The second control group
d2	The third control group
d3	The fourth control group

11 Motion Control API

mpMotTargetSend

[target]

Specify the pointer that indicates the target to be sent (or the array if there are multiple control groups).

```
typedef struct {
    int      id,                /* target ID. */
    MP_INTP_TYPE  intp;        /* interpolation type. */
    MP_POS  dst;               /* destination position. */
    MP_POS  aux;               /* passing(auxiliary) position. */
} MP_TARGET;

typedef enum {
    MP_MOV_NOP_TYPE,          /* NOP */
    MP_MOVJ_TYPE,             /* link interpolation type. */
    MP_MOVL_TYPE,             /* linear interpolation type. */
    MP_MOVC_TYPE,             /* circular interpolation type. */
} MP_INTP_TYPE;

typedef union {
    MP_COORD  coord;
    MP_JOINT  joint;
} MP_POS;

typedef struct {
    long      x, y, z;         /* XYZ position(0.001[mm]) */
    long      rx, ry, rz;      /* RxRyRz rotation(0.0001[deg]) */
    long      ex1, ex2;
} MP_COORD;

typedef long  MP_JOINT[MP_GRP_AXES_NUM]; /* pulse or angle(0.0001[deg]) */
```

NOTE

The angle can be specified in MP_JOINT. However, for robot controllers up to the DX200, there are axes on which the direction of the angle can be different from the one of the pulses. For the YRC1000/YRC1000micro, however, the direction of the angle is always the same as the direction of the pulses. For this reason, an application that moves a robot using absolute angles may reverse the movement direction of the robot if the DX200 program code runs on the YRC1000/YRC1000micro unmodified. When porting DX200 code to the YRC1000/YRC1000micro, check the direction of robot movement and correct the code if necessary.

A measure is available to keep the source code for DX200 and earlier robot controllers and for the YRC1000/YRC1000micro the same. Set parameter S2C1350 in the YRC1000/YRC1000micro to 0 (default value = 1) to use the same specification as the DX200 and earlier robot controllers. Then, the positive direction of the angle and the positive direction of the pulses for each axis become the same as those for the DX200.

[timeout]

Specify the waiting time in ticks. Able to specify "NO_WAIT" (doesn't wait more than the set number of ticks) or "WAIT_FOREVER" (keep waiting forever).

NO_WAIT	Doesn't wait
WAIT_FOREVER	Keep waiting forever

11 Motion Control API

mpMotTargetSend

■ Return Value

≥ 0 Success

< 0 Failure

Negative value means failure.

Return Value	Meaning	Remedy
E_MP_MOT_FAILURE (-1)	Error other than those listed below	
E_MP_MOT_TIMEOUT (-16)	Exceeded the waiting time.	Lengthen in waiting time. Execute or clear target to enable to send targets.
E_MP_MOT_GRP_NOT_EXIST (-32)	Specified control group doesn't exist.	Specify the correct control group.
E_MP_MOT_BASE_LACK (-33)	The base axis control group is lacking. The manipulator with base axis needs the target of both manipulator and the base axis.	Send the target to both manipulator and the base axis.
E_MP_MOT_INTP_OUTRNG (-34)	The interpolation command is out of range. The external axis cannot be moved by the APIs except for MOVJ(joint interpolation).	Specify the correct interpolation command.
E_MP_MOT_BASE_TASK_UNMATCH (-37)	The task number of the manipulator with base axis doesn't accord with the task number of the base axis.	Specify a same task number to the manipulator and the base axis.
E_MP_MOT_MASTER_INTP_OUTRNG (-40)	The coordinated motion is unable. The interpolation command of the master is out of range.	Specify MOVJ (joint interpolation) or MOVL (linear interpolation) to the master.
E_MP_MOT_SLAVE_INTP_OUTRNG (-41)	The coordinated motion is unable. The interpolation command of the slave is out of range.	Specify APIs except for MOVJ (joint interpolation) to the slave.
E_MP_MOT_MASTER_LACK (-42)	The coordinated motion is unable. Targets of master and slave are required for coordinated motion.	Send targets to the master and the slave.
E_MP_MOT_SYNC_TASK_UNMATCH (-43)	The coordinated motion is unable. The task number of the master doesn't accord with the task number of the slave.	Specify the same task number to the master and the slave.
E_MP_MOT_SYNC_ILLEGAL (-44)	Specification for coordinated motion is wrong. Multiple masters are specified to one task.	Specify one master to one task.

mpMotTargetReceive

Receives the report of having arrived to the target position.

■ Syntax

```
int mpMotTargetReceive
(
    int    grpNo,      /* control group number */
    int    id,         /* target ID to receive */
    int *   recvdId,   /* received target ID */
    int    timeout,    /* waiting time for the completion of receiving */
    int    options     /* option */
)
```

■ Description

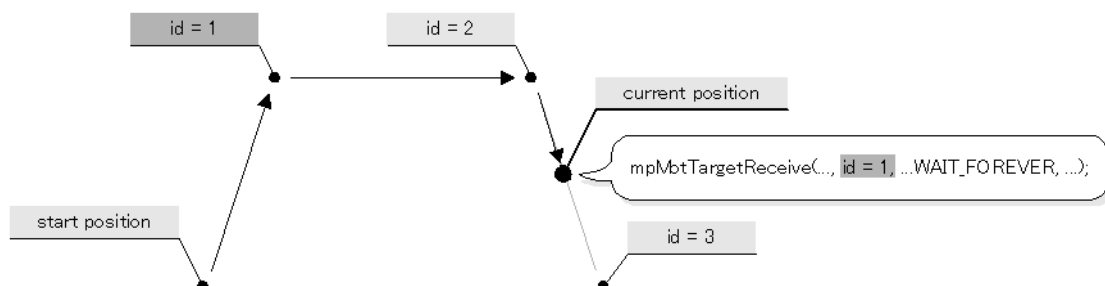
This routine receives the report of the control group specified in *grp* having arrived to the target of *id*. The received target ID is stored in the *recvdId*.

Specify the waiting time before receiving to *timeout*.

If operation is interrupted because of some factor (*mpMotTargetStop()*, emergency stop, alarm, etc.), the waiting status is canceled, and *E_MP_NOT_INTERRUPT* is returned.

Not all the arriving history, but only the latest arriving report can be received. The following figure shows the movement of manipulator (having arrived to *id=1*, then *id=2*, and moving toward *id=3*). At this time, if specify *id=1* and then call this API, receiving report is not done immediately and wait for the next arriving *id=1*. This is because *id=1* has already been arrived but the latest arrived target is *id=2*.

On the other hand, if specify *id=2* while moving toward *id=3*, receives the report immediately because the *id=2* is the latest arrived target.



11 Motion Control API

mpMotTargetReceive

■ Parameter

[grpNo]

Specify the target control group.

Specify the control group number retrieved by the mpCtrlGrpId2GrpNo().

Value	Meaning
0	The first control group
1	The second control group
2	The third control group
3	The fourth control group

[id]

Specify the target ID to receive.

[recvId]

Specify the destination of the received target ID.
NULL can be also specified.

[timeout]

Specify the waiting time in ticks. Able to specify "NO_WAIT" (doesn't wait more than the set number of ticks) or "WAIT_FOREVER" (keep waiting forever).

NO_WAIT End immediately even if the report is not received

WAIT_FOREVER Keep waiting without timeout

[options]

Specify 0.

11 Motion Control API

mpMotTargetReceive

■ Return Value

≥ 0 Success

< 0 Failure

Negative value means failure.

Return Value	Meaning	Remedy
E_MP_MOT_FAILURE (-1)	Error other than those listed below	
E_MP_MOT_TIMEOUT (-16)	Exceeded the waiting time.	Lengthen in waiting time. Specify the receivable target ID.
E_MP_MOT_INTERRUPT (-17)	Operation was interrupted.	Take steps not to occur the factor for interruption.
E_MP_MOT_GRP_NOT_EXIST (-32)	Specified control group doesn't exist.	Specify the correct control group.

mpMotTargetGetVa- cantBufNum

Get the number of targets that can be sent.

■ **Syntax**

```
int mpMotTargetGetVacantBufNum
(
    int grpNo,          /* control group number */
    int *vacantTargetNum /* number of targets that can be sent */
)
```

■ **Description**

This API gets the number of targets that can be sent to the control group specified in *grpNo*.

When a target is sent with mpMotTargetSend(), it is stored in the buffer inside the controller. If mpMotTargetSend() is executed when the buffer is full, execution waits for the buffer to empty for the amount of time specified by timeout in mpMotTargetSend().

The number of targets that can be sent with the remaining space in the buffer can be obtained by executing mpMotTargetGetVacantBufNum().

0 is obtained when the buffer is full and no targets can be sent.

■ **Parameter**

[grpNo]

Specify the target control group.

Specify the control group number retrieved by the mpCtrlGrpId2GrpNo().

Value	Meaning
0	The first control group
1	The second control group
2	The third control group
3	The fourth control group

[vacantTargetNum]

Specify the location to store the number of targets that can be sent.

11 Motion Control API

mpMotTargetGetVacantBufNum

■ Return Value

≥ 0	Success
< 0	Failure

Negative value means failure.

Return Value	Meaning	Remedy
E_MP_MOT_FAILURE (-1)	Error other than those listed below.	
E_MP_MOT_GRP_NOT_EXIST (-32)	Specified control group doesn't exist.	Specify the correct control group.

mpMotSetCoord

Specifies the coordinate system of the target position.

■ Syntax

```
int mpMotSetCoord
(
    int      grpNo, /* control group number */
    MP_COORD_TYPE type, /* target ID to receive */
    int      aux    /* option */
)
```

■ Description

Set the type of coordinate system to the control group specified in *grp*. The set details are reflected in the target that is sent after this API is called.

■ Parameter

[grpNo]

Specify the target control group.

Specify the control group number retrieved by the mpCtrlGrpId2GrpNo().

Constant	Meaning
0	The first control group
1	The second control group
2	The third control group
3	The fourth control group

[type]

Specify the type of the coordinate system.

Symbolic constant	Meaning	Auxiliary Data
MP_PULSE_TYPE	Pulse	Not used
MP_ANGLE_TYPE	Angle	Not used
MP_BASE_TYPE	Base coordinate system	Not used
MP_ROBOT_TYPE	Robot System	Not used
MP_USER_TYPE	User coordinate system	User coordinate system number 0~15(#01~#16)

After the controller starts, pulse (MP_PULSE_TYPE) is set as the default value for coordinate type for all groups.

11 Motion Control API

mpMotSetCoord



Angle can also be specified for the coordinate type. However, for robot controllers up to the DX200, there are axes on which the direction of the angle can be different from the one of the pulses. For the YRC1000/YRC1000micro, however, the direction of the angle is always the same as the direction of the pulses. For this reason, an application that moves a robot using absolute angles may reverse the movement direction of the robot if the DX200 program code runs on the YRC1000/YRC1000micro unmodified. When porting DX200 code to the YRC1000/YRC1000micro, check the direction of robot movement and correct the code if necessary.

A measure is available to keep the source code for DX200 and earlier robot controllers and for the YRC1000/YRC1000micro the same. Set parameter S2C1350 in the YRC1000/YRC1000micro to 0 (default value = 1) to use the same specification as the DX200 and earlier robot controllers. Then, the positive direction of the angle and the positive direction of the pulses for each axis become the same as those for the DX200.

[aux]

Specify the auxiliary data of the type of coordinate system.

When specify the user coordinate system to *type*, specify the user coordinate system number 0 - 15(#01 -#16) to *aux*.

■ Return Value

≥ 0 Success

< 0 Failure

Negative value means failure.

Return Value	Meaning	Remedy
E_MP_MOT_GRP_NOT_EXIST (-32)	Specified control group doesn't exist.	Specify the correct control group.
E_MP_MOT_USER_OUTRNG (-35)	The specified user coordinate system number is out of range.	Specify the correct user coordinate system number.

mpMotSetSpeed

Specifies speed of the target.

■ **Syntax**

```
int mpMotSetSpeed
(
    int      grpNo, /* control group number */
    MP_SPEED * spd  /* speed */
)
```

■ **Description**

Set the speed specified in *spd* to the control group specified in *grpNo*. *spd* includes joint speed, orbital speed, and posture speed in each member. This API sets all these speed. If the set value is 0, the speed is not changed and keeps the previous status.

The set details are reflected in the target that is sent after this API is called.

■ **Parameter**

[*grpNo*]

Specify the target control group.
Specify the control group number retrieved by the mpCtrlGrpId2GrpNo().

Value	Meaning
0	The first control group
1	The second control group
2	The third control group
3	The fourth control group

[*spd*]

Specify the type of the coordinate system.

```
typedef struct {
    long  vj, /* joint velocity(0.01[%]). */
    long  v,  /* trajectory velocity(0.1[mm/sec]). */
    long  vr, /* orientation velocity(0.1[deg/sec]). */
    MP_SPEED;
```

Member	Meaning	Units
MP_SPEED.vj	Axis speed	0.01[%]
MP_SPEED.v	Orbital speed	0.1[mm/sec]
MP_SPEED.vr	Posture speed	0.1[deg/sec]

After the controller starts, the axis speed and orbital speed are set to a medium speed (a speed of level 5 selected when registering a move instruction) and the posture speed is set to the maximum speed registered in a move instruction as the default speeds for all groups.

11 Motion Control API

mpMotSetSpeed

■ Return Value

≥ 0 Success

< 0 Failure

Negative value means failure.

Return Value	Meaning	Remedy
E_MP_MOT_GRP_NOT_EXIST (-32)	Specified control group doesn't exist.	Specify the correct control group.

mpMotSetTool

specifies the tool number for the target.

■ **Syntax**

```
int mpMotSetTool
(
    int  grpNo,      /* control group number */
    int  toolNo     /* tool number */
)
```

■ **Description**

Set the tool number specified in *toolNo* to the control group specified in *grpNo*.
The set details are reflected in the target that is sent after this API is called.

■ **Parameter**

[grpNo]

Specify the target control group.
Specify the control group number retrieved by mpCtrlGrpId2GrpNo().

Constant	Meaning
0	The first control group
1	The second control group
2	The third control group
3	The fourth control group

[toolNo]

specify the tool number. (0 to 63)
After the controller starts, 0 is set as the default value of the tool number for all control groups.

■ **Return Value**

>= 0 Success
< 0 Failure
Negative value means failure

Return Value (symbolic constant (value))	Meaning	Remedy
E_MP_MOT_GRP_NOT_EXIST (-32)	Specified control group doesn't exist.	Specify the correct control group.
E_MP_MOT_TOOL_OUTRNG (-45)	Specified tool number is out of range.	Specify the correct tool number.

mpMotSetOrigin

Specifies the basis of the target position.

■ **Syntax**

```
int mpMotSetOrigin
(
    int grpNo, /* control group number */
    int options /* option */
)
```

■ **Description**

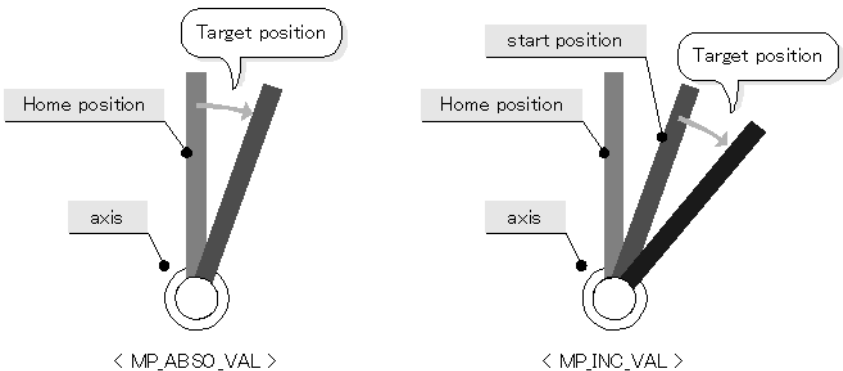
Set the basis specified in *options* to the control group specified in *grpNo*.
When specify MP_ABSO_VAL to *options*, base on the followings.
For example, when the each axis target position of the target is 10[deg],
the manipulator moves to the position of 10[deg].

Target position	Base position
Each axis value (pulse, angle, etc.)	0 pulse position of each axis (The position of absolute angle 0[deg])
Cartesian value (base coordinate system, user coordinate system, etc.)	The defined position of coordinate system

When specify the MP_INC_VAL to *options*, base on the start position.
For example, when the each axis target position of the target is 10[deg]
and each axis angle is 5[deg], the manipulator moves to the position of 15[deg].

Target position	Base position
Each axis value (pulse, angle, etc.)	Each axis start position
Cartesian value (base coordinate system, user coordinate system, etc.)	Cartesian value start position

The set details are reflected to the sent target after this API had been called.



11 Motion Control API

mpMotSetOrigin

■ Parameter

[grpNo]

Specify the target control group.

Specify the control group number retrieved by the mpCtrlGrpId2GrpNo().

Value	Meaning
0	The first control group
1	The second control group
2	The third control group
3	The fourth control group

[options]

Specify the basis.

MP_ABSO_VAL Absolute value

MP_INC_VAL Incremental value

After the controller starts, absolute value (MP_ABSO_VAL) is set as the default value of the basis of the target position for all control groups.

■ Return Value

>= 0 Success

< 0 Failure

Negative value means failure.

Return Value	Meaning	Remedy
E_MP_MOT_GRP_NOT_EXIST (-32)	Specified control group doesn't exist.	Specify the correct control group.

mpMotSetTask

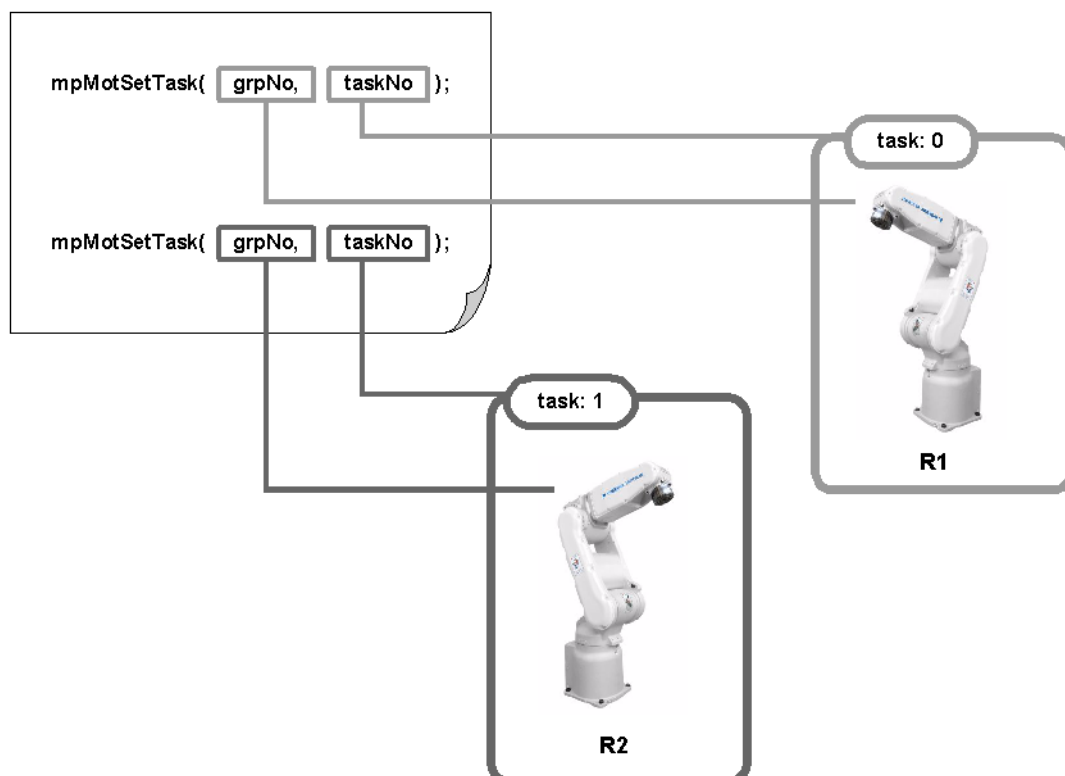
Specifies a task number of the target.
To use this function, the independent control function (optional) is required.

■ Syntax

```
int mpMotSetTask
(
    int grpNo, /* control group number */
    int taskNo /* task number */
)
```

■ Description

Set the task number specified in *taskNo* to the control group specified in *grpNo*.
The set details are reflected to the target that is sent after this API is called.



11 Motion Control API

mpMotSetTask

■ Parameter

[grpNo]

Specify the target control group.

Specify the control group number retrieved by the mpCtrlGrpId2GrpNo().

Value	Meaning
0	The first control group
1	The second control group
2	The third control group
3	The fourth control group

[taskNo]

Specify the task number.

■ Return Value

≥ 0 Success

< 0 Failure

Negative value means failure.

Return Value	Meaning	Remedy
E_MP_MOT_GRP_NOT_EXIST (-32)	Specified control group doesn't exist.	Specify the correct control group.
E_MP_MOT_TASK_OUTRNG (-36)	Specified task number is out of range.	Specify the correct task number.

NOTICE

- If specify any task number except for 0 when the independent control function (optional) is invalid, "E_MP_MOT_TASK_OUTRNG" is returned.
To use this function, the independent control function (optional) is required separately.

mpMotSetSync

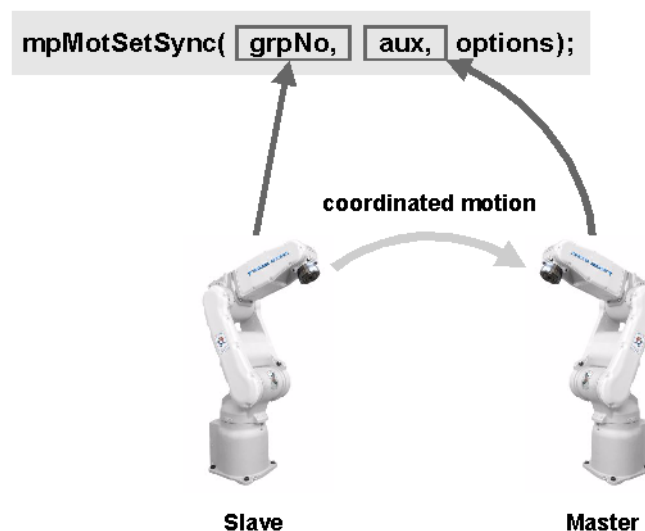
Specifies synchronized control information between target control groups (master and slave control group numbers).
To use this function, the coordinated control function (optional) is required.

■ Syntax

```
int mpMotSetSync
(
    int grpNo, /* Specify the slave control group number */
    int aux,    /* Specify the master control group number */
    int options /* option */
)
```

■ Description

Specify the control group, which is specified in *grpNo*, for coordinated motion. The target control group becomes a slave control group, and executes the coordinated motion on the basis of the master tool coordinates. The set details are reflected to the target that is sent after this API.



11 Motion Control API

mpMotSetSync

■ Parameter

[grpNo]

Specify the slave control group.

Specify the control group number retrieved by the mpCtrlGrpId2GrpNo().

Value	Meaning
0	The first control group
1	The second control group
2	The third control group
3	The fourth control group

[aux]

Specify the master control group.

Specify the control group number retrieved by the mpCtrlGrpId2GrpNo().

Value	Meaning
0	The first control group
1	The second control group
2	The third control group
3	The fourth control group

[options]

Specify 0.

■ Return Value

≥ 0 Success

< 0 Failure

Return Value	Meaning	Remedy
E_MP_MOT_MASTER_GRP_ILLEGAL (-38)	Specified coordinated master is wrong. The control group is out of range or the base axis is specified to the master.	Specify the correct control group.
E_MP_MOT_SLAVE_GRP_ILLEGAL (-39)	Specified coordinated slave is wrong. The control group is out of range or other than manipulator is specified to the slave.	Specify the correct control group.

NOTICE

- If specify or cancel coordinated motion when the coordinated control function (optional) is invalid, ERROR is returned. To use this function, the independent control function (optional) is required separately.

mpMotResetSync

Cancels the coordinated motion specification of the target.
To use this function, the coordinated control function (optional) is required.

■ Syntax

```
int mpMotResetSync
(
    int grpNo /* control group number */
)
```

■ Description

This routine cancels the coordinated motion of the control group specified in *grpNo*.
The set details are reflected to the target which is sent after this API.

■ Parameter

[grpNo]

Specify the target control group.
Specify the control group number retrieved by the mpCtrlGrpId2GrpNo().

Constant	Meaning
0	The first control group
1	The second control group
2	The third control group
3	The fourth control group

■ Return Value

=> 0 Success
< 0 Failure

Return Value	Meaning	Remedy
E_MP_MOT_GRP_NOT_EXIST (-32)	Specified control group doesn't exist.	Specify the correct control group.

NOTICE

- If specify or cancel coordinated motion when the coordinated control function (optional) is invalid, ERROR is returned.
To use this function, the independent control function (optional) is required separately.

mpMotSetConfig

Specify the robot configuration of the specified control group.

■ **Syntax**

```
int mpMotSetConfig
(
    int      grpNo, /* Control group number */
    ULONG    config /* Robot configuration */
)
```

■ **Description**

This API specifies the robot configuration of the control group specified in *grpNo*. The value set by this API is valid only when parameter S2C430 (Relative job operation method specification) is set to "TYPE prioritized" (1).

In *grpNo*, 0 indicates the first control group in the system, 1 indicates the second control group. Subsequent groups follow the same format.

In a system with only a manipulator (R1), 0 indicates R1.

■ **Parameter**

[*grpNo*]

Specify the target control group.
Specify the control group number retrieved by the mpCtrlGrpId2GrpNo().

Constant	Meaning
0	The first control group
1	The second control group
2	The third control group
3	The fourth control group

11 Motion Control API

mpMotSetConfig

[config]

Specify the robot configuration by bit units in the following table.

After the controller starts, “Front, Upper arm, Flip, R <180, T <180, S <180” are set as the default value of the configuration for all control groups. For details on the configuration, refer to “Manipulator Types” in chapter 3.9 of the operator's manual of the controller¹⁾.

Attribute (description of bits)	Description
D00	0: Front 1: Back
D01	0: Upper arm 1: Lower arm
D02	0: Flip 1: No flip
D03	0: R < 180 1: R >= 180
D04	0: T < 180 1: T >= 180
D05	0: S < 180 1: S >= 180
D06 - D15	Reserved by manufacturer

■ Return Value

>= 0 Success

< 0 Failure

Negative value means failure.

Return Value	Meaning	Remedy
E_MP_MOT_GRP_NOT_EXIST (-32)	The specified control group does not exist.	Specify the correct control group.

1 For the YRC1000:
YRC1000 GENERAL OPERATOR'S MANUAL (RE-CSO-A051)
For the YRC1000micro:
YRC1000micro OPERATOR'S MANUAL (RE-CSO-A058)

mpMotSetAccuracy

Specifies the positioning level of the specified control group.

■ **Syntax**

```
int mpMotSetAccuracy
(
    int    grpNo, /* Control group number */
    long   acr    /* Positioning level */
)
```

■ **Description**

This API specifies the positioning level of the control group specified in *grpNo*.
In *grpNo*, 0 indicates the first control group in the system, 1 indicates the second control group. Subsequent groups follow the same format.
In a system with only a manipulator (R1), 0 indicates R1.

■ **Parameter**

[grpNo]
Specify the target control group.
Specify the control group number retrieved by the mpCtrlGrpId2GrpNo().

Constant	Meaning
0	The first control group
1	The second control group
2	The third control group
3	The fourth control group

[acr]
Specify the positioning level in [μm] units. If *acr* is the negative number, the positioning level is set as invalid.
After the controller starts, the positioning level for all groups is set to invalid. If the positioning level is invalid, the deceleration motion from the previous step and the acceleration motion to the next step overlap for smooth motion when passing through a step. (The motion is the same as when a PL tag is not added to an INFORM move instruction.)

11 Motion Control API

mpMotSetAccuracy

■ Return Value

≥ 0 Success

< 0 Failure

Negative value means failure.

Return Value	Meaning	Remedy
E_MP_MOT_GRP_NOT_EXIST (-32)	The specified control group does not exist.	Specify the correct control group.

mpMotSetAccel

Sets the target acceleration.

■ Syntax

```
int mpMotSetAccel
(
    int    grpNo, /* Control group number */
    long   accel  /* Acceleration adjustment ratio [0.01%] */
)
```

■ Description

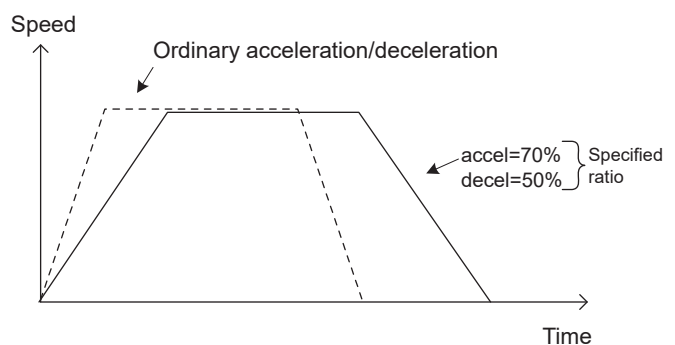
Set the ratio specified in *accel* to the control group specified in *grpNo*.
This routine reduces the amount of acceleration in the specified ratio.
The set details are reflected in all targets that are sent after this API is called.



Acceleration/deceleration adjustment ratio

The acceleration/deceleration adjustment ratio (accel/decel) reduces the amount of acceleration/deceleration in the specified ratio.

Using this function can reduce the load inertia on the tool and the workpiece.



11 Motion Control API

mpMotSetAccel

■ Parameter

[grpNo]

Specify the target control group.

Specify the control group number retrieved by the mpCtrlGrpId2GrpNo().

Constant	Meaning
0	The first control group
1	The second control group
2	The third control group
3	The fourth control group

[accel]

Specify the acceleration adjustment ratio in 0.01% units.

Specify the acceleration adjustment ratio in the range of 2000 (20%) to 10000 (100%).

After the controller starts, 100% is set as the default value of the acceleration adjustment ratio.

■ Return Value

≥ 0 Success

< 0 Failure

Negative value means failure.

Return Value	Meaning	Remedy
E_MP_MOT_GRP_NOT_EXIST (-32)	The specified control group does not exist.	Specify the correct control group.
E_MP_MOT_ACC_OUTRNG (-47)	The specified acceleration adjustment ratio is out of range.	Specify the correct acceleration.

■ Note

This API can be used in the following software configuration.

For the YRC1000:

- System software: YAS2.41.00A-00 or later
- MotoPlusIDE: Ver2.01 or later

For the YRC1000micro:

- System software: YBS1.31.00-00 or later
- MotoPlusIDE: Ver2.01 or later

mpMotSetDecel

Sets the target deceleration.

■ Syntax

```
int mpMotSetDecel
(
    int    grpNo, /* Control group number */
    long   decel  /* Deceleration adjustment ratio [0.01%] */
)
```

■ Description

Set the deceleration adjustment ratio in *decel* to the control group specified in *grpNo*.

This routine reduces the amount of deceleration in the specified ratio.

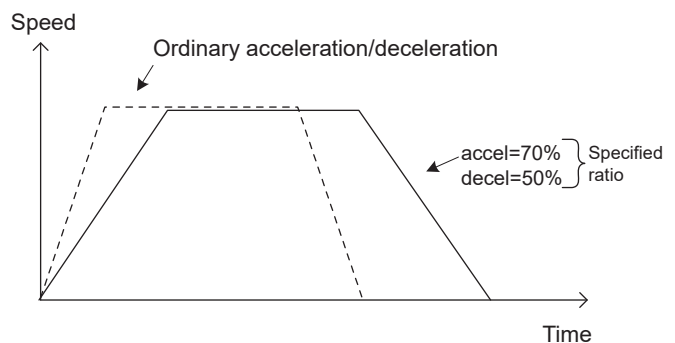
The set details are reflected in all targets that are sent after this API is called.



Acceleration/deceleration adjustment ratio

The acceleration/deceleration adjustment ratio (accel/decel) reduces the amount of acceleration/deceleration in the specified ratio.

Using this function can reduce the load inertia on the tool and the workpiece.



11 Motion Control API

mpMotSetDecel

■ Parameter

[grpNo]

Specify the target control group.

Specify the control group number retrieved by the mpCtrlGrpId2GrpNo().

Constant	Meaning
0	The first control group
1	The second control group
2	The third control group
3	The fourth control group

[decel]

Specify the deceleration adjustment ratio in 0.01% units.

Specify the deceleration adjustment ratio in the range of 2000 (20%) to 10000 (100%).

After the controller starts, 100% is set as the default value of the deceleration adjustment ratio.

■ Return Value

≥ 0 Success

< 0 Failure

Negative value means failure.

Return Value	Meaning	Remedy
E_MP_MOT_GRP_NOT_EXIST (-32)	The specified control group does not exist.	Specify the correct control group.
E_MP_MOT_DEC_OUTRNG (-48)	The specified deceleration adjustment ratio is out of range.	Specify the correct deceleration.

■ Note

This API can be used in the software configuration that satisfies all of the following conditions.

For the YRC1000:

- System software: YAS2.41.00A-00 or later
- MotoPlusIDE: Ver2.01 or later

For the YRC1000micro

- System software: YBS1.31.00-00 or later
- MotoPlusIDE: Ver2.01 or later

12 Motion Monitor/Control API

mplMOV

Moves the manipulator from the current position by the specified value in the Cartesian coordinates.

■ Syntax

```
LONG mplMOV (  
MP_IMOV_SEND_DATA*      sData,  
MP_STD_RSP_DATA*        rData  
);
```

■ Description

Moves the manipulator from the current position by the specified value in the Cartesian coordinates.

Only linear motion is available.

■ Parameter

[sData]

[in] The pointer to the data structure which transmits the information of increment move

- MP_IMOV_SEND_DATA

[in] The data structure which transmits the information of increment move

Syntax: #define MAX_NO_OF_AXES (12)

```
typedef struct  
{  
    ULONG      sCtrlGrp;  
    LONG       lSpeed;  
    SHORT      sVType;  
    SHORT      sFrame;  
    SHORT      sToolNo;  
    CHAR       reserved[2];  
    LONG       lPos[MAX_NO_OF_AXES];  
} MP_IMOV_SEND_DATA;
```

Member: <sCtrlGrp>
Control group

Constant	Description
0	R1 (Robot 1)
1	R2 (Robot 2)
2	R3 (Robot 3)
3	R4 (Robot 4)
4	R5 (Robot 5)
5	R6 (Robot 6)
6	R7 (Robot 7)
7	R8 (Robot 8)

12 Motion Monitor/Control API

mplMOV

Constant	Description
8	B1 (Base 1)
9	B2 (Base 2)
10	B3 (Base 3)
11	B4 (Base 4)
12	B5 (Base 5)
13	B6 (Base 6)
14	B7 (Base 7)
15	B8 (Base 8)
16	S1 (Station 1)
17	S2 (Station 2)
18	S3 (Station 3)
19	S4 (Station 4)
20	S5 (Station 5)
21	S6 (Station 6)
22	S7 (Station 7)
23	S8 (Station 8)
24	S9 (Station 9)
25	S10 (Station 10)
26	S11 (Station 11)
27	S12 (Station 12)
28	S13 (Station 13)
29	S14 (Station 14)
30	S15 (Station 15)
31	S16 (Station 16)
32	S17 (Station 17)
33	S18 (Station 18)
34	S19 (Station 19)
35	S20 (Station 20)
36	S21 (Station 21)
37	S22 (Station 22)
38	S23 (Station 23)
39	S24 (Station 24)

</Speed>

Motion speed

0.1 to S1C*G17 (linear speed for registration) [mm/s]

0.1 to S1C*G25 (position angle speed) [deg/s]

For more information on each parameter, refer to

“LINEAR SPEED FOR REGISTRATION” and
“POSITION ANGLE SPEED” in chapter 8.2 of
the operator's manual of the controller¹⁾.

¹ For the YRC1000:

YRC1000 GENERAL OPERATOR'S MANUAL (RE-CSO-A051)

For the YRC1000micro:

YRC1000micro OPERATOR'S MANUAL (RE-CSO-A058)

12 Motion Monitor/Control API

mpIMOV

<sVType>

Selection of motion speed

Constant	Description
0	Control point
1	Position angular

<sFrame>

Coordinate system ID

Constant	Description
0	Base coordinate
1	Robot coordinate
2, 3, ...64	User coordinate 1, 2, ...63

<sToolNo>

Tool number(0 to 63)

<IPos[MAX_NO_OF_AXES]>

Specified position (up to 12 arrays)

<<Robot axis>>

Array	Description
IPos[0]	X-axis coordinate (unit: micron)
IPos[1]	Y-axis coordinate (unit: micron)
IPos[2]	Z-axis coordinate (unit: micron)
IPos[3]	Wrist angle Rx (unit: 0.0001 deg)
IPos[4]	Wrist angle Ry (unit: 0.0001 deg)
IPos[5]	Wrist angle Rz (unit: 0.0001 deg)
IPos[6]	Elbow angle E (unit: 0.0001 deg)
IPos[7]	8th axis pulse value (micron in the case of traveling axis)
IPos[8]	Not in use
IPos[9]	Not in use
IPos[10]	Not in use
IPos[11]	Not in use

* Set "0" for data IPos[8] to IPos[11] if the system has no external axis.

<<External axis>>

Array	Description
IPos[0]	Not in use
IPos[1]	Not in use
IPos[2]	Not in use
IPos[3]	Not in use
IPos[4]	Not in use
IPos[5]	Not in use
IPos[6]	1st traveling axis pulse value
IPos[7]	2nd traveling axis pulse value
IPos[8]	3rd traveling axis pulse value

12 Motion Monitor/Control API

mpIMOV

Array	Description
IPos[9]	Not in use
IPos[10]	Not in use
IPos[11]	Not in use

* Set "0" for data IPos[0] to IPos[5], and IPos[9] to IPos[11].

[rData]

[out] The pointer to the data structure which receives the error information

• MP_STD_RSP_DATA

[out] The data structure which receives the error information

Syntax:

```
typedef struct
{
    USHORT    err_no;
    CHAR      reserved[2];
} MP_STD_RSP_DATA;
```

Member: <err_no>
Error number

Value	Description
0x0000	Normal end
0x2010	Robot is in operation
0x2030	In HOLD status (PP)
0x2040	In HOLD status (External)
0x2050	In HOLD status (Command)
0x2060	In error/alarm status
0x2070	In SERVO OFF status
0x2080	Wrong operation mode
0x3040	Home position is not registered
0x3050	Out of range (ABSO data)
0x3061	Each axis operation range limit function is temporarily disable ¹⁾
0x3062	Robot operation range limit function is temporarily disable ¹⁾
0x3063	Tool angle monitoring function is temporarily disable ¹⁾
0x3064	Tool switch monitoring function is temporarily disable ¹⁾
0x3065	External axis coasting value setting is not done ¹⁾
0x3350	User coordinate is not registered
0x3370	Wrong control group

1 Only when the safety function (option) is valid

12 Motion Monitor/Control API

mpIMOV

■ Return Value

- 0 API execution processor received the command.
Return value = 0 and err_no = 0 Normal end
Return value = 0 and err_no != 0 Error (Check err_no.)
- 1 API execution processor does not respond.



Two or more move instructions such as mpIMOV(), mpMOVJ(), mpMOVL(), mpPulseMOVJ(), and mpPulseMOVL(), mpManualMOV() cannot be called. If one of the manipulators is in operation by the move instruction above, the move instruction to the other manipulators cannot be issued. If done so, "0x2060: In error/alarm status" returns in err_no.



Exclusive control by using semaphores is executed between this API and the APIs listed below. Thus, if this API is executed while one of the following APIs is already being executed, the task may be suspended to wait for the API to be completed. (waiting for semaphore acquisition)

Therefore, DO NOT use the following APIs for the task which must be performed periodically with high priority.

mpCancelError(), mpResetAlarm(), mpSetCycle(), mpSetServoPower(), mpSetMasterJob(), mpSetCurJob(), mpStartJob(), mpHold(), mpWaitForJobEnd(), mpDeleteJob(), mpGetJogSpeed(), mpGetJogCoord(), mpIMOV(), mpMOVJ(), mpMOVL(), mpPulseMOVJ(), mpPulseMOVL(), mpManualMOV()

mpMOVJ

Moves the manipulator to the position specified with the link interpolation.

■ Syntax

```
LONG mpMOVJ (
MP_MOVJ_SEND_DATA*      sData,
MP_STD_RSP_DATA*        rData
);
```

■ Parameter

[*sData*]

[in] The pointer to the data structure which transmits the MOVJ information

- MP_MOVJ_SEND_DATA

[in] The data structure which transmits the MOVJ information

Syntax: #define MAX_NO_OF_AXES (12)

```
typedef struct
{
    ULONG      sCtrlGrp;
    LONG       lSpeed;
    SHORT      sFrame;
    SHORT      sConfig;
    SHORT      sToolNo;
    CHAR       reserved[2];
    LONG       lPos[MAX_NO_OF_AXES];
} MP_MOVJ_SEND_DATA;
```

Member: <*sCtrlGrp*>
Control group

Constant	Description
0	R1 (Robot 1)
1	R2 (Robot 2)
2	R3 (Robot 3)
3	R4 (Robot 4)
4	R5 (Robot 5)
5	R6 (Robot 6)
6	R7 (Robot 7)
7	R8 (Robot 8)
8	B1 (Base 1)
9	B2 (Base 2)
10	B3 (Base 3)
11	B4 (Base 4)
12	B5 (Base 5)
13	B6 (Base 6)
14	B7 (Base 7)

12 Motion Monitor/Control API

mpMOVJ

Constant	Description
15	B8 (Base 8)
16	S1 (Station 1)
17	S2 (Station 2)
18	S3 (Station 3)
19	S4 (Station 4)
20	S5 (Station 5)
21	S6 (Station 6)
22	S7 (Station 7)
23	S8 (Station 8)
24	S9 (Station 9)
25	S10 (Station 10)
26	S11 (Station 11)
27	S12 (Station 12)
28	S13 (Station 13)
29	S14 (Station 14)
30	S15 (Station 15)
31	S16 (Station 16)
32	S17 (Station 17)
33	S18 (Station 18)
34	S19 (Station 19)
35	S20 (Station 20)
36	S21 (Station 21)
37	S22 (Station 22)
38	S23 (Station 23)
39	S24 (Station 24)

</Speed>

Motion speed (1 - 10000 representing 0.01 to 100.0%)

<sFrame>

Coordinate system ID

Constant	Description
0	Base coordinate
1	Robot coordinate
2, 3, ... 64	User coordinate 1, 2, ... 63

<sConfig>

Configuration

Attribute	Description
D00	0: Front 1: Back
D01	0: Upper arm 1: Lower arm
D02	0: Flip 1: No flip
D03	0: R < 180 1: R >= 180
D04	0: T < 180 1: T >= 180
D05	0: S < 180 1: S >= 180
D06 - D15	Reserved by manufacturer

12 Motion Monitor/Control API

mpMOVJ

<sToolNo>

Tool number (0 to 63)

<IPos[*MAX_NO_OF_AXES*]>

<< Robot axis >>

Array	Description
IPos[0]	X-axis coordinate (unit: micron)
IPos[1]	Y-axis coordinate (unit: micron)
IPos[2]	Z-axis coordinate (unit: micron)
IPos[3]	Wrist angle Rx (unit: 0.0001 deg)
IPos[4]	Wrist angle Ry (unit: 0.0001 deg)
IPos[5]	Wrist angle Rz (unit: 0.0001 deg)
IPos[6]	Elbow angle E (unit: 0.0001 deg)
IPos[7]	8th axis pulse value (micron in the case of traveling axis)
IPos[8]	Not in use
IPos[9]	Not in use
IPos[10]	Not in use
IPos[11]	Not in use

* Set "0" for data IPos[8] to IPos[11] if the system has no external axis.

<< External axis >>

Array	Description
IPos[0]	Not in use
IPos[1]	Not in use
IPos[2]	Not in use
IPos[3]	Not in use
IPos[4]	Not in use
IPos[5]	Not in use
IPos[6]	1st traveling axis pulse value
IPos[7]	2nd traveling axis pulse value
IPos[8]	3rd traveling axis pulse value
IPos[9]	Not in use
IPos[10]	Not in use
IPos[11]	Not in use

* Set "0" for data IPos[0] to IPos[5], and IPos[9] to IPos[11].

12 Motion Monitor/Control API

mpMOVJ

[rData]

[out] The pointer to the data structure which receives the error information

- MP_STD_RSP_DATA

[out] The data structure which receives the error information

Syntax:

```
typedef struct
{
    USHORT    err_no;
    CHAR      reserved[2];
} MP_STD_RSP_DATA;
```

Member: `<err_no>`
Error number

Value	Description
0x0000	Normal end
0x2010	Robot is in operation
0x2030	In HOLD status (PP)
0x2040	In HOLD status (External)
0x2050	In HOLD status (Command)
0x2060	In error/alarm status
0x2070	In SERVO OFF status
0x2080	Wrong operation mode
0x3040	Home position is not registered
0x3050	Out of range (ABSO data)
0x3061	Each axis operation range limit function is temporarily disable ¹⁾
0x3062	Robot operation range limit function is temporarily disable ¹⁾
0x3063	Tool angle monitoring function is temporarily disable ¹⁾
0x3064	Tool switch monitoring function is temporarily disable ¹⁾
0x3065	External axis coasting value setting is not done ¹⁾
0x3350	User coordinate is not registered
0x3370	Wrong control group

1 Only when the safety function (option) is valid

■ Return Value

- 0 API execution processor received the command.
 Return value = 0 and err_no = 0 Normal end
 Return value = 0 and err_no != 0 Error (Check err_no.)
- 1 API execution processor does not respond.

12 Motion Monitor/Control API

mpMOVJ



Two or more move instructions such as mpIMOV(), mpMOVJ(), mpMOVL(), mpPulseMOVJ(), and mpPulseMOVL(), mpManualMOV() cannot be called. If one of the manipulators is in operation by the move instruction above, the move instruction to the other manipulators cannot be issued. If done so, "0x2060: In error/alarm status" returns in err_no.



Exclusive control by using semaphores is executed between this API and the APIs listed below. Thus, if this API is executed while one of the following APIs is already being executed, the task may be suspended to wait for the API to be completed. (waiting for semaphore acquisition)

Therefore, DO NOT use the following APIs for the task which must be performed periodically with high priority.

mpCancelError(), mpResetAlarm(), mpSetCycle(), mpSetServoPower(), mpSetMasterJob(), mpSetCurJob(), mpStartJob(), mpHold(), mpWaitForJobEnd(), mpDeleteJob(), mpGetJogSpeed(), mpGetJogCoord(), mpIMOV(), mpMOVJ(), mpMOVL(), mpPulseMOVJ(), mpPulseMOVL(), mpManualMOV()

mpMOVL

Moves the manipulator to the position specified with the linear interpolation.

■ **Syntax**

```
LONG mpMOVL (  
  MP_MOVL_SEND_DATA*      sData,  
  MP_STD_RSP_DATA*        rData  
);
```

■ **Parameter**

[sData]

[in] The pointer to the data structure which transmits the MOVL information

- MP_MOVL_SEND_DATA

[in] The data structure which transmits the MOVL information

Syntax: #define MAX_NO_OF_AXES (12)

```
typedef struct  
{  
  ULONG      sCtrlGrp;  
  LONG       ISpeed;  
  SHORT      sVType;  
  SHORT      sFrame;  
  SHORT      sConfig;  
  SHORT      sToolNo;  
  LONG       IPos[MAX_NO_OF_AXES];  
} MP_MOVL_SEND_DATA;
```

Member: <*sCtrlGrp*>
Control group

Constant	Description
0	R1 (Robot 1)
1	R2 (Robot 2)
2	R3 (Robot 3)
3	R4 (Robot 4)
4	R5 (Robot 5)
5	R6 (Robot 6)
6	R7 (Robot 7)
7	R8 (Robot 8)
8	B1 (Base 1)
9	B2 (Base 2)
10	B3 (Base 3)
11	B4 (Base 4)
12	B5 (Base 5)
13	B6 (Base 6)

12 Motion Monitor/Control API

mpMOVL

Constant	Description
14	B7 (Base 7)
15	B8 (Base 8)
16	S1 (Station 1)
17	S2 (Station 2)
18	S3 (Station 3)
19	S4 (Station 4)
20	S5 (Station 5)
21	S6 (Station 6)
22	S7 (Station 7)
23	S8 (Station 8)
24	S9 (Station 9)
25	S10 (Station 10)
26	S11 (Station 11)
27	S12 (Station 12)
28	S13 (Station 13)
29	S14 (Station 14)
30	S15 (Station 15)
31	S16 (Station 16)
32	S17 (Station 17)
33	S18 (Station 18)
34	S19 (Station 19)
35	S20 (Station 20)
36	S21 (Station 21)
37	S22 (Station 22)
38	S23 (Station 23)
39	S24 (Station 24)

</Speed>

Motion speed

0.1 to S1C*G17 (linear speed for registration) [mm/s]

0.1 to S1C*G25 (position angle speed) [deg/s]

For more information on each parameter, refer to "LINEAR SPEED FOR REGISTRATION" and "POSITION ANGLE SPEED" in chapter 8.2 of the operator's manual of the controller¹⁾.

<sVType>

Selection of motion speed

Constant	Description
0	Control point
1	Position angular

¹ For the YRC1000:
YRC1000 GENERAL OPERATOR'S MANUAL (RE-CSO-A051)
For the YRC1000micro:
YRC1000micro OPERATOR'S MANUAL (RE-CSO-A058)

12 Motion Monitor/Control API

mpMOVL

<sFrame>

Coordinate system ID

Constant	Description
0	Base coordinate
1	Robot coordinate
2, 3, ... 64	User coordinate 1, 2, ... 63

<sConfig>

Configuration

Attribute (description of bits)	Description
D00	0: Front 1: Back
D01	0: Upper arm 1: Lower arm
D02	0: Flip 1: No flip
D03	0: R < 180 1: R >= 180
D04	0: T < 180 1: T >= 180
D05	0: S < 180 1: S >= 180
D06 - D15	Reserved by manufacturer

<sToolNo>

Tool number (0 to 63)

<IPos[MAX_NO_OF_AXES]>

<< Robot axis >>

Array	Description
IPos[0]	X-axis coordinate (unit: micron)
IPos[1]	Y-axis coordinate (unit: micron)
IPos[2]	Z-axis coordinate (unit: micron)
IPos[3]	Wrist angle Rx (unit: 0.0001 deg)
IPos[4]	Wrist angle Ry (unit: 0.0001 deg)
IPos[5]	Wrist angle Rz (unit: 0.0001 deg)
IPos[6]	Elbow angle E (unit: 0.0001 deg)
IPos[7]	Reserved
IPos[8]	Not in use
IPos[9]	Not in use
IPos[10]	Not in use
IPos[11]	Not in use

* Set "0" for data IPos[8] to IPos[11] if the system has no external axis.

12 Motion Monitor/Control API

mpMOVL

<< External axis >>

Array	Description
IPos[0]	Not in use
IPos[1]	Not in use
IPos[2]	Not in use
IPos[3]	Not in use
IPos[4]	Not in use
IPos[5]	Not in use
IPos[6]	1st traveling axis pulse value
IPos[7]	2nd traveling axis pulse value
IPos[8]	3rd traveling axis pulse value
IPos[9]	Not in use
IPos[10]	Not in use
IPos[11]	Not in use

* Set "0" for data IPos[0] to IPos[5], and IPos[9] to IPos[11].

[rData]

[out] The pointer to the data structure which receives the error information

• MP_STD_RSP_DATA

[out] The data structure which receives the error information

```
Syntax:  typedef struct
          {
            USHORT   err_no;
            CHAR      reserved[2];
          } MP_STD_RSP_DATA;
```

Member: <err_no>
Error number

Value	Description
0x0000	Normal end
0x2010	Robot is in operation
0x2030	In HOLD status (PP)
0x2040	In HOLD status (External)
0x2050	In HOLD status (Command)
0x2060	In error/alarm status
0x2070	In SERVO OFF status
0x2080	Wrong operation mode
0x3040	Home position is not registered
0x3050	Out of range (ABS data)
0x3061	Each axis operation range limit function is temporarily disable ¹⁾
0x3062	Robot operation range limit function is temporarily disable ¹⁾
0x3063	Tool angle monitoring function is temporarily disable ¹⁾
0x3064	Tool switch monitoring function is temporarily disable ¹⁾
0x3065	External axis coasting value setting is not done ¹⁾
0x3350	User coordinate is not registered
0x3370	Wrong control group

1 Only when the safety function (option) is valid

12 Motion Monitor/Control API

mpMOVL

■ Return Value

- 0 API execution processor received the command.
Return value = 0 and err_no = 0 Normal end
Return value = 0 and err_no != 0 Error (Check err_no.)
- 1 API execution processor does not respond.



Two or more move instructions such as mpIMOV(), mpMOVJ(), mpMOVL(), mpPulseMOVJ(), and mpPulseMOVL(), mpManualMOV() cannot be called. If one of the manipulators is in operation by the move instruction above, the move instruction to the other manipulators cannot be issued. If done so, "0x2060: In error/alarm status" returns in err_no.



Exclusive control by using semaphores is executed between this API and the APIs listed below. Thus, if this API is executed while one of the following APIs is already being executed, the task may be suspended to wait for the API to be completed. (waiting for semaphore acquisition)

Therefore, DO NOT use the following APIs for the task which must be performed periodically with high priority.

mpCancelError(), mpResetAlarm(), mpSetCycle(), mpSetServoPower(), mpSetMasterJob(), mpSetCurJob(), mpStartJob(), mpHold(), mpWaitForJobEnd(), mpDeleteJob(), mpGetJogSpeed(), mpGetJogCoord(), mpIMOV(), mpMOVJ(), mpMOVL(), mpPulseMOVJ(), mpPulseMOVL(), mpManualMOV()

mpPulseMOVJ

Moves the manipulator to the pulse position specified with the link interpolation.

■ Syntax

```
LONG mpPulseMOVJ (
  MP_PMOVJ_SEND_DATA*   sData,
  MP_STD_RSP_DATA*      rData
);
```

■ Parameter

[*sData*]

[in] The pointer to the data structure which transmits the PMOVJ information

- MP_PMOVJ_SEND_DATA

[in] The data structure which transmits the PMOVJ information

Syntax: #define MAX_NO_OF_AXES (12)

```
typedef struct
{
  ULONG   sCtrlGrp;
  LONG    lSpeed;
  SHORT   sToolNo;
  CHAR    reserved 0[2];
  LONG    lPos[MAX_NO_OF_AXES];
  CHAR    reserved 1[4];
} MP_PMOVJ_SEND_DATA;
```

Member: <*sCtrlGrp*>
Control group

Constant	Description
0	R1 (Robot 1)
1	R2 (Robot 2)
2	R3 (Robot 3)
3	R4 (Robot 4)
4	R5 (Robot 5)
5	R6 (Robot 6)
6	R7 (Robot 7)
7	R8 (Robot 8)
8	B1 (Base 1)
9	B2 (Base 2)
10	B3 (Base 3)
11	B4 (Base 4)
12	B5 (Base 5)
13	B6 (Base 6)
14	B7 (Base 7)

12 Motion Monitor/Control API

mpPulseMOVJ

Constant	Description
15	B8 (Base 8)
16	S1 (Station 1)
17	S2 (Station 2)
18	S3 (Station 3)
19	S4 (Station 4)
20	S5 (Station 5)
21	S6 (Station 6)
22	S7 (Station 7)
23	S8 (Station 8)
24	S9 (Station 9)
25	S10 (Station 10)
26	S11 (Station 11)
27	S12 (Station 12)
28	S13 (Station 13)
29	S14 (Station 14)
30	S15 (Station 15)
31	S16 (Station 16)
32	S17 (Station 17)
33	S18 (Station 18)
34	S19 (Station 19)
35	S20 (Station 20)
36	S21 (Station 21)
37	S22 (Station 22)
38	S23 (Station 23)
39	S24 (Station 24)

</Speed>

Motion speed (1 - 10000 representing 0.01 to 100.0 %)

<sToolNo>

Tool number (0 to 63)

<IPos[MAX_NO_OF_AXES]>

<< Robot axis >>

Array	Description
IPos[0]	1st axis (S) pulse value
IPos[1]	2nd axis (L) pulse value
IPos[2]	3rd axis (U) pulse value
IPos[3]	4th axis (R) pulse value
IPos[4]	5th axis (B) pulse value
IPos[5]	6th axis (T) pulse value
IPos[6]	7th axis (E) pulse value
IPos[7]	8th axis pulse value
IPos[8]	Not in use
IPos[9]	Not in use
IPos[10]	Not in use
IPos[11]	Not in use

* Set "0" for data IPos[8] to IPos[11] if the system has no external axis.

12 Motion Monitor/Control API

mpPulseMOVJ

<< External axis >>

Array	Description
IPos[0]	Not in use
IPos[1]	Not in use
IPos[2]	Not in use
IPos[3]	Not in use
IPos[4]	Not in use
IPos[5]	Not in use
IPos[6]	Not in use
IPos[7]	Not in use
IPos[8]	1st external axis pulse value
IPos[9]	2nd external axis pulse value
IPos[10]	3rd external axis pulse value
IPos[11]	4th external axis pulse value

* Set "0" for data IPos[0] to IPos[7].

[rData]

[out] The pointer to the data structure which receives the error information

• MP_STD_RSP_DATA

[out] The data structure which receives the error information

```
Syntax:  typedef struct
          {
            USHORT    err_no;
            CHAR      reserved[2];
          } MP_STD_RSP_DATA;
```

Member: <err_no>
Error number

Value	Description
0x0000	Normal end
0x2010	Robot is in operation
0x2030	In HOLD status (PP)
0x2040	In HOLD status (External)
0x2050	In HOLD status (Command)
0x2060	In error/alarm status
0x2070	In SERVO OFF status
0x2080	Wrong operation mode
0x3040	Home position is not registered
0x3050	Out of range (ABS data)
0x3061	Each axis operation range limit function is temporarily disable ¹⁾
0x3062	Robot operation range limit function is temporarily disable ¹⁾
0x3063	Tool angle monitoring function is temporarily disable ¹⁾
0x3064	Tool switch monitoring function is temporarily disable ¹⁾
0x3065	External axis coasting value setting is not done ¹⁾
0x3350	User coordinate is not registered
0x3370	Wrong control group

1 Only when the safety function (option) is valid

12 Motion Monitor/Control API

mpPulseMOVJ

■ Return Value

- 0 API execution processor received the command.
 Return value = 0 and err_no = 0 Normal end
 Return value = 0 and err_no != 0 Error (Check err_no.)
- 1 API execution processor does not respond.



Two or more move instructions such as mpIMOV(), mpMOVJ(), mpMOVL(), mpPulseMOVJ(), and mpPulseMOVL(), mpManualMOV() cannot be called.
 If one of the manipulators is in operation by the move instruction above, the move instruction to the other manipulators cannot be issued.
 If done so, "0x2060: In error/alarm status" returns in err_no.



Exclusive control by using semaphores is executed between this API and the APIs listed below.
 Thus, if this API is executed while one of the following APIs is already being executed, the task may be suspended to wait for the API to be completed. (waiting for semaphore acquisition)

Therefore, DO NOT use the following APIs for the task which must be performed periodically with high priority.

mpCancelError(), mpResetAlarm(), mpSetCycle(),
 mpSetServoPower(), mpSetMasterJob(), mpSetCurJob(),
 mpStartJob(), mpHold(), mpWaitForJobEnd(),
 mpDeleteJob(), mpGetJogSpeed(), mpGetJogCoord(),
 mpIMOV(), mpMOVJ(), mpMOVL(), mpPulseMOVJ(),
 mpPulseMOVL(), mpManualMOV()

mpPulseMOVL

Moves the manipulator to the pulse position specified with the linear interpolation.

■ Syntax

```
LONG mpPulseMOVL (
  MP_PMOVL_SEND_DATA*   sData,
  MP_STD_RSP_DATA*      rData
);
```

■ Parameter

[*sData*]

[in] The pointer to the data structure which transmits the PMOVL information

- MP_PMOVL_SEND_DATA

[in] The data structure which transmits the PMOVL information

Syntax: #define MAX_NO_OF_AXES (12)

```
typedef struct
{
  ULONG      sCtrlGrp;
  LONG       lSpeed;
  SHORT      sVType;
  SHORT      sToolNo;
  LONG       lPos[MAX_NO_OF_AXES];
  CHAR       reserved[4];
} MP_PMOVL_SEND_DATA;
```

Member: <*sCtrlGrp*>
Control group

Constant	Description
0	R1 (Robot 1)
1	R2 (Robot 2)
2	R3 (Robot 3)
3	R4 (Robot 4)
4	R5 (Robot 5)
5	R6 (Robot 6)
6	R7 (Robot 7)
7	R8 (Robot 8)
8	B1 (Base 1)
9	B2 (Base 2)
10	B3 (Base 3)
11	B4 (Base 4)
12	B5 (Base 5)
13	B6 (Base 6)
14	B7 (Base 7)

12 Motion Monitor/Control API

mpPulseMOVL

Constant	Description
15	B8 (Base 8)
16	S1 (Station 1)
17	S2 (Station 2)
18	S3 (Station 3)
19	S4 (Station 4)
20	S5 (Station 5)
21	S6 (Station 6)
22	S7 (Station 7)
23	S8 (Station 8)
24	S9 (Station 9)
25	S10 (Station 10)
26	S11 (Station 11)
27	S12 (Station 12)
28	S13 (Station 13)
29	S14 (Station 14)
30	S15 (Station 15)
31	S16 (Station 16)
32	S17 (Station 17)
33	S18 (Station 18)
34	S19 (Station 19)
35	S20 (Station 20)
36	S21 (Station 21)
37	S22 (Station 22)
38	S23 (Station 23)
39	S24 (Station 24)

</Speed>

Motion speed

0.1 to S1C*G17 (linear speed for registration) [mm/s]

0.1 to S1C*G25 (position angle speed) [deg/s]

For more information on each parameter, refer to "LINEAR SPEED FOR REGISTRATION" and "POSITION ANGLE SPEED" in chapter 8.2 of the operator's manual of the controller¹⁾.

<sVType>

Selection of motion speed

Constant	Description
0	Control point
1	Position angular

1 For the YRC1000:
YRC1000 GENERAL OPERATOR'S MANUAL (RE-CSO-A051)
For the YRC1000micro:
YRC1000micro OPERATOR'S MANUAL (RE-CSO-A058)

12 Motion Monitor/Control API

mpPulseMOVL

<sToolNo>

Tool number (0 to 63)

<IPos[*MAX_NO_OF_AXES*]>

<< Robot axis >>

Array	Description
IPos[0]	1st axis (S) pulse value
IPos[1]	2nd axis (L) pulse value
IPos[2]	3rd axis (U) pulse value
IPos[3]	4th axis (R) pulse value
IPos[4]	5th axis (B) pulse value
IPos[5]	6th axis (T) pulse value
IPos[6]	7th axis (E) pulse value
IPos[7]	8th axis pulse value
IPos[8]	Not in use
IPos[9]	Not in use
IPos[10]	Not in use
IPos[11]	Not in use

* Set "0" for data IPos[8] to IPos[11] if the system has no external axis.

<< External axis >>

Array	Description
IPos[0]	Not in use
IPos[1]	Not in use
IPos[2]	Not in use
IPos[3]	Not in use
IPos[4]	Not in use
IPos[5]	Not in use
IPos[6]	Not in use
IPos[7]	Not in use
IPos[8]	1st external axis pulse value
IPos[9]	2nd external axis pulse value
IPos[10]	3rd external axis pulse value
IPos[11]	4th external axis pulse value

* Set "0" for data IPos[0] to IPos[5].

12 Motion Monitor/Control API

mpPulseMOVL

[rData]

[out] The pointer to the data structure which receives the error information

- MP_STD_RSP_DATA

[out] The data structure which receives the error information

Syntax:

```
typedef struct
{
    USHORT    err_no;
    CHAR      reserved[2];
} MP_STD_RSP_DATA;
```

Member: `<err_no>`
Error number

Value	Description
0x0000	Normal end
0x2010	Robot is in operation
0x2030	In HOLD status (PP)
0x2040	In HOLD status (External)
0x2050	In HOLD status (Command)
0x2060	In error/alarm status
0x2070	In SERVO OFF status
0x2080	Wrong operation mode
0x3040	Home position is not registered
0x3050	Out of range (ABSO data)
0x3061	Each axis operation range limit function is temporarily disable ¹⁾
0x3062	Robot operation range limit function is temporarily disable ¹⁾
0x3063	Tool angle monitoring function is temporarily disable ¹⁾
0x3064	Tool switch monitoring function is temporarily disable ¹⁾
0x3065	External axis coasting value setting is not done ¹⁾
0x3350	User coordinate is not registered
0x3370	Wrong control group

1 Only when the safety function (option) is valid

■ Return Value

- 0 API execution processor received the command.
 Return value = 0 and err_no = 0 Normal end
 Return value = 0 and err_no != 0 Error (Check err_no.)
- 1 API execution processor does not respond.



Two or more move instructions such as mpIMOV(), mpMOVJ(), mpMOVL(), mpPulseMOVJ(), and mpPulseMOVL(), mpManualMOV() cannot be called.
 If one of the manipulators is in operation by the move instruction above, the move instruction to the other manipulators cannot be issued.
 If done so, "0x2060: In error/alarm status" returns in err_no.

12 Motion Monitor/Control API

mpPulseMOVL



Exclusive control by using semaphores is executed between this API and the APIs listed below.

Thus, if this API is executed while one of the following APIs is already being executed, the task may be suspended to wait for the API to be completed. (waiting for semaphore acquisition)

Therefore, DO NOT use the following APIs for the task which must be performed periodically with high priority.

mpCancelError(), mpResetAlarm(), mpSetCycle(),
mpSetServoPower(), mpSetMasterJob(), mpSetCurJob(),
mpStartJob(), mpHold(), mpWaitForJobEnd(),
mpDeleteJob(), mpGetJogSpeed(), mpGetJogCoord(),
mpIMOV(), mpMOVJ(), mpMOVL(), mpPulseMOVJ(),
mpPulseMOVL(), mpManualMOV()

mpManualMOV

Moves the manipulator to the specified target position in the teach mode.



WARNING

- mpManualMOV() enables the jog operation by the external device, instead of the programming pendant.
DO NOT use this API for the controller to which the programming pendant is connected.
By the operation of this API, the manipulator may move without operator's intention and this may result in injury.

■ Syntax

```
LONG mpManualMOV (
MP_POS_DATA*   src_p,           /* Target position */
SHORT          Interpol[MP_GRP_NUM], /* Interpolation method */
SHORT          sSpeedLvl,       /* Speed level */
MP_STD_RSP_DATA* rData
);
```

■ Parameter

[src_p]

[in] The pointer to MP_POS_DATA

- MP_POS_DATA

Syntax:

```
typedef struct {
    CTRLG_T ctrl_grp; /* Control group to be controlled*/
    CTRLG_T m_ctrl_grp; /* Not used in this API */
    CTRLG_T s_ctrl_grp; /* Not used in this API */
    MP_GRP_POS_INFO grp_pos_info[MP_GRP_NUM];
} MP_POS_DATA;

typedef struct {
    MP_POS_TAG pos_tag;
    long pos[MP_GRP_AXES_NUM]; /* Correction amount data*/
} MP_GRP_POS_INFO;

typedef struct {
    UCHAR data[8]; /* Data attribution, detail is as follows */
    CTRLG_T ei_ctrl_grp; /* Not used in this API */
} MP_POS_TAG;
```

MP_GRP_AXES_NUM: Maximum axis number of the control group: 8
MP_GRP_NUM: Number of the control group: 32

12 Motion Monitor/Control API

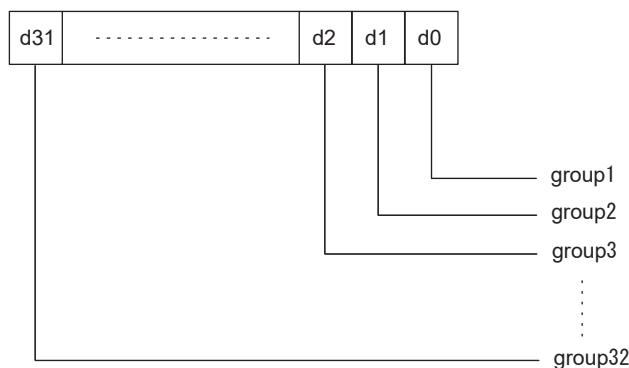
mpManualMOV

MP_POS_DATA.ctrl_grp

Specify target control groups in bits.

Retrieve the control group number of the target group by
mpCtrlGrpld2GrpNo().

Turn ON the bit which corresponds to the retrieved group number.



MP_POS_DATA.grp_pos_info[*]

Specify the data related to the target position of each group.

As the array number, specify the control group number retrieved by
mpCtrlGrpld2GrpNo().

(e.g.) In the case of a system "R1+B1+S1": the group number retrieved by
mpCtrlGrpld2GrpNo() is "R1=0, B1=1, S1=2".

To operate R1 and B1,

MP_POS_DATA.ctrl_grp=0x03(0000_0011)

Specify the R1 target position to MP_POS_DATA.grp_pos_info[0]

Specify the B1 target position to MP_POS_DATA.grp_pos_info[1]

To operate S1,

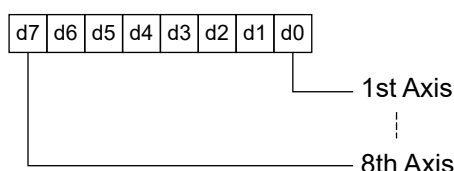
MP_POS_DATA.ctrl_grp=0x04(0000_0100)

Specify the S1 target position to MP_POS_DATA.grp_pos_info[2]

MP_GRP_POS_INFO[*].MP_POS_TAG.data[0]: axis configuration data

Specify the axis configuration of the target control group as bits.

When the standard type manipulator composed of 6-axes (S, L, U, R, B, T-axes) is used, the axis configuration data is
MP_POS_TAG.data[0]=0x3f(0011_1111).



MP_GRP_POS_INFO[*].MP_POS_TAG.data[2]: Tool file number

Specify the tool file number (0~63) of the manipulator position.

MP_GRP_POS_INFO[*].MP_POS_TAG.data[3]: Data type

Specify the data type of MP_GRP_POS_INFO.pos

Symbolic constant	Description
MP_PULSE_DTYPE	Pulse data
MP_ANGLE_DTYPE	Angle data ¹⁾

12 Motion Monitor/Control API

mpManualMOV

Symbolic constant	Description
MP_BF_DTYPE	Base coordinate system data
MP_RF_DTYPE	Robot coordinate system data
MP_UF_DTYPE	User coordinate system data

- 1 The "angle data" is different from the pulse data in the zero degree position and the plus/minus direction of each axis.

The valid data type of the base axis to specify is only "pulse data" and "base coordinate system data". For the station axis, only "pulse data" is valid. If other axes are specified, error #3630 (Error in data type) occurs.

MP_GRP_POS_INFO[*].MP_POS_TAG.data[4]: User coordinate number (1~63)
Specify the user coordinate number in case of the data type is the user coordinate number.

MP_GRP_POS_INFO[*].MP_POS_TAG.data[1], [5]~[7]: For future expansion

MP_GRP_POS_INFO[*].pos: Target position data

Array	Pulse type	Angle type	Cartesian type ¹⁾
pos[0]	1 st axis (S) pulse	1 st axis (S) angle [0.0001deg]	X-axis coordinate [μ m]
pos[1]	2 nd axis (L) pulse	2 nd axis (L) angle [0.0001deg]	Y-axis coordinate [μ m]
pos[2]	3 rd axis (U) pulse	3 rd axis (U) angle [0.0001deg]	Z-axis coordinate [μ m]
pos[3]	4 th axis (R) pulse	4 th axis (R) angle [0.0001deg]	Wrist angle Rx [0.0001deg]
pos[4]	5 th axis (B) pulse	5 th axis (B) angle [0.0001deg]	Wrist angle Ry [0.0001deg]
pos[5]	6 th axis (T) pulse	6 th axis (T) angle [0.0001deg]	Wrist angle Rz [0.0001deg]
pos[6]	7 th axis (E) pulse	7 th axis (E) angle [0.0001deg]	angle Re [0.0001deg]
pos[7]	8 th axis pulse	8 th angle [0.0001deg]	fig_ctrl (specify type)

1. As for the cartesian type, each value(e.g. x, y, z) and the type can be specified by casting MP_GRP_POS_INFO.pos with the following structure.

```
typedef struct {
    LONG    x;
    LONG    y;
    LONG    z;
    LONG    rx;
    LONG    ry;
    LONG    rz;
    LONG    exp;
    UCHAR   reserved[3];
    UCHAR   fig_ctrl;          /* type specification */
} MP_F_COORD;
```

12 Motion Monitor/Control API

mpManualMOV

<fig_ctrl>

Description of bit	Description
D0	0: Front 1:Back
D1	0: Upper arm 1:Lower arm
D2	0: Flip 1:Noflip
D3	0: R<180 1:R>=180
D4	0: T<180 1:T>=180
D5	0: S<180 1:S>=180

The position type variable data can be retrieved by mpGetPosVarData(). If the variable is a cartesian value, its figure specification can be also retrieved.

Set the state of D8~D13 (rData[0], retrieved by mpGetPosVarData()) to the fig_ctrl (the aforesaid MP_F_COORD type).

In addition, whether controlling the operation by referring to the given "form specification" or operating the manipulator by giving priority to the current figure is decided according to "S2C430: Relative job operation method specification".

S2C430: Specify the relative job operation method

- 0: Previous step regarded (give priority to the current manipulator figure)
- 1: TYPE prioritized (give priority to the given figure specification)

[sInterpol[*]]

[in] Specify the interpolation method

Specify the interpolation method of a manipulator to move to the target position. The specification of the "linear interpolation" to the base axis and the station axis is invalid.

Symbolic constant	Description
MP_JOINT_INTERPOLATE	Link interpolation
MP_LINEAR_INTERPOLATE	Linear interpolation

[sSpeedLv]

[in] Specify the speed level

Specify the operation speed level which corresponds to the controller's manual speed level.

Symbolic constant	Description
MP_MANUALSPD_LOW	Low speed
MP_MANUALSPD_MID	Middle speed
MP_MANUALSPD_HIGH	High speed
MP_MANUALSPD_MAX	Maximum speed

[rData]

[out] The pointer to the data structure which receives the standard data

- MP_STD_RSP_DATA

12 Motion Monitor/Control API

mpManualMOV

[out] The data structure which receives the standard data

Syntax:

```
typedef struct
{
    USHORT    err_no;
    CHAR      reserved[2];
}    MP_STD_RSP_DATA;
```

Member: <err_no>

Error number

Value	Description
0x0000	Normal end
0x2010	Robot is in operation
0x2030	In HOLD status (PP)
0x2040	In HOLD status (External)
0x2050	In HOLD status (Command)
0x2060	In error/alarm status
0x2070	In SERVO OFF status
0x2080	Wrong operation mode
0x3040	The home position is not registered
0x3050	Out of range (ABSO data)
0x3350	User coordinate is not registered
0x3360	Error in user coordinate file
0x3370	Wrong control group
0x3600	User coordinate is out of range.
0x3610	Error in axis construction data
0x3620	Error in specifying tool file number
0x3630	Error in data type
0x3640	Error in interpolation method
0x3650	Error in speed level specification

■ Return Value

- 0 API execution processor received the command.
 Return value = 0 and err_no = 0 Normal end
 Return value = 0 and err_no != 0 Error (Check err_no.)
- 1 API execution processor does not respond.

12 Motion Monitor/Control API

mpManualMOV

- By this API, only one group is available to be operated. (e.g. The operation of only R1, B1, or S1 is possible. The simultaneous operation of R1 and S1 is impossible.) However, a simultaneous operation of a pair manipulators and a base axis is possible. (e.g. It is possible to set the target position of R1 and B1, and operate them at the same time.)
- Two or more move instructions such as mpIMOV(), mpMOVJ(), mpMOVL(), mpPulseMOVJ(), and mpPulseMOVL(), mpManualMOV() cannot be called. If one of the manipulators is in operation by the move instruction above, the move instruction to the other manipulators cannot be issued. If done so, "0x2060: In error/alarm status" returns in err_no.

NOTE

Exclusive control by using semaphores is executed between this API and the APIs listed below. Thus, if this API is executed while one of the following APIs is already being executed, the task may be suspended to wait for the API to be completed. (waiting for semaphore acquisition)

Therefore, DO NOT use the following APIs for the task which must be performed periodically with high priority.

mpCancelError(), mpResetAlarm(), mpSetCycle(), mpSetServoPower(), mpSetMasterJob(), mpSetCurJob(), mpStartJob(), mpHold(), mpWaitForJobEnd(), mpDeleteJob(), mpGetJogSpeed(), mpGetJogCoord(), mpIMOV(), mpMOVJ(), mpMOVL(), mpPulseMOVJ(), mpPulseMOVL(), mpManualMOV()

13 Sensor Control API

mpReceiveSkillCommand

Receives a sensor command sent by executing SKILLSND instruction.

■ Syntax

```
STATUS mpReceiveSkillCommand
(
  int          sl_id,          /* Specifies communication area with the system*/
  SYS2MP_SENS_MSG *msg_p      /* Structure of sensor command data */
)
```

■ Description

This API receives the command (sensor command), which is sent from the system when SKILLSND instruction is executed or when the JOB execution after that is suspended, from the command area specified by *sl_id*. The character string specified by SKILLSND instruction is sent to SYS2MP_SENS_MSG structure specified by *msg_p*, so it is necessary to describe the process to distinguish its content.

Note that when an operation which interferes with the JOB execution (e.g. jog operation, JOB cursor move, and JOB selection) is performed after suspending the JOB, and then the JOB is started again, a sensor command is sent as the sub command “MP_SKILL_END”. After this command is sent, the suspended interpolation motion cannot be continued even by restarting, so stop the path correction process and initialize the path correction information.

■ Parameter

[*sl_id*]

Specifies the communication area with the system.
The following values can be specified.
Make sure to specify MP_SL_ID1 for the robot 1 (R1), and MP_SL_ID2 for the robot 2 (R2).

Symbolic constant	Description
MP_SL_ID1	Communication area 1
MP_SL_ID2	Communication area 2

13 Sensor Control API

mpReceiveSkillCommand

[msg_p]

Structure receiving the message from the system

typedef struct

```
{
int main_comm;           // main command
int sub_comm;            // sub command
int exe_tsk;             // task number of job in execution (0-15)
int exe_apl;             // application number of execution control group
char cmd[256];           // text command data sent by SKILLSND
int usr_opt;             // for future addition (reserved)
} SYS2MP_SENS_MSG;
```

To main_comm, one of the following values is sent from the system.

Value	Description
MP_SKILL_COMM 0	Skill command
Other than 0	Reserved by the system Do not perform the process to this command.

To sub_comm, one of the following values is sent from the system.

main_comm value	Value		Description
MP_SKILL_COMM	MP_SKILL_SEND	1	SKILLSEND
	MP_SKILL_END	2	SKILL command force-quit

SYS2MP_SENS_MSG Structure

• main_comm

(1) MP_SKILL_COMM

This is sent when the SKILLSND command is executed or force-quit.

Refer to sub_comm to check which is sent.

(2) Other than 0

This value is reserved by the system. Do not perform the process even if this is sent.

• sub_comm

(1) MP_SKILL_SEND

This is sent when the SKILLSND command is executed. After receiving this, the character string which is set as the tag of the SKILLSND command will be stored in cmd. Refer to this and perform the process according to the contents of the character string.

13 Sensor Control API

mpReceiveSkillCommand

(2) MP_SKILL_END

After completing MP_SKILL_SEND command process, if one of the operations listed below which makes the process unable to continue is performed and then the JOB execution is started, this command is sent.

- Job selection
- Job or file edit operation (including changing the cursor location of the job)
- Jog operation
- Step, [FWD], or [BWD] switching

(For example, when the start operation is performed after moving the cursor location of the job after switching the mode from the play mode to the teach mode during playback after executing the MP_SKILL_SEND command.)

After receiving the MP_SKILL_END command, stop the processes performed in real-time such as the path correction process.

- exe_tsk

The local task of the job to which SKILLSND is executed is stored.

- exe_apl

The application number of the control group (the application number used to refer to the specific I/O) to which SKILLSND is executed is stored.

“0” is set when SKILLSND is executed at the JOB of R1 and sub_comm is MP_SKILL_SEND. And, “1” is set when it is executed at the JOB of R2.

However, “0” is set when sub_comm is MP_SKILL_END.

- cmd

The pointer of the command (character string) specified by the SKILLSND command is stored.

“NULL” is set when sub_comm is MP_SKILL_END.

- usr_opt

This is provided for future addition (reserved).

Be sure to pass the content of *msg_p* to mpEndSkillCommandProcess. If the content of another member is modified, the completion of command receiving process may not be properly notified to the system.

■ Return Value

Symbolic constant: OK (value: 0) : Normal end

Symbolic constant: ERROR (value: -1) : Error

13 Sensor Control API

mpReceiveSkillCommand

SKILLSND instruction, as a control group, can be executed only by the JOB to which a manipulator or an external axis is defined. In case executing SKILLSND by a JOB to which a robot is not defined (without a control group), an alarm "4475: CANNOT EXECUTE JOB(NO ROBOT)" occurs. Also, when it is executed by a concurrent JOB, "4524:CANNOT EXECUTE INST(COUCUR JOB)" alarm occurs.

The communication area with the MotoPlus application which sends a command from SKILLSND is automatically selected to MP_SL_ID1 to the Robot 1 JOB and MP_SL_ID2 to the Robot 2 JOB.



For the coordinated JOB, the communication area is determined by the slave side manipulator.

JOB: R1+R2:R1(R1:master):

MP_SL_ID2 is selected because R2 is the slave.

JOB: R1+R2:R2(R2:master):

MP_SL_ID1 is selected because R1 is the slave.

For the exclusive job to the external axis, the communication area is not automatically selected. SLn tag should be put to SKILLSEND, and this tag selects the communication area.

SKILLSEND SL5 "command": MP_SL_ID1 is selected.

SKILLSEND SL6 "command": MP_SL_ID2 is selected.

mpEndSkillCommandProcess

After receiving the sensor command, this API notifies the system that the command process in the application is completed.

■ **Syntax**

```
void mpEndSkillCommandProcess
(
  int          sl_id,      /* Specifies communication area with the system */
  SYS2MP_SENS_MSG *msg_p /* Structure of sensor command data */
)
```

■ **Description**

This API is called when the task which received a command completes preparations for path correction process according to the content of the command. The system waits without executing SKILLSND instruction until it receives the notification by this API. Upon receiving the notification by this API, the system completes SKILLSND instruction and proceed to the next instruction. With this mechanism, it is possible to prevent the move instruction under path correction or speed change from starting before the application is ready.

■ **Parameter**

[*sl_id*]

Specifies the communication area with the system.
The following values can be specified.
Make sure to specify MP_SL_ID1 for the robot 1 (R1), and MP_SL_ID2 for the robot 2 (R2).

Symbolic constant	Description
MP_SL_ID1	Communication area 1
MP_SL_ID2	Communication area 2

[*msg_p*]

Structure receiving the message from the system

```
typedef struct
{
  int main_comm;      // main command
  int sub_comm;       // sub command
  int exe_tsk;        // task number of job in execution (0-15)
  int exe_apl;        // application number of execution control group
  char cmd[256];      // text command data sent by SKILLSND
  int usr_opt;        // for future addition (reserved)
} SYS2MP_SENS_MSG;
```

13 Sensor Control API

mpEndSkillCommandProcess

■ Sample Code

The following is the sample of process to receive the command from the system by using mpReceiveSkillCommand() and mpEndSkillCommandProcess().

```
void sensCommRcvTask(void)
{
    SYS2MP_SENS_MSG msg;
    memset(&msg, CLEAR, sizeof(SYS2MP_SENS_MSG));

    FOREVER
    {
        // Notification of command process completion. SKILLSND instruction
        // completes the process, and executes the next instruction.
        mpEndSkillCommandProcess (MP_SL_ID1, &msg);

        // Waits for command from the system.
        if (mpReceiveSkillCommand (MP_SL_ID1, &msg) == ERROR)
        {
            printf("mpReceiveSensComm Error\n\r");
            break;
        }
        else
        { // Command process is described in the following.
            :
        }
    }
}
```

13 Sensor Control API

mpEndSkillCommandProcess

In the programmer's manual, it is described that mpEndSkillCommandProcess() and mpReceiveSkillCommand() are combined to create the API referred to as receiveSkillCommand() and called as follows:

```
void sensCommandRcvTask(void)
{
    SYS2MP_SENS_MSG msg;
    int rc;
    memset(&msg, CLEAR, sizeof(SYS2MP_SENS_MSG));
    FOREVER
    {
        if (receiveSkillCommand (&msg) == ERROR)
            break;
        :
    }
}

// Process to receive command from SKILLSND
STATUS receiveSkillCommand (SYS2MP_SENS_MSG *msg)
{
    // Returns as command-completed response. Execution proceeds to the next step of SKILLSND.
    int rc;
    mpEndSkillCommandProcess (MP_SL_ID1, msg);
    // Receives command sent by SKILLSND.
    rc= mpReceiveSkillCommand(MP_SL_ID1, msg) ;
    return(rc);
}
```

■ Return Value

None

mpMeiGetJobExecTask

Retrieves the task (local task) number of the JOB in execution.

■ Syntax

```
int mpMeiGetJobExecTask
(
  int          sl_id,      /* Specifies communication area with the system */
  EXEJT_T      *dst_p     /* Pointer to data structure returned by the API */
)
```

■ Description

This routine returns the task number (master task and sub 1 task to sub 15 task) of the current JOB in execution as 32-bit status (d0 to d15: ignoring higher 16-bit).

The task executing a job in the independent control function may be called “local task”.

This word is used descriptively to avoid confusion (conflict) with the application task that the operating system executes, but both words (“task” and “local task”) are synonymous.

■ Parameter

[*sl_id*]

Specifies the communication area with the system.

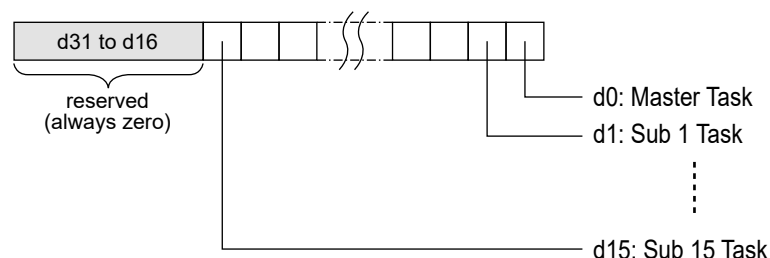
The following values can be specified.

Make sure to specify MP_SL_ID1 for the robot 1 (R1), and MP_SL_ID2 for the robot 2 (R2).

Symbolic constant	Description
MP_SL_ID1	Communication area 1
MP_SL_ID2	Communication area 2

[*dst_p*]

The task (local task) executing JOB.



■ Return Value

Symbolic constant: OK (value: 0) : Normal end

Symbolic constant: ERROR (value: -1) : Error

mpMeiGetInterpolation

Retrieves interpolation type of the move instruction of JOB in execution.

■ Syntax

```
int mpMeiGetInterpolation
(
    int      sl_id,          /* Specifies communication area with the system */
    int      exe_tsk        /* Task number of JOB in execution */
)
```

■ Description

This routine returns the interpolation of the move instruction executed in the task (local task) specified by *exe_tsk*.

■ Parameter

[*sl_id*]

Specifies the communication area with the system.

The following values can be specified.

Make sure to specify MP_SL_ID1 for the robot 1 (R1), and MP_SL_ID2 for the robot 2 (R2).

Symbolic constant	Description
MP_SL_ID1	Communication area 1
MP_SL_ID2	Communication area 2

[*exe_tsk*]

Specified task (local task)

Symbolic constant	Description
0	Master task
1	Sub task 1
2	Sub task 2
...	...
15	Sub task 15

■ Return Value

One of the following values Normal end

Symbolic constant	Value
MP_MOVJ_COM	0
MP_MOVL_COM	1
MP_MOVC_COM	2
MP_EIMOVL_COM	3
MP_EIMOVC_COM	4
MP_MOVS_COM	5

-1

Error

mpMeiGetExecControlGroup

Retrieves the control group of the path of the move instruction in execution (instruction in interpolation process).

■ Syntax

```
int mpMeiGetExecControlGroup
(
  int          sl_id,          /* Specifies communication area with the system */
  int          exe_tsk,        /* Task number of JOB in execution */
  CTRLG_T      *ctrl_grp      /* Pointer to data structure returned by the API */
)
```

■ Description

This routine returns the control group information executed in the task (local task) specified by *exe_tsk*.

■ Parameter

[*sl_id*]

Specifies the communication area with the system.

The following values can be specified.

Make sure to specify MP_SL_ID1 for the robot 1 (R1), and MP_SL_ID2 for the robot 2 (R2).

Symbolic constant	Description
MP_SL_ID1	Communication area 1
MP_SL_ID2	Communication area 2

[*exe_tsk*]

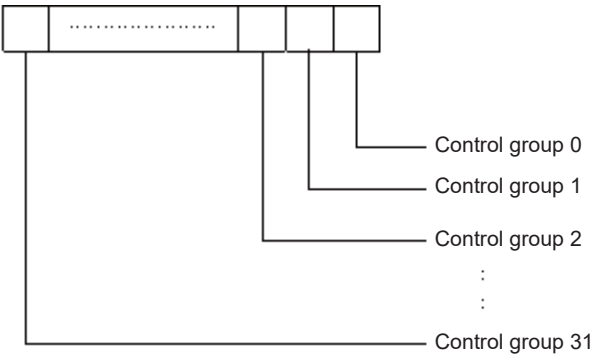
Specified task (local task)

Constant	Description
0	Master task
1	Sub task 1
2	Sub task 2
...	...
15	Sub task 15

13 Sensor Control API
mpMeiGetExecControlGroup

[ctrl_grp]

Control group
Specify the target group in bits.
Retrieve the control group number of the target group by
mpCtrlGrpId2GrpNo().



■ Return Value

Symbolic constant: OK (value: 0) : Normal end
Symbolic constant: ERROR (value: -1) : Error

mpMeiPutCorrPath

Performs position correction for the path that the robot is moving on with the move instruction in the JOB.

■ Syntax

```
int mpMeiPutCorrPath
(
  int          sl_id,      /* Specifies communication area with the system */
  MP_POS_DATA *src_p      /* Pointer of structure to set path correction data */
)
```

■ Description

This routine sets the amount of path correction to the robot of the control group indicated by *src_p* -> *ctrl_grp*. The specified correction amount is calculated as the path correction amount, and the corrected value is added by the system. For example, let the interpolation cycle be 4 msec and the correction amount be 1 mm in Y-direction per interpolation cycle, and then the path is corrected 250 mm in one second.

■ Parameter

[*sl_id*]

Specifies the communication area with the system.

The following values can be specified.

Make sure to specify MP_SL_ID1 for the robot 1 (R1), and MP_SL_ID2 for the robot 2 (R2).

Symbolic constant	Description
MP_SL_ID1	Communication area 1
MP_SL_ID2	Communication area 2

[*src_p*]

Path correction information

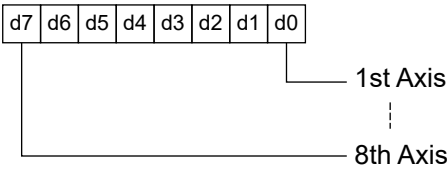
```
typedef struct {
  CTRLG_T  ctrl_grp;      /* Target control group for increment value move */
  CTRLG_T  m_ctrl_grp;     /* It's not used in this API(Used in mpMeiIncrementMove*/
  CTRLG_T  s_ctrl_grp;     /* It's not used in this API(Used in mpMeiIncrementMove*/
  MP_GRP_POS_INFO grp_pos_info[MP_GRP_NUM];
} MP_POS_DATA;
```

13 Sensor Control API
mpMeiPutCorrPath

```
typedef struct {
MP_POS_TAG      pos_tag;
long            pos[MP_GRP_AXES_NUM]; /* Details of data attribute
                                         are shown below */
} MP_GRP_POS_INFO;

typedef struct {
UCHAR          data[8]; /* Correction amount data */
CTRLG_T        ei_ctrl_grp; /* For EIMOV robot control group specification */
} MP_POS_TAG;
```

MP_GRP_AXES_NUM: Maximum number of control axes of control group 8
MP_GRP_NUM: Maximum number of control groups 32
MP_POS_TAG.data[0]: Axis configuration information



MP_POS_TAG.data[2]: Not in use
MP_POS_TAG.data[3]: Data type

Symbolic constant	Description
MP_CORR_BF_DTYPE	MP_FBASE_TYPE MP_MASK_INC_DATA
MP_CORR_RF_DTYPE	MP_FROBOT_TYPE MP_MASK_INC_DATA
MP_CORR_TF_DTYPE	MP_FTOOL_TYPE MP_MASK_INC_DATA
MP_CORR_UF_DTYPE	MP_FUSER_TYPE MP_MASK_INC_DATA

MP_POS_TAG.data[4]: User coordinate number (0 to 62: valid only for user coordinate)
MP_POS_TAG.data[1], [5] to [7]: For future extensions
MP_GRP_POS_INFO pos

Array	Pulse	Cartesian
pos [0]	1st axis (S) pulse value	X-axis coordinate (unit: micron)
pos [1]	2nd axis (L) pulse value	Y-axis coordinate (unit: micron)
pos [2]	3rd axis (U) pulse value	Z-axis coordinate (unit: micron)
pos [3]	4th axis (R) pulse value	Wrist angle Rx (unit: 0.0001 deg)
pos [4]	5th axis (B) pulse value	Wrist angle Ry (unit: 0.0001 deg)
pos [5]	6th axis (T) pulse value	Wrist angle Rz (unit: 0.0001 deg)
pos [6]	7th axis (E) pulse value	Angle Re (unit: 0.0001 deg)
pos [7]	8th axis pulse value	8th axis pulse value (micron in the case of traveling axis)

13 Sensor Control API

mpMeiPutCorrPath

■ Return Value

0 Normal end

One of the following negative values Error

Symbolic constant (value)	Description
E_MP_ARG_OBJ_ID (-1)	Incorrect sl_id
E_MP_CTRL_GRP (-3)	Incorrect control group
E_MP_AXES_CONFIG (-4)	Incorrect axis configuration information
E_MP_TOOL_NO (-5)	Incorrect tool number
E_MP_POS_DTYPE (-6)	Incorrect data type
E_MP_USER_FRAME (-7)	Incorrect user coordinate setting
E_MP_INACT_SKILLCMD (-14)	Error indicating the API was called without executing a SKILLSND instruction



The correction data received with the system, by calling this API, is volatile (disappears, once read and retrieved). Then usually, the source task of calling this API needs to be activated with a constant cycle (such as the interpolation cycle), to achieve real-time tracking. Consequently, the program structure that periodically calls this API is achieved while the path correction is necessary. However, if the program calls the API irregularly (asynchronously), no contradiction occurs in the API.

■ Sample Code

The following is the sample code for path correction. When correct_req is set by the command receiving task, the path is corrected 1 mm in Y-direction per interpolation cycle. (For the sample including the command receiving task, refer to the programmer's manual.)

Although the path correction process is FOREVER (infinite loop), mpClkAnnounce(MP_INTERPOLATION_CLK); is described at the starting of the loop so that the path correction process synchronizes with the interpolation cycle.

```
int correct_req;
void pathCorrectionTask(void)
{
    MP_POS_DATA corrpath_src_p;

    // Initializes correction amount data
    memset(&corrpath_src_p, CLEAR, sizeof(MP_POS_DATA));
    corrpath_src_p.ctrl_grp = 1; // Robot1
    corrpath_src_p.grp_pos_info[0].pos_tag.data[0] = 0x3f; // Effective axis
                                                             specification (6 axes)
```

13 Sensor Control API

mpMeiPutCorrPath

```
corrpath_src_p.grp_pos_info[0].pos_tag.data[2] = 0;
corrpath_src_p.grp_pos_info[0].pos_tag.data[3] = MP_CORR_BF_DTYPE;
// Corrects in base coordinates

corrpath_src_p.grp_pos_info[0].pos[0] = 0; //X
corrpath_src_p.grp_pos_info[0].pos[1] = 0; //Y
corrpath_src_p.grp_pos_info[0].pos[2] = 0; //Z
corrpath_src_p.grp_pos_info[0].pos[3] = 0; //Rx
corrpath_src_p.grp_pos_info[0].pos[4] = 0; //Ry
corrpath_src_p.grp_pos_info[0].pos[5] = 0; //Rz

FOREVER
{
    mpClkAnnounce(MP_INTERPOLATION_CLK); // Performs the following
                                         process per interpolation cycle

    if ( correct_req == ON ) // Request for correction? Set by command receiving task.
    {
        //PutCorrPath (1 [mm] correction in Y-direction per interpolation cycle
        corrpath_src_p.grp_pos_info[0].pos[1] = 1000;
        mpMeiPutCorrPath(MP_SL_ID1, &corrpath_src_p);
    }
}
}
```

mpMeiPutForcePathEnd

Force-quits the move instruction in execution, and changes the motion to the next step.

However, using this API is not recommended.

If this API is used, an unintended move instruction may be canceled.

Adding an UNTIL instruction to the move instruction in the JOB is recommended instead of using this API.

This API remains available to maintain backwards compatibility with existing MotoPlus applications.

■ Syntax

```
int mpMeiPutForcePathEnd
(
    int          sl_id,          /* Specifies communication area with the system */
    CTRLG_T ctrl_grp /* Effective control groups specification */
)
```

■ Description

This routine force-quits the move instruction of the robot specified by *ctrl_grp* currently in execution.

Thus, the software of the system (simply called “system” hereafter) stops calculating the position command data of the current step, and starts calculating the position command data from the current position to the next step (teaching position). At this time, the current step terminates without deceleration, and motion to the next step starts.

This request is received only once by the move instruction in execution (current step). Thus, the following step is not affected.

■ Parameter

[sl_id]

Specifies the communication area with the system.

The following values can be specified.

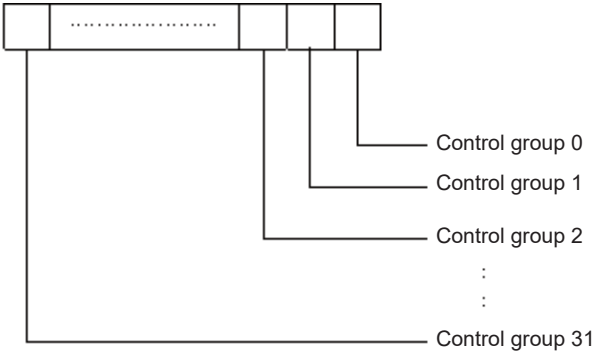
Make sure to specify MP_SL_ID1 for the robot 1 (R1), and MP_SL_ID2 for the robot 2 (R2).

Symbolic constant	Description
MP_SL_ID1	Communication area 1
MP_SL_ID2	Communication area 2

13 Sensor Control API
mpMeiPutForcePathEnd

[ctrl_grp]

Control group
Specify the target group in bits.
Retrieve the control group number of the target group by
mpCtrlGrpId2GrpNo().



■ Return Value

Symbolic constant: OK (value: 0) : Normal end
Symbolic constant: ERROR (value: -1) : Error

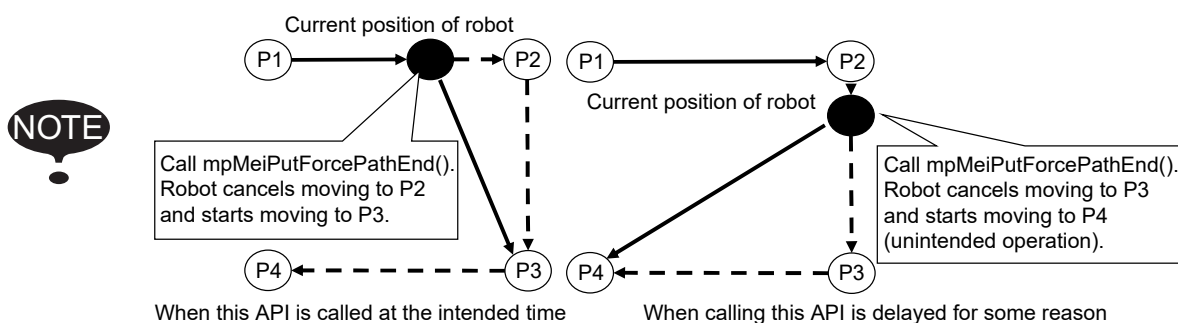
13 Sensor Control API mpMeiPutForcePathEnd

To forcibly change the interpolated motion, adding an UNTIL instruction to the move instruction in the JOB is recommended instead of using this API.

If this API is used, the move instruction for which the interpolated motion was force-quit cannot be identified and the robot may move in an unintended manner.

For example, take a JOB that moves the current position of the robot to four points in the following order: P1, P2, P3, and P4. The API is to be called by inputting an external signal to move the robot to P3 while this JOB is moving from P1 to P2. If this API is called while the current position of the robot is between P1 and P2, the robot cancels moving to P2 and moves to P3. Now assume that this API was called when the external signal input was delayed for some reason and the robot was moving from P2 to P3. In this case, the robot cancels moving to P3 and starts moving to P4.

Since the timing at which the interpolated motion is switched with this API cannot be restricted to a specific segment, the robot may move in an unintended manner.



If the UNTIL instruction is used, there is no risk of the above behavior because the motion to cancel while moving between segments can be specified.

<Example JOB Code>

```

:
MOVL                               ; Move to point P1.
MOVL UNTIL IN#(1)=ON              ; Move to point P2. (When IN#(1)=ON, forcibly complete
                                ; this instruction and execute the next instruction.)
MOVL                               ; Move to point P3.
MOVL                               ; Move to point P4.
:

```


mpMeiPutSpdOverride

Sets a speed override amount.

■ **Syntax**

```
int mpMeiPutSpdOverride
(
    int          sl_id,          /* Specifies communication area with the system */
    CTRLG_T      ctrl_grp,       /* Effective control groups specification */
    long         src_p[MP_GRP_NUM] /* Speed override ratio(1000~15000) */
)
```

MP_GRP_NUM: Maximum number of control groups 32

■ **Description**

This routine sets a speed override amount (ratio) to the robot specified by *ctrl_grp*, in the range of 10.00% to 150.00% (minimum resolution: 1/100 [%], valid integer value: 1,000 to 15,000).

Once this value is set, the value is used consecutively for the move instruction in motion. To cancel this, turn OFF all the bits specified by *ctrl_grp* and call this API. Then, the speed override amount from the application is canceled.

```
/* speed override cancel */
rc = mpMeiPutSpdOverride(sl_id, OFF, NULL);
```

■ **Parameter**

[sl_id]

Specifies the communication area with the system.
The following values can be specified.
Make sure to specify MP_SL_ID1 for the robot 1 (R1), and MP_SL_ID2 for the robot 2 (R2).

Symbolic constant	Description
MP_SL_ID1	Communication area 1
MP_SL_ID2	Communication area 2

13 Sensor Control API

mpMeiPutSpdOverride

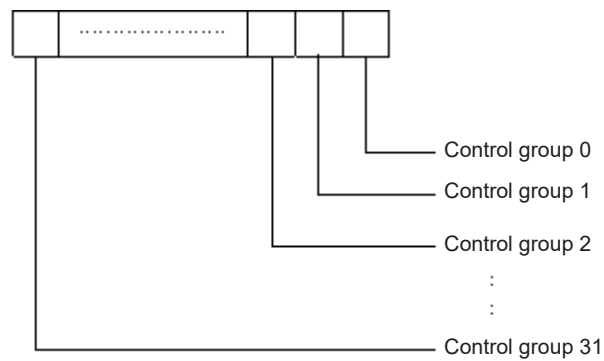
[ctrl_grp]

Control group

Specify the target group in bits.

Retrieve the control group number of the target group by
mpCtrlGrpId2GrpNo().

Specify OFF (all bits turned OFF) to make all the speed override amount
from MotoPlus invalid.



[src_p[MP_GRP_NUM]]

Speed override amount (0 to 15,000, unit: 1/100 [%])

■ Return Value

Symbolic constant: OK (value: 0) : Normal end

Symbolic constant: ERROR (value: -1) : Error

mpMeilIncrementMove

Sets an increment value move per each interpolation cycle to the manipulator which is executing WAIT instruction. The coordinated and synchronized operation functions of the slave side over the master side control group operation is equipped.



To use the increment value move function, using the mpExRcsIncrementMove() API is recommended.

Using mpMeilIncrementMove() involves additional procedures because the SKILLSND instruction is required in the JOB that operates in conjunction with the API. mpExRcsIncrementMove() is a version of the API that fixes this issue.

mpMeilIncrementMove() remains available to maintain backwards compatibility with existing MotoPlus applications.

■ Syntax

```
int mpMeilIncrementMove
(
  int          sl_id,          /* Specifies the communication area with the system */
  MP_POS_DATA *src_p          /* Pointer to the structure that sets the data
                               for path correction*/
)
```

■ Description

This routine sets the increment move value to src_p->ctrl_grp indicating control group manipulators. This API adds the specified increment move value to the manipulator's present position. By this operation, the manipulator moves for increment value move like JOG operation.

To execute this operation, SKILLSND instruction had been executed and WAIT instruction is being executed by the JOB execution.

In case JOB execution is stopped by the HOLD operation or change of the mode, this correcting operation also stops. And then, the increment value move is resumed when the JOB is re-started.

SKILLSND instruction should be executed again in case the cursor of the JOB is moved or a JOB is newly selected.

In the following JOB example, by the execution of "INCMOVE", it prepares to receive the specification of increment value move from the MotoPlus application. And then, the increment value is specified by the execution of "mpMeilIncrementMove()". The specification of increment value is received while the JOB is executing WAIT instruction. However, in case input IN#(1) is turned ON, the following MOVL instruction is executed after WAIT instruction is passed through.

At this time, the correction is ignored even if the increment value is specified by mpMeilIncrementMove(). (mpMeilIncrementMove() API returns as E_MP_MOVE_BUSY (-9) to its return value.)

For example, when setting 4msec as its interpolation cycle and gives 1mm per interpolation cycle in Y-direction, the manipulator moves 250mm in 1 second.

13 Sensor Control API mpMeilIncrementMove

<JOB example>

:

MOVL C00000 V=100.0

SKILLSND "INCMOVE"

WAIT IN#(1) = ON // Increment value move is possible
only while this WAIT instruction is being executed.

MOVL C00001 V=100.0

This specification is available not only for the manipulator but also to the external axis (station axis). When executing the increment value move using this API from an exclusive JOB to the external axis, specification of communication area indicating tag "SLn (SL5 or SL6)" is required. This is because the external axis has no specified communication area whereas Robot 1 is specified with communication area 1 and Robot 2 is specified with communication area 2 naturally.

As for the tag which indicates the communication area to SKILLSND instruction, SL5 indicates the communication area 1 and SL6 indicates the communication area 2.

In the system with one manipulator and one external axis, the exclusive JOB to the external axis specifies SL6 as its communication area because, to avoid the area to overlap, the communication area 1 is used when SKILLSND instruction is executed at Robot 1 side.

<JOB example>

:

MOVL EC00000 V=100.0

SKILLSND SL6 "INCMOVE"

WAIT IN#(1) = ON // Increment value move is possible
only while this WAIT instruction is being executed.

■ Parameter [sl_id]

Specifies the communication area with the system.
The following values can be specified.

Specify MP_SL_ID1 to the Robot 1 (R1) and MP_SL_ID2 to the Robot 1 (R2).

When it is a coordinated JOB, specification of the area depends on the slave side manipulator.

When it is an exclusive JOB to the external axis, specify MP_SL_ID1 when SL5 is specified by SKILLSND in the JOB and specify MP_SL_ID2 when SL6 is specified.

Symbolic constant	Description
MP_SL_ID1	Communication area 1
MP_SL_ID2	Communication area 2

13 Sensor Control API

mpMeilncrementMove

[src_p]

Increased move value information

```
typedef struct {
    CTRLG_T  ctrl_grp;    /* Object control group for increment value move*/
    CTRLG_T  m_ctrl_grp; /* Master side control group for coordinated (synchronized) operation */
    CTRLG_T  s_ctrl_grp; /* Slave side control group for coordinated (synchronized) working operation */
    MP_GRP_POS_INFO  grp_pos_info[MP_GRP_NUM];
} MP_POS_DATA;
```

- ctrl_grp
Control group of the object operation
- m_ctrl_grp
Master side control group when the synchronized operation is executed. It should be the same control group as (ctrl_grp).
Set "0" when the synchronized operation is not executed.
- s_ctrl_grp
Slave side control group which is synchronized with the master side control group when the synchronized operation is executed.
Set "0" when the synchronized operation is not executed.

< Errors>

Followings are the API errors when setting the control group information.

1.

- When the specified m_ctrl_grp is not "0" and the specified m_ctrl_grp is not found (does not exist) in the system configuration.
- When s_ctrl_grp is not "0" and the specified m_ctrl_grp is "0".
- When more than one control groups are specified.
- When more than one control groups are specified as a master.
→E_MP_M_CTRL_GRP (-12)
Master side control group specifying error

2.

- When the specified m_ctrl_grp is not "0" and the specified s_ctrl_grp is not found (does not exist or "0" is set) in the system configuration.
→E_MP_S_CTRL_GRP (-13)
Slave side control group specifying error

3. When overlapped control groups exist in the specified m_ctrl_grp and s_ctrl_grp.

- E_MP_SYNC_MOVE_CTRL_GRP (-11)
Control group specifying error in the coordinated operation

13 Sensor Control API

mpMeilIncrementMove

4. When "0" is set to either specified m_ctrl_grp or s_ctrl_grp.

→E_MP_M_CTRL_GRP (-12)

Master side control group specifying error

→E_MP_S_CTRL_GRP (-13)

Slave side control group specifying error

5. When R2 is operated to start in the MotoPlus application in which MP_SL_ID1 received the SKILLSND instruction executed from R1 JOB but the JOB of R2 is already executed.

→E_MP_USING_CTRL_GRP (-15)

A specified control group JOB is being executed.

R1 and R2 system control group setting and behavior

<setting example>

m_ctrl_grp	m_ctrl_grp	s_ctrl_grp	Operation
1	0	0	R1 independent operation
1	1	2	Operate R1 and R2 synchronizes
1	2	1	Error E_MP_SYNC_MOVE_CTRL_GRP (-11) Control group specifying error in the coordinated operation
1	0	1	Error E_MP_M_CTRL_GRP (-12) Master side control group specifying error
1	1	0	Error E_MP_S_CTRL_GRP (-13) Slave side control group specifying error
1	1	1	Error E_MP_SYNC_MOVE_CTRL_GRP (-11) Control group specifying error in the coordinated operation
1	0	2	Error E_MP_M_CTRL_GRP (-12) Master side control group specifying error
1	2	0	Error E_MP_S_CTRL_GRP (-13) Slave side control group specifying error
2	0	0	R2 independent operation
2	1	2	Error E_MP_SYNC_MOVE_CTRL_GRP (-11) Control group specifying error in the coordinated operation
2	2	1	Operate R1 and R2 synchronizes
2	0	1	Error E_MP_M_CTRL_GRP (-12) Master side control group specifying error
2	1	0	Error E_MP_S_CTRL_GRP (-13) Slave side control group specifying error
2	0	2	Error E_MP_M_CTRL_GRP (-12) Master side control group specifying error

13 Sensor Control API
mpMeilIncrementMove

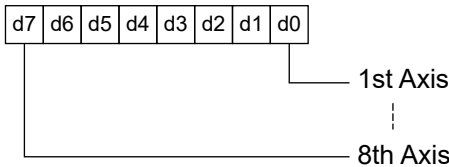
m_ctrl_grp	m_ctrl_grp	s_ctrl_grp	Operation
2	2	0	Error E_MP_S_CTRL_GRP (-13) Slave side control group specifying error
2	2	2	Error E_MP_SYNC_MOVE_CTRL_GRP (-11) Control group specifying error in the coordinated operation
3	3	1	Error E_MP_M_CTRL_GRP (-12) Master side control group specifying error

```
typedef struct {  
    MP_POS_TAG    pos_tag;  
    long          pos[MP_GRP_AXES_NUM];  
                /* For the property and details of the data, see below*/  
} MP_GRP_POS_INFO;
```

```
typedef struct {  
    UCHAR        data[8];    /* Correction amount data */  
    CTRLG_T      ei_ctrl_grp;  
                /* Control group specification when EIMOV robot */  
} MP_POS_TAG;
```

MP_GRP_AXES_NUM: Maximum axis number of the control group (8)
MP_GRP_NUM: Number of the control group (32)

MP_POS_TAG.data[0]: Information of axis configuration



MP_POS_TAG.data[2]: Tool file number
The tool file number used when increment value move is executed in the tool coordinate system.

13 Sensor Control API

mpMeilIncrementMove

MP_POS_TAG.data[3]: Data type

Symbolic constant	Description
MP_INC_PULSE_DTYPE	Pulse data (correction of the present position (INC operation))
MP_INC_ANGLE_DTYPE	Angle data (correction of the present position (INC operation))
MP_INC_BF_DTYPE	Base coordinate system data (correction of the present position (INC operation))
MP_INC_RF_DTYPE	Robot coordinate system data (correction of the present position (INC operation))
MP_INC_TF_DTYPE	Tool coordinate system data (correction of the present position (INC operation))
MP_INC_UF_DTYPE	User coordinate system data (correction of the present position (INC operation))

MP_POS_TAG.data[4]: User coordinate number
(0 to 62: valid only when the user coordinate is specified)



Angle can also be specified for the data type. However, for robot controllers up to the DX200, there are axes on which the direction of the angle can be different from the one of the pulses. For the YRC1000/YRC1000micro, however, the direction of the angle is always the same as the direction of the pulses. For this reason, an application that moves a robot using absolute angles may reverse the movement direction of the robot if the DX200 program code runs on the YRC1000/YRC1000micro unmodified. When porting DX200 code to the YRC1000/YRC1000micro, check the direction of robot movement and correct the code if necessary.

A measure is available to keep the source code for DX200 and earlier robot controllers and for the YRC1000/YRC1000micro the same. Set parameter S2C1350 in the YRC1000/YRC1000micro to 0 (default value = 1) to use the same specification as the DX200 and earlier robot controllers. Then, the positive direction of the angle and the positive direction of the pulses for each axis become the same as those for the DX200.

MP_POS_TAG.data[1], [5] to [7]: For the future extension

13 Sensor Control API mpMeilncrementMove

MP_GRP_POS_INFO pos

Array	Pulse type	Angle type	Cartesian type
pos[0]	1st axis (S) pulse	1st axis (S) angle [0.0001deg]	X-axis coordinate [μm]
pos[1]	2nd axis (L) pulse	2nd axis (L) angle [0.0001deg]	Y-axis coordinate [μm]
pos[2]	3rd axis (U) pulse	3rd axis (U) angle [0.0001deg]	Z-axis coordinate [μm]
pos[3]	4th axis (R) pulse	4th axis (R) angle [0.0001deg]	Wrist angle Rx [0.0001deg]
pos[4]	5th axis (B) pulse	5th axis (B) angle [0.0001deg]	Wrist angle Ry [0.0001deg]
pos[5]	6th axis (T) pulse	6th axis (T) angle [0.0001deg]	Wrist angle Rz [0.0001deg]
pos[6]	7th axis (E) pulse	7th axis (E) angle [0.0001deg]	Wrist angle Rx [0.0001deg]
pos[7]	8th axis pulse	8th axis angle [0.0001deg]	8th axis pulse value (micron in the case of traveling axis)

■ Return Value

0 Normal end

One of the following negative values Error

Symbolic constant (value)	Description
E_MP_ARG_OBJ_ID (-1)	sl_id error
E_MP_ACCESS_LIMIT (-2)	Access limited
E_MP_CTRL_GRP (-3)	Control group error
E_MP_AXES_CONFIG (-4)	Axis configuration information error
E_MP_TOOL_NO (-5)	Tool number error
E_MP_POS_DTYPE (-6)	Data type error
E_MP_USER_FRAME (-7)	User coordinate setting error
E_MP_JOG_BUSY (-8)	JOG is in operation
E_MP_MOVE_BUSY (-9)	Move command is being executed
E_MP_NOT_UNDER_PLAYBACK-10	Not being playback operation
E_MP_SYNC_MOVE_CTRL_GRP-11	Control group error in coordinated operation
E_MP_M_CTRL_GRP (-12)	Master side control group specifying error
E_MP_S_CTRL_GRP (-13)	Slave side control group specifying error
E_MP_INACT_SKILLCMD (-14)	Calling API when SKILLSND instruction is not executed
E_MP_USING_CTRL_GRP (-15)	Specified control group JOB is being executed
E_MP_ROB_USE_SL_ID (-16)	Wrong SL specification is used in SKILLSND

13 Sensor Control API mpMeilIncrementMove

In the environment using the independent control function, when a specified control group which executes the increment value move by this API in a job local task, the control group can execute the increment value move only in the same local task until the job local task completes. When the control group tries to execute the increment value move in the different job local tasks, this API returns “-3: Control group error”, and does not execute the increment value move.



For example, when R1 executes the increment value move by this API in the master local task and then R1 executes the increment value move by this API in the sub 1 local task, this API returns “-3: Control group error”.

When this API is executed in the environment using the independent control function, create a job so as to make the control group and the job local task to execute the increment value move the same.

For example, when executing the increment value move using this API in the dual-robot system, create a job so as to make the control group and the job local task to execute the increment value move the same by executing the increment value move respectively by R1 to execute the WAIT command only in the sub 1 local task and by R2 in the sub 2.



The correction data received by the system after calling of this API is valid only once and for all. This means that the calling task for this API should be periodical (For example, it starts in the interpolation cycle.).

Consequently, the program structure which can periodically call this API for the increment value move is realized.

13 Sensor Control API

mpMeilIncrementMove

■ Sample code

Followings are the sample code of the increment value move operation. When incmove_req is set, the manipulator moves 1mm to Y-direction per interpolation cycle by the task received by the command.

The increment value move process is FOEVER (exists in the infinite loop). And this process should be executed in coordination with the interpolation cycle, thus mpClkAnnounce(MP_INTERPORLATION_CLK) is indicated to the head of the loop.

```
int incmove_req;
void pathCorrectionTask(void)
{
    MP_POS_DATA corrpah_src_p;

    // Initialization of the correction data
    memset(&corrpah_src_p, CLEAR, sizeof(MP_POS_DATA));
    corrpah_src_p.ctrl_grp = 1; // Robot1
    corrpah_src_p.grp_pos_info[0].pos_tag.data[0] = 0x3f;
                                //Specification of the available axis (for 6 axes)
    corrpah_src_p.grp_pos_info[0].pos_tag.data[2] = 0;
    corrpah_src_p.grp_pos_info[0].pos_tag.data[3] =
MP_CORR_BF_DTYPE;
    //Increment value move operation on the base coordinate system
    corrpah_src_p.grp_pos_info[0].pos[0] = 0; //X
    corrpah_src_p.grp_pos_info[0].pos[1] = 0; //Y
    corrpah_src_p.grp_pos_info[0].pos[2] = 0; //Z
    corrpah_src_p.grp_pos_info[0].pos[3] = 0; //Rx
    corrpah_src_p.grp_pos_info[0].pos[4] = 0; //Ry
    corrpah_src_p.grp_pos_info[0].pos[5] = 0; //Rz

    FOEVER
    {
        mpClkAnnounce(MP_INTERPOLATION_CLK);
                                //Execute following process per interpolation
                                cycle

        if (incmove_req == ON )
            //Correction required? It is set by the task received by the command
        {
            //PutCorrPath (Correct 1[mm] in Y-direction per interpolation cycle
            corrpah_src_p.grp_pos_info[0].pos[1] = 1000;
            mpMeilIncrementMove (MP_SL_ID1, &corrpah_src_p);
        }
    }
}
```

14 Robot Motion Control API

mpExRcsIncrementMove

Sets the increment move amount per interpolation period to the manipulator executing the WAIT instruction. This API has a synchronized operation function, but a control group that includes an external axis cannot be specified as the master control group.



To use the increment value move function, using the mpExRcsIncrementMove() API is recommended. This API is an improved version of mpMeilIncrementMove() that can be used more easily and no longer requires additional procedures involving the SKILLSND instruction.

■ Syntax

```
int mpExRcsIncrementMove
(
    MP_EXPOS_DATA *src_p /* pointer of the structure which sets the path
                           correcting data */
)
```

■ Description

This API sets the increment move amount to the manipulator of the control group that `src_p->ctrl_grp` indicates. Since this API adds the specified increment move amount to the current position, the increment move (such as the jog movement) is available. However, as a necessary condition, the target control group needs to be executing the WAIT instruction in the job. When the job execution stops because of the emergency stop, the hold ON, or the mode change, this correction also stops. When the job is started again, this API starts the increment move again.

In the following job example, by the execution of “SET B000 1”, it prepares to receive the specification of increment value move from the MotoPlus application. And then, the increment value is specified by the execution of “mpRcsIncrementMove()”. The specification of increment value is received while the job is executing WAIT instruction. However, in case input IN#(1) is turned ON, the following MOVL instruction is executed after WAIT instruction is passed through.

At this time, the correction is ignored even if the increment value is specified by mpRcsIncrementMove(). (mpRcsIncrementMove() API returns “E_EXRCS_IMOV_UNREADY(-1): not under executing WAIT” as its return value.)

14 Robot Motion Control API

mpExRcsIncrementMove

<JOB example>

```

:
MOVL C000 V=100.0
SET B000 1

WAIT IN#(1) = ON    // The increment value move is possible
                    // only while this WAIT instruction is being executed.
MOVL C001 V=100.0
:

```

This specification is available not only for the manipulator but also to the external axis (station axis).

■ Parameter

[src_p]

The increment move value data

```

typedef struct {
CTRLG_T ctrl_grp;      /* Target control group which executes the
                        increment value move*/
CTRLG_T m_ctrl_grp;    /* Master side control group for coordinated
                        (synchronized) operation */
CTRLG_T s_ctrl_grp;    /* Slave side control group for coordinated
                        (synchronized) operation */
MP_GRP_POS_INFO grp_pos_info[MP_GRP_NUM];
}    MP_EXPOS_DATA;

```

- ctrl_grp
Control group of the object operation
- m_ctrl_grp
The master group when performing coordinated operation. This must be the same control group as ctrl_grp. Set 0 if not performing coordinated operation. A control group with an external axis cannot be set.
- s_ctrl_grp
Slave side control group for coordinated (synchronized) operation. Set "0" in case synchronized operation is not executed.

14 Robot Motion Control API

mpExRcsIncrementMove

NOTICE

< Errors>

Followings are the functional errors when setting the control group data.

1.

- When the specified “m_ctrl_grp” is not found (does not exist) in the system configuration.
- When “s_ctrl_grp” is set, “m_ctrl_grp” is “0”, or more than one control groups are specified.
- Symbolic constant: E_EXRCS_M_CTRL_GRP (value: -11)

2. When “m_ctrl_grp” is set, “s_ctrl_grp” is “0”, or it is not found (does not exist or “0” is set) in the system configuration.

- Symbolic constant: E_EXRCS_S_CTRL_GRP (value: -12)

3. When overlapped control groups exist in the specified “m_ctrl_grp” and “s_ctrl_grp”.

- Symbolic constant: E_EXRCS_SYNC_MOVE_CTRL_GRP (value: -10)

(e.g.) The control group setting and the action on the R1+R2 system

ctrl_grp	m_ctrl_grp	s_ctrl_grp	Operation
1	0	0	R1 independent operation
1	1	2	Operate R1 and R2 synchronizes
1	2	1	Error: E_EXRCS_SYNC_MOVE_CTRL_GRP(-10) Control group specification error in the coordinated operation
1	0	1	Error: E_EXRCS_M_CTRL_GRP(-11) Master side control group specification error
1	1	0	Error: E_EXRCS_S_CTRL_GRP(-12) Slave side control group specification error
1	1	1	Error: E_EXRCS_SYNC_MOVE_CTRL_GRP(-10) Control group specification error in the coordinated operation
1	0	2	Error: E_EXRCS_M_CTRL_GRP(-11) Master side control group specification error
1	2	0	Error: E_EXRCS_S_CTRL_GRP(-12) Slave side control group specification error
2	0	0	R2 independent operation
2	1	2	Error: E_EXRCS_SYNC_MOVE_CTRL_GRP(-10) Control group specification error in the coordinated operation
2	2	1	Operate R1 and R2 synchronizes
2	0	1	Error: E_EXRCS_M_CTRL_GRP(-11) Master side control group specification error
2	1	0	Error: E_EXRCS_S_CTRL_GRP(-12) Slave side control group specification error
2	0	2	Error: E_EXRCS_M_CTRL_GRP(-11) Master side control group specification error
2	2	0	Error: E_EXRCS_S_CTRL_GRP(-12) Slave side control group specification error
2	2	2	Error: E_EXRCS_SYNC_MOVE_CTRL_GRP(-10) Control group specification error in the coordinated operation
3	3	1	Error: E_EXRCS_M_CTRL_GRP(-11) Master side control group specification error

14 Robot Motion Control API
mpExRcsIncrementMove

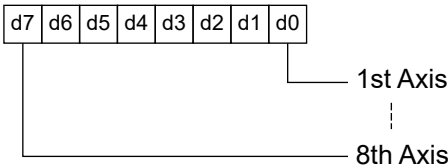
```
typedef struct {  
    MP_POS_TAG pos_tag;  
    long pos[MP_GRP_AXES_NUM]; /* Correction amount data*/  
} MP_GRP_POS_INFO;
```

```
typedef struct {  
    UCHAR data[8]; /* For the property and details of the data, see below */  
    CTRLG_T ei_ctrl_grp; /* Control group specification when EIMOV robot */  
} MP_POS_TAG;
```

Symbolic constant: MP_GRP_AXES_NUM (value: 8)
Description: Maximum axis number of the control groups

Symbolic constant: MP_GRP_NUM (value: 32)
Description: Number of the control groups

MP_POS_TAG.data[0]: Information of axis configuration



MP_POS_TAG.data[2]: Tool file number

The tool file number used when increment value move is executed in the tool coordinate system

MP_POS_TAG.data[3]: Data type

Symbolic constant	real value	Description
MP_INC_PULSE_DTYPE	0x80	Increment value specification by pulse data
MP_INC_ANGLE_DTYPE	0x81	Increment value specification by angle data
MP_INC_BF_DTYPE	0x90	Increment value specification by base coordinate system cartesian data
MP_INC_RF_DTYPE	0x91	Increment value specification by robot coordinate system cartesian data
MP_INC_TF_DTYPE	0x92	Increment value specification by tool coordinate system cartesian data
MP_INC_UF_DTYPE	0x93	Increment value specification by user coordinate system cartesian data

MP_POS_TAG.data[4]: User coordinate number
(0-15: valid only when user coordinate is specified)

MP_POS_TAG.data[1], [5]-[7]: For future expansion

14 Robot Motion Control API

mpExRcsIncrementMove



Angle can also be specified for the data type. However, for robot controllers up to the DX200, there are axes on which the direction of the angle can be different from the one of the pulses. For the YRC1000/YRC1000micro, however, the direction of the angle is always the same as the direction of the pulses. For this reason, an application that moves a robot using absolute angles may reverse the movement direction of the robot if the DX200 program code runs on the YRC1000/YRC1000micro unmodified. When porting DX200 code to the YRC1000/YRC1000micro, check the direction of robot movement and correct the code if necessary.

A measure is available to keep the source code for DX200 and earlier robot controllers and for the YRC1000/YRC1000micro the same. Set parameter S2C1350 in the YRC1000/YRC1000micro to 0 (default value = 1) to use the same specification as the DX200 and earlier robot controllers. Then, the positive direction of the angle and the positive direction of the pulses for each axis become the same as those for the DX200.

MP_GRP_POS_INFO pos

Array	Pulse type	Angle type	Cartesian type
pos[0]	1 st axis (S) pulse	1 st axis (S) angle [0.0001deg]	X-axis coordinate [μm]
pos[1]	2 nd axis (L) pulse	2 nd axis (L) angle [0.0001deg]	Y-axis coordinate [μm]
pos[2]	3 rd axis (U) pulse	3 rd axis (U) angle [0.0001deg]	Z-axis coordinate [μm]
pos[3]	4 th axis (R) pulse	4 th axis (R) angle [0.0001deg]	Wrist angle Rx [0.0001deg]
pos[4]	5 th axis (B) pulse	5 th axis (B) angle [0.0001deg]	Wrist angle Ry [0.0001deg]
pos[5]	6 th axis (T) pulse	6 th axis (T) angle [0.0001deg]	Wrist angle Rz [0.0001deg]
pos[6]	7 th axis (E) pulse	7 th axis (E) angle [0.0001deg]	angle Re [0.0001deg]
pos[7]	8 th axis pulse	8 th axis angle [0.0001deg]	8th axis pulse value (micron in the case of traveling axis)

14 Robot Motion Control API

mpExRcsIncrementMove

■ Return value

0 : Normal end

One of the following negative values : Error

Value	Description
E_EXRCS_IMOV_UNREADY(-1)	Not executing WAIT instruction
E_EXRCS_CTRL_GRP(-3)	Error in control group setting (It occurs when the operation target control group specification is overlapped, too. e.g. When a specified control group is used in other task of the independent control.)
E_EXRCS_AXES_CONFIG(-4)	Error in axis construction data
E_EXRCS_TOOL_NO(-5)	Error in tool number
E_EXRCS_POS_DTYPE(-6)	Error in data type
E_EXRCS_USER_FRAME(-7)	Error in user coordinate setting
E_EXRCS_JOG_BUSY(-8)	Operating jog
E_EXRCS_MOVE_BUSY(-9)	Executing move instruction
E_EXRCS_SYNC_MOVE_CTRL_GRP(-10)	Error in control group specification in coordinated operation
E_EXRCS_M_CTRL_GRP(-11)	Error in master side control group specification
E_EXRCS_S_CTRL_GRP(-12)	Error in slave side control group specification
E_EXRCS_USING_CTRL_GRP(-13)	Job of the control group specified in mpExRcsIncrementMove() is executing.



If this API is executed in the state that the job which executes the increment value move is not executed, it returns "OK(0): Normal end", and doesn't execute the increment value move.

(e.g.) When the job execution is stopped during the manipulator's increment value move by this API because of the emergency stop, hold ON, or mode switch, the increment value move of the manipulator also stops. All the while this API returns "OK(0): Normal end" as the return value. When the JOB is started again and executes the WAIT instruction, the manipulator starts the increment value move again while this API keeps returning "OK(0): Normal end".

14 Robot Motion Control API

mpExRcsIncrementMove

NOTE

In the environment using the independent control function, when a specified control group which executes the increment value move by this API in a job local task, the control group can execute the increment value move only in the same local task until the job local task completes.

When the control group tries to execute the increment value move in the different job local tasks, this API returns “-3: Error in control group setting”, and does not execute the increment value move.

For example, when R1 executes the increment value move by this API in the master local task and then R1 executes the increment value move by this API in the sub 1 local task, this API returns “-3: Error in control group setting”.

When this API is executed in the environment using the independent control function, create a job so as to make the control group and the job local task to execute the increment value move the same.

For example, when executing the increment value move using this API in the dual-robot system, create a job so as to make the control group and the job local task to execute the increment value move the same by executing the increment value move respectively by R1 to execute the WAIT command only in the sub 1 local task and by R2 in the sub 2.

NOTE

The correction data that the system received by calling this API is valid only once. So normally the task which calls this API needs to be periodical (For example, it starts in the interpolation cycle.). That realizes the program structure which periodically calls this API when the increment value move is needed.

14 Robot Motion Control API

mpExRcsIncrementMove

■ Sample code

The following is the sample code for increment value move operation.

When incmove_req is set by the command receiving task, the manipulator moves 1mm to Y-direction per interpolation cycle.

Although the increment value move process is FOREVER (infinite loop), mpClkAnnounce(MP_INTERPOLATION_CLK);” is described at the starting of the loop so that the increment value move process synchronizes with the interpolation cycle.

```
int incmove_req;
void pathCorrectionTask(void)
{
    MP_EXPOS_DATA corrp_path_src_p;

    //
    memset(&corrp_path_src_p, CLEAR, sizeof(MP_EXPOS_DATA));
    corrp_path_src_p.ctrl_grp = 1;    // Robot1
    corrp_path_src_p.grp_pos_info[0].pos_tag.data[0] = 0x3f;
                                    // Effective axis specification (6 axes)
    corrp_path_src_p.grp_pos_info[0].pos_tag.data[2] = 0;
    corrp_path_src_p.grp_pos_info[0].pos_tag.data[3] = MP_CORR_BF_DTYPE;
    // Increment value move operation on the base coordinate system
    corrp_path_src_p.grp_pos_info[0].pos[0] = 0;    //X
    corrp_path_src_p.grp_pos_info[0].pos[1] = 0;    //Y
    corrp_path_src_p.grp_pos_info[0].pos[2] = 0;    //Z
    corrp_path_src_p.grp_pos_info[0].pos[3] = 0;    //Rx
    corrp_path_src_p.grp_pos_info[0].pos[4] = 0;    //Ry
    corrp_path_src_p.grp_pos_info[0].pos[5] = 0;    //Rz

    FOREVER
    {
        mpClkAnnounce(MP_INTERPOLATION_CLK);
        / Execute following process per interpolation cycle
        if (incmove_req == ON)                //Correction required?
                                            Set by command receiving task .

        {
            // PutCorrPath (Correct 1[mm] in Y-direction per interpolation cycle
            corrp_path_src_p.grp_pos_info[0].pos[1] = 1000;
            mpExRcsIncrementMove(&corrp_path_src_p);
        }
    }
}
```

15 Servo Monitor/Control API

mpSvsGetVelTrqFb

Retrieves the feedback speed and torque of all control groups and axes.

■ Syntax

```
int mpSvsGetVelTrqFb
(
    MP_GRP_AXES_T dst_vel, /* destination for velocity feedback */
    MP_TRQ_CTL_VAL *dst_trq /* destination for Torque reference */
)
```

■ Description

This API writes the feedback speed and torque of all control groups and axes in the area specified by the parameter. Since the area specified by the parameter is not initialized (0 clear) by this API, initialize the area by the application as necessary. If NULL is set for the area specification of the parameter, the target data are not written.

The system monitor APIs mpGetFBSpeed() and mpGetTorque() can also return the feedback speed and torque, but by this API, mpSvsGetVelTrqFb(), the data of speed and torque are updated at shorter intervals.

Also, the units of the data of speed and torque are different from those of mpGetFBSpeed() and mpGetTorque(). Regarding the data of torque, the direction (sign) may be different from that of mpGetTorque(). For details, refer to the following description about the parameter.

■ Parameter

[dst_vel]

Specify the pointer to the feedback speed receiving array. The feedback speed is written in the array in units of 0.1 pulse/sec.

- MP_GRP_AXES_T
Data array for all control groups and axes
Syntax: typedef long MP_GRP_AXES_T[MP_GRP_NUM][MP_GRP_AXES_NUM];
Data: <dst_vel>
Feedback speed

Array	Description
dst_vel[x][y] x: control group number y: axis number	The feedback speed of the y-th axis of the x-th control group. Use the value retrieved by mpCtrlGrpId2GrpNo() as the control group number.

[dst_trq]

Specify the pointer to the torque receiving array.

- MP_TRQ_CTL_VAL

Torque data structure

Syntax: typedef struct {

```
int                                     unit;
MP_TRQCTL_DATA                         data;
}                                     MP_TRQ_CTL_VAL;
```

Member: <unit>

Torque data unit

Symbolic constant	Description
TRQ_PERCENTAGE	The torque data can be retrieved in the percentage of the rated torque of the motor. The data is in units of 0.01%.
TRQ_NEWTON_METER	The torque value of the motor can be retrieved. The data is in units of 0.000001 Nm.

<data>

- MP TRQCTL DATA

Torque data array

The torque data are written in this array in the units specified in "unit" above.

```
Syntax: #define MP_TRQCTL_DATA MP_GRP_AXES_T
typedef long
        MP_GRP_AXES_T[MP_GRP_NUM][MP_GRP_AXES_NUM];
```

Array	Description
dataI[x][y] x: control group number y: axis number	The torque data of the y-th axis of the x-th control group. Use the value retrieved by mpCtrlGrpId2GrpNo() as the control group number.

- The direction (sign) of the torque retrieved by `mpSvsGetVelTrqFb()` is the same as the direction of the axis rotation. Also, it coincides with the direction of the speed.

However, it may be different from the direction of the torque retrieved by the system monitor API, `mpGetTorque()`.

For `mpGetTorque()`, the torque's normal (forward) direction is not the direction of the axis rotation but the direction of the motor's normal (forward) rotation.

The direction of the axis rotation and the direction of the motor's normal rotation may be reversed. If so, the direction of the torque retrieved by `mpGetTorque()` is the reverse of the direction of the speed and the torque for the servo control API.

- The torque feedback value is the value of {Torque reference} in the pendant oscilloscope function, a separate option.

- **Return Value**

Symbolic constant: OK (value: 0) : Normal end

Symbolic constant: ERROR (value: -1) : Error

mpSvsStartTrqLimit

Starts the torque limit. (Only for the external axis)



CAUTION

- By using this API, the motor may not be able to maintain its position and may move in an unexpected direction. Carefully read MotoPlus Programmer's Manual¹⁾, and be sure to confirm safety before using this API.

¹ For the YRC1000:
YRC1000 OPTIONS INSTRUCTIONS Programmer's Manual For New Language Environment MotoPlus (HW1483598)
For the YRC1000micro:
YRC1000micro OPTIONS INSTRUCTIONS Programmer's Manual For New Language Environment MotoPlus (HW1484526)

■ Syntax

```
MP_SVS_HANDLE mpSvsStartTrqLimit
(
MP_GRP_CONFIG *arg_grp_config; /* specified effective group axes */
)
```

■ Description

The execution of this API sets the control group and axis for which the torque limit is executed, and the torque limit is made ready to start. However, the torque limit cannot be started under one of the following conditions:

- The manipulator or the base axis is specified as the target group.
- The axis for which the torque limit or the torque control is already executed is present in the specified control group.
- The move command is output to the specified control group.
- One of the following functions, which cannot be executed simultaneously with the torque limit, is already executed:
 - Servo float function
 - Speed control function
 - Search function
 - Endless function
 - Motor gun pressure instructions (Only YRC1000)

Note that the actual torque limit is started after executing this API and then mpSvsSetTrqLimit().

■ Parameter

[arg_grp_config]

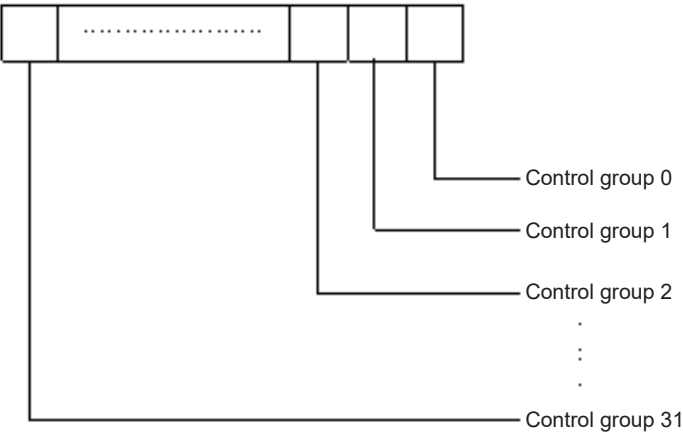
Specify the control group and axis for which the torque limit is executed.

• MP_GRP_CONFIG

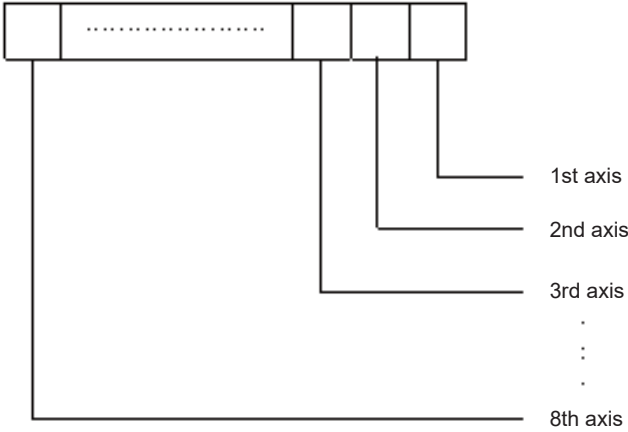
Control group and control axis structure

Syntax: typedef struct {
 CTRLG_T ctrl_grp;
 UCHAR axes_config[MP_GRP_NUM];
 }
 MP_GRP_CONFIG;

Member: <ctrl_grp>
Control group
Specify the target group in bits.
Retrieve the control group number of the target group by
mpCtrlGrpId2GrpNo().



<axes_config[]>
Axis configuration information
Specify the target axis in bits.
Specify the control group number of the target group
retrieved by mpCtrlGrpId2GrpNo() as the array number.



15 Servo Monitor/Control API

mpSvsStartTrqLimit

■ Return Value

0 or more Normal end
The handle value is returned.

Less than 0 Error
One of the following values is returned according to the error content.

Symbolic constant	Value	Description
E_SVS_HANDLE_FULL	-2	The number of the handle values issued already reaches the limit.
E_SVS_UNINITIALIZED	-5	The area for the servo control is not initialized.
E_SVS_USING_CTRL_GRP	-6	The axis for which the torque limit or the torque control is already executed is present in the specified control group.
E_SVS_NON_CTRL_GRP	-7	No group is specified.
E_SVS_CTRL_GRP_NOT_EXIST	-8	Non-existent group is specified.
E_SVS_AXES_SPECIFIED_NON	-9	Non-existent axis is specified.
E_SVS_MOVE_BUSY	-10	This API is executed for the group to which the move command is output.
E_SVS_AX_COMM_OBJ_FULL	-11	It was impossible to command the servo-control part to move to the torque limit.
E_SVS_AX_COMM_TIME_OVER	-12	
E_SVS_AX_COMM_FAILURE	-13	
E_SVS_INVALID_ROB_TRQ_FUNC	-14	This API was executed for the manipulator or the base axis.
E_SVS_INVALID_ROB_TRQ_AXIS	-15	
E_SVS_AX_TRQ_LMT_SET	-16	The servo-control part was not able to move to the torque limit mode.
E_SVS_RTP_NOT_SUPPORTED	-25	The system software does not support this API.
E_SVS_JOG_BUSY	-26	This API was executed for the group in JOG operation.

mpSvsSetTrqLimit

Sets the torque limit value. (Only for the external axis)



CAUTION

- By using this API, the motor may not be able to maintain its position and may move in an unexpected direction. Carefully read MotoPlus Programmer's Manual¹⁾, and be sure to confirm safety before using this API.

1 For the YRC1000:
YRC1000 OPTIONS INSTRUCTIONS Programmer's Manual For New Language Environment MotoPlus (HW1483598)
For the YRC1000micro:
YRC1000micro OPTIONS INSTRUCTIONS Programmer's Manual For New Language Environment MotoPlus (HW1484526)

■ Syntax

```
int mpSvsSetTrqLimit
(
    MP_SVS_HANDLE handle,          /* specified SVS handle */
    MP_GRP_CONFIG *arg_grp_config, /* torque limit set group axes */
    MP_TRQ_LMT_VAL *arg_lmt_val    /* torque limit max/min value */
)
```

■ Description

This API sets the torque limit value for the control group and axis for which the torque limit is started by mpSvsStartTrqLimit().
The value within $\pm 100\%$ can be set as the torque limit value.

15 Servo Monitor/Control API

mpSvsSetTrqLimit

■ Parameter

[handle]

Specify the handle value which is returned after executing mpSvsStartTrqLimit().

[arg_grp_config]

Specify the control group and axis for which the torque limit value is set.

It is not necessary to specify all of the control groups and axes set when executing mpSvsStartTrqLimit(), so it is possible to specify only some of them. However, the groups and axes to be specified must be included in the control groups and axes set when executing mpSvsStartTrqLimit().

• MP_GRP_CONFIG

Control group and control axis structure

Syntax: typedef struct {

```

        CTRLG_T      ctrl_grp;
        UCHAR         axes_config[MP_GRP_NUM];
    }
    MP_GRP_CONFIG;

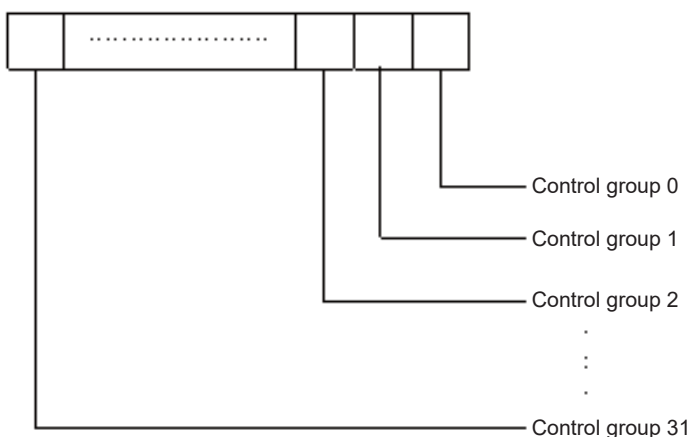
```

Member: <ctrl_grp>

Control group

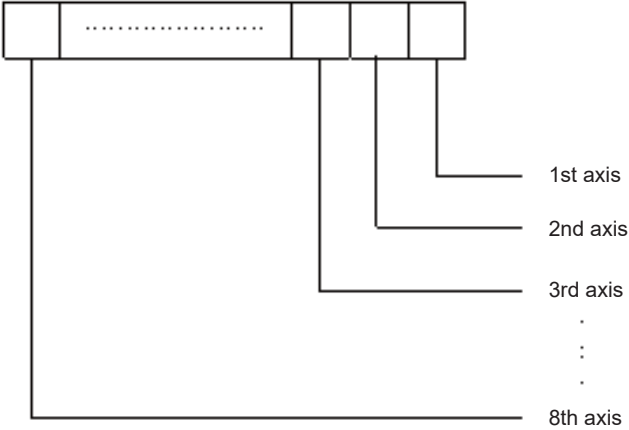
Specify the target group in bits.

Retrieve the control group number of the target group by mpCtrlGrpId2GrpNo().



15 Servo Monitor/Control API
mpSvsSetTrqLimit

<axes_config[]>
Axis configuration information
Specify the target axis in bits.
Specify the control group number of the target group
retrieved by mpCtrlGrpId2GrpNo() as the array number.



[arg_lmt_val]

Specify the pointer to the torque limit value structure.

- MP_TRQ_LMT_VAL
Torque limit value structure
Syntax: typedef struct {
 int unit;
 MP_TRQLMT_RANGE data[MP_GRP_NUM];
} MP_TRQ_LMT_VAL;
Member: <unit>
Torque data unit

Symbolic constant	Description
TRQ_PERCENTAGE	The torque limit value is specified in the percentage of the rated torque of the motor. The data is in units of 0.01%.
TRQ_NEWTON_METER	The torque limit value of the motor is specified. The data is in units of 0.000001 Nm.

15 Servo Monitor/Control API

mpSvsSetTrqLimit

<data[]>

- **MP_TRQLMT_RANGE**

Torque limit value structure

Syntax: typedef struct {

```

    long    max[MP_GRP_AXES_NUM];
    long    min[MP_GRP_AXES_NUM];
}          MP_TRQLMT_RANGE;
```

Member: <max[]>

Torque upper limit value

<min[]>

Torque lower limit value

Array	Description
data[x].max[y] x: control group number y: axis number	The torque upper limit value of the y-th axis of the x-th control group. Use the value retrieved by mpCtrlGrpId2GrpNo() as the control group number.
data[x].min[y] x: control group number y: axis number	The torque lower limit value of the y-th axis of the x-th control group. Use the value retrieved by mpCtrlGrpId2GrpNo() as the control group number.



- It is possible to set positive (plus) values to both of the torque upper limit value and the torque lower limit value, or negative (minus) values to both of them. However, if those values are set as such, the torque cannot take the value 0 and a certain torque is always applied to the motor, thus the motor may move.
- The direction (sign) of the torque limit value is the same as the direction of the axis rotation. Also, it coincides with the direction of the speed.
However, it may be different from the direction of the torque retrieved by the system monitor API, mpGetTorque().
For mpGetTorque(), the torque's normal (forward) direction is not the direction of the axis rotation but the direction of the motor's normal (forward) rotation.
The direction of the axis rotation and the direction of the motor's normal rotation may be reversed. If so, the direction of the torque retrieved by mpGetTorque() is the reverse of the direction of the speed and the torque for this API.

15 Servo Monitor/Control API

mpSvsSetTrqLimit

■ Return Value

0 Normal end

Less than 0 Error
One of the following values is returned according to the error content.

Symbolic constant	Value	Description
E_SVS_BAD_HANDLE	-1	The handle value for servo control is not specified.
E_SVS_NOT_FUNC_CONCERNED	-3	The handle value for torque limit is not specified.
E_SVS_NON_CTRL_GRP	-7	No group is specified.
E_SVS_CTRL_GRP_NOT_EXIST	-8	Non-existent group is specified.
E_SVS_NOT_TRQ_LMT_AXIS	-18	An invalid axis is specified.
E_SVS_TRQ_LMT_LARGER_THAN_UL	-20	A value larger than the upper limit is set to the lower limit.
E_SVS_TRQ_LMT_RANGE_MAX	-21	The value set to the upper limit exceeds the allowable range.
E_SVS_TRQ_LMT_RANGE_MIN	-22	The value set to the lower limit exceeds the allowable range.
E_SVS_RTP_NOT_SUPPORTED	-25	The system software does not support this API.

mpSvsEndTrqLimit

Ends the torque limit. (Only for the external axis)



CAUTION

- By using this API, the motor may not be able to maintain its position and may move in an unexpected direction. Carefully read MotoPlus Programmer's Manual¹⁾, and be sure to confirm safety before using this API.

1 For the YRC1000:
YRC1000 OPTIONS INSTRUCTIONS Programmer's Manual For New Language Environment MotoPlus (HW1483598)
For the YRC1000micro:
YRC1000micro OPTIONS INSTRUCTIONS Programmer's Manual For New Language Environment MotoPlus (HW1484526)

■ Syntax

```
int mpSvsEndTrqLimit
(
  MP_SVS_HANDLE handle    /* specified SVS handle */
)
```

■ Description

This API ends the torque limit for all control groups and axes started by mpSvsStartTrqLimit(). During the torque limit, the position is not controlled, thus the command position and the feedback position do not coincide. Therefore, create the command position (current position) of the target group again after the torque limit ends, and adjust it to the feedback position.

Note that this API cannot be executed if the move command is output to the target group.



Do not output the move command to the target group until the execution of this API is completed. Since the command position (current position) is created again after the execution of this API is completed, the move command may be changed considerably.

■ Parameter

[handle]

Specify the handle value which is returned after executing mpSvsStartTrqLimit().

15 Servo Monitor/Control API mpSvsEndTrqLimit

■ Return Value

0 Normal end

Less than 0 Error
One of the following values is returned according to the error content.

Symbolic constant	Value	Description
E_SVS_BAD_HANDLE	-1	The handle value for servo control is not specified.
E_SVS_NOT_FUNC_CONCERNED	-3	The handle value for torque limit is not specified.
E_SVS_MOVE_BUSY	-10	This API is executed for the group to which the move command is output.
E_SVS_AX_COMM_OBJ_FULL	-11	It was impossible to command the servo-control part to end the torque limit.
E_SVS_AX_COMM_TIME_OVER	-12	
E_SVS_AX_COMM_FAILURE	-13	
E_SVS_AX_TRQ_LMT_CANCEL	-17	The servo-control part was not able to end the torque limit mode.
E_SVS_PR_MAKE_FAILURE	-24	Re-creation of the current position is failed.
E_SVS_RTP_NOT_SUPPORTED	-25	The system software does not support this API.
E_SVS_JOG_BUSY	-26	This API was executed for the group in JOG operation.

mpSvsStartTrqCtrl

Starts the torque control. (Only for the external axis)



CAUTION

- By using this API, the motor may not be able to maintain its position and may move in an unexpected direction. Carefully read MotoPlus Programmer's Manual¹⁾, and be sure to confirm safety before using this API.

1 For the YRC1000:
YRC1000 OPTIONS INSTRUCTIONS Programmer's Manual For New Language Environment MotoPlus (HW1483598)
For the YRC1000micro:
YRC1000micro OPTIONS INSTRUCTIONS Programmer's Manual For New Language Environment MotoPlus (HW1484526)

■ Syntax

```
MP_SVS_HANDLE mpSvsStartTrqCtrl
(
  MP_GRP_CONFIG *arg_grp_config; /* specified effective group axes */
)
```

■ Description

The execution of this API sets the control group and axis for which the torque control is executed, and the torque control is made ready to start. However, the torque control cannot be started under one of the following conditions:

- The manipulator or the base axis is specified as the target group.
- The axis for which the torque limit or the torque control is already executed is present in the specified control group.
- The move command is output to the specified control group.
- One of the following functions, which cannot be executed simultaneously with the torque control, is already executed:
 - Servo float function
 - Speed control function
 - Search function
 - Endless function
 - Motor gun pressure instructions (Only YRC1000)

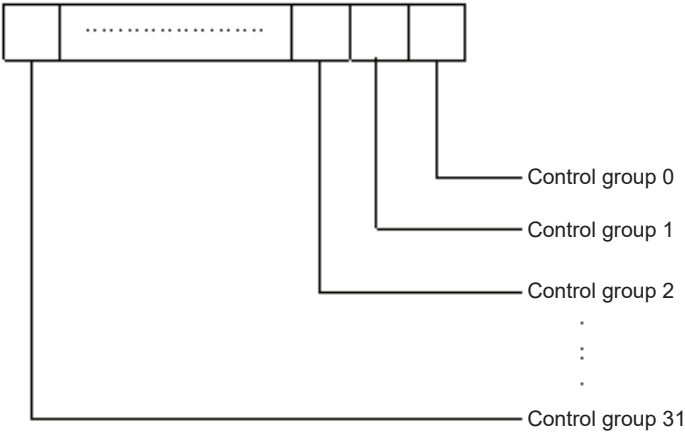
Note that the actual torque control is started after executing this API and then mpSvsSetTrqCtrl().

■ Parameter

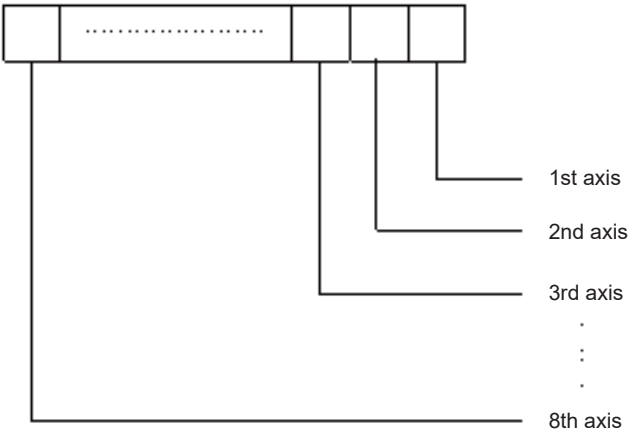
[arg_grp_config]

Specify the control group and axis for which the torque control is executed.

- MP_GRP_CONFIG
Control group and control axis structure
Syntax: typedef struct {
 CTRLG_T ctrl_grp;
 UCHAR axes_config[MP_GRP_NUM];
 }
 MP_GRP_CONFIG;
Member: <ctrl_grp>
 Control group
 Specify the target group in bits.
 Retrieve the control group number of the target group by
 mpCtrlGrpId2GrpNo().



- <axes_config[]>
Axis configuration information
Specify the target axis in bits.
Specify the control group number of the target group
retrieved by mpCtrlGrpId2GrpNo() as the array number.



15 Servo Monitor/Control API

mpSvsStartTrqCtrl

■ Return Value

- 0 or more Normal end
The handle value is returned.
- Less than 0 Error
One of the following values is returned according to the error content.

Symbolic constant	Value	Description
E_SVS_HANDLE_FULL	-2	The number of the handle values issued already reaches the limit.
E_SVS_UNINITIALIZED	-5	The area for the servo control is not initialized.
E_SVS_USING_CTRL_GRP	-6	The axis for which the torque limit or the torque control is already executed is present in the specified control group.
E_SVS_NON_CTRL_GRP	-7	No group is specified.
E_SVS_CTRL_GRP_NOT_EXIST	-8	Non-existent group is specified.
E_SVS_AXES_SPECIFIED_NON	-9	Non-existent axis is specified.
E_SVS_MOVE_BUSY	-10	This API is executed for the group to which the move command is output.
E_SVS_AX_COMM_OBJ_FULL	-11	It was impossible to command the servo-control part to move to the torque control.
E_SVS_AX_COMM_TIME_OVER	-12	
E_SVS_AX_COMM_FAILURE	-13	
E_SVS_INVALID_ROB_TRQ_FUNC	-14	This API was executed for the manipulator or the base axis.
E_SVS_INVALID_ROB_TRQ_AXIS	-15	
E_SVS_AX_TRQ_LMT_SET	-16	The servo-control part was not able to move to the torque control mode.
E_SVS_RTP_NOT_SUPPORTED	-25	The system software does not support this API.
E_SVS_JOG_BUSY	-26	This API was executed for the group in JOG operation.

mpSvsSetTrqCtrl

Sets the torque command value. (Only for the external axis)



CAUTION

- By using this API, the motor may not be able to maintain its position and may move in an unexpected direction. Carefully read MotoPlus Programmer's Manual¹⁾, and be sure to confirm safety before using this API.

1 For the YRC1000:
YRC1000 OPTIONS INSTRUCTIONS Programmer's Manual For New Language Environment MotoPlus (HW1483598)
For the YRC1000micro:
YRC1000micro OPTIONS INSTRUCTIONS Programmer's Manual For New Language Environment MotoPlus (HW1484526)

■ Syntax

```
int mpSvsSetTrqCtrl
(
    MP_SVS_HANDLE handle,          /* specified SVS handle */
    MP_GRP_CONFIG *arg_grp_config, /* torque limit set group axes */
    MP_TRQ_CTL_VAL *arg_ctl_val    /* torque control values */
)
```

■ Description

This API sets the torque command value for the control group and axis for which the torque control is started by mpSvsStartTrqCtrl(). The value within $\pm 100\%$ can be set as the torque command value.

■ Parameter

[handle]

Specify the handle value which is returned after executing mpSvsStartTrqCtrl().

[arg_grp_config]

Specify the control group and axis for which the torque command value is set.

It is not necessary to specify all of the control groups and axes set when executing mpSvsStartTrqCtrl(), so it is possible to specify only some of them. However, the groups and axes to be specified must be included in the control groups and axes set when executing mpSvsStartTrqCtrl().

- MP_GRP_CONFIG
Control group and control axis structure
Syntax: typedef struct {
 CTRLG_T ctrl_grp;
 UCHAR axes_config[MP_GRP_NUM];
} MP_GRP_CONFIG;

15 Servo Monitor/Control API

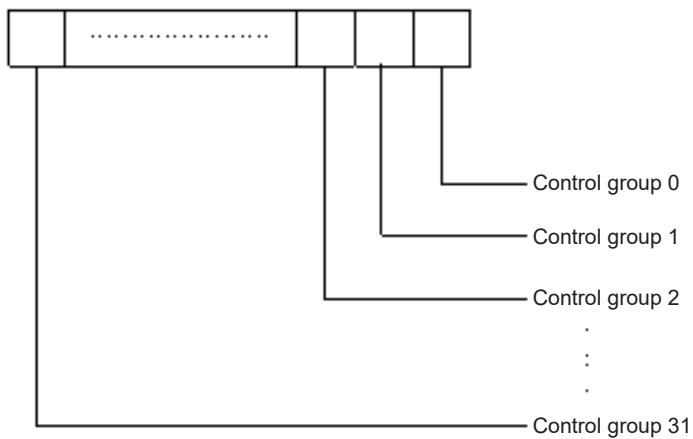
mpSvsSetTrqCtrl

Member: <ctrl_grp>

Control group

Specify the target group in bits.

Retrieve the control group number of the target group by mpCtrlGrpId2GrpNo().

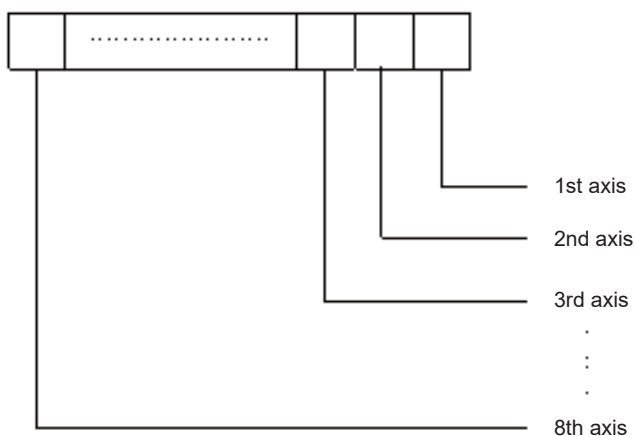


<axes_config[]>

Axis configuration information

Specify the target axis in bits.

Specify the control group number of the target group retrieved by mpCtrlGrpId2GrpNo() as the array number.



[arg_ctl_val]

Specify the pointer to the torque data structure.

- MP_TRQ_CTL_VAL
Torque data structure

```
Syntax: typedef struct {
    int                unit;
    MP_TRQCTL_DATA     data;
} MP_TRQ_CTL_VAL;
```

Member: <unit>
Torque data unit

Symbolic constant	Description
TRQ_PERCENTAGE	The torque command value is specified in the percentage of the rated torque of the motor. The data is in units of 0.01%.
TRQ_NEWTON_METER	The torque command value of the motor is specified. The data is in units of 0.000001 Nm.

<data>

- MP_TRQCTL_DATA
Torque data array

Syntax:

```
#define MP_TRQCTL_DATA MP_GRP_AXES_T
typedef long
MP_GRP_AXES_T[MP_GRP_NUM][MP_GRP_AXES_NUM];
```

Array	Description
dataI[x][y] x: control group number y: axis number	The torque command value of the y-th axis of the x-th control group. Use the value retrieved by mpCtrlGrpId2GrpNo() as the control group number.

NOTE

The direction (sign) of the torque command value is the same as the direction of the axis rotation. Also, it coincides with the direction of the speed.

However, it may be different from the direction of the torque retrieved by the system monitor API, `mpGetTorque()`.

For `mpGetTorque()`, the torque's normal (forward) direction is not the direction of the axis rotation but the direction of the motor's normal (forward) rotation.

The direction of the axis rotation and the direction of the motor's normal rotation may be reversed. If so, the direction of the torque retrieved by `mpGetTorque()` is the reverse of the direction of the speed and the torque for this API.

15 Servo Monitor/Control API

mpSvsSetTrqCtrl

■ Return Value

0 Normal end

Less than 0 Error

One of the following values is returned according to the error content.

Symbolic constant	Value	Description
E_SVS_BAD_HANDLE	-1	The handle value for servo control is not specified.
E_SVS_NOT_FUNC_CONCERNED	-3	The handle value for torque limit is not specified.
E_SVS_NON_CTRL_GRP	-7	No group is specified.
E_SVS_CTRL_GRP_NOT_EXIST	-8	Non-existent group is specified.
E_SVS_NOT_TRQ_CTL_AXIS	-19	An invalid axis is specified.
E_SVS_TRQ_CTL_RANGE	-23	The value set as the torque command value exceeds the allowable range.
E_SVS_RTP_NOT_SUPPORTED	-25	The system software does not support this API.

mpSvsEndTrqCtrl

Ends the torque control. (Only for the external axis)



CAUTION

- By using this API, the motor may not be able to maintain its position and may move in an unexpected direction. Carefully read MotoPlus Programmer's Manual¹⁾, and be sure to confirm safety before using this API.

1 For the YRC1000:
YRC1000 OPTIONS INSTRUCTIONS Programmer's Manual For New Language Environment MotoPlus (HW1483598)
For the YRC1000micro:
YRC1000micro OPTIONS INSTRUCTIONS Programmer's Manual For New Language Environment MotoPlus (HW1484526)

■ Syntax

```
int mpSvsEndTrqCtrl  
(  
    MP_SVS_HANDLE handle    /* specified SVS handle */  
)
```

■ Description

This API ends the torque control for all control groups and axes started by mpSvsStartTrqCtrl().

During the torque control, the position is not controlled, thus the command position and the feedback position do not coincide. Therefore, create the command position (current position) of the target group again after the torque control ends, and adjust it to the feedback position.

Note that this API cannot be executed if the move command is output to the target group.



Do not output the move command to the target group until the execution of this API is completed.
Since the command position (current position) is created again after the execution of this API is completed, the move command may be changed considerably.

■ Parameter

[handle]

Specify the handle value which is returned after executing mpSvsStartTrqCtrl().

15 Servo Monitor/Control API

mpSvsEndTrqCtrl

■ Return Value

0 Normal end

Less than 0 Error

One of the following values is returned according to the error content.

Value		Description
E_SVS_BAD_HANDLE	-1	The handle value for servo control is not specified.
E_SVS_NOT_FUNC_CONCERNED	-3	The handle value for torque control is not specified.
E_SVS_MOVE_BUSY	-10	This API is executed for the group to which the move command is output.
E_SVS_AX_COMM_OBJ_FULL	-11	It was impossible to command the servo-control part to end the torque control.
E_SVS_AX_COMM_TIME_OVER	-12	
E_SVS_AX_COMM_FAILURE	-13	
E_SVS_AX_TRQ_LMT_CANCEL	-17	The servo-control part was not able to end the torque control mode.
E_SVS_PR_MAKE_FAILURE	-24	Re-creation of the current position is failed.
E_SVS_RTP_NOT_SUPPORTED	-25	The system software does not support this API.
E_SVS_JOG_BUSY	-26	This API was executed for the group in JOG operation.

mpSvsForceInit

Force-quits the servo control (torque limit, torque control). (Only for the external axis)

■ Syntax

STATUS mpSvsForceInit(void)

■ Description

This API force-quits the servo control (torque limit, torque control) in execution if any, and initializes the area for servo control (including the handle value or internal objects).

Use this API in the case when an error occurred during servo control due to the system and recovery is difficult.

Note that if it is during the execution of the torque limit or torque control, the command position (current position) of the target group is created again to end the torque limit or torque control. Thus, if the move command is output to the control group for which the torque limit or torque control is executed (the command position is changing), the torque limit or torque control cannot be ended. Do not use this API when the move command is output to the control group for which the torque limit or torque control is executed.

■ Parameter

None

■ Return Value

Symbolic constant: OK (value: 0) : Normal end

Symbolic constant: ERROR (value: -1) : Error
In the case when it was impossible to end the servo control in execution, and the like.

16 Kinematics API

mpConvAxesToCartPos

Converts an angle position of the specified control group to a cartesian coordinate position (robot coordinate systems).



For robot controllers up to the DX200, there are axes on which the direction of the angle can be different from the one of the pulses. For the YRC1000/YRC1000micro, however, the direction of the angle is always the same as the direction of the pulses. For this reason, an application that moves a robot using absolute angles may reverse the movement direction of the robot if the DX200 program code runs on the YRC1000/YRC1000micro unmodified. When porting DX200 code to the YRC1000/YRC1000micro, check the direction of robot movement and correct the code if necessary.

A measure is available to keep the source code for DX200 and earlier robot controllers and for the YRC1000/YRC1000micro the same. Set parameter S2C1350 in the YRC1000/YRC1000micro to 0 (default value = 1) to use the same specification as the DX200 and earlier robot controllers. Then, the positive direction of the angle and the positive direction of the pulses for each axis become the same as those for the DX200.

■ Syntax

```
int mpConvAxesToCartPos(
    unsigned int    grp_no,
    long            angle[MP_GRP_AXES_NUM],
    unsigned int    tool_no,
    BITSTRING*     fig_ctrl,
    MP_COORD*       coord
);
```

■ Parameter

[grp_no]

Control group number

Acquire the control group number by mpCtrlGrpId2GrpNo().

Constant	Description
0	First control group
1	Second control group
...	...
31	Thirty-second control group

16 Kinematics API

mpConvAxesToCartPos

[angle]

Angle (degree) position

Syntax: #define MP_GRP_AXES_NUM (8)

Array	Description
angle[0]	1st axis (S) angle value (unit: 0.0001 deg)
angle[1]	2nd axis (L) angle value (unit: 0.0001 deg)
angle[2]	3rd axis (U) angle value (unit: 0.0001 deg)
angle[3]	4th axis (R) angle value (unit: 0.0001 deg)
angle[4]	5th axis (B) angle value (unit: 0.0001 deg)
angle[5]	6th axis (T) angle value (unit: 0.0001 deg)
angle[6]	7th axis (E) angle value (unit: 0.0001 deg)
angle[7]	8th axis (W) angle value (unit: 0.0001 deg)

[tool_no]

Tool number

Constant	Description
0	Tool number 0
1	Tool number 1
...	...
63	Tool number 63

[fig_ctrl]

Pointer to the figure information (output)

Syntax: typedef unsigned int BITSTRING;

Bit	Description	
D00	0:Front	1:Back
D01	0:Upper arm	1:Lower arm
D02	0:Flip	1:No flip
D03	0:R<180	1:R>=180
D04	0:T<180	1:T>=180
D05	0:S<180	1:S>=180
D06 - D31	Reserved by manufacturer	

16 Kinematics API

mpConvAxesToCartPos

[coord]

Pointer to the cartesian coordinate position structure (output)

- MP_COORD
Cartesian coordinate position structure
Syntax: typedef struct

```
{
    long  x, y, z;
    long  x, ry, rz;
    long  ex1, ex2;
} MP_COORD;
```

Member	Description
x	X-axis coordinate (unit: micron)
y	Y-axis coordinate (unit: micron)
z	Z-axis coordinate (unit: micron)
rx	Wrist angle Rx (unit: 0.0001 deg)
ry	Wrist angle Ry (unit: 0.0001 deg)
rz	Wrist angle Rz (unit: 0.0001 deg)
ex1	Elbow angle Re (unit: 0.0001 deg)
ex2	Reserved by manufacturer

■ Return Value

0 Normal end
One of the following negative values Error

Symbolic constant (value)	Description
E_KINEMA_FAILURE (-1)	Failed in conversion
E_KINEMA_CONV_IMPOSSIBLE (-2)	Conversion impossible
E_KINEMA_GRP_OUT_RANGE (-3)	Specified control group out of range
E_KINEMA_TOOL_OUT_RANGE (-4)	Specified tool number out of range



In case executing the API in each of the following condition, because the system is busy, correct conversion result cannot be acquired, and then E_KINEMA_FAILURE(-1) is returned to "Return Value".

- When setting a parameter
- When setting a tool file
- When setting an user coordinate
- When setting a calibration

When E_KINEMA_FAILURE(-1) is returned to "Return Value", try executing the API again after an interval.

mpConvCartPosToAxes

Converts a cartesian coordinate position (robot coordinate systems) of the specified control group to an angle position.

■ **Syntax**

```
int mpConvCartPosToAxes(  
    unsigned int      grp_no,  
    MP_COORD*         coord,  
    unsigned int      tool_no,  
    BITSTRING         fig_ctrl,  
    long              prev_angle[MP_GRP_AXES_NUM],  
    MP_KINEMA_TYPE     type,  
    long              angle[MP_GRP_AXES_NUM]  
);
```

■ **Parameter**

[*grp_no*]

Control group number

Acquire the control group number by mpCtrlGrpId2GrpNo()

Constant	Description
0	First control group
1	Second control group
...	...
31	Thirty-second control group

16 Kinematics API

mpConvCartPosToAxes

[coord]

Pointer to the cartesian coordinate position structure (input)

- MP_COORD
Cartesian coordinate position structure

Syntax: typedef struct

```
{
    long  x, y, z;
    long  ry, rz;
    long  ex1, ex2;
} MP_COORD;
```

Member	Description
x	X-axis coordinate (unit: micron)
y	Y-axis coordinate (unit: micron)
z	Z-axis coordinate (unit: micron)
rx	Wrist angle Rx (unit: 0.0001 deg)
ry	Wrist angle Ry (unit: 0.0001 deg)
rz	Wrist angle Rz (unit: 0.0001 deg)
ex1	Elbow angle Re (unit: 0.0001 deg)
ex2	Reserved by manufacturer

[tool_no]

Tool number

Constant	Description
0	Tool number 0
1	Tool number 1
...	...
63	Tool number 63

[fig_ctrl]

Figure information

Syntax: typedef unsigned int BITSTRING;

Bit	Description	
D00	0:Front	1:Back
D01	0:Upper arm	1:Lower arm
D02	0:Flip	1:No flip
D03	0:R<180	1:R>=180
D04	0:T<180	1:T>=180
D05	0:S<180	1:S>=180
D06 - D31	Reserved by manufacturer	

[prev_angle]

Previous angle

It is expressed by the unit “angle”. NULL can be specified to the angle. When NULL is specified, it is regarded as “0” is set to all elements.

Syntax: #define MP_GRP_AXES_NUM (8)

Value	Description
prev_angle[0]	1st axis (S) angle value (unit: 0.0001 deg)
prev_angle[1]	2nd axis (L) angle value (unit: 0.0001 deg)
prev_angle[2]	3rd axis (U) angle value (unit: 0.0001 deg)
prev_angle[3]	4th axis (R) angle value (unit: 0.0001 deg)
prev_angle[4]	5th axis (B) angle value (unit: 0.0001 deg)
prev_angle[5]	6th axis (T) angle value (unit: 0.0001 deg)
prev_angle[6]	7th axis (E) angle value (unit: 0.0001 deg)
prev_angle[7]	8th axis (W) angle value (unit: 0.0001 deg)

[type]

Inverse kinematics method specification

- MP_KINEMA_TYPE
Inverse kinematics method each

Syntax: typedef enum

```
{  
    MP_KINEMA_DEFAULT,  
    MP_KINEMA_DELTA,  
    MP_KINEMA_FIG,  
} MP_KINEMA_TYPE;
```

Symbolic constant	Description
MP_KINEMA_DEFAULT	Default inverse kinematics
MP_KINEMA_DELTA	Inverse kinematics according to the previous position
MP_KINEMA_FIG	Inverse kinematics to realize the figure

[angle]

Angle (degree) position

Array	Description
angle[0]	1st axis (S) angle value (unit: 0.0001 deg)
angle[1]	2nd axis (L) angle value (unit: 0.0001 deg)
angle[2]	3rd axis (U) angle value (unit: 0.0001 deg)
angle[3]	4th axis (R) angle value (unit: 0.0001 deg)
angle[4]	5th axis (B) angle value (unit: 0.0001 deg)
angle[5]	6th axis (T) angle value (unit: 0.0001 deg)
angle[6]	7th axis (E) angle value (unit: 0.0001 deg)
angle[7]	8th axis (W) angle value (unit: 0.0001 deg)

16 Kinematics API

mpConvCartPosToAxes

■ Return Value

0 Normal end
 One of the following negative values Error

Symbolic constant (value)	Description
E_KINEMA_FAILURE (-1)	Failed in conversion
E_KINEMA_CONV_IMPOSSIBLE (-2)	Conversion impossible
E_KINEMA_GRP_OUT_RANGE (-3)	Specified control group out of range
E_KINEMA_TOOL_OUT_RANGE (-4)	Specified tool number out of range
E_KINEMA_KINEMA_TYPE_ERR (-5)	Inverse kinematics wrong
E_KINEMA_INTERPOLATION_INVALID (-6)	Interpolation operation at the cartesian position before conversion is invalid.



In case executing the API in each of the following condition, because the system is busy, correct conversion result cannot be acquired, and then E_KINEMA_FAILURE(-1) is returned to "Return Value".

- When setting a parameter
- When setting a tool file
- When setting an user coordinate
- When setting a calibration

When E_KINEMA_FAILURE(-1) is returned to "Return Value", try executing the API again after an interval.

mpConvPulseToAngle

Converts a pulse position of the specified control group to an angle position.



For robot controllers up to the DX200, there are axes on which the direction of the angle can be different from the one of the pulses. For the YRC1000/YRC1000micro, however, the direction of the angle is always the same as the direction of the pulses. For this reason, an application that moves a robot using absolute angles may reverse the movement direction of the robot if the DX200 program code runs on the YRC1000/YRC1000micro unmodified. When porting DX200 code to the YRC1000/YRC1000micro, check the direction of robot movement and correct the code if necessary.

A measure is available to keep the source code for DX200 and earlier robot controllers and for the YRC1000/YRC1000micro the same. Set parameter S2C1350 in the YRC1000/YRC1000micro to 0 (default value = 1) to use the same specification as the DX200 and earlier robot controllers. Then, the positive direction of the angle and the positive direction of the pulses for each axis become the same as those for the DX200.

■ **Syntax**

```
int mpConvPulseToAngle(  
    unsigned int    grp_no,  
    long            pulse[MP_GRP_AXES_NUM],  
    long            angle[MP_GRP_AXES_NUM]  
);
```

■ **Parameter**

[grp_no]

Control group number

Acquire the control group number by mpCtrlGrpId2GrpNo().

Constant	Description
0	First control group
1	Second control group
...	...
31	Thirty-second control group

16 Kinematics API

mpConvPulseToAngle

[pulse]

Pulse position

Syntax: #define MP_GRP_AXES_NUM (8)

Value	Description
pulse[0]	1st axis (S) pulse value
pulse[1]	2nd axis (L) pulse value
pulse[2]	3rd axis (U) pulse value
pulse[3]	4th axis (R) pulse value
pulse[4]	5th axis (B) pulse value
pulse[5]	6th axis (T) pulse value
pulse[6]	7th axis (E) pulse value
pulse[7]	8th axis (W) pulse value

[angle]

Angle (degree) position

Array	Description
angle[0]	1st axis (S) angle value (unit: 0.0001 deg)
angle[1]	2nd axis (L) angle value (unit: 0.0001 deg)
angle[2]	3rd axis (U) angle value (unit: 0.0001 deg)
angle[3]	4th axis (R) angle value (unit: 0.0001 deg)
angle[4]	5th axis (B) angle value (unit: 0.0001 deg)
angle[5]	6th axis (T) angle value (unit: 0.0001 deg)
angle[6]	7th axis (E) angle value (unit: 0.0001 deg)
angle[7]	8th axis (W) angle value (unit: 0.0001 deg)

Return Value

0 Normal end
One of the following negative values Error

Symbolic constant (value)	Description
E_KINEMA_FAILURE (-1)	Failed in conversion
E_KINEMA_CONV_IMPOSSIBLE (-2)	Conversion impossible
E_KINEMA_GRP_OUT_RANGE (-3)	Specified control group out of range



In case executing the API in each of the following condition, because the system is busy, correct conversion result cannot be acquired, and then E_KINEMA_FAILURE(-1) is returned to "Return Value".

- When setting a parameter
- When setting a tool file
- When setting an user coordinate
- When setting a calibration

When E_KINEMA_FAILURE(-1) is returned to "Return Value", try executing the API again after an interval.

mpConvAngleToPulse

Converts an angle position of the specified control group to a pulse position.



For robot controllers up to the DX200, there are axes on which the direction of the angle can be different from the one of the pulses. For the YRC1000/YRC1000micro, however, the direction of the angle is always the same as the direction of the pulses. For this reason, an application that moves a robot using absolute angles may reverse the movement direction of the robot if the DX200 program code runs on the YRC1000/YRC1000micro unmodified. When porting DX200 code to the YRC1000/YRC1000micro, check the direction of robot movement and correct the code if necessary.

A measure is available to keep the source code for DX200 and earlier robot controllers and for the YRC1000/YRC1000micro the same. Set parameter S2C1350 in the YRC1000/YRC1000micro to 0 (default value = 1) to use the same specification as the DX200 and earlier robot controllers. Then, the positive direction of the angle and the positive direction of the pulses for each axis become the same as those for the DX200.

■ **Syntax**

```
int mpConvAngleToPulse(  
    unsigned int    grp_no,  
    long            angle[MP_GRP_AXES_NUM],  
    long            pulse[MP_GRP_AXES_NUM]  
);
```

■ **Parameter**

[grp_no]

Control group number

Acquire the control group number by mpCtrlGrpId2GrpNo().

Constant	Description
0	First control group
1	Second control group
...	...
31	Thirty-second control group

16 Kinematics API

mpConvAngleToPulse

[angle]

Angle (degree) position

Syntax: #define MP_GRP_AXES_NUM (8)

Array	Description
angle[0]	1st axis (S) angle value (unit: 0.0001 deg)
angle[1]	2nd axis (L) angle value (unit: 0.0001 deg)
angle[2]	3rd axis (U) angle value (unit: 0.0001 deg)
angle[3]	4th axis (R) angle value (unit: 0.0001 deg)
angle[4]	5th axis (B) angle value (unit: 0.0001 deg)
angle[5]	6th axis (T) angle value (unit: 0.0001 deg)
angle[6]	7th axis (E) angle value (unit: 0.0001 deg)
angle[7]	8th axis (W) angle value (unit: 0.0001 deg)

[pulse]

Pulse position

Value	Description
pulse[0]	1st axis (S) pulse value
pulse[1]	2nd axis (L) pulse value
pulse[2]	3rd axis (U) pulse value
pulse[3]	4th axis (R) pulse value
pulse[4]	5th axis (B) pulse value
pulse[5]	6th axis (T) pulse value
pulse[6]	7th axis (E) pulse value
pulse[7]	8th axis (W) pulse value

Return Value

0 Normal end
One of the following negative values Error

Symbolic constant (value)	Description
E_KINEMA_FAILURE (-1)	Failed in conversion
E_KINEMA_CONV_IMPOSSIBLE (-2)	Conversion impossible
E_KINEMA_GRP_OUT_RANGE (-3)	Specified control group out of range



In case executing the API in each of the following condition, because the system is busy, correct conversion result cannot be acquired, and then E_KINEMA_FAILURE(-1) is returned to "Return Value".

- When setting a parameter
- When setting a tool file
- When setting an user coordinate
- When setting a calibration

When E_KINEMA_FAILURE(-1) is returned to "Return Value", try executing the API again after an interval.

mpConvFBPulseToPulse

Converts motor axis feedback pulses to load axis feedback pulses.

■ **Syntax**

```
int mpConvFBPulseToPulse(  
    unsigned int    grp_no,  
    long            fbpulse[MP_GRP_AXES_NUM],  
    long            pulse[MP_GRP_AXES_NUM]  
);
```

■ **Parameter**

[grp_no]

Control group number
Acquire the control group number by mpCtrlGrpId2GrpNo().

Constant	Description
0	First control group
1	Second control group
...	...
31	Thirty-second control group

[fbpulse]

Feedback pulse position
Syntax: #define MP_GRP_AXES_NUM (8)

Value	Description
pulse[0]	1st axis (S) pulse value
pulse[1]	2nd axis (L) pulse value
pulse[2]	3rd axis (U) pulse value
pulse[3]	4th axis (R) pulse value
pulse[4]	5th axis (B) pulse value
pulse[5]	6th axis (T) pulse value
pulse[6]	7th axis (E) pulse value
pulse[7]	8th axis (W) pulse value

16 Kinematics API

mpConvFBPulseToPulse

[pulse]

Pulse position

Value	Description
pulse[0]	1st axis (S) pulse value
pulse[1]	2nd axis (L) pulse value
pulse[2]	3rd axis (U) pulse value
pulse[3]	4th axis (R) pulse value
pulse[4]	5th axis (B) pulse value
pulse[5]	6th axis (T) pulse value
pulse[6]	7th axis (E) pulse value
pulse[7]	8th axis (W) pulse value

Return Value

0 Normal end

One of the following negative values Error

Symbolic constant (value)	Description
E_KINEMA_FAILURE (-1)	Failed in conversion
E_KINEMA_CONV_IMPOSSIBLE (-2)	Conversion impossible
E_KINEMA_GRP_OUT_RANGE (-3)	Specified control group out of range



In case executing the API in each of the following condition, because the system is busy, correct conversion result cannot be acquired, and then E_KINEMA_FAILURE(-1) is returned to "Return Value".

- When setting a parameter
- When setting a tool file
- When setting an user coordinate
- When setting a calibration

When E_KINEMA_FAILURE(-1) is returned to "Return Value", try executing the API again after an interval.



Depending on the mechanisms of the robot, the motor axis feedback pulses and the load axis feedback pulses may not match.

For example, when the controller moves only B-axis, the T-axis motor may also move a small amount together with the B-axis motor depending on the mechanisms.

The robot's reference pulses are all values for the load axes, so the motor axis positions change for the B-axis and T-axis, but only the position of load axis for the B-axis changes and the T-axis does not change.

To confirm arrival at the target pulse position that was given with MOVx instructions in the JOB or with mpPulseMOVx APIs, retrieve the feedback pulses of the motor axis with mpGetFBPulsePos() and convert those pulses to the feedback pulses of the load axis with mpConvFBPulseToPulse().

mpMakeFrame

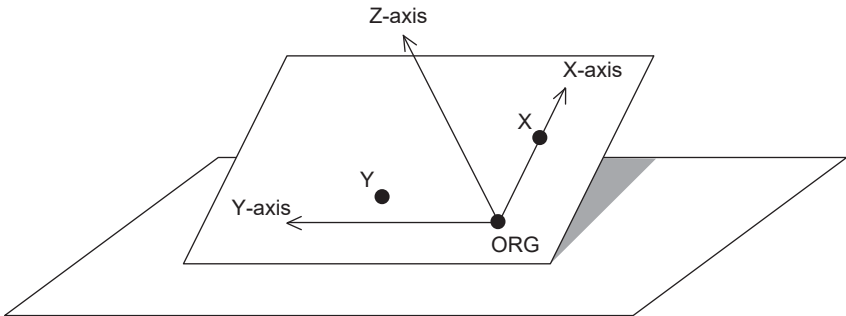
Creates a homogeneous transformation matrix from a three-point position.

■ **Syntax**

```
int mpMakeFrame(  
    MP_XYZ*    org_vector,  
    MP_XYZ*    x_vector,  
    MP_XYZ*    y_vector,  
    MP_FRAME*  frame  
);
```

■ **Description**

“org_vector” indicates the home position and “x_vector” indicates a point on the x-axis. By defining “y_vector, which is a point on the y-axis side, the directions of both y-axis and z-axis are determined.



■ **Parameter**

[org_vector]

Pointer to the home position vector structure of the coordinate (input)

- MP_XYZ
Position vector structure
Syntax: typedef struct
double x;
double y;
double z;
} MP_XYZ;

Member	Description
x	X-component value (unit: micron)
y	Y-component value (unit: micron)
z	Z-component value (unit: micron)

16 Kinematics API

mpMakeFrame

[x_vector]

Pointer to the position vector structure on the x-axis of the coordinate (input)

[y_vector]

Pointer to the position vector structure on the y-axis of the coordinate (input)

[frame]

Pointer to the homogeneous transformation matrix structure (output)

- **MP_FRAME**

Homogeneous transformation matrix structure

Syntax: typedef struct

```
{
double  nx, ny, nz;
double  ox, oy, oz;
double  ax, ay, az;
double  px, py, pz;
} MP_FRAME;
```

Member	Description
nx	normal vector X-component value
ny	normal vector Y-component value
nz	normal vector Z-component value
ox	orientation vector X-component value
oy	orientation vector Y-component value
oz	orientation vector Z-component value
ax	approach vector X-component value
ay	approach vector Y-component value
az	approach vector Z-component value
px	position vector X-component value (unit: micron)
py	position vector Y-component value (unit: micron)
pz	position vector Z-component value (unit: micron)

■ Return Value

0 Normal end

One of the following negative values Error

Symbolic constant (value)	Description
E_KINEMA_FAILURE (-1)	Failed in conversion

mplInvFrame

Create an inverse matrix of an optional homogeneous transformation matrix.

■ **Syntax**

```
int mplInvFrame(  
    MP_FRAME*  org_frame,  
    MP_FRAME*  frame  
);
```

■ **Parameter**

[org_frame]

Pointer to the homogeneous transformation matrix structure (input)

- MP_FRAME
homogeneous transformation matrix structure
Syntax: typedef struct

```
{  
    double    nx, ny, nz;  
    double    ox, oy, oz;  
    double    ax, ay, az;  
    double    ox, py, pz;  
} MP_FRAME;
```

Member	Description
nx	normal vector X-component value
ny	normal vector Y-component value
nz	normal vector Z-component value
ox	orientation vector X-component value
oy	orientation vector Y-component value
oz	orientation vector Z-component value
ax	approach vector X-component value
ay	approach vector Y-component value
az	approach vector Z-component value
px	position vector X-component value (unit: micron)
py	position vector Y-component value (unit: micron)
pz	position vector Z-component value (unit: micron)

16 Kinematics API

mplInvFrame

[frame]

Pointer to the homogeneous transformation matrix structure (output)

■ Return Value

0 Normal end
One of the following negative values Error

Symbolic constant (value)		Description
E_KINEMA_FAILURE	(-1)	Failed in conversion

mpRotFrame

Finds a rotated homogeneous transformation matrix around an optional vector.

■ **Syntax**

```
int mpRotFrame(  
    MP_FRAME*  org_frame,  
    double     angle,  
    MP_XYZ*    vector,  
    MP_FRAME*  frame  
);
```

■ **Parameter**

[org_frame]

Pointer to the homogeneous transformation matrix structure (input)

- MP_FRAME
homogeneous transformation matrix structure
Syntax: typedef struct
{
 double nx, ny, nz;
 double ox, oy, oz;
 double ax, ay, az;
 double px, py, pz;
} MP_FRAME;

Member	Description
nx	normal vector X-component value
ny	normal vector Y-component value
nz	normal vector Z-component value
ox	orientation vector X-component value
oy	orientation vector Y-component value
oz	orientation vector Z-component value
ax	approach vector X-component value
ay	approach vector Y-component value
az	approach vector Z-component value
px	position vector X-component value (unit: micron)
py	position vector Y-component value (unit: micron)
pz	position vector Z-component value (unit: micron)

[angle]

Rotation angle (degree)

16 Kinematics API

mpRotFrame

[vector]

Pointer to the position vector structure (input)

- MP_XYZ
Position vector structure
Syntax: typedef struct
{
double x;
double y;
double z;
} MP_XYZ;

Member	Description
x	X-component value (unit: micron)
y	Y-component value (unit: micron)
z	Z-component value (unit: micron)

[frame]

Pointer to the homogeneous transformation matrix structure (output)

■ Return Value

0 Normal end

One of the following negative values Error

Symbolic constant (value)	Description
E_KINEMA_FAILURE (-1)	Failed in conversion

mpMulFrame

Finds a product of two optional homogeneous transformation matrix.

■ **Syntax**

```
int mpMulFrame(  
    MP_FRAME*   frame1,  
    MP_FRAME*   frame2,  
    MP_FRAME*   prod  
);
```

■ **Description**

Multiplies frame1 by frame2 from the right and stores the result in prod.
([frame1] [frame2] = [prod])

■ **Parameter**

[org_frame]

Pointer to the homogeneous transformation matrix structure (input)

- MP_FRAME
homogeneous transformation matrix structure
Syntax: typedef struct
{
 double nx, ny, nz;
 double ox, oy, oz;
 double ax, ay, az;
 double px, py, pz;
} MP_FRAME;

Member	Description
nx	normal vector X-component value
ny	normal vector Y-component value
nz	normal vector Z-component value
ox	orientation vector X-component value
oy	orientation vector Y-component value
oz	orientation vector Z-component value
ax	approach vector X-component value
ay	approach vector Y-component value
az	approach vector Z-component value
px	position vector X-component value (unit: micron)
py	position vector Y-component value (unit: micron)
pz	position vector Z-component value (unit: micron)

16 Kinematics API

mpMulFrame

[frame2]

Pointer to the homogeneous transformation matrix structure (input)

[prod]

Pointer to the homogeneous transformation matrix structure (output)

■ Return Value

0 Normal end

One of the following negative values Error

Symbolic constant (value)	Description
E_KINEMA_FAILURE (-1)	Failed in conversion

mpZYXeulerToFrame

Converts an optional XYZ eulerian expression cartesian value to a homogeneous transformation matrix.

■ **Syntax**

```
int mpZYXeulerToFrame(  
    MP_COORD*    coord,  
    MP_FRAME*    frame  
);
```

■ **Parameter**

[coord]

Pointer to the cartesian coordinate position structure (input)

- MP_COORD
Cartesian coordinate position structure
Syntax: typedef struct
 {
 long x, y, z;
 long rx, ry, rz;
 } MP_COORD;

Member	Description
x	X-axis coordinate (unit: micron)
y	Y-axis coordinate (unit: micron)
z	Z-axis coordinate (unit: micron)
rx	Wrist angle Rx (unit: 0.0001 deg)
ry	Wrist angle Ry (unit: 0.0001 deg)
rz	Wrist angle Rz (unit: 0.0001 deg)
ex1	Elbow angle Re (unit: 0.0001 deg)
ex2	Reserved by manufacturer

[frame]

Pointer to the homogeneous transformation matrix structure (output)

```
• MP_FRAME
Homogeneous transformation matrix structure
Syntax: typedef struct
{
double    nx, ny, nz;
double    ox, oy, oz;
double    ax, ay, az;
double    px, py, pz;
} MP_FRAME;
```

Member	Description
nx	normal vector X-component value
ny	normal vector Y-component value
nz	normal vector Z-component value
ox	orientation vector X-component value
oy	orientation vector Y-component value
oz	orientation vector Z-component value
ax	approach vector X-component value
ay	approach vector Y-component value
az	approach vector Z-component value
px	position vector X-component value (unit: micron)
py	position vector Y-component value (unit: micron)
pz	position vector Z-component value (unit: micron)

■ Return Value

0 Normal end
One of the following negative values Error

Symbolic constant (value)	Description
E_KINEMA_FAILURE (-1)	Failed in conversion

mpFrameToZYXeuler

Converts an optional XYZ eulerian expression cartesian value to a homogeneous transformation matrix.

■ **Syntax**

```
int mpFrameToZYXeuler(  
    MP_FRAME*   frame,  
    MP_COORD*   coord  
);
```

■ **Parameter**

[frame]

Pointer to the homogeneous transformation matrix structure (input)

- **MP_FRAME**
Homogeneous transformation matrix structure
Syntax: typedef struct
{
 double nx, ny, nz;
 double ox, oy, oz;
 double ax, ay, az;
 double px, py, pz;
} MP_FRAME;

Member	Description
nx	normal vector X-component value
ny	normal vector Y-component value
nz	normal vector Z-component value
ox	orientation vector X-component value
oy	orientation vector Y-component value
oz	orientation vector Z-component value
ax	approach vector X-component value
ay	approach vector Y-component value
az	approach vector Z-component value
px	position vector X-component value (unit: micron)
py	position vector Y-component value (unit: micron)
pz	position vector Z-component value (unit: micron)

16 Kinematics API
mpFrameToZYXeuler

[coord]

Pointer to the cartesian coordinate position structure (output)

- MP_COORD
Cartesian coordinate position structure
Syntax: typedef struct

```
{
    long  x, y, z;
    long  rx, ry, rz;
    long  ex1, ex2;
} MP_COORD;
```

Member	Description
x	X-axis coordinate (unit: micron)
y	Y-axis coordinate (unit: micron)
z	Z-axis coordinate (unit: micron)
rx	Wrist angle Rx (unit: 0.0001 deg)
ry	Wrist angle Ry (unit: 0.0001 deg)
rz	Wrist angle Rz (unit: 0.0001 deg)
ex1	Elbow angle Re (unit: 0.0001 deg)
ex2	Reserved by manufacturer

■ Return Value

- 0 Normal end
- One of the following negative values Error

Symbolic constant (value)	Description
E_KINEMA_FAILURE (-1)	Failed in conversion

mpCrossProduct

Finds an outer product of two vectors.

■ **Syntax**

```
int mpCrossProduct(  
    MP_XYZ*  vector1,  
    MP_XYZ*  vector2,  
    MP_XYZ*  prod  
);
```

■ **Description**

Returns the multiplication of vector 2 and vector1 to prod.

■ **Parameter**

[vector1]

Pointer to the position vector structure (input)

- MP_XYZ
Position vector structure
Syntax: typedef struct
{
 double x;
 double y;
 double z;
} MP_XYZ;

Member	Description
x	X-component value (unit: micron)
y	Y-component value (unit: micron)
z	Z-component value (unit: micron)

[vector2]

Pointer to the position vector structure (input)

[prod]

Pointer to the position vector structure (output)

■ **Return Value**

- 0 Normal end
- One of the following negative values Error

Symbolic constant (value)	Description
E_KINEMA_FAILURE (-1)	Failed in conversion

mplInnerProduct

Finds an inner product of two vectors.

■ **Syntax**

```
int mplInnerProduct(  
  MP_XYZ*  vector1,  
  MP_XYZ*  vector2,  
  double*   prod  
);
```

■ **Parameter**

[vector1]

Pointer to the position vector structure (input)

- MP_XYZ
Position vector structure
Syntax: typedef struct
{
 double x;
 double y;
 double z;
} MP_XYZ;

Member	Description
x	X-component value (unit: micron)
y	Y-component value (unit: micron)
z	Z-component value (unit: micron)

[vector2]

Pointer to the position vector structure (input)

[prod]

Pointer to the cosine (output)

■ **Return Value**

- 0 Normal end
- One of the following negative values Error

Symbolic constant (value)	Description
E_KINEMA_FAILURE (-1)	Failed in conversion

17 Alarm List

Alarm Number	Alarm Name	Subcode	Meaning
1020	MOTOPLUS APPLICATION LOAD ERROR	1	The number of loaded files exceeds the limit.
		2	The memory is insufficient. (Available memory area is less than 3 Mbyte.)
		3	Failed to open the directory.
		4	Load failure (Failed to open the file.)
		5	Load failure (Undefined symbol)
		6	Load failure (Others: application overloaded)
		7	Failed to initialize the API library.
		8	No user root task
		9	Failed to create the user root task.
		10	Failed to create RAM-Disk.
4478	MM TASK NO RESPONSE (MotoPlus)		The process requested from the MotoPlus application to the MM task was not completed within the specified time.
4479	MOTOPLUS MM TASK WATCHDOG ERROR		The man-machine task does not run for 3 seconds or more.

Error Number	Error Name	Meaning	Remedy
300	Undefined user frame	An operation was executed on user coordinates with uncompleted settings.	- Complete the settings of the user coordinates to use. - Check if the user coordinates number being controlled from the programming pendant or specified in MotoPlus is a user coordinate number with completed settings.
3090	Transmission error with the media	Transmission to the target external memory failed.	- Check that the device specified for "Device" is inserted. - Check that nothing is wrong with the inserted device.
3640	Limit of file number	Detected that the maximum number of *.out files that can be loaded already exist in the controller. No more *.out file can be loaded.	Delete existing *.out files in the controller to stay within the limit for the number of files loaded.

18 Version Information for the MotoPlus API

18.1 Version Information for the YRC1000 MotoPlus API

Each MotoPlus API is supported from the system software version given in the following table.

API Name		API Rev.	MotoPlusIDE Version	System Software Version	Note
Task control API	mpCreateTask	1st	1.00	YAS1.00.00-00	
	mpDeleteTask	1st	1.00	YAS1.00.00-00	
	mpTaskSuspend	1st	1.00	YAS1.00.00-00	
	mpTaskResume	1st	1.00	YAS1.00.00-00	
	mpMsgQCreate	1st	1.00	YAS1.00.00-00	
	mpMsgQDelete	1st	1.00	YAS1.00.00-00	
	mpMsgQReceive	1st	1.00	YAS1.00.00-00	
	mpMsgQSend	1st	1.00	YAS1.00.00-00	
	mpErrMsgQRcv	1st	1.00	YAS1.00.00-00	
	mpErrMsgQSnd	1st	1.00	YAS1.00.00-00	
	mpSemBCreate	1st	1.00	YAS1.00.00-00	
	mpSemDelete	1st	1.00	YAS1.00.00-00	
	mpSemTake	1st	1.00	YAS1.00.00-00	
	mpSemGive	1st	1.00	YAS1.00.00-00	
	mpErrSemTake	1st	1.00	YAS1.00.00-00	
	mpTaskDelay	1st	1.00	YAS1.00.00-00	
	mpGetRtc	1st	1.00	YAS1.00.00-00	
	mpGetSysClkRate	1st	1.00	YAS1.00.00-00	
	mpClkAnnounce	1st	1.00	YAS1.00.00-00	
	mpClkGetPeriod	1st	2.52	YAS5.60.00-00	MotoPlusSDK 3.10
	mpStopWatchCreate	1st	1.00	YAS1.00.00-00	
	mpStopWatchDelete	1st	1.00	YAS1.00.00-00	
	mpStopWatchStart	1st	1.00	YAS1.00.00-00	
	mpStopWatchStop	1st	1.00	YAS1.00.00-00	
	mpStopWatchLap	1st	1.00	YAS1.00.00-00	
	mpStopWatchReset	1st	1.00	YAS1.00.00-00	
	mpStopWatchGetTime	1st	1.00	YAS1.00.00-00	
	mpStopWatchGetLapNum	1st	1.00	YAS1.00.00-00	
	mpStopWatchGetLapTime	1st	1.00	YAS1.00.00-00	
	mpStopWatchGetAliveLapNo	1st	1.00	YAS1.00.00-00	
User watchdog API	mpUsrWdogCreate	1st	1.00	YAS1.00.00-00	
	mpUsrWdogDelete	1st	1.00	YAS1.00.00-00	
	mpUsrWdogStart	1st	1.00	YAS1.00.00-00	
	mpUsrWdogClear	1st	1.00	YAS1.00.00-00	

18 Version Information for the MotoPlus API

API Name		API Rev.	MotoPlusIDE Version	System Software Version	Note
Network API	mpSocket	1st	1.00	YAS1.00.00-00	
	mpListen	1st	1.00	YAS1.00.00-00	
	mpAccept	1st	1.00	YAS1.00.00-00	
	mpBind	1st	1.00	YAS1.00.00-00	
	mpConnect	1st	1.00	YAS1.00.00-00	
	mpRecv	1st	1.00	YAS1.00.00-00	
	mpRecvFrom	1st	1.00	YAS1.00.00-00	
	mpSend	1st	1.00	YAS1.00.00-00	
	mpSendTo	1st	1.00	YAS1.00.00-00	
	mpClose	1st	1.00	YAS1.00.00-00	
	mpHtonl	1st	1.00	YAS1.00.00-00	
	mpHtons	1st	1.00	YAS1.00.00-00	
	mpNtohl	1st	1.00	YAS1.00.00-00	
	mpNtohs	1st	1.00	YAS1.00.00-00	
	mplnetAddr	1st	1.00	YAS1.00.00-00	
	mplnetNtoa	1st	1.00	YAS1.00.00-00	
	mplnetNtoaB	1st	1.00	YAS1.00.00-00	
	mpGetsockname	1st	1.00	YAS1.00.00-00	
	mpGetpeername	1st	1.00	YAS1.00.00-00	
	mpSetsockopt	1st	1.00	YAS1.00.00-00	
	mpIoctl	1st	1.00	YAS1.00.00-00	
	mpSelect	1st	1.00	YAS1.00.00-00	
Serial communication API	mpRsInit	1st	1.00	YAS1.00.00-00	
	mpRsOpen	1st	1.00	YAS1.00.00-00	
	mpRsClose	1st	1.00	YAS1.00.00-00	
	mpRsSend	1st	1.00	YAS1.00.00-00	
	mpRsRecv	1st	1.00	YAS1.00.00-00	
Memory pool control API	mpMalloc	1st	1.00	YAS1.00.00-00	
	mpFree	1st	1.00	YAS1.00.00-00	

18 Version Information for the MotoPlus API

API Name		API Rev.	MotoPlusIDE Version	System Software Version	Note
File control API	mpCreate	1st	1.00	YAS1.00.00-00	
	mpOpen	1st	1.00	YAS1.00.00-00	
	mpRemove	1st	1.00	YAS1.00.00-00	
	mpClose	1st	1.00	YAS1.00.00-00	
	mpRename	1st	1.00	YAS1.00.00-00	
	mpRead	1st	1.00	YAS1.00.00-00	
	mpWrite	1st	1.00	YAS1.00.00-00	
	mpIoctl	1st	1.00	YAS1.00.00-00	
	mpLseek	1st	1.00	YAS1.00.00-00	
	mpFstat	1st	1.00	YAS1.00.00-00	
	mpStat	1st	1.00	YAS1.00.00-00	
	mpOpendir	1st	1.00	YAS1.00.00-00	
	mpReaddir	1st	1.00	YAS1.00.00-00	
	mpRewinddir	1st	1.00	YAS1.00.00-00	
	mpClosedir	1st	1.00	YAS1.00.00-00	
	mpCreatedir	1st	1.00	YAS1.00.00-00	
	mpRemovedir	1st	1.00	YAS1.00.00-00	
	mpRenamedir	1st	1.00	YAS1.00.00-00	
Existing file access API	mpLoadFile	1st	1.00	YAS1.00.00-00	
	mpSaveFile	1st	1.00	YAS1.00.00-00	
	mpRefreshFileList	1st	1.00	YAS1.00.00-00	
	mpGetFileCount	1st	1.00	YAS1.00.00-00	
	mpGetFileName	1st	1.00	YAS1.00.00-00	
	mpFdWriteFile	1st	1.00	YAS1.00.00-00	
	mpFdReadFile	1st	1.00	YAS1.00.00-00	
	mpFdGetJobList	1st	1.00	YAS1.00.00-00	
System monitor API	mpGetVarData	1st	1.00	YAS1.00.00-00	
	mpGetSVarInfo	1st	1.00	YAS1.00.00-00	
	mpReadIO	1st	1.00	YAS1.00.00-00	
	mpMonitor	1st	1.00	YAS1.00.00-00	
	mpGetPosVarData	1st	1.00	YAS1.00.00-00	
	mpGetUserVars	1st	1.00	YAS1.00.00-00	
	mpGetAlarmStatus	1st	1.00	YAS1.00.00-00	
	mpGetAlarmCode	1st	1.00	YAS1.00.00-00	
	mpGetMode	1st	1.00	YAS1.00.00-00	
	mpGetCycle	1st	1.00	YAS1.00.00-00	
	mpGetServoPower	1st	1.00	YAS1.00.00-00	
	mpGetPlayStatus	1st	1.00	YAS1.00.00-00	
	mpGetMasterJob	1st	1.00	YAS1.00.00-00	
	mpGetCurJob	1st	1.00	YAS1.00.00-00	
	mpGetSpecialOpStatus	1st	1.00	YAS1.00.00-00	
	mpGetJobDate	1st	1.00	YAS1.00.00-00	
	mpGetCartPos	1st	1.00	YAS1.00.00-00	

18 Version Information for the MotoPlus API

API Name		API Rev.	MotoPlusIDE Version	System Software Version	Note
System monitor API	mpGetCartPosEx	1st	1.00	YAS1.00.00-00	
		2nd	2.41	YAS4.40.00-00	Added the feature to retrieve the current position at the current tool number and to retrieve the current tool number. Refer to the description of the API for details.
	mpGetPulsePos	1st	1.00	YAS1.00.00-00	
	mpGetRadPos	1st	1.00	YAS1.00.00-00	
	mpGetRadPosEx	1st	1.00	YAS1.00.00-00	
	mpGetDegPos	1st	1.00	YAS1.00.00-00	
	mpGetDegPosEx	1st	1.00	YAS1.00.00-00	
	mpGetFBPulsePos	1st	1.00	YAS1.00.00-00	
	mpGetFBPulsePosEx	1st	1.00	YAS1.00.00-00	
	mpGetServoSpeed	1st	1.00	YAS1.00.00-00	
	mpGetFBSpeed	1st	1.00	YAS1.00.00-00	
	mpGetTorque	1st	1.00	YAS1.00.00-00	
	mpGetTorqueEx	1st	1.00	YAS1.00.00-00	
	mpGetSysTimes	1st	1.00	YAS1.00.00-00	
	mpGetSysVersionNo	1st	1.00	YAS1.00.00-00	
	mpGetJogSpeed	1st	1.00	YAS1.00.00-00	
	mpGetJogCoord	1st	1.00	YAS1.00.00-00	
		2nd	2.22	YAS2.60.00-00	Added a feature to retrieve the user coordinates number. Refer to the description of the API for details.
	mpCtrlGrpId2GrpNo	1st	1.00	YAS1.00.00-00	
	mpGetToolData	1st	1.00	YAS1.00.00-00	
	mpGetJobStepData	1st	1.00	YAS1.00.00-00	
	mpGetToolNo	1st	1.00	YAS1.00.00-00	
	mpGetEncoderTemp	1st	2.01	YAS2.41.00-00	

18 Version Information for the MotoPlus API

API Name		API Rev.	MotoPlusIDE Version	System Software Version	Note
System control API	mpPutVarData	1st	1.00	YAS1.00.00-00	
	mpPutSVarInfo	1st	1.00	YAS1.00.00-00	
	mpWriteIO	1st	1.00	YAS1.00.00-00	
	mpPutPosVarData	1st	1.00	YAS1.00.00-00	
	mpPutUserVars	1st	1.00	YAS1.00.00-00	
	mpSetAlarm	1st	1.00	YAS1.00.00-00	
	mpCancelError	1st	1.00	YAS1.00.00-00	
	mpResetAlarm	1st	1.00	YAS1.00.00-00	
	mpSetCycle	1st	1.00	YAS1.00.00-00	
	mpSetServoPower	1st	1.00	YAS1.00.00-00	
	mpSetMasterJob	1st	1.00	YAS1.00.00-00	
	mpSetCurJob	1st	1.00	YAS1.00.00-00	
	mpStartJob	1st	1.00	YAS1.00.00-00	
	mpHold	1st	1.00	YAS1.00.00-00	
	mpWaitForJobEnd	1st	1.00	YAS1.00.00-00	
	mpDeleteJob	1st	1.00	YAS1.00.00-00	
	mpSetJogCoord	1st	1.00	YAS1.00.00-00	
	mpSetToolNo	1st	1.00	YAS1.00.00-00	
	mpSetJobPosData	1st	1.00	YAS1.00.00-00	
	mpParallelShiftJob	1st	1.00	YAS1.00.00-00	
	mpApplicationInfoNotify	1st	1.00	YAS1.00.00-00	
Motion control API	mpMotStart	1st	1.00	YAS1.00.00-00	
	mpMotStop	1st	1.00	YAS1.00.00-00	
	mpMotTargetClear	1st	1.00	YAS1.00.00-00	
	mpMotTargetSend	1st	1.00	YAS1.00.00-00	
	mpMotTargetReceive	1st	1.00	YAS1.00.00-00	
	mpMotTargetGetVacantBufNum	1st	2.51	YAS5.23.00-00	
	mpMotSetCoord	1st	1.00	YAS1.00.00-00	
	mpMotSetSpeed	1st	1.00	YAS1.00.00-00	
	mpMotSetTool	1st	1.00	YAS1.00.00-00	
	mpMotSetOrigin	1st	1.00	YAS1.00.00-00	
	mpMotSetTask	1st	1.00	YAS1.00.00-00	
	mpMotSetSync	1st	1.00	YAS1.00.00-00	
	mpMotResetSync	1st	1.00	YAS1.00.00-00	
	mpMotSetConfig	1st	1.00	YAS1.00.00-00	
	mpMotSetAccuracy	1st	1.00	YAS1.00.00-00	
	mpMotSetAccel	1st	2.01	YAS2.41.00-00	
	mpMotSetDecel	1st	2.01	YAS2.41.00-00	
Motion monitor/control API	mpIMOV	1st	1.00	YAS1.00.00-00	
	mpMOVJ	1st	1.00	YAS1.00.00-00	
	mpMOVL	1st	1.00	YAS1.00.00-00	
	mpPulseMOVJ	1st	1.00	YAS1.00.00-00	
	mpPulseMOVL	1st	1.00	YAS1.00.00-00	
	mpManualMOV	1st	1.00	YAS1.00.00-00	

18 Version Information for the MotoPlus API

API Name		API Rev.	MotoPlusIDE Version	System Software Version	Note
Sensor control API	mpReceiveSkillCommand	1st	1.00	YAS1.00.00-00	
	mpEndSkillCommand Process	1st	1.00	YAS1.00.00-00	
	mpMeiGetJobExecTask	1st	1.00	YAS1.00.00-00	
	mpMeiGetInterpolation	1st	1.00	YAS1.00.00-00	
	mpMeiGetExecControlGroup	1st	1.00	YAS1.00.00-00	
	mpMeiPutCorrPath	1st	1.00	YAS1.00.00-00	
	mpMeiPutForcePathEnd	1st	1.00	YAS1.00.00-00	
	mpMeiPutSpdOverride	1st	1.00	YAS1.00.00-00	
	mpMeiIncrementMove	1st	1.00	YAS1.00.00-00	
Robot Control API	mpExRcsIncrementMove	1st	1.00	YAS1.00.00-00	
Servo monitor/ control API	mpSvsGetVelTrqFb	1st	1.00	YAS1.00.00-00	
	mpSvsStartTrqLimit	1st	1.00	YAS1.00.00-00	
	mpSvsSetTrqLimit	1st	1.00	YAS1.00.00-00	
	mpSvsEndTrqLimit	1st	1.00	YAS1.00.00-00	
	mpSvsStartTrqCtrl	1st	1.00	YAS1.00.00-00	
	mpSvsSetTrqCtrl	1st	1.00	YAS1.00.00-00	
	mpSvsEndTrqCtrl	1st	1.00	YAS1.00.00-00	
	mpSvsForcelInit	1st	1.00	YAS1.00.00-00	
Kinematics API	mpConvAxesToCartPos	1st	1.00	YAS1.00.00-00	
	mpConvCartPosToAxes	1st	1.00	YAS1.00.00-00	
	mpConvPulseToAngle	1st	1.00	YAS1.00.00-00	
	mpConvAngleToPulse	1st	1.00	YAS1.00.00-00	
	mpConvFBPulseToPulse	1st	1.00	YAS1.00.00-00	
	mpMakeFrame	1st	1.00	YAS1.00.00-00	
	mpInvFrame	1st	1.00	YAS1.00.00-00	
	mpRotFrame	1st	1.00	YAS1.00.00-00	
	mpMulFrame	1st	1.00	YAS1.00.00-00	
	mpZYXeulerToFrame	1st	1.00	YAS1.00.00-00	
	mpFrameToZYXeuler	1st	1.00	YAS1.00.00-00	
	mpCrossProduct	1st	1.00	YAS1.00.00-00	
	mpInnerProduct	1st	1.00	YAS1.00.00-00	

18 Version Information for the MotoPlus API

18.2 Version Information for the YRC1000micro MotoPlus API

Each MotoPlus API is supported from the system software version given in the following table.

API Name		API Rev.	MotoPlusIDE Version	System Software Version (YRC1000micro)	Note
Task control API	mpCreateTask	1st	1.00	YBS1.00.00-00	
	mpDeleteTask	1st	1.00	YBS1.00.00-00	
	mpTaskSuspend	1st	1.00	YBS1.00.00-00	
	mpTaskResume	1st	1.00	YBS1.00.00-00	
	mpMsgQCreate	1st	1.00	YBS1.00.00-00	
	mpMsgQDelete	1st	1.00	YBS1.00.00-00	
	mpMsgQReceive	1st	1.00	YBS1.00.00-00	
	mpMsgQSend	1st	1.00	YBS1.00.00-00	
	mpErrMsgQRcv	1st	1.00	YBS1.00.00-00	
	mpErrMsgQSnd	1st	1.00	YBS1.00.00-00	
	mpSemBCreate	1st	1.00	YBS1.00.00-00	
	mpSemDelete	1st	1.00	YBS1.00.00-00	
	mpSemTake	1st	1.00	YBS1.00.00-00	
	mpSemGive	1st	1.00	YBS1.00.00-00	
	mpErrSemTake	1st	1.00	YBS1.00.00-00	
	mpTaskDelay	1st	1.00	YBS1.00.00-00	
	mpGetRtc	1st	1.00	YBS1.00.00-00	
	mpGetSysClkRate	1st	1.00	YBS1.00.00-00	
	mpClkAnnounce	1st	1.00	YBS1.00.00-00	
	mpClkGetPeriod	1st	2.52	YBS4.21.00-00	MotoPlusSDK 3.10
	mpStopWatchCreate	1st	1.00	YBS1.00.00-00	
	mpStopWatchDelete	1st	1.00	YBS1.00.00-00	
	mpStopWatchStart	1st	1.00	YBS1.00.00-00	
	mpStopWatchStop	1st	1.00	YBS1.00.00-00	
	mpStopWatchLap	1st	1.00	YBS1.00.00-00	
	mpStopWatchReset	1st	1.00	YBS1.00.00-00	
	mpStopWatchGetTime	1st	1.00	YBS1.00.00-00	
	mpStopWatchGetLapNum	1st	1.00	YBS1.00.00-00	
	mpStopWatchGetLapTime	1st	1.00	YBS1.00.00-00	
	mpStopWatchGetAliveLapNo	1st	1.00	YBS1.00.00-00	
User watchdog API	mpUsrWdogCreate	1st	1.00	YBS1.00.00-00	
	mpUsrWdogDelete	1st	1.00	YBS1.00.00-00	
	mpUsrWdogStart	1st	1.00	YBS1.00.00-00	
	mpUsrWdogClear	1st	1.00	YBS1.00.00-00	
Network API	mpSocket	1st	1.00	YBS1.00.00-00	
	mpListen	1st	1.00	YBS1.00.00-00	
	mpAccept	1st	1.00	YBS1.00.00-00	
	mpBind	1st	1.00	YBS1.00.00-00	
	mpConnect	1st	1.00	YBS1.00.00-00	
	mpRecv	1st	1.00	YBS1.00.00-00	
	mpRecvFrom	1st	1.00	YBS1.00.00-00	

18 Version Information for the MotoPlus API

API Name		API Rev.	MotoPlusIDE Version	System Software Version (YRC1000micro)	Note
Network API	mpSend	1st	1.00	YBS1.00.00-00	
	mpSendTo	1st	1.00	YBS1.00.00-00	
	mpClose	1st	1.00	YBS1.00.00-00	
	mpHtonl	1st	1.00	YBS1.00.00-00	
	mpHtons	1st	1.00	YBS1.00.00-00	
	mpNtohl	1st	1.00	YBS1.00.00-00	
	mpNtohs	1st	1.00	YBS1.00.00-00	
	mplnetAddr	1st	1.00	YBS1.00.00-00	
	mplnetNtoa	1st	1.00	YBS1.00.00-00	
	mplnetNtoaB	1st	1.00	YBS1.00.00-00	
	mpGetsockname	1st	1.00	YBS1.00.00-00	
	mpGetpeername	1st	1.00	YBS1.00.00-00	
	mpSetsockopt	1st	1.00	YBS1.00.00-00	
	mpioctl	1st	1.00	YBS1.00.00-00	
	mpSelect	1st	1.00	YBS1.00.00-00	
Memory pool control API	mpMalloc	1st	1.00	YBS1.00.00-00	
	mpFree	1st	1.00	YBS1.00.00-00	
File control API	mpCreate	1st	1.00	YBS1.00.00-00	
	mpOpen	1st	1.00	YBS1.00.00-00	
	mpRemove	1st	1.00	YBS1.00.00-00	
	mpClose	1st	1.00	YBS1.00.00-00	
	mpRename	1st	1.00	YBS1.00.00-00	
	mpRead	1st	1.00	YBS1.00.00-00	
	mpWrite	1st	1.00	YBS1.00.00-00	
	mpioctl	1st	1.00	YBS1.00.00-00	
	mpLseek	1st	1.00	YBS1.00.00-00	
	mpFstat	1st	1.00	YBS1.00.00-00	
	mpStat	1st	1.00	YBS1.00.00-00	
	mpOpendir	1st	1.00	YBS1.00.00-00	
	mpReaddir	1st	1.00	YBS1.00.00-00	
	mpRewinddir	1st	1.00	YBS1.00.00-00	
	mpClosedir	1st	1.00	YBS1.00.00-00	
	mpCreatedir	1st	1.00	YBS1.00.00-00	
	mpRemovedir	1st	1.00	YBS1.00.00-00	
	mpRenamedir	1st	1.00	YBS1.00.00-00	
Existing file access API	mpLoadFile	1st	1.00	YBS1.00.00-00	
	mpSaveFile	1st	1.00	YBS1.00.00-00	
	mpRefreshFileList	1st	1.00	YBS1.00.00-00	
	mpGetFileCount	1st	1.00	YBS1.00.00-00	
	mpGetFileName	1st	1.00	YBS1.00.00-00	
	mpFdWriteFile	1st	1.00	YBS1.00.00-00	
	mpFdReadFile	1st	1.00	YBS1.00.00-00	
	mpFdGetJobList	1st	1.00	YBS1.00.00-00	

18 Version Information for the MotoPlus API

API Name		API Rev.	MotoPlusIDE Version	System Software Version (YRC1000micro)	Note
System monitor API	mpGetVarData	1st	1.00	YBS1.00.00-00	
	mpGetSVarInfo	1st	1.00	YBS1.00.00-00	
	mpReadIO	1st	1.00	YBS1.00.00-00	
	mpMonitor	1st	1.00	YBS1.00.00-00	
	mpGetPosVarData	1st	1.00	YBS1.00.00-00	
	mpGetUserVars	1st	1.00	YBS1.00.00-00	
	mpGetAlarmStatus	1st	1.00	YBS1.00.00-00	
	mpGetAlarmCode	1st	1.00	YBS1.00.00-00	
	mpGetMode	1st	1.00	YBS1.00.00-00	
	mpGetCycle	1st	1.00	YBS1.00.00-00	
	mpGetServoPower	1st	1.00	YBS1.00.00-00	
	mpGetPlayStatus	1st	1.00	YBS1.00.00-00	
	mpGetMasterJob	1st	1.00	YBS1.00.00-00	
	mpGetCurJob	1st	1.00	YBS1.00.00-00	
	mpGetSpecialOpStatus	1st	1.00	YBS1.00.00-00	
	mpGetJobDate	1st	1.00	YBS1.00.00-00	
	mpGetCartPos	1st	1.00	YBS1.00.00-00	
	mpGetCartPosEx	1st	1.00	YBS1.00.00-00	
		2nd	2.41	YBS2.54.00-00	Added the feature to retrieve the current position at the current tool number and to retrieve the current tool number. Refer to the description of the API for details.
	mpGetPulsePos	1st	1.00	YBS1.00.00-00	
	mpGetRadPos	1st	1.00	YBS1.00.00-00	
	mpGetRadPosEx	1st	1.00	YBS1.00.00-00	
	mpGetDegPos	1st	1.00	YBS1.00.00-00	
	mpGetDegPosEx	1st	1.00	YBS1.00.00-00	
	mpGetFBPulsePos	1st	1.00	YBS1.00.00-00	
	mpGetFBPulsePosEx	1st	1.00	YBS1.00.00-00	
	mpGetServoSpeed	1st	1.00	YBS1.00.00-00	
	mpGetFBSpeed	1st	1.00	YBS1.00.00-00	
	mpGetTorque	1st	1.00	YBS1.00.00-00	
	mpGetTorqueEx	1st	1.00	YBS1.00.00-00	
	mpGetSysTimes	1st	1.00	YBS1.00.00-00	
	mpGetSysVersionNo	1st	1.00	YBS1.00.00-00	
	mpGetJogSpeed	1st	1.00	YBS1.00.00-00	
	mpGetJogCoord	1st	1.00	YBS1.00.00-00	
		2nd	2.22	YBS2.10.00-00	Added a feature to retrieve the user coordinates number. Refer to the description of the API for details.

18 Version Information for the MotoPlus API

API Name		API Rev.	MotoPlusIDE Version	System Software Version (YRC1000micro)	Note
System monitor API	mpCtrlGrpId2GrpNo	1st	1.00	YBS1.00.00-00	
	mpGetToolData	1st	1.00	YBS1.00.00-00	
	mpGetJobStepData	1st	1.00	YBS1.00.00-00	
	mpGetToolNo	1st	1.00	YBS1.00.00-00	
	mpGetEncoderTemp	1st	2.01	YBS1.31.00-00	
System control API	mpPutVarData	1st	1.00	YBS1.00.00-00	
	mpPutSVarInfo	1st	1.00	YBS1.00.00-00	
	mpWriteIO	1st	1.00	YBS1.00.00-00	
	mpPutPosVarData	1st	1.00	YBS1.00.00-00	
	mpPutUserVars	1st	1.00	YBS1.00.00-00	
	mpSetAlarm	1st	1.00	YBS1.00.00-00	
	mpCancelError	1st	1.00	YBS1.00.00-00	
	mpResetAlarm	1st	1.00	YBS1.00.00-00	
	mpSetCycle	1st	1.00	YBS1.00.00-00	
	mpSetServoPower	1st	1.00	YBS1.00.00-00	
	mpSetMasterJob	1st	1.00	YBS1.00.00-00	
	mpSetCurJob	1st	1.00	YBS1.00.00-00	
	mpStartJob	1st	1.00	YBS1.00.00-00	
	mpHold	1st	1.00	YBS1.00.00-00	
	mpWaitForJobEnd	1st	1.00	YBS1.00.00-00	
	mpDeleteJob	1st	1.00	YBS1.00.00-00	
	mpSetJogCoord	1st	1.00	YBS1.00.00-00	
	mpSetToolNo	1st	1.00	YBS1.00.00-00	
	mpSetJobPosData	1st	1.00	YBS1.00.00-00	
	mpParallelShiftJob	1st	1.00	YBS1.00.00-00	
	mpApplicationInfoNotify	1st	1.00	YBS1.00.00-00	
Motion control API	mpMotStart	1st	1.00	YBS1.00.00-00	
	mpMotStop	1st	1.00	YBS1.00.00-00	
	mpMotTargetClear	1st	1.00	YBS1.00.00-00	
	mpMotTargetSend	1st	1.00	YBS1.00.00-00	
	mpMotTargetReceive	1st	1.00	YBS1.00.00-00	
	mpMotTargetGetVacantBufNum	1st	2.42	YBS4.11.00-00	
	mpMotSetCoord	1st	1.00	YBS1.00.00-00	
	mpMotSetSpeed	1st	1.00	YBS1.00.00-00	
	mpMotSetTool	1st	1.00	YBS1.00.00-00	
	mpMotSetOrigin	1st	1.00	YBS1.00.00-00	
	mpMotSetTask	1st	1.00	YBS1.00.00-00	
	mpMotSetSync	1st	1.00	YBS1.00.00-00	
	mpMotResetSync	1st	1.00	YBS1.00.00-00	
	mpMotSetConfig	1st	1.00	YBS1.00.00-00	
	mpMotSetAccuracy	1st	1.00	YBS1.00.00-00	
	mpMotSetAccel	1st	2.01	YBS1.31.00-00	
	mpMotSetDecel	1st	2.01	YBS1.31.00-00	

18 Version Information for the MotoPlus API

API Name		API Rev.	MotoPlusIDE Version	System Software Version (YRC1000micro)	Note
Motion monitor/ control API	mpIMOV	1st	1.00	YBS1.00.00-00	
	mpMOVJ	1st	1.00	YBS1.00.00-00	
	mpMOVL	1st	1.00	YBS1.00.00-00	
	mpPulseMOVJ	1st	1.00	YBS1.00.00-00	
	mpPulseMOVL	1st	1.00	YBS1.00.00-00	
	mpManualMOV	1st	1.00	YBS1.00.00-00	
Sensor control API	mpReceiveSkillCommand	1st	1.00	YBS1.00.00-00	
	mpEndSkillCommandProcess	1st	1.00	YBS1.00.00-00	
	mpMeiGetJobExecTask	1st	1.00	YBS1.00.00-00	
	mpMeiGetInterpolation	1st	1.00	YBS1.00.00-00	
	mpMeiGetExecControlGroup	1st	1.00	YBS1.00.00-00	
	mpMeiPutCorrPath	1st	1.00	YBS1.00.00-00	
	mpMeiPutForcePathEnd	1st	1.00	YBS1.00.00-00	
	mpMeiPutSpdOverride	1st	1.00	YBS1.00.00-00	
	mpMeiIncrementMove	1st	1.00	YBS1.00.00-00	
Robot Control API	mpExRcsIncrementMove	1st	1.00	YBS1.00.00-00	
Servo monitor/ control API	mpSvsGetVelTrqFb	1st	1.00	YBS1.00.00-00	
	mpSvsStartTrqLimit	1st	1.00	YBS1.00.00-00	
	mpSvsSetTrqLimit	1st	1.00	YBS1.00.00-00	
	mpSvsEndTrqLimit	1st	1.00	YBS1.00.00-00	
	mpSvsStartTrqCtrl	1st	1.00	YBS1.00.00-00	
	mpSvsSetTrqCtrl	1st	1.00	YBS1.00.00-00	
	mpSvsEndTrqCtrl	1st	1.00	YBS1.00.00-00	
	mpSvsForceInit	1st	1.00	YBS1.00.00-00	
Kinematics API	mpConvAxesToCartPos	1st	1.00	YBS1.00.00-00	
	mpConvCartPosToAxes	1st	1.00	YBS1.00.00-00	
	mpConvPulseToAngle	1st	1.00	YBS1.00.00-00	
	mpConvAngleToPulse	1st	1.00	YBS1.00.00-00	
	mpConvFBPulseToPulse	1st	1.00	YBS1.00.00-00	
	mpMakeFrame	1st	1.00	YBS1.00.00-00	
	mpInvFrame	1st	1.00	YBS1.00.00-00	
	mpRotFrame	1st	1.00	YBS1.00.00-00	
	mpMulFrame	1st	1.00	YBS1.00.00-00	
	mpZYX eulerToFrame	1st	1.00	YBS1.00.00-00	
	mpFrameToZYX euler	1st	1.00	YBS1.00.00-00	
	mpCrossProduct	1st	1.00	YBS1.00.00-00	
	mpInnerProduct	1st	1.00	YBS1.00.00-00	

YRC1000 OPTIONS INSTRUCTIONS

**Reference Manual For New Language Environment MotoPlus
(API Function Specifications)**

YASKAWA

YASKAWA ELECTRIC CORPORATION

© 2016 YASKAWA ELECTRIC CORPORATION
Published by YASKAWA

July 2025 16-09

MANUAL NO.

HW1483602 

425/425